UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

NovaSol Salud

Sistema Web de Gestión de Citas Médicas

Caso de estudio: Clínica Vida Salud

Área: Ingeniería en Requisitos y Modelado

Docente: M.s.c Ing. Angelica Garzon Cuellar

Estudiante:

GRUPO # 20

Integrantes	Registro
Sergio Estaca Paco	221089421
Frank Miguel Yerbales Molina	218164629

Octubre de 2026

Repositorio GitHub
github.com/Kato-001/novasol-salud

Sistema en línea
novasol-salud.vercel.app

Índice

1. Perfil....5
1.1 Introducción....5
1.2 Antecedente....5
1.3 Justificación....6
1.4 Descripción del problema....7
1.4.1 Desafíos identificados....8
1.5 Formulación del problema....8
1.6 Objetivos....9
1.6.1 Objetivo general....9
1.6.2 Objetivos específicos....9
1.7 Alcance....10
1.8 Entrevista....11
2. Elementos del sistema basado en computadoras....13
2.1 Hardware....13
2.1.1 Servidor....13
2.1.2 Cliente....13
2.1.3 Medios de comunicación....14
2.1.4 Otros dispositivos....14
2.2 Software....15
2.2.1 Servidor....15
2.2.2 Cliente....15
2.2.3 Otro software adicional....16
2.3 Datos....16
2.4 Procesos....17
2.5 Gente/Usuarios....17
2.6 Documento....18
3. Tecnología para el desarrollo del software....19
3.1 Estrategia para el desarrollo del software....19
3.2 Metodología para el desarrollo del software....19
3.2.1 Características del PUDS....19
3.2.2 Características del UML....20
3.3 Herramientas del desarrollo....21
3.3.1 Software....21
3.3.2 Hardware....21
4. Posibles costos....22
4.1 Costos de desarrollo....22
4.2 Costos de software....22
4.3 Costo de infraestructura....23
5. Posibles beneficios....24

5.1 Tiempo....24
5.2 Esfuerzo....24
5.3 Costos....24
6. Posibles clientes....25
7. Modelo de dominio....26
8. Modelo de negocio....28
9. Marco teórico....30
9.1 Sistemas de información web....30
9.2 Ingeniería de requisitos....30
9.3 Método de Ishikawa....30
9.4 Proceso Unificado de Desarrollo de Software (PUDS)....30
9.5 Lenguaje Unificado de Modelado (UML)....31
9.6 Bases de datos relacionales y normalización....31
9.7 PHP y PDO....31
9.8 PostgreSQL....31
9.9 Arquitectura Modelo-Vista-Controlador (MVC)....31
9.10 Seguridad en aplicaciones web....32
9.11 Computación en la nube....32
Capítulo 1: Método de Ishikawa....33
1. Identificar problemas....33
1.1 Lista de problemas....33
1.2 Depurar problemas....34
1.3 Lista final de problemas....34
1.4 Lista de propietarios de problemas....35
1.5 Análisis de problemas....36
1.6 Estimación y cuantificación....36
1.7 Alternativas de cambio....37
1.8 Conclusión y recomendación....37
1.9 Diseñar el diagrama de Ishikawa....38
2. Identificar las principales categorías....39
3. Identificar las causas....39
4. Analizar y discutir el diagrama....40
C1. Métodos y procesos....40
C2. Personal....40
C3. Tecnología....40
C4. Información y datos....40
C5. Pacientes....40
C6. Gestión administrativa....41
5. Modelo de negocio....41
5.1 Diagrama de actividad....41
Capítulo 2. Flujo de trabajo: captura de requisitos....45

2.1 Identificar actores y casos de uso....45
Actores....45
Casos de uso....45
2.2 Priorizar los casos de uso....50
2.3 Detallar un caso de uso....51
2.4 Prototipar interfaz de usuario....65
2.5 Estructurar modelo de casos de uso....114
Capítulo 3. Flujo de trabajo: análisis....121
3.1 Análisis de la arquitectura....121
3.1.1 Identificar paquetes....121
3.1.2 Relacionar paquetes y casos de uso....122
3.1.3 Vista de paquetes....125
3.2 Analizar un caso de uso....126
3.3 Análisis de clase....137
3.4 Análisis de paquetes....140
Capítulo 4. Flujo de trabajo: diseño....142
4.1 Diseño de arquitectura....142
4.1.1 Diseño físico de la arquitectura (diagrama de despliegue)....142
4.1.2 Diseño lógico de la arquitectura (diagrama organizado en capas)....144
Capítulo 5. Diseño de datos....146
5.1 Diseño de datos lógico....146
Diagrama de clase....146
Mapeo....147
Normalización....148
5.2 Diseño de datos físico....149
Tabla de volumen....149
Script....150
Diagrama relacional....152
Actualización de tuplas (población de datos)....153
Consultas SQL....155
Procedimientos almacenados....166
Triggers (disparadores)....170
5.3 Diseñar casos de uso....174
Capítulo 6. Flujo de trabajo: implementación....194
6.1 Elección de plataforma de desarrollo del software....194
6.2 Implementación de la arquitectura del sistema....194
6.3 Implementación de la arquitectura del subsistema....196
7. Conclusiones....200
8. Recomendaciones....201
9. Bibliografía....202
Anexos....204

Anexo A. Repositorio y sistema en línea.....204
Anexo B. Cuentas de acceso.....204
Anexo C. Pruebas del sistema.....205
Anexo D. Manual breve de uso.....206
Anexo E. Modelo en Enterprise Architect.....206
Anexo F. Script de creación de las tablas.....207

1. Perfil

1.1 Introducción

La Clínica Vida Salud es un establecimiento médico que atiende las especialidades de medicina general, pediatría, ginecología, odontología, cardiología y dermatología. Administra sus citas con cuadernos, llamadas telefónicas y hojas de cálculo, lo que provoca citas duplicadas, pérdida de datos de los pacientes y falta de información para la administración.

El presente documento describe el análisis, diseño e implementación de **NovaSol Salud**, un sistema web de gestión de citas médicas desarrollado con PHP 8 y PostgreSQL, siguiendo el Proceso Unificado de Desarrollo de Software (PUDS) con notación UML (Jacobson et al., 2000; Booch et al., 2006). El trabajo se organizó en dos ciclos: el **Ciclo I** construye la base del sistema (seguridad, datos maestros de médicos y pacientes) y el **Ciclo II** construye el núcleo del negocio (citas, atención médica, reportes y el portal del paciente).

El sistema centraliza la información de pacientes, médicos, especialidades y horarios; programa citas mostrando únicamente los horarios realmente disponibles; controla los estados de las citas; y genera reportes administrativos. Se encuentra publicado en https://novasol-salud.vercel.app y su código fuente en https://github.com/Kato-001/novasol-salud.

1.2 Antecedente

La Clínica Vida Salud (caso de estudio planteado en la materia) es un centro de atención ambulatoria de la ciudad de Santa Cruz de la Sierra que cuenta con cuatro médicos, dos recepcionistas y un administrador. Atiende de lunes a sábado en horarios de mañana y tarde, y recibe a la mayoría de sus pacientes por teléfono o de forma presencial.

Desde su apertura, la recepción registra las citas en un cuaderno por médico y mantiene una hoja de cálculo con los datos de los pacientes. Este método fue suficiente mientras el volumen de atención era bajo; con el crecimiento de la clínica, la información quedó repartida entre varios cuadernos y archivos, y la recepción ya no puede saber con rapidez qué horarios están libres ni cuál es el historial de un paciente.

A continuación se presenta el análisis FODA de la organización:

Tabla 1

Análisis FODA de la Clínica Vida Salud (fortalezas y oportunidades)

Fortalezas	Oportunidades
• Médicos con especialidades diversas. • Pacientes fieles y recomendaciones de boca en boca. • Personal con experiencia en atención al público.	• Uso creciente de celulares e internet por los pacientes. • Servicios en la nube de bajo costo o gratuitos. • Pocas clínicas pequeñas con reservas en línea.

Tabla 2

Análisis FODA de la Clínica Vida Salud (debilidades y amenazas)

Debilidades	Amenazas
• Registro manual de citas en cuadernos. • Datos de pacientes duplicados o incompletos. • Sin reportes para la toma de decisiones.	• Clínicas competidoras con reservas por internet. • Pérdida de pacientes por largas esperas al teléfono. • Riesgo de pérdida de información sin respaldo.

1.3 Justificación

La programación manual de citas genera errores que afectan directamente la atención: dos pacientes citados a la misma hora con el mismo médico, citas olvidadas, cancelaciones que

no liberan el horario y pacientes cuyos datos no se encuentran. Un sistema centralizado elimina estas fallas y libera tiempo del personal para atender a los pacientes.

El desarrollo del sistema se justifica por los siguientes motivos:

- **Justificación operativa:** el sistema calcula la disponibilidad real de cada médico y la base de datos impide las citas superpuestas, por lo que desaparecen las citas duplicadas.
- **Justificación técnica:** PHP y PostgreSQL son tecnologías libres, maduras y conocidas por el equipo; la publicación en la nube (Vercel y Supabase) no requiere servidores propios.
- **Justificación económica:** se usan herramientas gratuitas, y el ahorro de tiempo en la recepción y la reducción de inasistencias compensan el costo de desarrollo.
- **Justificación social:** los pacientes pueden reservar y cancelar sus citas desde el celular, sin esperar al teléfono, y consultar sus resultados de estudios.

1.4 Descripción del problema

La recepción de la clínica registra a los pacientes, consulta la disponibilidad de los médicos, programa las citas y comunica la fecha y hora de atención. Todo se realiza de forma manual: para saber si un médico está libre, la recepcionista revisa el cuaderno de ese médico; si el paciente quiere cambiar la cita, se tacha y se vuelve a escribir; y para preparar un informe mensual se cuentan las citas a mano.

Como consecuencia, se presentan citas duplicadas para un mismo médico y horario, pérdida o registro incorrecto de los datos de los pacientes, falta de control sobre las cancelaciones y las inasistencias, demoras en la búsqueda del historial de un paciente y ausencia de información actualizada para la administración.

1.4.1 Desafíos identificados

- **Evitar citas superpuestas:** garantizar que un médico (y un paciente) no tenga dos citas a la misma hora, incluso si dos recepcionistas reservan al mismo tiempo.
- **Conocer la disponibilidad real:** combinar los horarios semanales, los feriados, las vacaciones y las citas ya tomadas para mostrar solo los horarios libres.
- **Controlar el ciclo de vida de la cita:** programada, confirmada, reprogramada, cancelada, atendida o inasistencia, conservando el historial de cada cambio.
- **Centralizar la información clínica:** historia clínica, consultas, tratamientos y estudios del paciente en un solo lugar.
- **Proteger la información:** acceso por roles, contraseñas cifradas y registro de las operaciones en una bitácora.
- **Generar reportes:** citas por fecha, médico, especialidad y estado para la toma de decisiones.

1.5 Formulación del problema

La Clínica Vida Salud ha crecido en número de pacientes y médicos, pero mantiene un proceso manual de programación de citas que provoca duplicidad de citas, pérdida de datos, falta de control de cancelaciones e inasistencias y ausencia de información para la administración. Ante esta situación surge la siguiente pregunta:

> *¿Cómo desarrollar un sistema web utilizando PHP y PostgreSQL que permita administrar pacientes, médicos, especialidades y citas médicas, evitando conflictos de horarios y facilitando el control de las atenciones de la Clínica Vida Salud?*

1.6 Objetivos

1.6.1 Objetivo general

Desarrollar un sistema web de gestión de citas médicas utilizando PHP como lenguaje del lado del servidor y PostgreSQL como sistema gestor de base de datos, con el propósito de mejorar el registro, la programación, el seguimiento y el control de las citas médicas de la Clínica Vida Salud.

1.6.2 Objetivos específicos

- Analizar los procesos de registro y programación de citas de la clínica mediante el método de Ishikawa y el modelo de negocio.
- Capturar y especificar los requisitos con casos de uso, priorizados y distribuidos en dos ciclos del Proceso Unificado.
- Diseñar una base de datos relacional normalizada hasta la tercera forma normal, con claves primarias, foráneas y restricciones.
- Implementar la seguridad del sistema: inicio de sesión, roles, permisos, recuperación de contraseña y bitácora (Ciclo I).
- Implementar la gestión de especialidades, médicos, horarios y pacientes (Ciclo I).
- Programar, reprogramar y cancelar citas verificando la disponibilidad del médico en el servidor y en la base de datos (Ciclo II).
- Registrar la atención médica, los tratamientos y los estudios, y generar reportes de citas (Ciclo II).
- Verificar el funcionamiento del sistema mediante pruebas funcionales, de integridad y de seguridad.

1.7 Alcance

El sistema es una aplicación web accesible desde cualquier navegador (computadora o celular). Está destinado principalmente al personal de la clínica (administrador, recepcionistas y médicos) y ofrece además un portal para los pacientes, todos con el mismo inicio de sesión y un menú distinto según el rol.

Tabla 3

Alcance del sistema por ciclo y paquete

Ciclo	Paquete	Alcance
Ciclo I	P1 Seguridad y acceso	Inicio y cierre de sesión, recuperación de contraseña por correo, roles y permisos, usuarios, configuración y bitácora.
Ciclo I	P2 Gestión médica	Especialidades, médicos con una o varias especialidades, horarios de atención y bloqueos de agenda.
Ciclo I	P3 Gestión de pacientes	Registro, búsqueda, modificación y desactivación de pacientes; historia clínica y contactos de emergencia.
Ciclo II	P3 Gestión de pacientes	Autoregistro del paciente en el portal.
Ciclo II	P4 Gestión de citas	Disponibilidad, reserva, reprogramación, cancelación, asistencia y agenda médica sin citas superpuestas.
Ciclo II	P5 Atención médica	Consulta médica, tratamientos, estudios con resultados e historial clínico.
Ciclo II	P6 Reportes	Reportes por fecha, médico, especialidad y estado; exportación a CSV y PDF.
Ciclo II	Página pública	Portada de la clínica con especialidades, médicos y horarios. Para reservar se inicia sesión y el sistema vuelve a la reserva elegida.

Fuera del alcance: facturación y cobros, envío de recordatorios por WhatsApp o SMS (los recordatorios se muestran dentro del sistema), videollamadas y la integración con equipos de laboratorio.

1.8 Entrevista

Para la captura de requisitos se preparó una entrevista dirigida a la administración y a la recepción de la clínica. Como la Clínica Vida Salud es un caso de estudio propuesto por la docente, las respuestas se elaboraron a partir del enunciado del proyecto y de la situación descrita en él.

Tabla 4

Datos de la entrevista

Entrevistados	Administrador de la clínica y recepcionista (roles del caso de estudio)
Entrevistadores	Sergio Estaca Paco y Frank Miguel Yerbales Molina
Objetivo	Conocer el proceso actual de programación de citas, sus problemas y lo que se espera del sistema.
Técnica	Entrevista semiestructurada con preguntas abiertas.

Tabla 5

Preguntas y respuestas de la entrevista

Pregunta	Respuesta
¿Cómo registran hoy las citas?	En un cuaderno por médico. Los datos del paciente se anotan en una hoja de cálculo que no siempre se actualiza.
¿Qué problemas son los más frecuentes?	Citas duplicadas a la misma hora, pacientes que no encontramos porque se registraron con otro nombre y cancelaciones que nadie anota.
¿Cómo saben si un médico está libre?	Se revisa el cuaderno del médico y se le pregunta por teléfono si cambió su horario o si tiene permiso ese día.
¿Qué información necesita la administración?	Cuántas citas se atienden por médico y especialidad, cuántas se cancelan y cuántos pacientes no asisten.
¿Quiénes usarían el sistema?	La recepción para las citas, los médicos para ver su agenda y registrar la consulta, la administración para la configuración y los reportes, y los pacientes para reservar.
¿Qué datos se registran del paciente?	CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo, dirección, tipo de sangre, alergias y un contacto de emergencia.

Pregunta	Respuesta
¿Cuánto dura una cita?	Treinta minutos, aunque nos gustaría poder cambiarlo.
¿Qué pasa cuando un paciente cancela?	El horario debería quedar libre para otro paciente, pero muchas veces queda ocupado en el cuaderno.

2. Elementos del sistema basado en computadoras

Un sistema basado en computadoras está formado por hardware, software, datos, procesos, personas y documentos que trabajan juntos para cumplir un objetivo (Pressman y Maxim, 2021). A continuación se describe cada elemento en NovaSol Salud.

2.1 Hardware

2.1.1 Servidor

El sistema se publica en la nube, por lo que la clínica no necesita comprar un servidor. La aplicación se ejecuta como función *serverless* en Vercel y la base de datos en Supabase, ambas en la región de São Paulo.

Tabla 6

Hardware del servidor

Componente	Proveedor / versión	Características
Servidor de aplicación	Vercel (función serverless)	PHP 8.3, 1 GB de memoria por función, región gru1 (São Paulo), CDN global para los archivos estáticos
Servidor de base de datos	Supabase (plan gratuito)	PostgreSQL 17, 500 MB de almacenamiento, 2 núcleos compartidos, región sa-east-1
Servidor local alternativo	Laragon / XAMPP	Procesador de 2 GHz, 4 GB de RAM, 20 GB de disco

2.1.2 Cliente

El sistema funciona en cualquier computadora o celular con un navegador actual. El equipo usado en el desarrollo y las pruebas tiene las siguientes características:

Tabla 7

Hardware del cliente

Componente	Versión	Características
Procesador	Intel Core i5-1035G1 (10.ª generación)	4 núcleos, 1,0 GHz (hasta 3,6 GHz)
Memoria RAM	DDR4	24 GB
Disco	SSD MSI S270	960 GB
Pantalla	Laptop	1366 × 768 px
Requisito mínimo del cliente	Cualquier PC o celular	2 GB de RAM, navegador actualizado y conexión a internet

2.1.3 Medios de comunicación

- Conexión a internet en la clínica (mínimo 5 Mbps) y en los celulares de los pacientes.
- Router inalámbrico (Wi-Fi) y red local para las computadoras de recepción y consultorios.
- Protocolo HTTPS entre el navegador y Vercel, y conexión cifrada (TLS) entre Vercel y PostgreSQL.
- Correo electrónico (SMTP de Gmail) para enviar el código de recuperación de contraseña.

2.1.4 Otros dispositivos

Tabla 8

Otros dispositivos

Dispositivo	Uso
Impresora	Imprimir reportes y la agenda del día (opción Imprimir / PDF).

Dispositivo	Uso
Celulares y tabletas	Uso del portal del paciente y consulta de la agenda por los médicos.
Lector de código QR (cámara del celular)	Acceso rápido al sistema y al repositorio desde los códigos QR del informe.

2.2 Software

2.2.1 Servidor

Tabla 9

Software del servidor

Nombre	Versión	Uso
Sistema operativo	Amazon Linux 2023 (gestionado por Vercel)	Ejecución de las funciones serverless
Lenguaje del servidor	PHP 8.3 (vercel-php 0.7.4)	Lógica de la aplicación (MVC propio)
Gestor de base de datos	PostgreSQL 17 (Supabase)	Almacenamiento de los datos
Extensión	pdo_pgsql	Conexión de PHP con PostgreSQL
Extensión de PostgreSQL	btree_gist	Restricciones de exclusión contra citas superpuestas

2.2.2 Cliente

Tabla 10

Software del cliente

Nombre	Versión
Sistema operativo	Windows 10/11, Android, iOS o Linux
Navegador	Google Chrome, Microsoft Edge, Firefox o Safari (versiones actuales)

2.2.3 Otro software adicional

Tabla 11

Software adicional

Nombre	Descripción
Visual Studio Code	Editor de código
Git y GitHub	Control de versiones y publicación del código
Enterprise Architect 17.2 (Sparx Systems)	Modelo UML del proyecto (Docs/EA/NovaSol_Salud.qea)
PlantUML y Graphviz	Figuras de los diagramas con la notación y el estilo de Enterprise Architect
Laragon	Servidor local con PHP para desarrollo
Microsoft Word / LibreOffice	Elaboración de la documentación
Supabase Studio	Administración de la base de datos en la nube

2.3 Datos

El sistema almacena los siguientes datos, organizados en 25 tablas del esquema `clinica`:

- **Usuario:** nombre, usuario, correo, contraseña cifrada, estado, último acceso y rol.
- **Rol y permiso:** nombre del rol y permisos asignados por módulo.
- **Paciente:** código, CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo, dirección, tipo de sangre y estado; además, su historia clínica y contactos de emergencia.
- **Médico:** código, registro profesional, nombres, apellidos, teléfono, correo, estado y especialidades.
- **Especialidad:** nombre, descripción y estado.

- **Horario y bloqueo de agenda:** día de la semana, hora de inicio y fin; feriados, vacaciones y permisos.
- **Cita:** código, fecha, hora de inicio y fin, modalidad, motivo, paciente, médico, especialidad, estado y su historial de cambios.
- **Consulta, tratamiento y estudio:** signos vitales, diagnóstico, medicamentos indicados y estudios con su resultado.
- **Bitácora y notificación:** operaciones realizadas por cada usuario y avisos a los pacientes.

2.4 Procesos

- **Autenticación y control de acceso:** inicio de sesión, permisos por rol y recuperación de contraseña por correo.
- **Gestión de datos maestros:** registro de especialidades, médicos, horarios y pacientes.
- **Programación de citas:** consulta de disponibilidad, reserva, reprogramación y cancelación sin cruces de horario.
- **Atención médica:** registro de asistencia, consulta, tratamiento y estudios.
- **Reportes:** citas por fecha, médico, especialidad y estado, y resumen del período.

2.5 Gente/Usuarios

Tabla 12

Usuarios del sistema

Usuario	Descripción
Administrador	Configura el sistema: usuarios, roles, permisos, especialidades, médicos, horarios y catálogo de estudios; consulta la bitácora y los reportes.
Recepcionista	Registra pacientes, programa, reprograma y cancela citas, registra la asistencia y los resultados de estudios.

Usuario	Descripción
Médico	Consulta su agenda, atiende las citas, registra la consulta, el tratamiento y los estudios.
Paciente	Se registra en el portal, reserva y cancela sus citas, y consulta su historial y resultados.

2.6 Documento

- Ficha del paciente con su historia clínica.
- Comprobante de la cita (código, fecha, hora, médico y especialidad).
- Agenda diaria del médico.
- Reportes de citas en CSV y PDF.
- Bitácora de operaciones del sistema.
- Manual breve de uso (Anexo D).

3. Tecnología para el desarrollo del software

3.1 Estrategia para el desarrollo del software

Se adoptó la estrategia de **desarrollo a medida**, iterativo e incremental. El sistema se construyó en dos ciclos: el primero entrega la base (seguridad y datos maestros) y el segundo el núcleo del negocio (citas, atención y reportes). Cada ciclo recorre los flujos de trabajo de captura de requisitos, análisis, diseño, implementación y pruebas, de modo que al final de cada uno existe un sistema funcionando que puede revisarse.

Tabla 13

Estrategia de desarrollo por ciclos

Ciclo	Casos de uso	Entregable
Ciclo I	CU1, CU2, CU3, CU4, CU5, CU6, CU7	Ingreso al sistema, administración de usuarios, especialidades, médicos, horarios y pacientes.
Ciclo II	CU8, CU9, CU10, CU11, CU12, CU13, CU14, CU15, CU16, CU17, CU18, CU19, CU20	Programación de citas sin cruces, agenda, atención médica, estudios, reportes y portal del paciente.

3.2 Metodología para el desarrollo del software

Se utilizó el **Proceso Unificado de Desarrollo de Software (PUDS)** con el **Lenguaje Unificado de Modelado (UML)** como lenguaje de especificación (Jacobson et al., 2000).

3.2.1 Características del PUDS

- **Dirigido por casos de uso:** los casos de uso guían el análisis, el diseño, la implementación y las pruebas.

- **Centrado en la arquitectura:** la arquitectura en capas (MVC con servicios) se define en los primeros casos de uso y se mantiene estable.
- **Iterativo e incremental:** el sistema crece en ciclos; cada ciclo agrega casos de uso sobre una base que ya funciona.

Tabla 14

Fases del Proceso Unificado en el proyecto

Fase	Actividades	Resultado en el proyecto
Inicio	Perfil, entrevista, Ishikawa, modelo de negocio y casos de uso principales.	Capítulo 1 y modelo de casos de uso inicial.
Elaboración	Análisis de la arquitectura, paquetes, base de datos y casos de uso críticos (CU1, CU10).	Modelo de análisis y diseño, script de la base de datos.
Construcción	Implementación de los casos de uso del Ciclo I y del Ciclo II.	Sistema funcionando y probado.
Transición	Publicación en Vercel y Supabase, cuentas de prueba y documentación.	Sistema en línea e informe final.

3.2.2 Características del UML

UML es un lenguaje gráfico estándar para especificar, visualizar, construir y documentar los sistemas de software (Object Management Group [OMG], 2017). En este proyecto se utilizaron los diagramas de casos de uso, actividades, comunicación, clases, estados, paquetes, secuencia, componentes y despliegue.

3.3 Herramientas del desarrollo

3.3.1 Software

Tabla 15

Herramientas de software

Categoría	Herramienta	Versión	Uso
Lenguaje	PHP	8.3	Servidor (MVC propio, sin framework)
Base de datos	PostgreSQL	17	Gestor de base de datos
Interfaz	HTML5, CSS3, JavaScript	—	Pantallas, diseño adaptable y validaciones
Editor	Visual Studio Code	1.9x	Escritura del código
Versiones	Git y GitHub	2.4x	Control de versiones
Modelado	Enterprise Architect · PlantUML	17.2 · 1.2024	Modelo UML (.qea) y figuras de los diagramas
Publicación	Vercel y Supabase	—	Aplicación y base de datos en la nube

3.3.2 Hardware

Tabla 16

Hardware para el desarrollo

Componente	Requisito mínimo para desarrollar
Procesador	Intel Core i5 de 8.ª generación o AMD Ryzen 5
Memoria RAM	8 GB
Almacenamiento	256 GB SSD
Sistema operativo	Windows 10/11
Conexión	Internet de 10 Mbps

4. Posibles costos

Los costos se estiman en bolivianos (Bs), con un tipo de cambio de Bs 6,96 por dólar. Al ser un proyecto académico, el desarrollo lo realizaron los integrantes del grupo; los montos indican cuánto costaría el sistema en un entorno real.

4.1 Costos de desarrollo

Tabla 17

Costos de desarrollo estimados

Concepto	Cantidad	Costo unitario (Bs)	Total (Bs)
Analista-programador	2 personas × 3 meses	4 000 / mes	24 000
Diseño de la base de datos y diagramas UML	Incluido	—	—
Pruebas y documentación	1 mes (1 persona)	4 000	4 000
Total desarrollo			**28 000**

4.2 Costos de software

Tabla 18

Costos de software

Software	Licencia	Costo (Bs)
PHP 8.3	Libre (PHP License)	0
PostgreSQL 17	Libre (PostgreSQL License)	0
Visual Studio Code	Gratuito	0
Git, GitHub, PlantUML	Libres / gratuitos	0

Software	Licencia	Costo (Bs)
Enterprise Architect	Versión de prueba (30 días)	0
Dominio propio (opcional)	Anual	≈ 100
Total software		**≈ 100 / año**

4.3 Costo de infraestructura

Tabla 19

Costos de infraestructura

Servicio	Plan gratuito	Plan profesional (mensual)	Costo anual profesional (Bs)
Vercel (aplicación)	Hobby: Bs 0	Pro: USD 20	1 670
Supabase (base de datos)	Free: Bs 0	Pro: USD 25	2 088
Correo (Gmail)	Bs 0	Bs 0	0
Total	**Bs 0**	**USD 45**	**3 758**

El sistema funciona actualmente con los planes gratuitos. Los planes profesionales solo serían necesarios con un volumen alto de pacientes o si se requieren copias de seguridad diarias garantizadas.

5. Posibles beneficios

5.1 Tiempo

Tabla 20

Ahorro de tiempo estimado

Tarea	Proceso manual (estimado)	Con NovaSol Salud
Programar una cita	5 a 10 minutos (revisar cuadernos y llamar al médico)	Menos de 1 minuto
Buscar el historial de un paciente	5 minutos o más	Inmediato (búsqueda por CI o nombre)
Reporte mensual de citas	2 a 3 horas	Segundos
Reservar una cita (paciente)	Espera al teléfono	Desde el celular, a cualquier hora

5.2 Esfuerzo

- La recepción deja de revisar varios cuadernos: el sistema muestra solo los horarios libres.
- Las cancelaciones liberan el horario automáticamente.
- Los médicos ven su agenda del día y registran la consulta en un solo formulario.
- La administración obtiene reportes sin contar citas a mano.

5.3 Costos

- Menos pérdidas por citas duplicadas o pacientes que se van por largas esperas.
- Ahorro en papel y archivo físico.
- Sin compra de servidores: la infraestructura en la nube es gratuita en el volumen actual.

- El seguimiento de inasistencias permite tomar medidas y aprovechar mejor los horarios de los médicos.

6. Posibles clientes

NovaSol Salud puede adaptarse a otros establecimientos que atienden con cita previa:

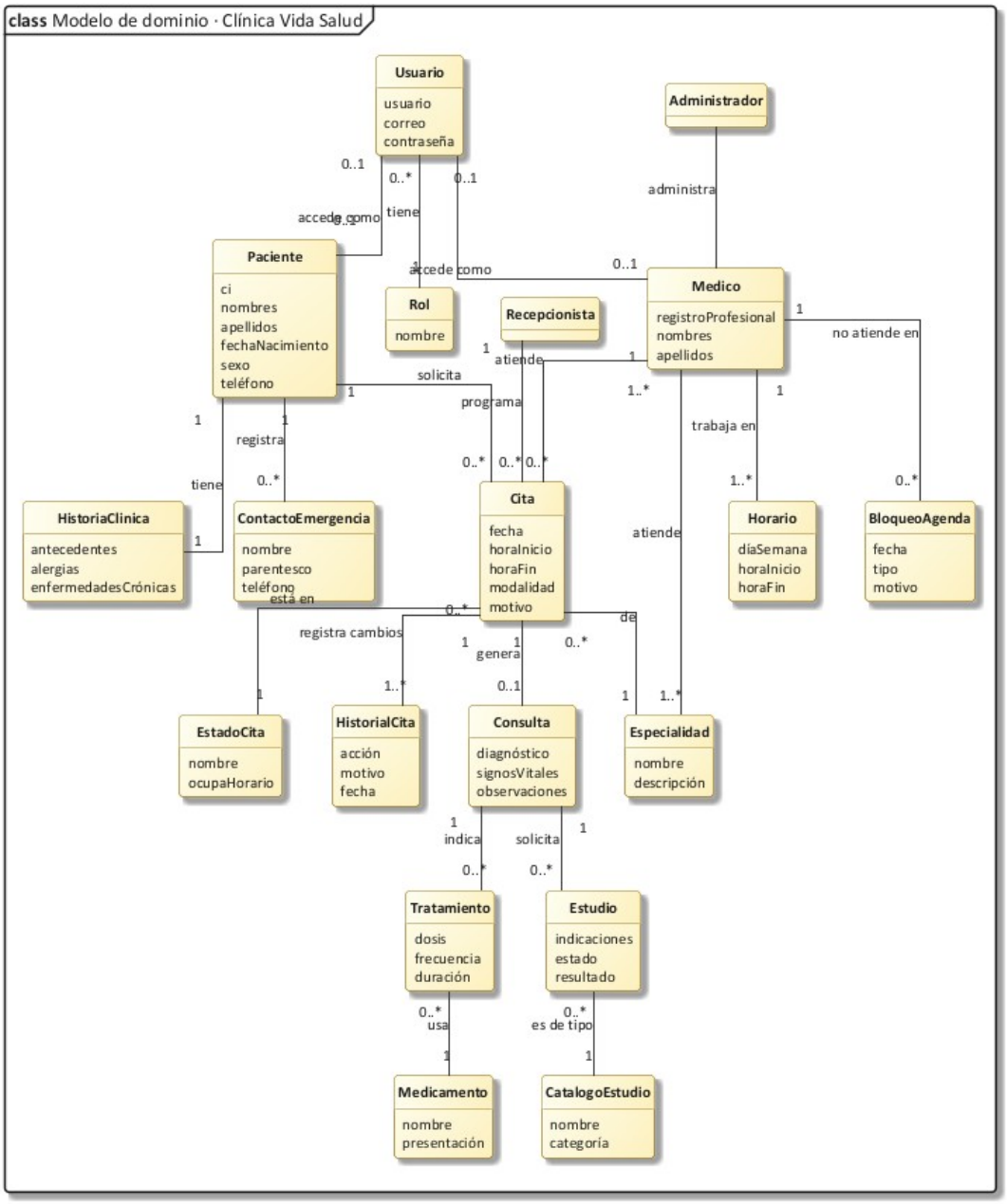
- **Clínicas privadas pequeñas y medianas** de Santa Cruz que todavía programan sus citas de forma manual.
- **Consultorios médicos y odontológicos** con uno o varios profesionales.
- **Centros de especialidades** (pediatría, ginecología, dermatología) que necesitan controlar la agenda de cada especialista.
- **Laboratorios y centros de imagen** que programan estudios y entregan resultados a los pacientes.
- **Centros de salud de instituciones** (universidades, empresas, ONG) que atienden a su personal o beneficiarios.

7. Modelo de dominio

El modelo de dominio representa los conceptos más importantes del negocio y sus relaciones, sin detalles de implementación (Larman, 2003). El concepto central es la **Cita**, que relaciona a un paciente con un médico en una especialidad, tiene un estado y puede generar una consulta con tratamientos y estudios.

Figura 1

Modelo de dominio de la Clínica Vida Salud



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

8. Modelo de negocio

El modelo de negocio describe cómo trabaja la organización. Se identifican los actores del negocio y los casos de uso del negocio, y se representan los procesos con diagramas de actividades (Jacobson et al., 2000).

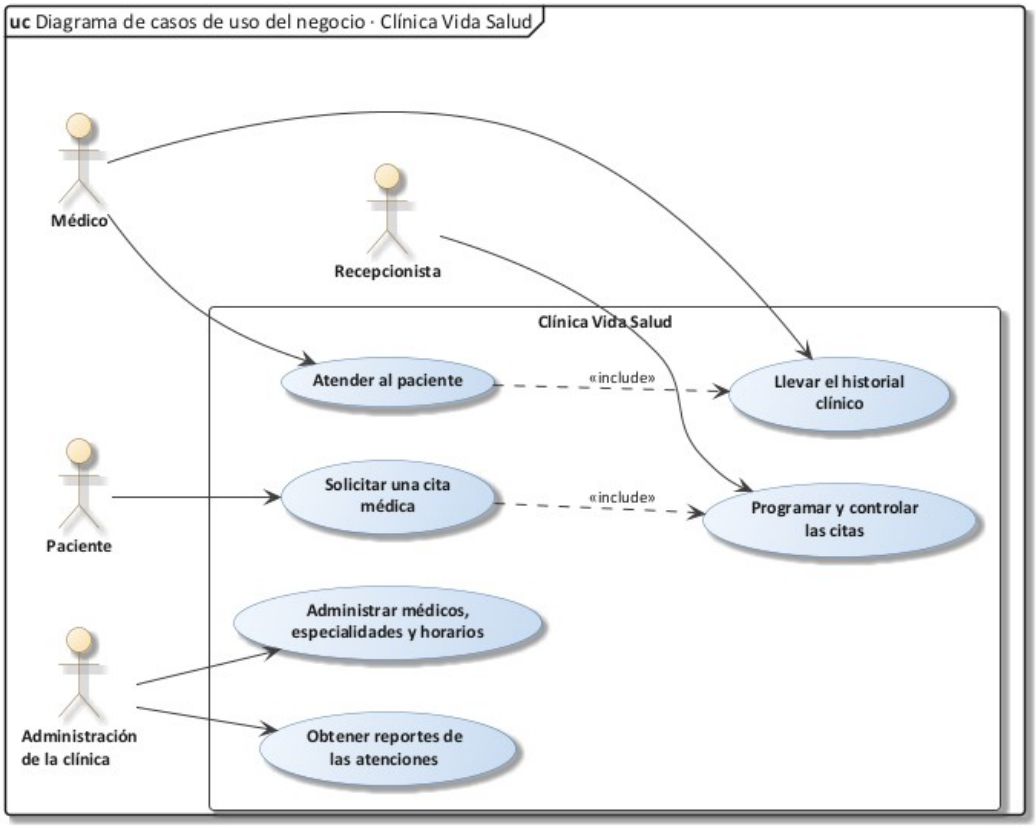
Tabla 21

Actores y trabajadores del negocio

Actor / trabajador del negocio	Responsabilidad
Paciente	Solicita citas, asiste a la consulta y recibe tratamientos y estudios.
Recepcionista	Registra pacientes, consulta la disponibilidad, programa, reprograma y cancela citas y registra la asistencia.
Médico	Atiende a los pacientes según su agenda, registra la consulta e indica tratamientos y estudios.
Administración de la clínica	Gestiona médicos, especialidades y horarios, y consulta los reportes.

Figura 2

Diagrama de casos de uso del negocio



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

9. Marco teórico

9.1 Sistemas de información web

Un sistema de información web se ejecuta en un servidor y se usa desde un navegador mediante el protocolo HTTP. Permite el acceso desde distintos dispositivos sin instalar software, centraliza la información en una base de datos y facilita la actualización del sistema, porque los cambios se aplican en un solo lugar (Pressman y Maxim, 2021).

9.2 Ingeniería de requisitos

Es el proceso de descubrir, analizar, documentar y validar los servicios que el sistema debe ofrecer y sus restricciones (Sommerville, 2011). Distingue los requisitos funcionales, que describen lo que el sistema debe hacer, de los no funcionales, que describen cualidades como la seguridad, el rendimiento o la usabilidad.

9.3 Método de Ishikawa

El diagrama de Ishikawa, o de causa y efecto, organiza las causas de un problema en categorías principales que convergen en el efecto estudiado, lo que facilita identificar la causa raíz y proponer soluciones (Ishikawa, 1986).

9.4 Proceso Unificado de Desarrollo de Software (PUDS)

Es un proceso dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental. Se organiza en flujos de trabajo (captura de requisitos, análisis, diseño, implementación y pruebas) que se repiten en cada ciclo, y en cuatro fases: inicio, elaboración, construcción y transición (Jacobson et al., 2000).

9.5 Lenguaje Unificado de Modelado (UML)

UML es el lenguaje gráfico estándar para modelar sistemas de software (OMG, 2017). Las clases de análisis se clasifican en interfaz («boundary»), control («control») y entidad («entity») (Booch et al., 2006).

9.6 Bases de datos relacionales y normalización

El modelo relacional organiza los datos en tablas relacionadas mediante claves (Codd, 1970). La normalización elimina redundancias aplicando la primera, segunda y tercera forma normal (Elmasri y Navathe, 2007).

9.7 PHP y PDO

PHP es un lenguaje del lado del servidor diseñado para la web. PDO es su capa de acceso a datos; con consultas parametrizadas separa el código SQL de los datos del usuario y previene la inyección SQL (The PHP Group, 2026).

9.8 PostgreSQL

PostgreSQL es un gestor de bases de datos relacional de código abierto que garantiza las propiedades ACID y ofrece restricciones de exclusión, disparadores y procedimientos en PL/pgSQL (The PostgreSQL Global Development Group, 2026).

9.9 Arquitectura Modelo-Vista-Controlador (MVC)

MVC separa la aplicación en modelo (datos y reglas), vista (presentación) y controlador (coordinación de las peticiones), lo que facilita el mantenimiento y las pruebas (Larman, 2003).

9.10 Seguridad en aplicaciones web

El proyecto aplica las recomendaciones de OWASP Foundation (2021): contraseñas cifradas, consultas parametrizadas, protección contra XSS y CSRF, sesiones seguras y control de acceso en el servidor.

9.11 Computación en la nube

Vercel ejecuta el código como funciones serverless que se crean bajo demanda (Vercel, 2026) y Supabase ofrece bases de datos PostgreSQL administradas (Supabase, 2026).

Capítulo 1: Método de Ishikawa

1. Identificar problemas

1.1 Lista de problemas

A partir de la entrevista y del análisis del proceso actual se identificaron los siguientes problemas:

Tabla 22

Lista inicial de problemas

Código	Problema
P1	Las citas se registran en cuadernos por médico.
P2	Se programan dos citas a la misma hora con el mismo médico.
P3	No se sabe con rapidez qué horarios están libres.
P4	Los datos de los pacientes se duplican o están incompletos.
P5	Las cancelaciones no liberan el horario.
P6	No se registran las inasistencias.
P7	El historial de citas y consultas de un paciente está disperso.
P8	No existen reportes de citas por médico, especialidad o estado.
P9	Los cambios de horario y los permisos de los médicos no se comunican a tiempo.
P10	Cualquier persona puede modificar los cuadernos y archivos (sin control de acceso).
P11	Los pacientes esperan al teléfono para obtener una cita.
P12	No existe respaldo de la información.

1.2 Depurar problemas

Tabla 23

Depuración de problemas

Problemas	Decisión	Problema depurado
P1 y P12	Se unen	Registro manual sin sistema ni respaldo
P2 y P3	Se unen	Citas superpuestas por desconocer la disponibilidad
P5 y P6	Se unen	Sin control de cancelaciones e inasistencias
P4 y P7	Se unen	Datos de pacientes duplicados e historial disperso
P8	Se mantiene	Sin reportes para la administración
P9	Se mantiene	Cambios de horario no comunicados
P10	Se mantiene	Sin control de acceso
P11	Se mantiene	Pacientes sin forma de reservar por sí mismos

1.3 Lista final de problemas

Tabla 24

Lista final de problemas

Código	Problema
PF1	Registro manual sin sistema ni respaldo
PF2	Citas superpuestas por desconocer la disponibilidad
PF3	Sin control de cancelaciones e inasistencias
PF4	Datos de pacientes duplicados e historial disperso
PF5	Sin reportes para la administración
PF6	Cambios de horario no comunicados
PF7	Sin control de acceso
PF8	Pacientes sin forma de reservar por sí mismos

1.4 Lista de propietarios de problemas

Tabla 25

Propietarios de los problemas

Propietario	Descripción
Prop1	Administrador de la clínica
Prop2	Recepcionista
Prop3	Médico
Prop4	Paciente

Tabla 26

Matriz de problemas y propietarios

Problema	Prop1	Prop2	Prop3	Prop4
PF1	X	X	X	
PF2	X	X	X	X
PF3	X	X	X	
PF4		X	X	X
PF5	X			
PF6	X	X	X	X
PF7	X	X		
PF8		X		X

1.5 Análisis de problemas

PF1 es el problema raíz: al no existir un sistema, la información queda en cuadernos y archivos aislados. De él derivan **PF2** (nadie ve la disponibilidad completa), **PF3** (las cancelaciones no se registran en un lugar común), **PF4** (los datos se copian varias veces) y **PF5** (no hay de dónde obtener estadísticas). **PF6** y **PF7** son problemas de organización que un sistema con roles y bloqueos de agenda resuelve, y **PF8** limita la atención a la capacidad del teléfono de la recepción.

1.6 Estimación y cuantificación

Tabla 27

Estimación y cuantificación de los problemas

Problema	Tipo	Cuantificación estimada
PF1	Cualitativo	Toda la información depende de cuadernos; una pérdida o daño borra el historial.
PF2	Cuantitativo	Entre 2 y 4 citas duplicadas por semana, que obligan a reprogramar.
PF3	Cuantitativo	Alrededor del 10 % de las citas se cancelan o no asisten sin registro.
PF4	Cuantitativo	5 minutos o más para encontrar el historial de un paciente.
PF5	Cuantitativo	2 a 3 horas de trabajo manual para un reporte mensual.
PF6	Cualitativo	Pacientes citados en días de permiso o feriado del médico.
PF7	Cualitativo	Cualquier persona puede alterar las citas sin dejar rastro.
PF8	Cuantitativo	Esperas de varios minutos al teléfono en horas pico.

Nota. Valores estimados a partir de la entrevista del caso de estudio.

1.7 Alternativas de cambio

Tabla 28

Alternativas de cambio

Problema	Alternativa de cambio
PF1	ALT1. Sistema web con base de datos PostgreSQL publicado en la nube, con respaldo del proveedor.
PF2	ALT2. Cálculo automático de horarios libres y restricción en la base de datos que impide citas superpuestas.
PF3	ALT3. Estados de la cita (cancelada, inasistencia) con motivo e historial de cambios.
PF4	ALT4. Registro único de pacientes por CI, con historia clínica y historial de citas.
PF5	ALT5. Módulo de reportes por fecha, médico, especialidad y estado, exportable a CSV y PDF.
PF6	ALT6. Horarios semanales y bloqueos de agenda (feriados, vacaciones y permisos) por médico.
PF7	ALT7. Inicio de sesión con roles, permisos y bitácora de operaciones.
PF8	ALT8. Portal del paciente para reservar y cancelar citas en línea.

1.8 Conclusión y recomendación

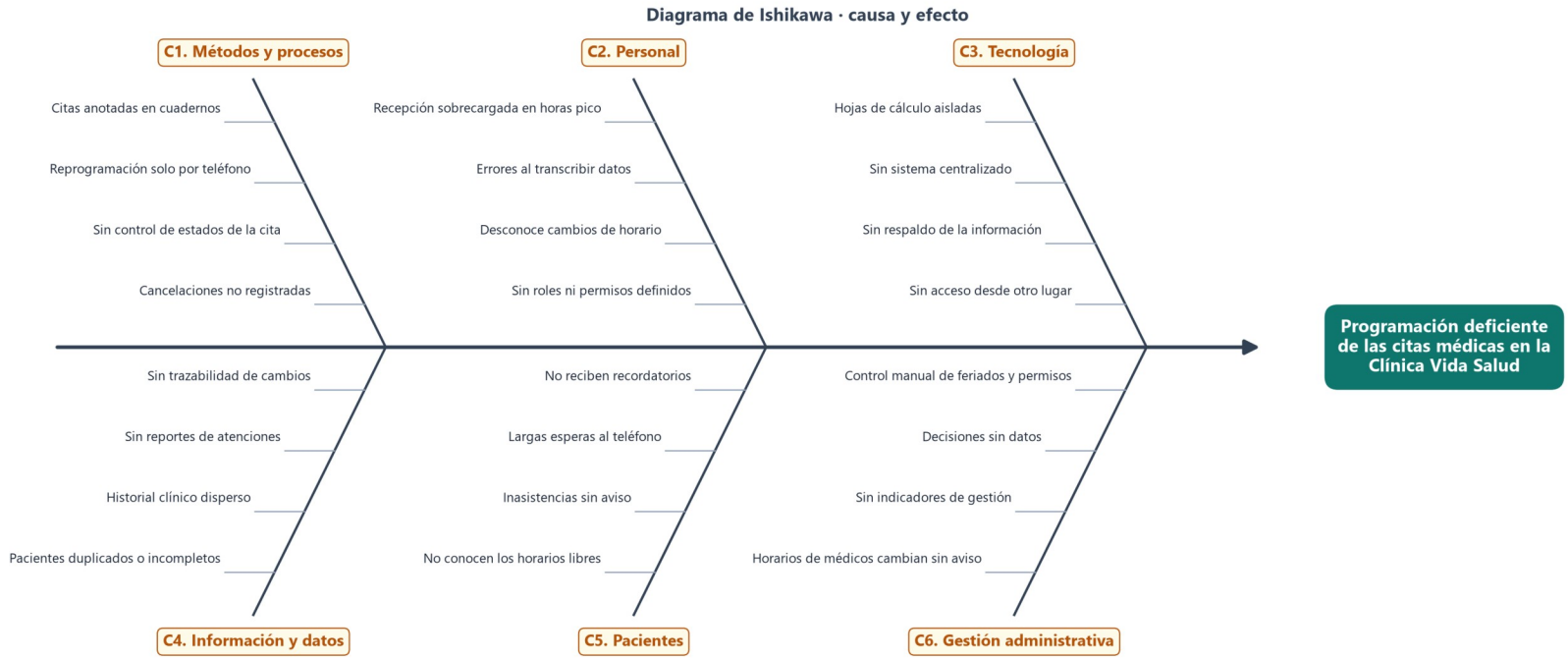
El análisis muestra que la mayoría de los problemas tienen una causa común: la falta de un sistema centralizado. Las alternativas ALT1 a ALT8 se resuelven con un único sistema web que controle la disponibilidad de los médicos, el ciclo de vida de las citas, la información de los pacientes y el acceso del personal.

Se recomienda desarrollar NovaSol Salud en dos ciclos: primero la seguridad y los datos maestros (ALT1, ALT4, ALT6 y ALT7) y luego las citas, la atención, los reportes y el portal (ALT2, ALT3, ALT5 y ALT8), capacitando al personal antes de su puesta en marcha.

1.9 Diseñar el diagrama de Ishikawa

Figura 3

Diagrama de Ishikawa del problema de programación de citas



Nota. Elaboración propia a partir de la entrevista y del enunciado del caso de estudio.

2. Identificar las principales categorías

Tabla 29

Categorías del diagrama de Ishikawa

Categoría	Descripción
C1. Métodos y procesos	Forma en que se registran, reprograman y cancelan las citas.
C2. Personal	Carga de trabajo, errores y responsabilidades del personal de recepción.
C3. Tecnología	Herramientas usadas para guardar y consultar la información.
C4. Información y datos	Calidad, unicidad y disponibilidad de los datos de pacientes y citas.
C5. Pacientes	Relación del paciente con la clínica para obtener y cumplir sus citas.
C6. Gestión administrativa	Control de horarios de los médicos e indicadores para decidir.

3. Identificar las causas

Tabla 30

Causas por categoría

Categoría	Causas
C1. Métodos y procesos	Citas anotadas en cuadernos; reprogramación solo por teléfono; sin control de estados; cancelaciones no registradas.
C2. Personal	Recepción sobrecargada en horas pico; errores al transcribir; desconoce cambios de horario; sin roles definidos.
C3. Tecnología	Hojas de cálculo aisladas; sin sistema centralizado; sin respaldo; sin acceso remoto.
C4. Información y datos	Pacientes duplicados o incompletos; historial disperso; sin reportes; sin trazabilidad de cambios.
C5. Pacientes	No conocen los horarios libres; inasistencias sin aviso; esperas al teléfono; sin recordatorios.
C6. Gestión administrativa	Horarios que cambian sin aviso; sin indicadores; decisiones sin datos; control manual de feriados y permisos.

4. Analizar y discutir el diagrama

C1. Métodos y procesos

Problema: El proceso manual no deja historial ni controla el estado de la cita.

Solución propuesta: Estados de la cita con transiciones controladas e historial de cada cambio (CU11, CU12, CU13).

C2. Personal

Problema: El personal pierde tiempo revisando cuadernos y comete errores al transcribir.

Solución propuesta: Formularios con validación y selección de horarios libres; roles y permisos (CU1 a CU3).

C3. Tecnología

Problema: La información está en herramientas aisladas y sin respaldo.

Solución propuesta: Sistema web con PostgreSQL en la nube (Vercel y Supabase).

C4. Información y datos

Problema: Los datos se duplican y no se pueden consultar ni resumir.

Solución propuesta: Paciente único por CI, historia clínica y reportes (CU7, CU16, CU20).

C5. Pacientes

Problema: El paciente depende del teléfono para obtener una cita.

Solución propuesta: Portal del paciente con reserva, cancelación y recordatorios (CU8 a CU12, CU19).

C6. Gestión administrativa

Problema: La administración no controla los horarios ni tiene indicadores.

Solución propuesta: Horarios y bloqueos de agenda por médico, y resumen del período (CU6, CU20).

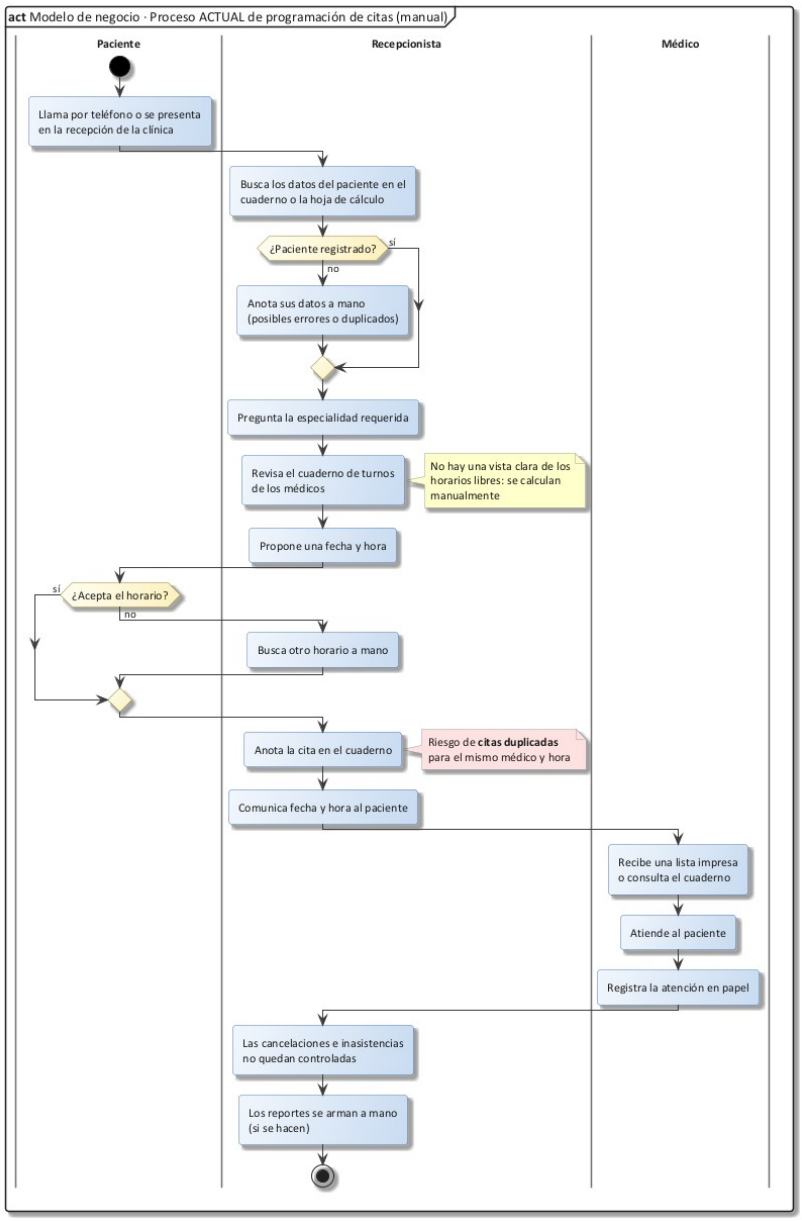
5. Modelo de negocio

5.1 Diagrama de actividad

Los siguientes diagramas de actividades muestran el proceso actual de programación de citas, el proceso propuesto con el sistema y el proceso de atención médica.

Figura 4

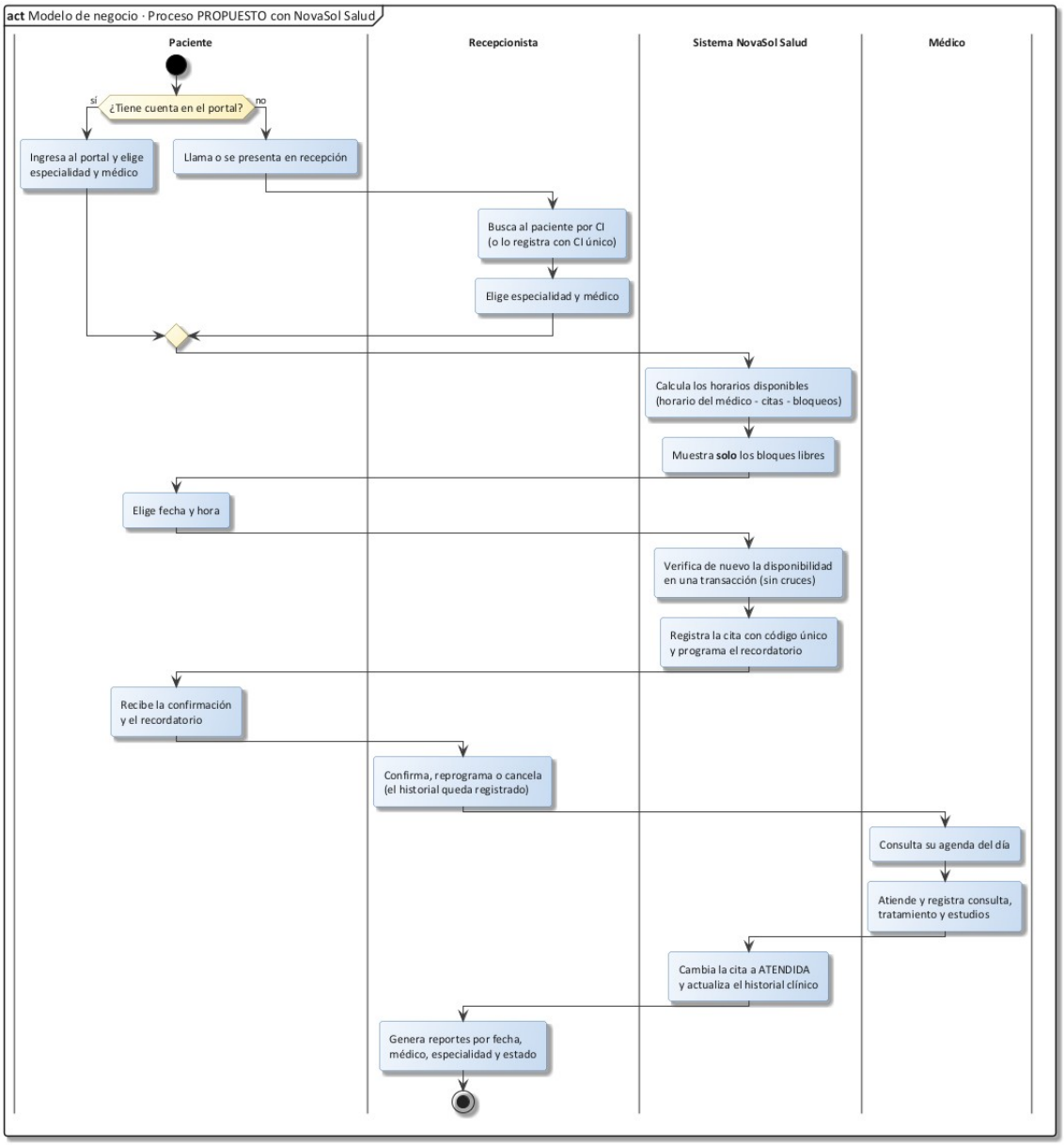
Diagrama de actividades del proceso actual de programación de citas



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 5

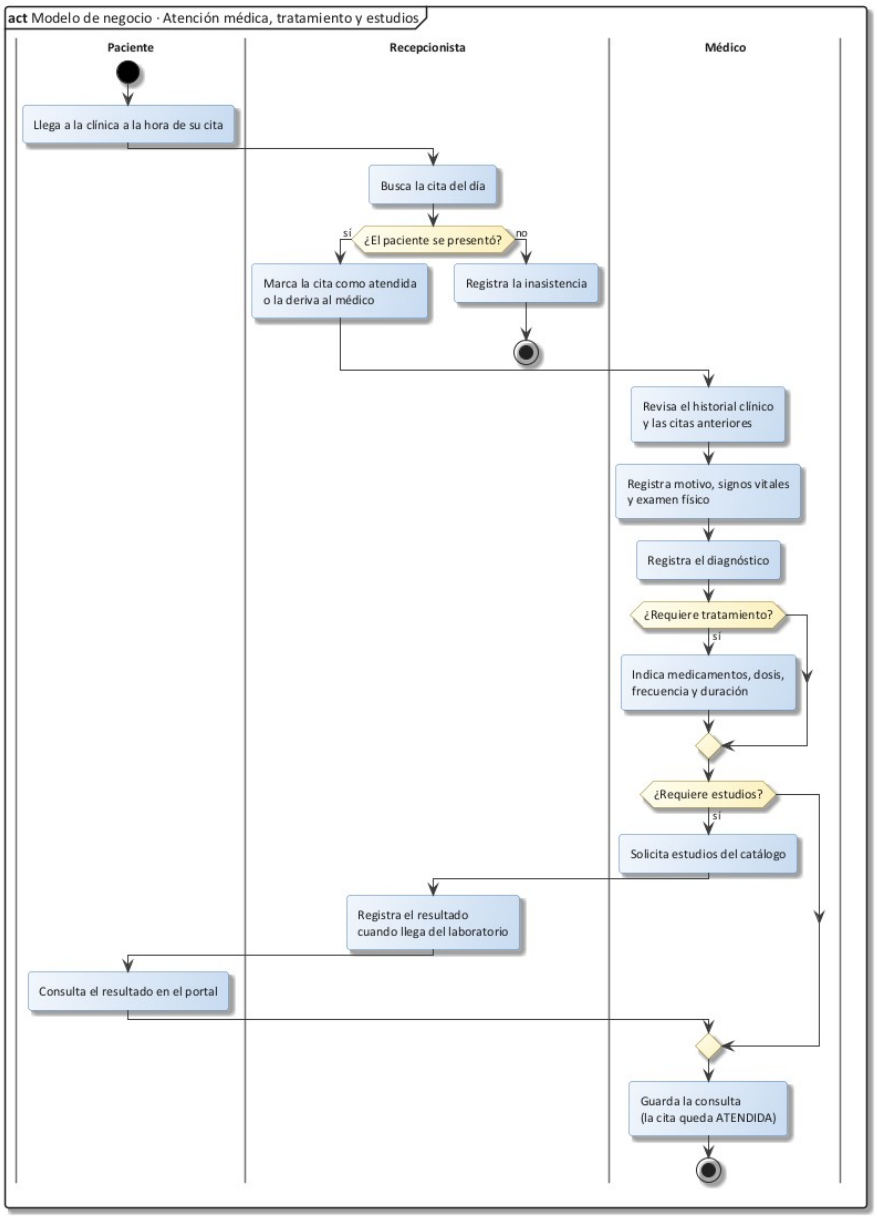
Diagrama de actividades del proceso propuesto con NovaSol Salud



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 6

Diagrama de actividades de la atención médica, tratamiento y estudios



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Capítulo 2. Flujo de trabajo: captura de requisitos

2.1 Identificar actores y casos de uso

Actores

Tabla 31

Actores del sistema

Actor	Descripción	Ciclo en que interviene
A1 Administrador	Configura y supervisa el sistema. Hereda los casos de uso del recepcionista.	Ciclo I y Ciclo II
A2 Recepcionista	Gestiona a los pacientes y las citas.	Ciclo I y Ciclo II
A3 Médico	Atiende a los pacientes y registra la consulta.	Ciclo I (inicio de sesión) y Ciclo II
A4 Paciente	Persona que se atiende en la clínica y usa el portal.	Ciclo I (inicio de sesión) y Ciclo II

Casos de uso

CICLO I · CU1 a CU7: seguridad y datos maestros

Tabla 32

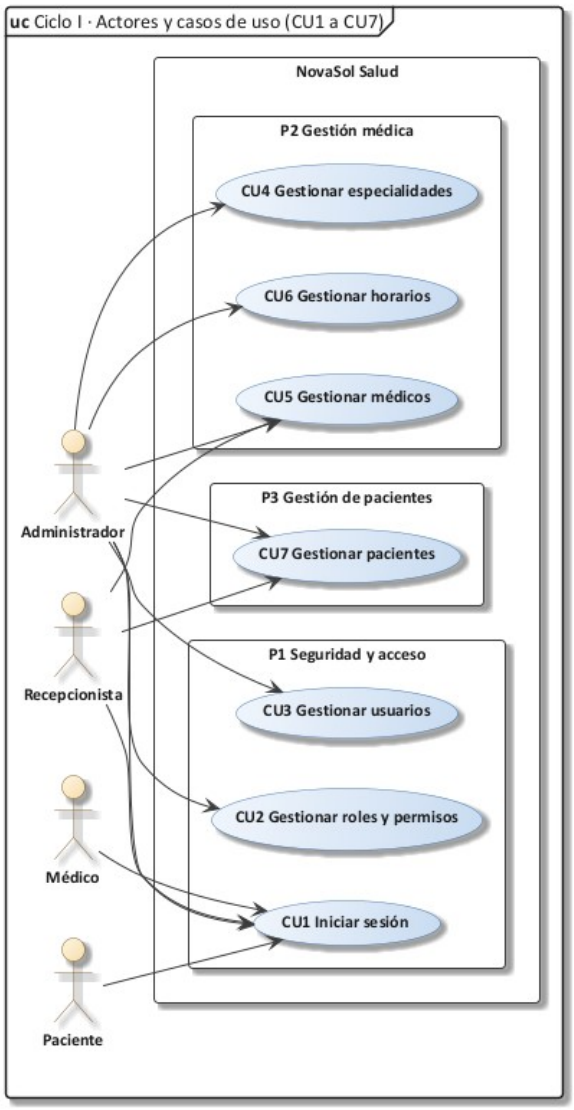
Casos de uso del Ciclo I

ID	Caso de uso	Paquete	Actores
CU1	Iniciar sesión	P1 Seguridad y acceso	Administrador, Recepcionista, Médico, Paciente (Usuario)
CU2	Gestionar roles y permisos	P1 Seguridad y acceso	Administrador
CU3	Gestionar usuarios	P1 Seguridad y acceso	Administrador
CU4	Gestionar especialidades	P2 Gestión médica	Administrador

ID	Caso de uso	Paquete	Actores
CU5	Gestionar médicos	P2 Gestión médica	Administrador (registra), Recepcionista (consulta)
CU6	Gestionar horarios	P2 Gestión médica	Administrador
CU7	Gestionar pacientes	P3 Gestión de pacientes	Recepcionista, Administrador

Figura 7

Diagrama de casos de uso del Ciclo I



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

CICLO II · CU8 a CU20: citas, atención, reportes y portal

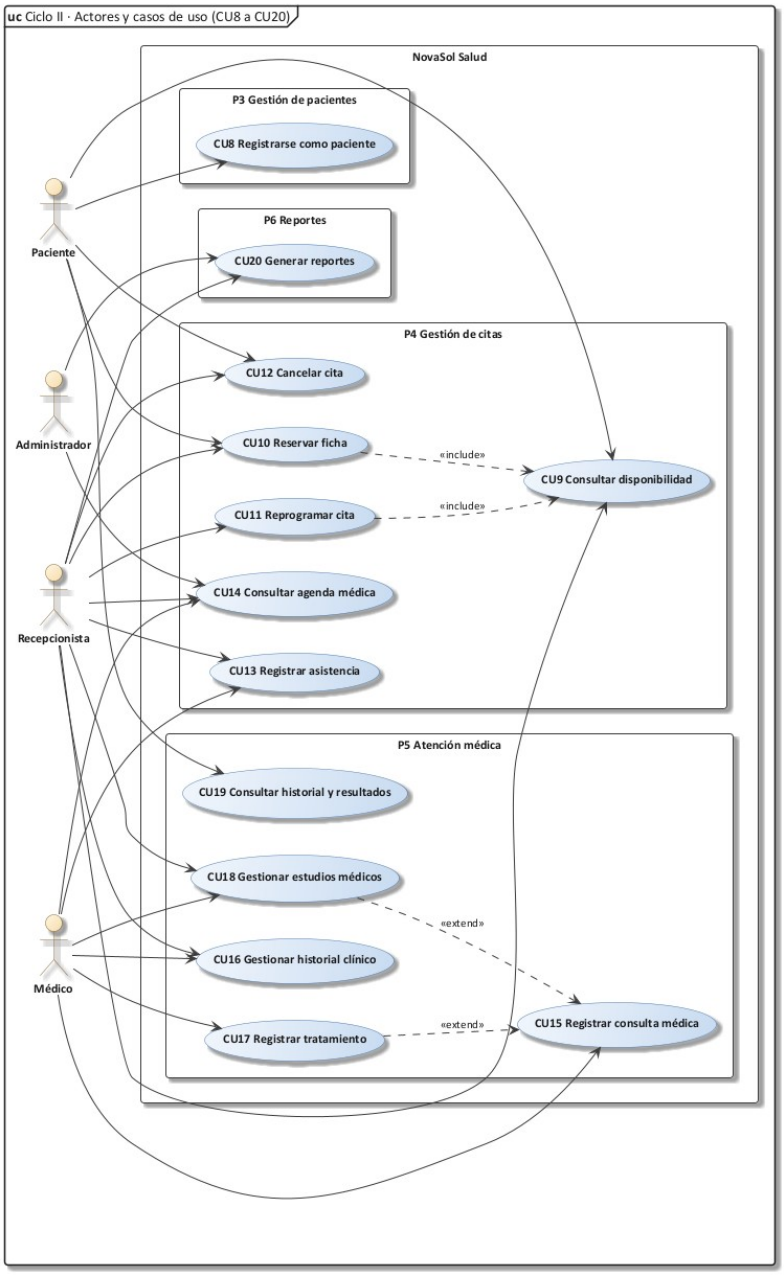
Tabla 33

Casos de uso del Ciclo II

ID	Caso de uso	Paquete	Actores
CU8	Registrarse como paciente	P3 Gestión de pacientes	Paciente
CU9	Consultar disponibilidad	P4 Gestión de citas	Recepcionista, Paciente
CU10	Reservar ficha	P4 Gestión de citas	Recepcionista, Administrador, Paciente (portal)
CU11	Reprogramar cita	P4 Gestión de citas	Recepcionista
CU12	Cancelar cita	P4 Gestión de citas	Recepcionista, Paciente (portal)
CU13	Registrar asistencia	P4 Gestión de citas	Recepcionista, Médico
CU14	Consultar agenda médica	P4 Gestión de citas	Médico, Recepcionista, Administrador
CU15	Registrar consulta médica	P5 Atención médica	Médico
CU16	Gestionar historial clínico	P5 Atención médica	Médico, Recepcionista, Paciente
CU17	Registrar tratamiento	P5 Atención médica	Médico
CU18	Gestionar estudios médicos	P5 Atención médica	Médico (solicita), Recepcionista (registra resultado), Administrador (catálogo)
CU19	Consultar historial y resultados	P5 Atención médica	Paciente
CU20	Generar reportes	P6 Reportes	Administrador, Recepcionista

Figura 8

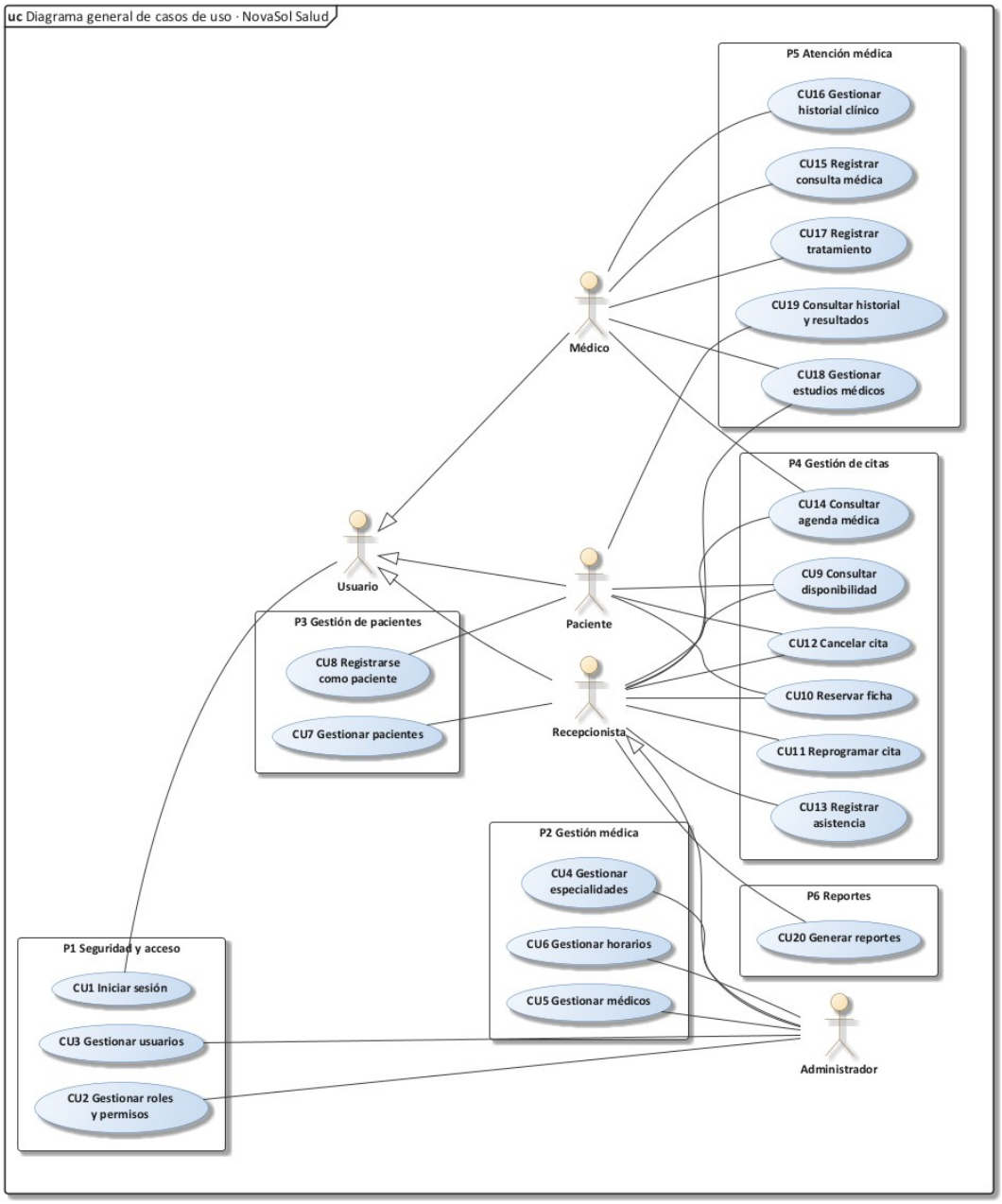
Diagrama de casos de uso del Ciclo II



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 9

Diagrama general de casos de uso (Ciclo I y Ciclo II)



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

2.2 Priorizar los casos de uso

Se priorizan primero los casos de uso arquitectónicamente significativos, que obligan a definir la base de datos, la seguridad y la regla central del negocio (sin citas superpuestas), y después los que dependen de ellos.

CICLO I · CU1 a CU7: seguridad y datos maestros

Tabla 34

Priorización de los casos de uso del Ciclo I

Caso de uso	Prioridad	Justificación
CU1: Iniciar sesión	Alta	Toda operación exige sesión y permisos (RF01, RN12).
CU2: Gestionar roles y permisos	Media	Define qué puede hacer cada rol.
CU3: Gestionar usuarios	Media	Crea las cuentas del personal.
CU4: Gestionar especialidades	Media	Una cita necesita una especialidad (RF04).
CU5: Gestionar médicos	Alta	Una cita necesita un médico activo (RF03, RN04).
CU6: Gestionar horarios	Alta	Sin horarios no hay disponibilidad (RF05).
CU7: Gestionar pacientes	Alta	Una cita necesita un paciente válido (RF02).

CICLO II · CU8 a CU20: citas, atención, reportes y portal

Tabla 35

Priorización de los casos de uso del Ciclo II

Caso de uso	Prioridad	Justificación
CU8: Registrarse como paciente	Media	Permite que el paciente use el portal.
CU9: Consultar disponibilidad	Alta	CU10 y CU11 dependen de él (RF05).
CU10: Reservar ficha	Crítica	Módulo principal: concentra RN03 a RN09 y RN13.
CU11: Reprogramar cita	Alta	Cambio de fecha controlado (RF08).

Caso de uso	Prioridad	Justificación
CU12: Cancelar cita	Alta	Libera el horario y conserva el historial (RF09).
CU13: Registrar asistencia	Alta	Control de asistencia y estados (RF10).
CU14: Consultar agenda médica	Alta	Consulta de la agenda (RF11).
CU15: Registrar consulta médica	Media	Registro de la atención.
CU16: Gestionar historial clínico	Media	Historial clínico del paciente (RF12).
CU17: Registrar tratamiento	Media	Complementa la consulta.
CU18: Gestionar estudios médicos	Media	Solicitud y resultados de estudios.
CU19: Consultar historial y resultados	Media	Acceso del paciente a su información.
CU20: Generar reportes	Alta	Control administrativo (RF13).

2.3 Detallar un caso de uso

A continuación se presenta la descripción expandida de cada caso de uso, con su flujo principal, flujos alternativos y las reglas que cumple.

CICLO I · CU1 a CU7: seguridad y datos maestros

Tabla 36

Descripción del caso de uso CU1 Iniciar sesión

Caso de uso: **CU1: Iniciar sesión**	
Paquete	P1 Seguridad y acceso
Propósito	Permitir que un usuario autorizado ingrese al sistema con su usuario, correo o CI y contraseña, y acceda solo a las opciones de su rol.
Actores	Administrador, Recepcionista, Médico, Paciente (Usuario)
Actor iniciador	Usuario
Precondición	El usuario tiene una cuenta registrada y ACTIVA.
Flujo principal	1. El usuario abre la página de ingreso (/login). 2. Escribe su usuario, correo electrónico o CI, y su contraseña.

	3. El sistema valida que los campos no estén vacíos (JavaScript y PHP) y verifica el token CSRF. 4. El sistema comprueba que la IP no tenga 5 o más intentos fallidos en los últimos 10 minutos. 5. El sistema busca la cuenta y verifica la contraseña con password_verify(). 6. El sistema regenera el identificador de sesión, actualiza el último acceso y registra el ingreso en la bitácora. 7. El sistema muestra el panel o el portal con el menú según los permisos del rol.
Flujos alternativos	A1. Credenciales incorrectas: se registra LOGIN_FALLIDO en la bitácora y se muestra "Usuario o contraseña incorrectos". A2. Cuenta inactiva: se muestra "Su cuenta está desactivada". A3. Demasiados intentos: se bloquea el ingreso desde esa IP durante 10 minutos. A4. Cerrar sesión: el usuario pulsa Salir, el sistema destruye la sesión y registra CERRAR_SESION. A5. Olvidó su contraseña: elige "¿Olvidaste tu contraseña?" y escribe su correo. El sistema genera un código de 6 dígitos (guardado con password_hash, vence en 15 minutos) y lo envía por correo (Gmail SMTP). El usuario escribe el código y su nueva contraseña; el sistema la cambia, invalida el código y cierra sus sesiones abiertas. Siempre responde lo mismo exista o no el correo, admite 5 intentos por código y limita las solicitudes por cuenta e IP.
Postcondición	Existe una sesión válida con el rol y los permisos del usuario.
Requisitos y reglas	RF01, RN12

Tabla 37

Descripción del caso de uso CU2 Gestionar roles y permisos

Caso de uso: **CU2: Gestionar roles y permisos**	
Paquete	P1 Seguridad y acceso
Propósito	Crear roles y asignar o quitar los permisos que determinan el menú y las operaciones de cada rol.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso roles.gestionar.
Flujo principal	1. El administrador abre Roles y permisos. 2. El sistema muestra los roles con su cantidad de usuarios y la matriz de permisos agrupada por módulo. 3. El administrador registra un nuevo rol (nombre en mayúsculas y descripción) o edita la descripción de uno existente. 4. El administrador marca o desmarca permisos en la columna de un rol y pulsa Guardar.

	5. El sistema reemplaza los permisos del rol en una transacción y registra la operación en la bitácora.
Flujos alternativos	A1. Nombre repetido o con caracteres no permitidos: el sistema rechaza el registro. A2. Rol del sistema (ADMINISTRADOR, RECEPCIONISTA, MEDICO, PACIENTE): no se puede renombrar ni eliminar. A3. El ADMINISTRADOR conserva siempre los permisos de usuarios y roles, para no quedar sin acceso. A4. Un rol solo se elimina si no tiene usuarios.
Postcondición	Los permisos del rol quedan actualizados y se aplican en la siguiente petición de sus usuarios.
Requisitos y reglas	RN12

Tabla 38

Descripción del caso de uso CU3 Gestionar usuarios

Caso de uso: **CU3: Gestionar usuarios**	
Paquete	P1 Seguridad y acceso
Propósito	Registrar, modificar, activar y desactivar las cuentas del personal y asignarles un rol. Incluye la configuración del sistema y la bitácora.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso usuarios.gestionar.
Flujo principal	1. El administrador abre Usuarios. El sistema lista las cuentas con búsqueda, filtro por rol y por estado, y paginación. 2. El administrador elige Nuevo usuario e ingresa nombre, usuario, correo, rol y contraseña con su confirmación. 3. El sistema valida los datos (formato, unicidad de usuario y correo, contraseña de mínimo 8 caracteres con letras y números). 4. El sistema guarda la cuenta con la contraseña cifrada mediante password_hash() y registra la bitácora. 5. Para desactivar, el administrador confirma en el diálogo. El sistema cambia el estado y cierra las sesiones abiertas de esa cuenta.
Flujos alternativos	A1. Usuario o correo ya existentes: se muestra el error en el campo. A2. El administrador no puede desactivarse ni cambiarse el rol a sí mismo. A3. Configuración: el administrador modifica parámetros (duración de cita, días máximos de reserva, etc.) con validación de rangos. A4. Bitácora: el administrador filtra los registros por texto, acción y rango de fechas.
Postcondición	La cuenta queda registrada o actualizada; una cuenta inactiva no puede iniciar sesión.

Requisitos y reglas	RF01, RN12

Tabla 39

Descripción del caso de uso CU4 Gestionar especialidades

Caso de uso: **CU4: Gestionar especialidades**	
Paquete	P2 Gestión médica
Propósito	Administrar las especialidades que ofrece la clínica.
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso especialidades.gestionar.
Flujo principal	1. El administrador abre Especialidades. El sistema lista cada especialidad con sus médicos activos, citas y estado. 2. El administrador ingresa nombre y descripción opcional. 3. El sistema compara el nombre normalizado (sin tildes, mayúsculas ni espacios repetidos) con los existentes. 4. El sistema guarda la especialidad y registra la bitácora.
Flujos alternativos	A1. Nombre repetido ("pediatria" frente a "Pediatría"): se rechaza. A2. Desactivar: la especialidad deja de ofrecerse al programar citas; las citas existentes se conservan.
Postcondición	La especialidad queda disponible (o no) para programar citas.
Requisitos y reglas	RF04

Tabla 40

Descripción del caso de uso CU5 Gestionar médicos

Caso de uso: **CU5: Gestionar médicos**	
Paquete	P2 Gestión médica
Propósito	Registrar y mantener la información de los médicos, sus especialidades y su cuenta de acceso.
Actores	Administrador (registra), Recepcionista (consulta)
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso medicos.gestionar (o medicos.ver para consultar).

	Existen especialidades.
Flujo principal	1. El administrador abre Médicos y elige Registrar médico. El sistema propone el siguiente código (MED-###). 2. Ingresa código, registro profesional, nombres, apellidos, teléfono, correo, una o varias especialidades, y usuario y contraseña de la cuenta. 3. El sistema valida formatos y unicidad (código, registro profesional, usuario, correo). 4. En una transacción, el sistema crea la cuenta (rol MEDICO), el médico y sus especialidades. 5. El sistema redirige a la definición de horarios del médico.
Flujos alternativos	A1. Buscar: por nombre, código o especialidad, y por estado. A2. Desactivar: el médico no recibe nuevas citas ni puede iniciar sesión; sus citas históricas se conservan. Si tiene citas pendientes, el sistema lo advierte. A3. Ver ficha: datos, horario semanal, bloqueos, estadísticas y agenda por fecha.
Postcondición	El médico queda registrado con su cuenta y sus especialidades.
Requisitos y reglas	RF03, RN02, RN04, RN11

Tabla 41

Descripción del caso de uso CU6 Gestionar horarios

Caso de uso: **CU6: Gestionar horarios**	
Paquete	P2 Gestión médica
Propósito	Definir los horarios de atención de cada médico y bloquear fechas (feriados, vacaciones, permisos).
Actores	Administrador
Actor iniciador	Administrador
Precondición	Sesión iniciada con el permiso horarios.gestionar. El médico existe.
Flujo principal	1. El administrador abre Horarios y elige un médico. El sistema muestra su semana y los bloques de cita por día. 2. El administrador ingresa día, hora inicial y hora final. 3. El sistema valida que la hora final sea posterior a la inicial, que cubra al menos una cita y que no se cruce con otro horario activo del mismo día. 4. El sistema guarda el horario. 5. Opcionalmente, el administrador bloquea una fecha (todo el día o un rango de horas) con tipo y motivo.
Flujos alternativos	A1. Horario cruzado o inválido: se rechaza con un mensaje explicativo. A2. Bloqueo sobre citas existentes: se crea y se advierte cuántas citas debe reprogramar la recepción. A3. Un horario con citas pendientes no se elimina: solo se desactiva.

Postcondición	La disponibilidad del médico refleja los horarios y bloqueos vigentes.
Requisitos y reglas	RF05, RN05, RN08

Tabla 42

Descripción del caso de uso CU7 Gestionar pacientes

Caso de uso: **CU7: Gestionar pacientes**	
Paquete	P3 Gestión de pacientes
Propósito	Registrar, buscar, modificar y desactivar pacientes y consultar su historial de citas.
Actores	Recepcionista, Administrador
Actor iniciador	Recepcionista
Precondición	Sesión iniciada con el permiso pacientes.gestionar (o pacientes.ver para consultar).
Flujo principal	1. La recepcionista busca al paciente por CI, nombre o apellido. 2. Si no existe, elige Registrar paciente e ingresa CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono y, opcionalmente, correo, dirección y tipo de sangre. 3. El sistema valida los datos en el navegador y en el servidor, y verifica que el CI no esté registrado. 4. En una transacción, el sistema crea el paciente (código PAC-####) y su historia clínica (HC-####). Opcionalmente crea su cuenta para el portal. 5. El sistema muestra la ficha del paciente con su historial de citas.
Flujos alternativos	A1. CI duplicado: "Ya existe un paciente registrado con ese CI". A2. Desactivar: el paciente no recibe nuevas citas y su historial se conserva. Si tiene citas pendientes, se advierte. A3. Contactos de emergencia: agregar o quitar nombre, parentesco y teléfono.
Postcondición	El paciente queda registrado y disponible para programar citas.
Requisitos y reglas	RF02, RF12, RN01, RN11

CICLO II · CU8 a CU20: citas, atención, reportes y portal

Tabla 43

Descripción del caso de uso CU8 Registrarse como paciente

Caso de uso: **CU8: Registrarse como paciente**	
Paquete	P3 Gestión de pacientes

Propósito	Permitir que una persona cree su propia cuenta en el portal para reservar y seguir sus citas.
Actores	Paciente
Actor iniciador	Paciente
Precondición	La persona no tiene una cuenta registrada.
Flujo principal	1. El visitante elige "Regístrese aquí" en la página de ingreso. 2. Completa sus datos personales, de contacto y de la cuenta (usuario, correo y contraseña) y acepta el uso de sus datos. 3. El sistema valida los datos y la unicidad del CI, el usuario y el correo. 4. En una transacción, el sistema crea la cuenta (rol PACIENTE), el paciente y su historia clínica. 5. El sistema inicia la sesión y muestra el portal.
Flujos alternativos	A1. Datos inválidos o duplicados: el sistema muestra el error en cada campo.
Postcondición	El paciente tiene una cuenta y accede a su portal.
Requisitos y reglas	Sección 3.4 del enunciado, RN01

Tabla 44

Descripción del caso de uso CU9 Consultar disponibilidad

Caso de uso: **CU9: Consultar disponibilidad**	
Paquete	P4 Gestión de citas
Propósito	Mostrar los horarios realmente libres de un médico en una fecha.
Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista o Paciente
Precondición	El médico está activo y tiene horarios de atención.
Flujo principal	1. El actor elige especialidad, médico y fecha. El sistema marca los días en que el médico atiende. 2. El sistema divide cada horario del día en bloques de la duración configurada (30 minutos). 3. Descarta los bloques pasados, los ocupados por citas que ocupan horario y los bloqueados en la agenda. 4. Muestra los bloques; solo los libres se pueden elegir.
Flujos alternativos	A1. Fecha pasada o mayor al máximo de días de reserva: se informa el motivo. A2. El médico no atiende ese día o tiene la agenda bloqueada: se informa. A3. Médico inactivo: no se ofrecen horarios.
Postcondición	El actor conoce los horarios libres.

Requisitos y reglas	RF05, RN04, RN05, RN06, RN08, RN09

Tabla 45

Descripción del caso de uso CU10 Reservar ficha

Caso de uso: **CU10: Reservar ficha**	
Paquete	P4 Gestión de citas
Propósito	Registrar una cita para un paciente con un médico en un horario realmente disponible.
Actores	Recepcionista, Administrador, Paciente (portal)
Actor iniciador	Recepcionista
Precondición	Sesión iniciada con citas.gestionar (o portal.paciente). Existen especialidades, médicos activos y horarios.
Flujo principal	1. El actor elige Programar cita. El sistema muestra el asistente: paciente, especialidad y médico, fecha, hora. 2. El actor busca al paciente por CI o nombre; el sistema muestra solo pacientes activos. 3. El actor elige la especialidad; el sistema carga los médicos activos que la atienden. 4. El actor elige médico y fecha; el sistema ejecuta CU9 y muestra solo los bloques libres. 5. El actor elige la hora, la modalidad y el motivo opcional, y confirma. 6. El sistema, dentro de una transacción, bloquea al médico (SELECT … FOR UPDATE) y vuelve a verificar la disponibilidad. 7. El sistema registra la cita con código único CM-#### y fecha de registro, el historial (PROGRAMAR), el aviso y el recordatorio, y la bitácora. 8. El sistema muestra el detalle de la cita.
Flujos alternativos	A1. El paciente no existe: se registra primero (CU7). A2. Otro usuario tomó el mismo horario segundos antes: "El horario ya no está disponible: el médico tiene otra cita". A3. Datos manipulados (fecha pasada, hora fuera de horario, médico que no atiende la especialidad): el servidor rechaza la operación. A4. Como última defensa, PostgreSQL rechaza cualquier cruce con la restricción EXCLUDE.
Postcondición	La cita queda PROGRAMADA y el horario deja de estar disponible.
Requisitos y reglas	RF06, RF07, RN03–RN09, RN13

Tabla 46

Descripción del caso de uso CU11 Reprogramar cita

Caso de uso: **CU11: Reprogramar cita**	
Paquete	P4 Gestión de citas
Propósito	Cambiar la fecha y hora de una cita programada o confirmada.
Actores	Recepcionista
Actor iniciador	Recepcionista
Precondición	La cita está en estado PROGRAMADA o CONFIRMADA.
Flujo principal	1. La recepcionista abre el detalle de la cita y elige Reprogramar. 2. El sistema recupera la cita y muestra los días de atención del mismo médico. 3. La recepcionista elige la nueva fecha y hora libres (CU9, excluyendo la propia cita) y escribe el motivo. 4. En una transacción, el sistema verifica la disponibilidad, actualiza la cita (vuelve a PROGRAMADA) y registra el historial con la fecha anterior y la nueva. 5. El sistema reemplaza el recordatorio del paciente.
Flujos alternativos	A1. Cita atendida, cancelada o con inasistencia: no se reprograma; el sistema ofrece crear una cita nueva con los mismos datos. A2. Horario ocupado: se rechaza.
Postcondición	La cita tiene la nueva fecha y hora; el historial conserva el cambio.
Requisitos y reglas	RF08, RN13

Tabla 47

Descripción del caso de uso CU12 Cancelar cita

Caso de uso: **CU12: Cancelar cita**	
Paquete	P4 Gestión de citas
Propósito	Cancelar una cita sin eliminarla y liberar el horario.
Actores	Recepcionista, Paciente (portal)
Actor iniciador	Recepcionista
Precondición	La cita está en estado PROGRAMADA o CONFIRMADA.
Flujo principal	1. El actor elige Cancelar cita. El sistema pide confirmación y el motivo. 2. En una transacción, el sistema cambia el estado a CANCELADA, registra el historial con el motivo, descarta los recordatorios y envía un aviso.

	3. El sistema muestra el detalle con el historial actualizado.
Flujos alternativos	A1. Cita atendida o en estado final: no se puede cancelar. A2. Desde el portal, el paciente solo cancela con la anticipación mínima configurada (2 horas) y solo sus propias citas.
Postcondición	La cita queda CANCELADA, permanece en el historial y deja de ocupar el horario.
Requisitos y reglas	RF09, RN09, RN10

Tabla 48

Descripción del caso de uso CU13 Registrar asistencia

Caso de uso: **CU13: Registrar asistencia**	
Paquete	P4 Gestión de citas
Propósito	Confirmar la cita y registrar si el paciente asistió (atendida) o no (inasistencia).
Actores	Recepcionista, Médico
Actor iniciador	Recepcionista
Precondición	La cita está PROGRAMADA o CONFIRMADA. Para asistencia o inasistencia, la fecha de la cita ya llegó.
Flujo principal	1. El actor abre el detalle de la cita. El sistema muestra solo las acciones válidas para su estado. 2. El actor elige Confirmar asistencia, Marcar como atendida o El paciente no asistió, y confirma. 3. El sistema valida la transición y la fecha, actualiza el estado y registra el historial y la bitácora.
Flujos alternativos	A1. Transición no permitida (por ejemplo, de Cancelada a Atendida): se rechaza en PHP y en el trigger de la base de datos. A2. Cita futura: no se registra asistencia.
Postcondición	La cita cambia de estado según el diagrama de estados.
Requisitos y reglas	RF10

Tabla 49

Descripción del caso de uso CU14 Consultar agenda médica

Caso de uso: **CU14: Consultar agenda médica**	
Paquete	P4 Gestión de citas

Propósito	Consultar las citas de un médico para una fecha, con filtros por médico, fecha y estado.
Actores	Médico, Recepcionista, Administrador
Actor iniciador	Médico
Precondición	Sesión iniciada con agenda.ver (todos los médicos) o agenda.propia (médico).
Flujo principal	1. El actor abre la Agenda; por defecto, la del día de hoy. 2. Elige la fecha (o navega al día anterior o siguiente), el médico y el estado. 3. El sistema muestra hora, código, paciente, especialidad, estado y motivo de cada cita, intercalados con los bloques libres del médico.
Flujos alternativos	A1. El médico solo ve su propia agenda, aunque intente pedir la de otro. A2. Sin citas: el sistema lo informa. A3. La agenda se puede imprimir.
Postcondición	El actor conoce la agenda solicitada.
Requisitos y reglas	RF11

Tabla 50

Descripción del caso de uso CU15 Registrar consulta médica

Caso de uso: **CU15: Registrar consulta médica**	
Paquete	P5 Atención médica
Propósito	Registrar la atención del paciente: signos vitales, examen físico, diagnóstico y observaciones.
Actores	Médico
Actor iniciador	Médico
Precondición	Sesión de médico con consultas.registrar. La cita es suya, está pendiente y es del día (o anterior).
Flujo principal	1. El médico abre la cita desde su agenda y elige Atender y registrar consulta. 2. El sistema muestra el formulario con los antecedentes, alergias y consultas anteriores del paciente. 3. El médico registra signos vitales, examen físico, diagnóstico (obligatorio) y observaciones; opcionalmente agrega tratamientos (CU17) y estudios (CU18). 4. El sistema valida rangos (peso, talla, temperatura, frecuencia) y el formato de presión (120/80). 5. En una transacción, el sistema guarda la consulta, los tratamientos y los estudios, y pasa la cita a ATENDIDA.
Flujos alternativos	A1. Cita de otro médico: acceso denegado (403). A2. Cita futura o ya cerrada: se informa y no se permite.

Postcondición	La consulta queda registrada y la cita queda ATENDIDA.
Requisitos y reglas	RF10, RN10

Tabla 51

Descripción del caso de uso CU16 Gestionar historial clínico

Caso de uso: **CU16: Gestionar historial clínico**	
Paquete	P5 Atención médica
Propósito	Mantener los antecedentes del paciente y consultar sus consultas anteriores.
Actores	Médico, Recepcionista, Paciente
Actor iniciador	Médico
Precondición	El paciente existe y tiene historia clínica.
Flujo principal	1. El actor abre la ficha del paciente, pestaña Historia clínica. 2. El sistema muestra antecedentes personales y familiares, alergias, enfermedades crónicas, medicación habitual y las consultas con sus tratamientos y estudios. 3. El actor autorizado actualiza los antecedentes y guarda.
Flujos alternativos	A1. El paciente actualiza su propia información médica desde el portal.
Postcondición	La historia clínica queda actualizada.
Requisitos y reglas	Sección 3.4 del enunciado

Tabla 52

Descripción del caso de uso CU17 Registrar tratamiento

Caso de uso: **CU17: Registrar tratamiento**	
Paquete	P5 Atención médica
Propósito	Indicar medicamentos con dosis, frecuencia, duración e indicaciones durante la consulta.
Actores	Médico
Actor iniciador	Médico
Precondición	El médico está registrando una consulta (CU15).
Flujo principal	1. En el formulario de consulta, el médico pulsa Agregar medicamento. 2. Elige el medicamento y completa dosis, frecuencia, duración e indicaciones.

	Puede agregar varias filas. 3. Al guardar la consulta, el sistema valida cada fila y registra los tratamientos.
Flujos alternativos	A1. Fila incompleta: el sistema indica qué fila completar.
Postcondición	Los tratamientos quedan asociados a la consulta y visibles en el historial.
Requisitos y reglas	«extend» de CU15

Tabla 53

Descripción del caso de uso CU18 Gestionar estudios médicos

Caso de uso: **CU18: Gestionar estudios médicos**	
Paquete	P5 Atención médica
Propósito	Solicitar estudios de laboratorio o imagen, registrar sus resultados y administrar el catálogo.
Actores	Médico (solicita), Recepcionista (registra resultado), Administrador (catálogo)
Actor iniciador	Médico
Precondición	Para solicitar: consulta en curso. Para resultados: permiso estudios.gestionar. Para el catálogo: estudios.catalogo.
Flujo principal	1. En la consulta, el médico pulsa Solicitar estudio, elige el estudio del catálogo y escribe las indicaciones. 2. Al guardar la consulta, el estudio queda SOLICITADO. 3. La recepción abre la bandeja Estudios médicos, filtra por estado y elige Registrar resultado. 4. Registra el estado (en proceso, con resultado o cancelado), el resultado y, opcionalmente, un enlace al archivo. 5. El sistema guarda el resultado y envía un aviso al portal del paciente.
Flujos alternativos	A1. Resultado vacío con estado "Con resultado": se rechaza. A2. El médico solo ve los estudios de sus propios pacientes. A3. Catálogo: agregar, editar, activar o desactivar estudios (sin nombres repetidos).
Postcondición	El estudio queda con su resultado y el paciente es notificado.
Requisitos y reglas	«extend» de CU15

Tabla 54

Descripción del caso de uso CU19 Consultar historial y resultados

Caso de uso: **CU19: Consultar historial y resultados**

Paquete	P5 Atención médica
Propósito	Permitir que el paciente consulte sus consultas, diagnósticos, tratamientos y resultados de estudios.
Actores	Paciente
Actor iniciador	Paciente
Precondición	Sesión de paciente con portal.paciente.
Flujo principal	1. El paciente abre Historial médico en su portal. 2. El sistema muestra su resumen médico, los resultados de sus estudios y sus consultas con tratamientos. 3. Desde Inicio, el paciente ve sus recordatorios y avisos (incluido "resultado disponible") y los marca como leídos.
Flujos alternativos	A1. El paciente solo accede a su propia información; cualquier otra ruta interna devuelve 403.
Postcondición	El paciente conoce su historial.
Requisitos y reglas	Sección 3.4 del enunciado

Tabla 55

Descripción del caso de uso CU20 Generar reportes

Caso de uso: **CU20: Generar reportes**	
Paquete	P6 Reportes
Propósito	Obtener reportes administrativos de las citas por fecha, médico, especialidad y estado, pacientes registrados y agenda diaria.
Actores	Administrador, Recepcionista
Actor iniciador	Administrador
Precondición	Sesión iniciada con reportes.ver.
Flujo principal	1. El actor abre Reportes y elige el tipo: resumen del período, citas por fecha, por médico, por especialidad, por estado, pacientes registrados o agenda diaria. 2. Elige el rango de fechas (o un atajo: este mes, mes pasado, este año) y los filtros. 3. El sistema muestra la tabla con totales y porcentajes, además de gráficos de barras y de anillo. 4. Opcionalmente, el actor exporta a CSV o imprime a PDF.
Flujos alternativos	A1. Rango invertido: el sistema lo corrige. A2. Rango mayor a un año: se limita.

	A3. Cada exportación se registra en la bitácora.
Postcondición	El actor obtiene el reporte con sus totales.
Requisitos y reglas	RF13

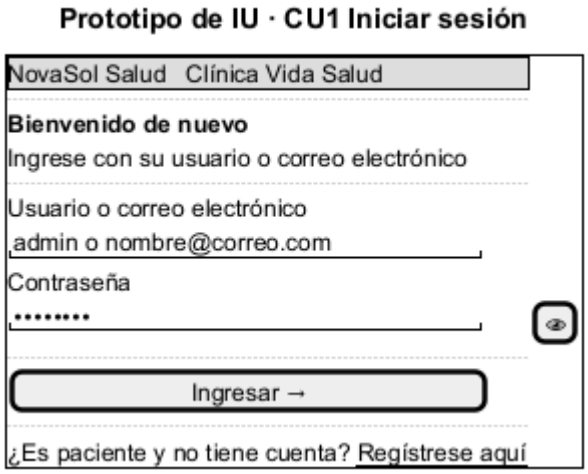
2.4 Prototipar interfaz de usuario

Los prototipos se elaboraron primero como bocetos (wireframes) y luego se construyeron como pantallas funcionales. Se presentan las pantallas implementadas de cada ciclo.

CICLO I · CU1 a CU7: seguridad y datos maestros

Figura 10

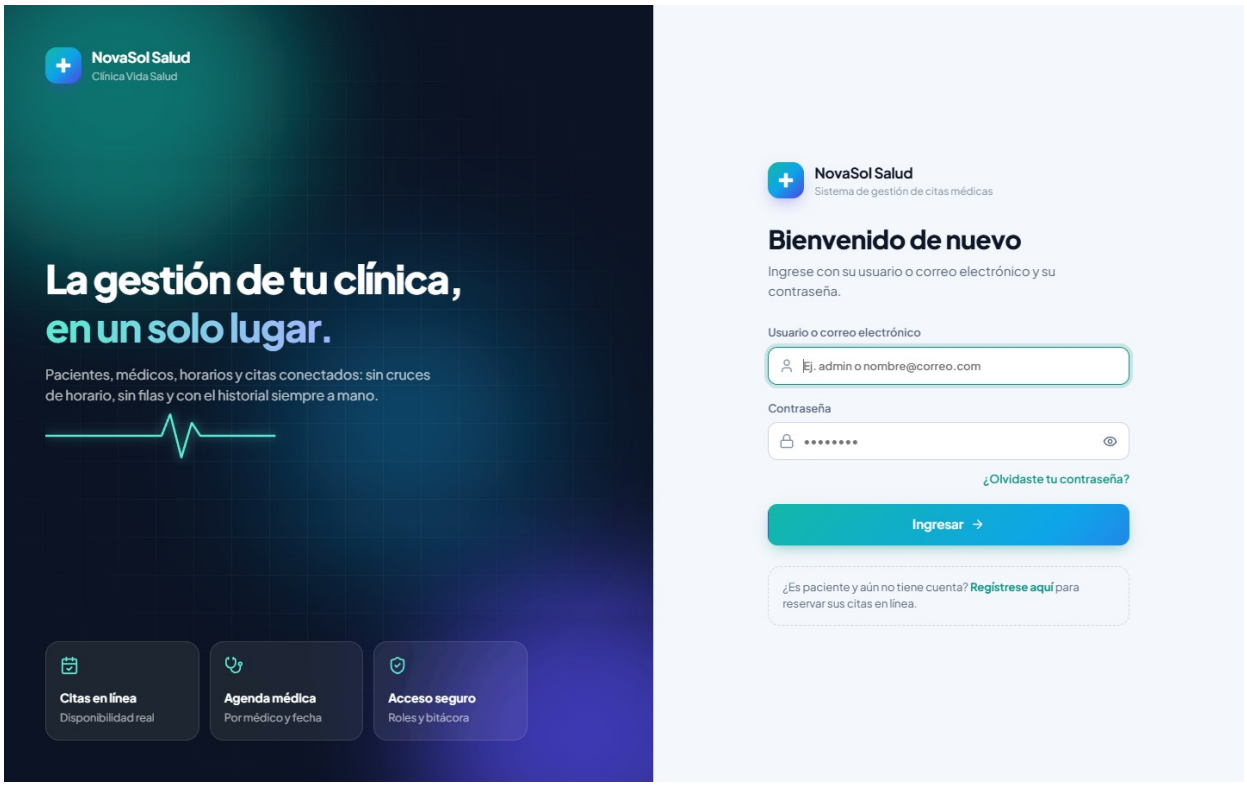
Boceto de la interfaz de CU1 Iniciar sesión



Nota. Boceto elaborado con PlantUML (Salt).

Figura 11

CU1: inicio de sesión



Nota. Captura del sistema en funcionamiento.

Figura 12

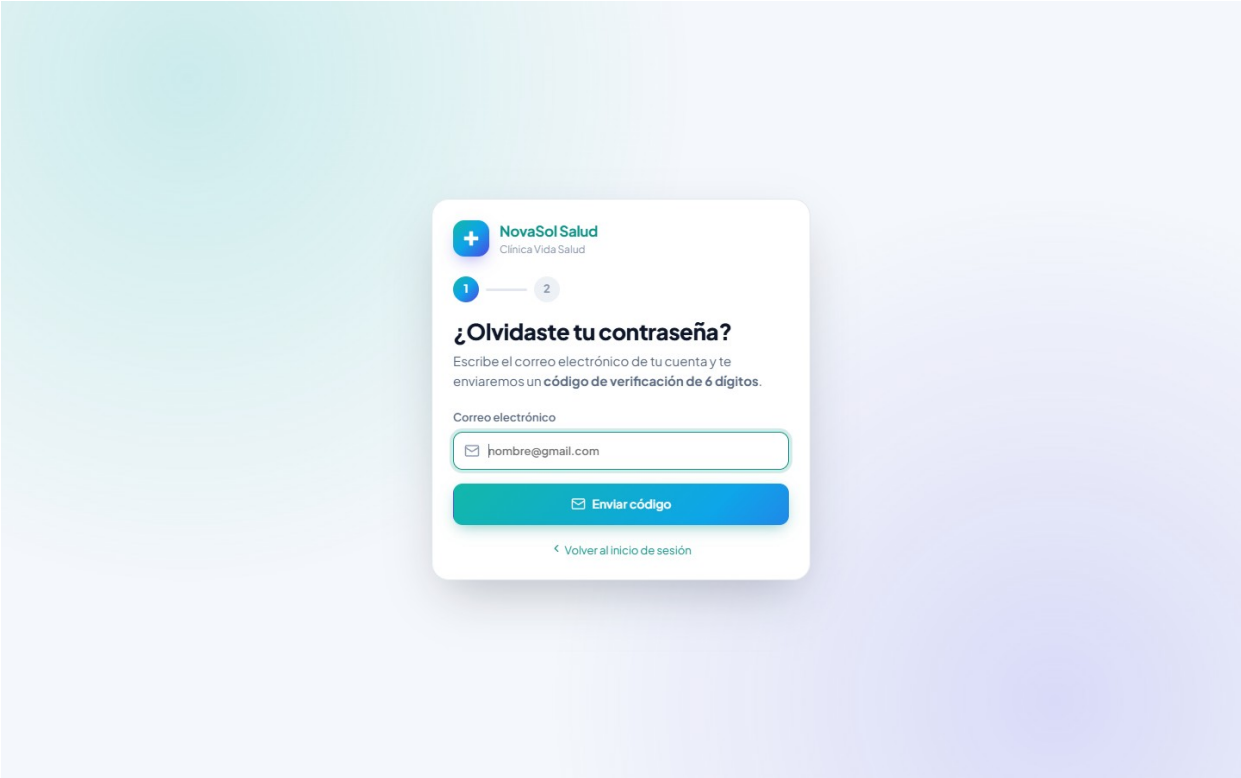
CU1: credenciales incorrectas



Nota. Captura del sistema en funcionamiento.

Figura 13

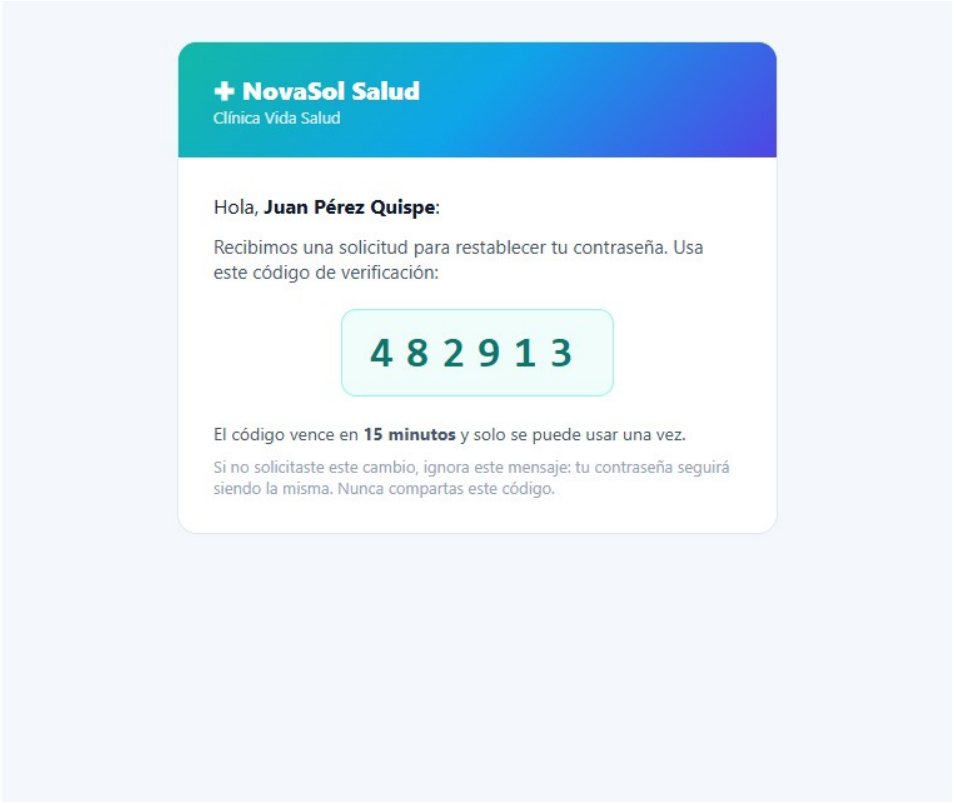
CU1: recuperar contraseña, solicitud del código



Nota. Captura del sistema en funcionamiento.

Figura 14

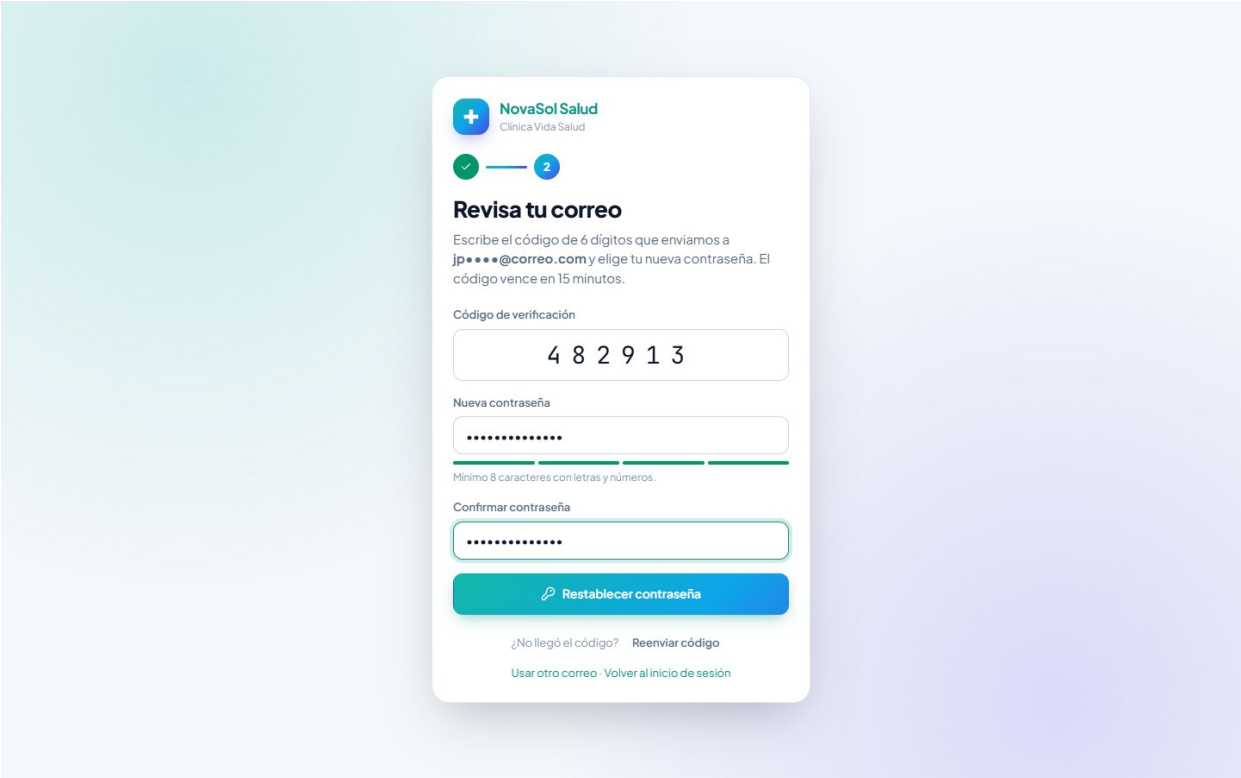
CU1: correo con el código de verificación



Nota. Captura del sistema en funcionamiento.

Figura 15

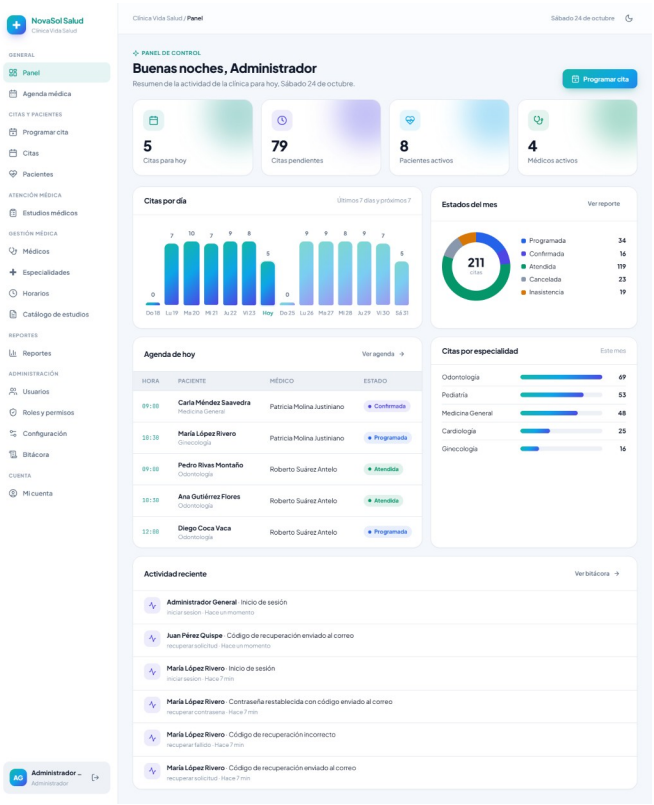
CU1: código y nueva contraseña



Nota. Captura del sistema en funcionamiento.

Figura 16

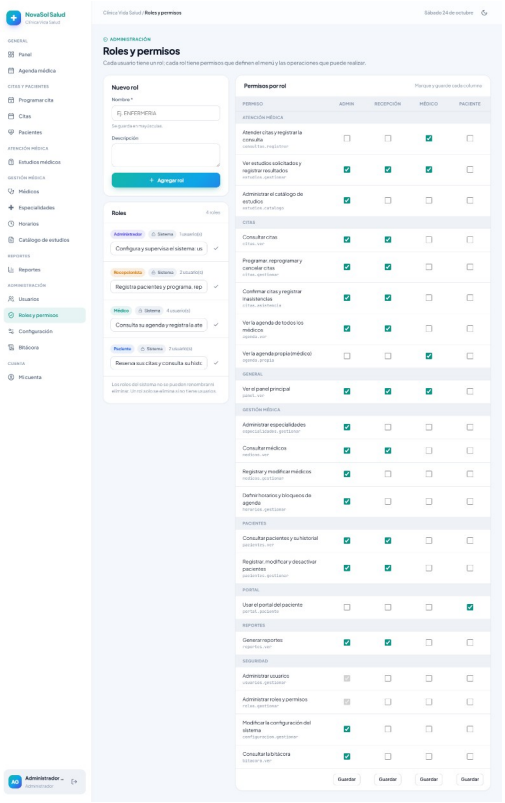
Panel principal del administrador



Nota. Captura del sistema en funcionamiento.

Figura 17

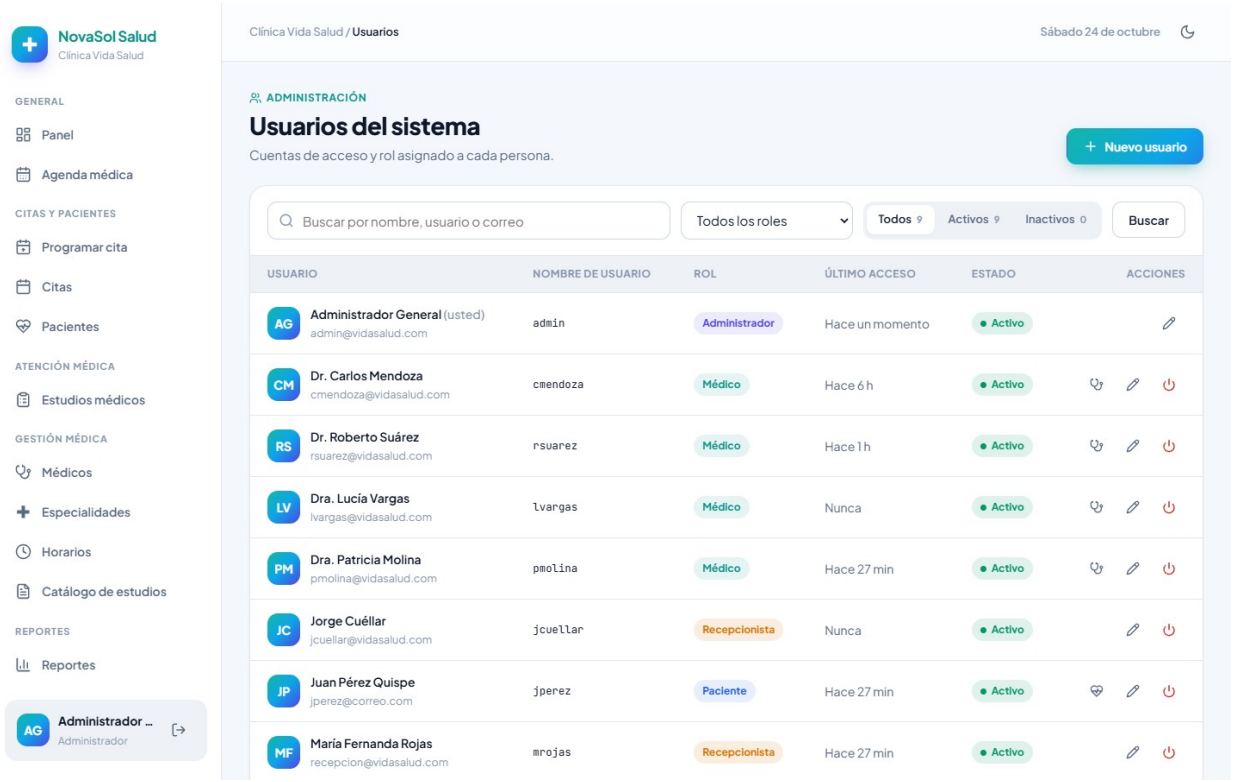
CU2: roles y matriz de permisos



Nota. Captura del sistema en funcionamiento.

Figura 18

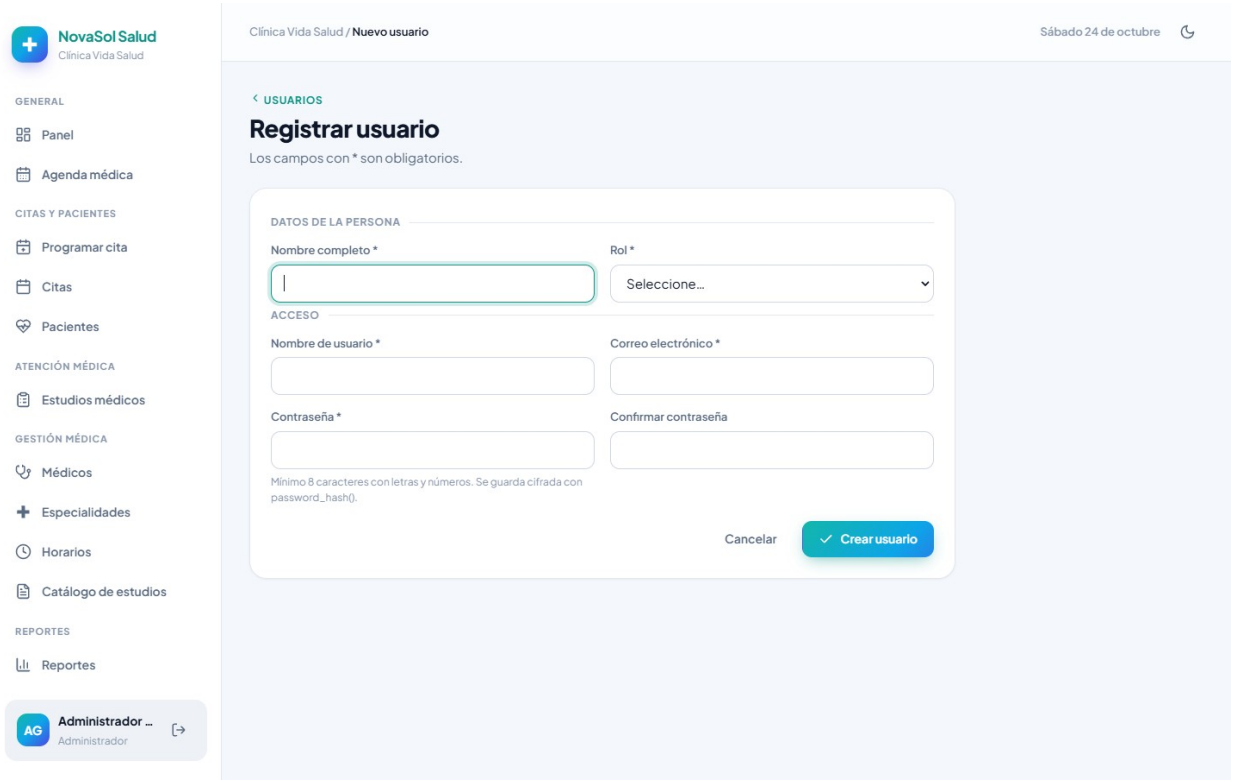
CU3: listado de usuarios



Nota. Captura del sistema en funcionamiento.

Figura 19

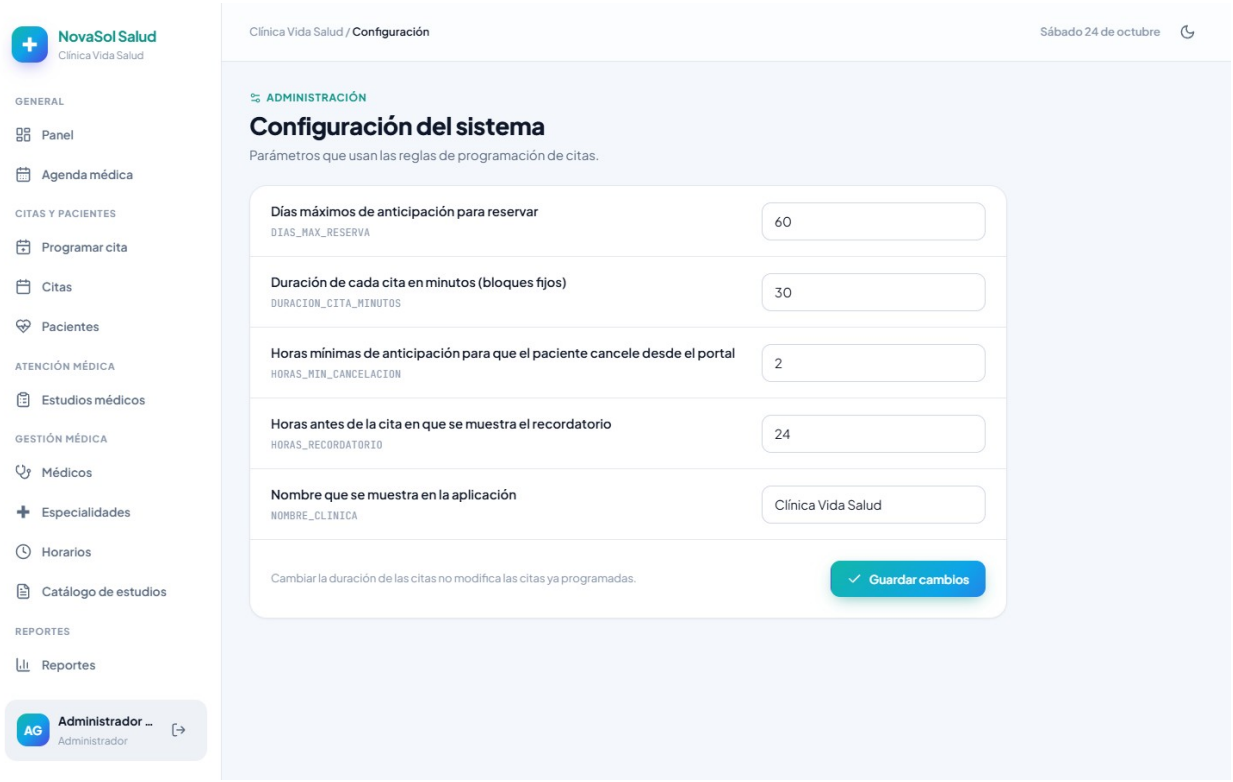
CU3: registro de usuario



Nota. Captura del sistema en funcionamiento.

Figura 20

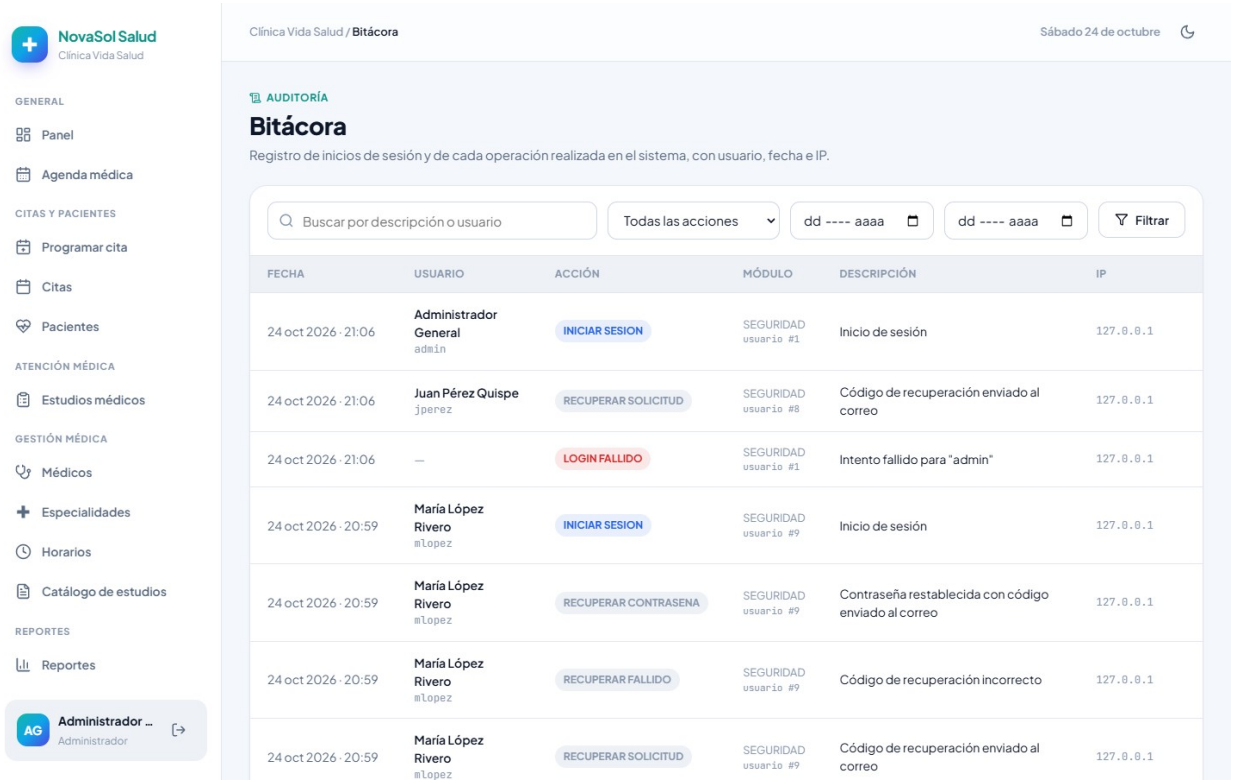
CU3: configuración del sistema



Nota. Captura del sistema en funcionamiento.

Figura 21

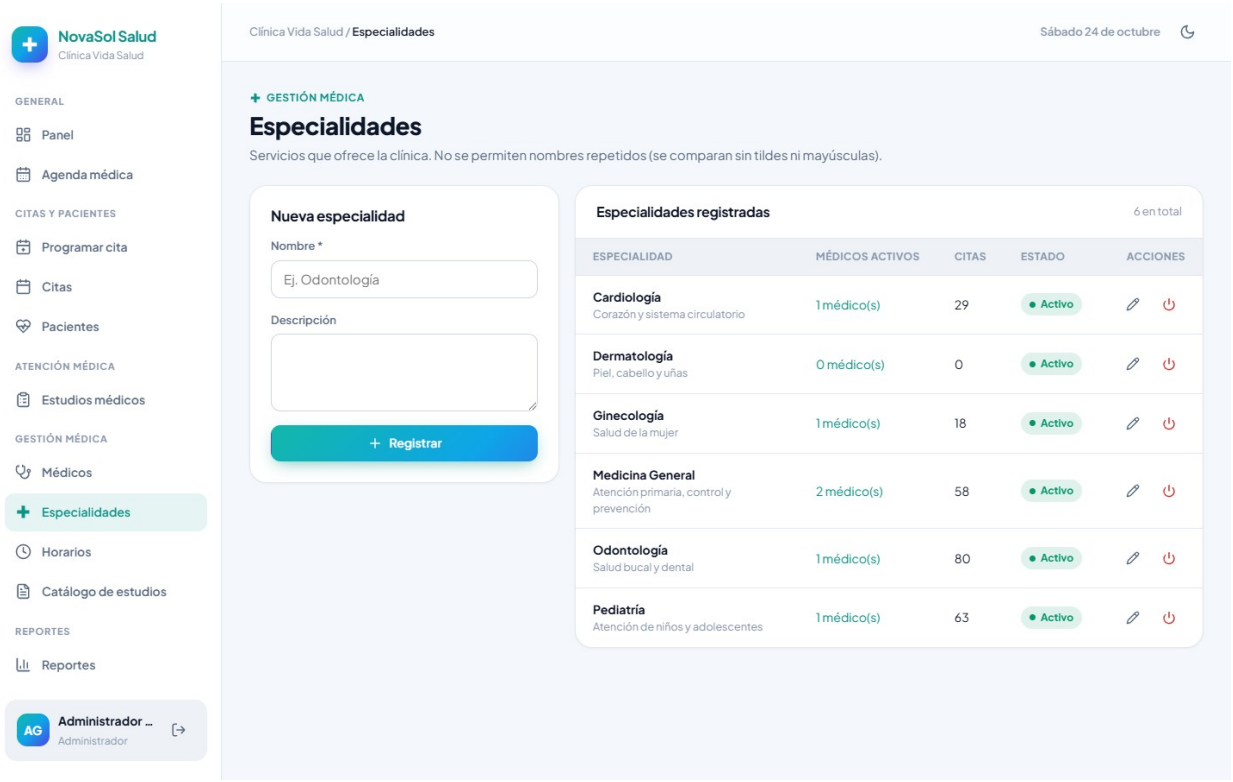
CU3: bitácora de auditoría



Nota. Captura del sistema en funcionamiento.

Figura 22

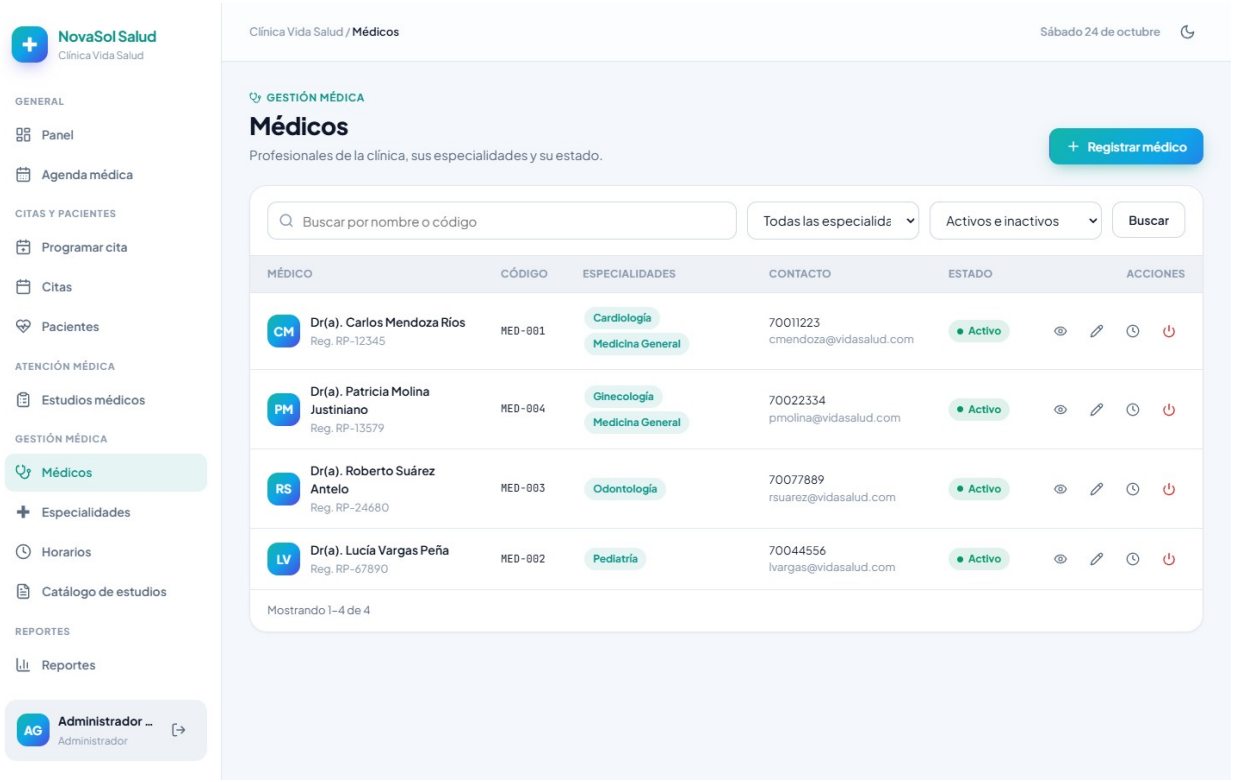
CU4: especialidades



Nota. Captura del sistema en funcionamiento.

Figura 23

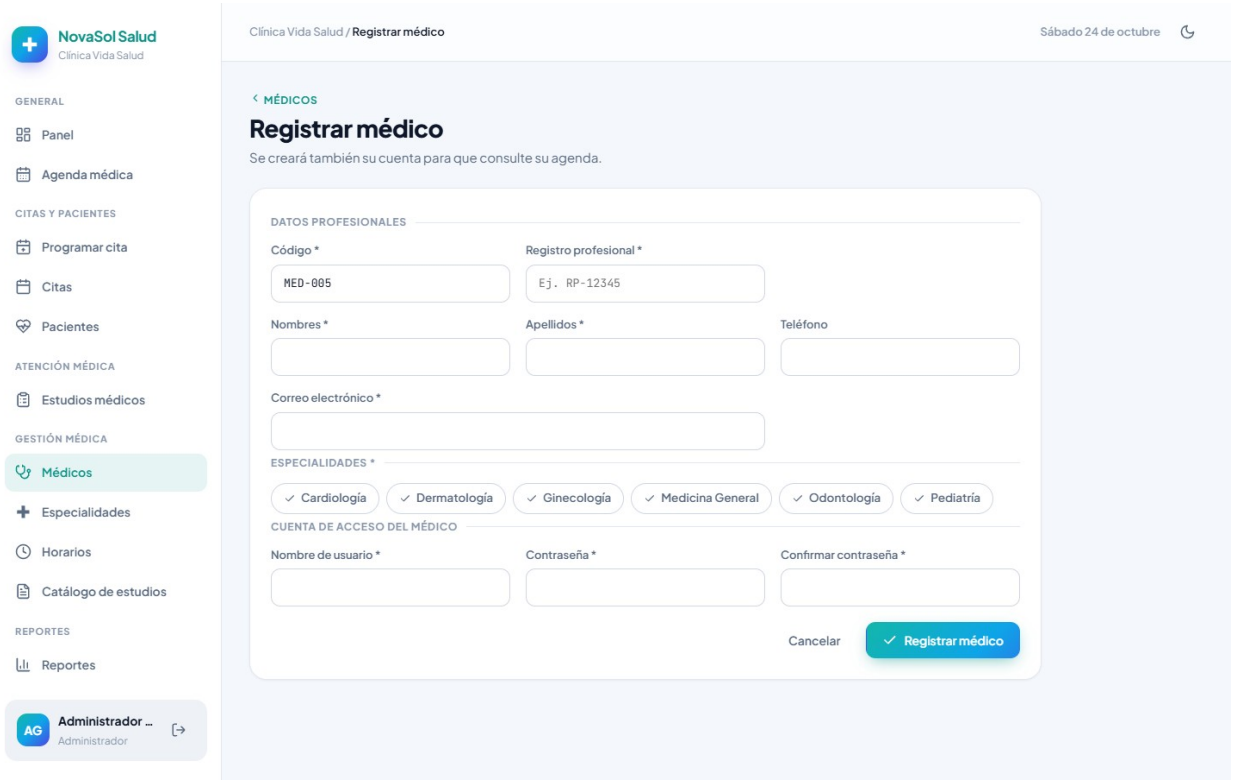
CU5: listado de médicos



Nota. Captura del sistema en funcionamiento.

Figura 24

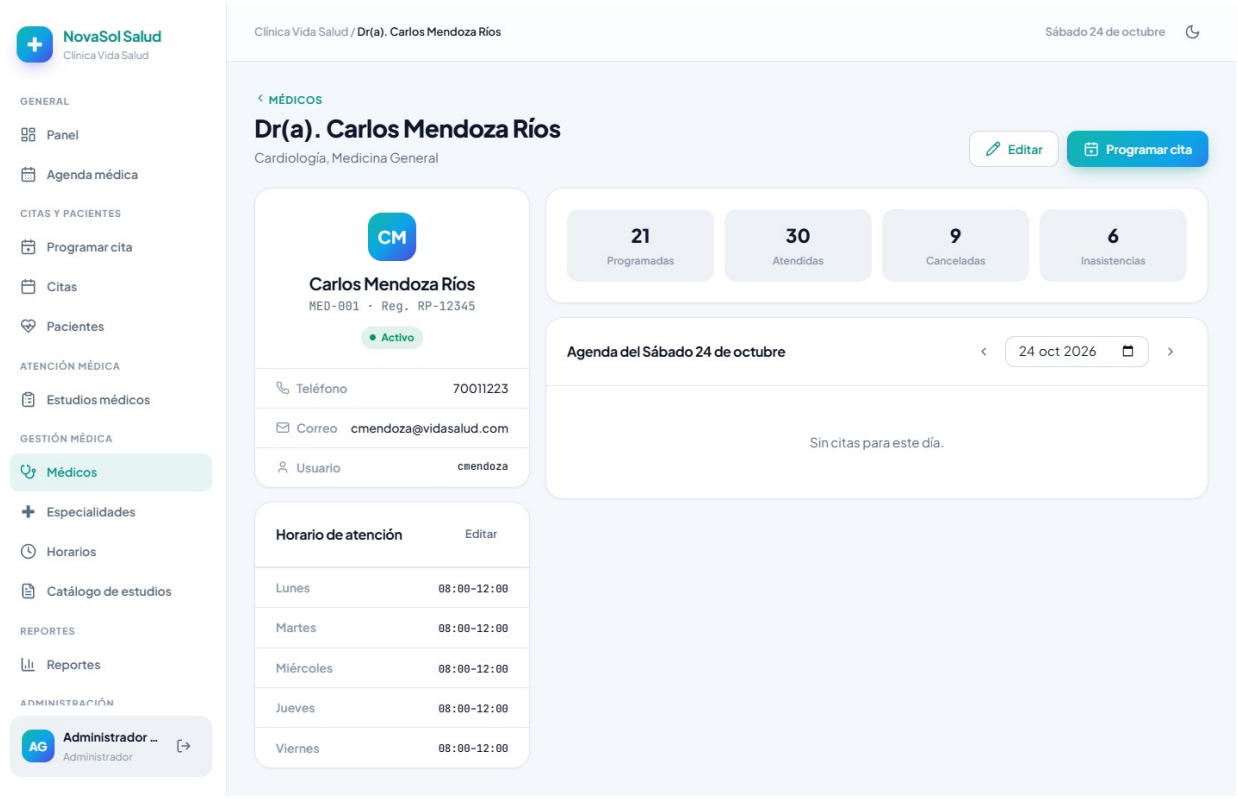
CU5: registro de médico



Nota. Captura del sistema en funcionamiento.

Figura 25

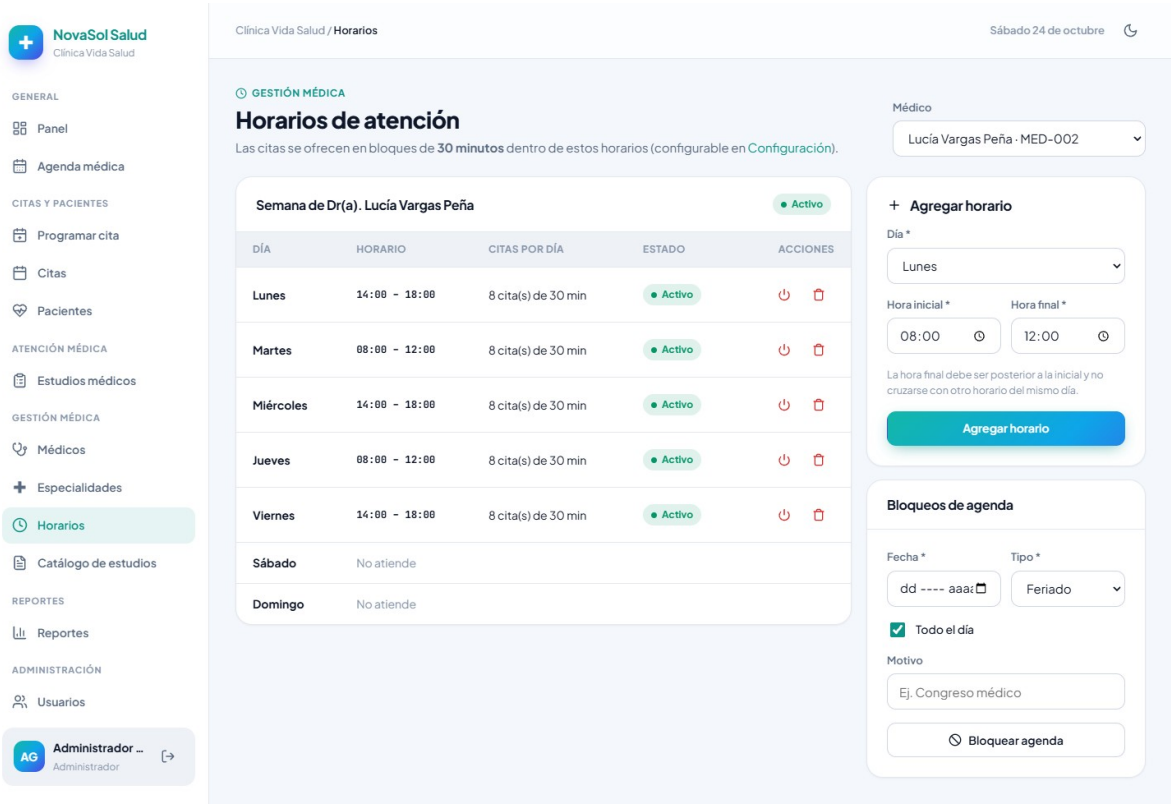
CU5: ficha del médico



Nota. Captura del sistema en funcionamiento.

Figura 26

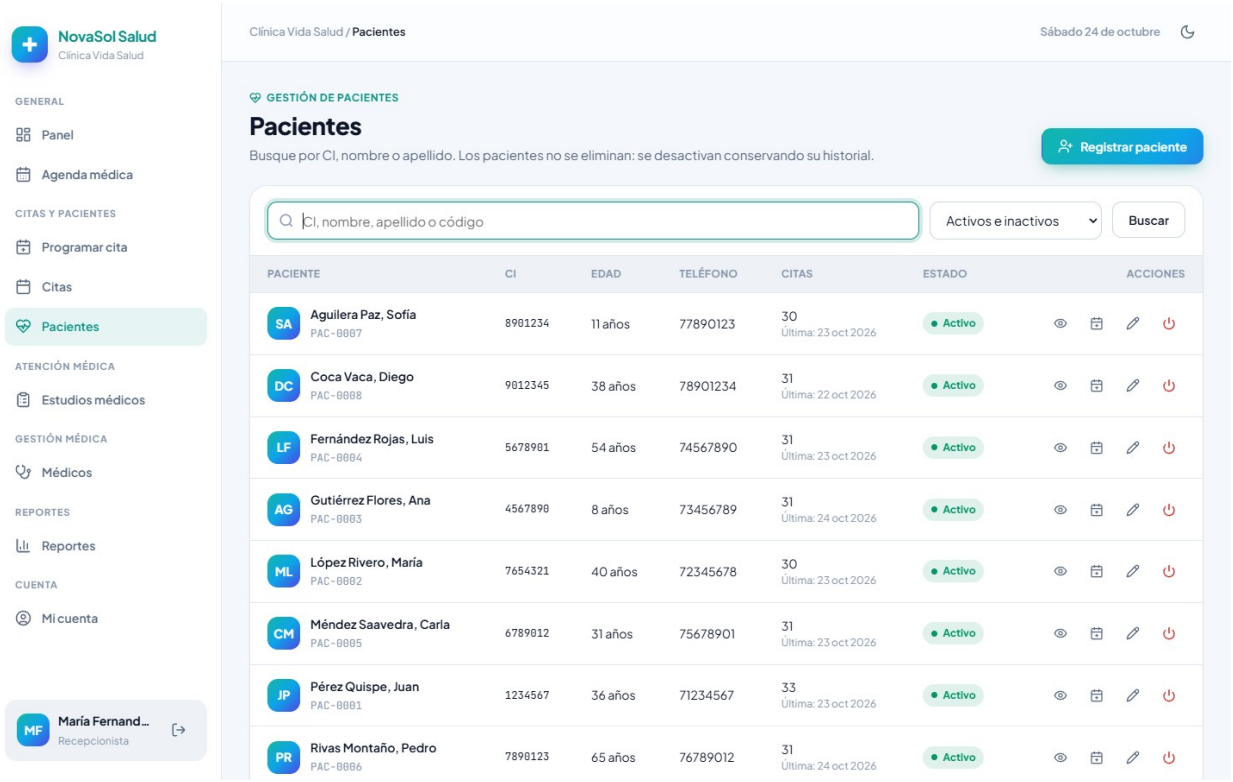
CU6: horarios de atención y bloqueos



Nota. Captura del sistema en funcionamiento.

Figura 27

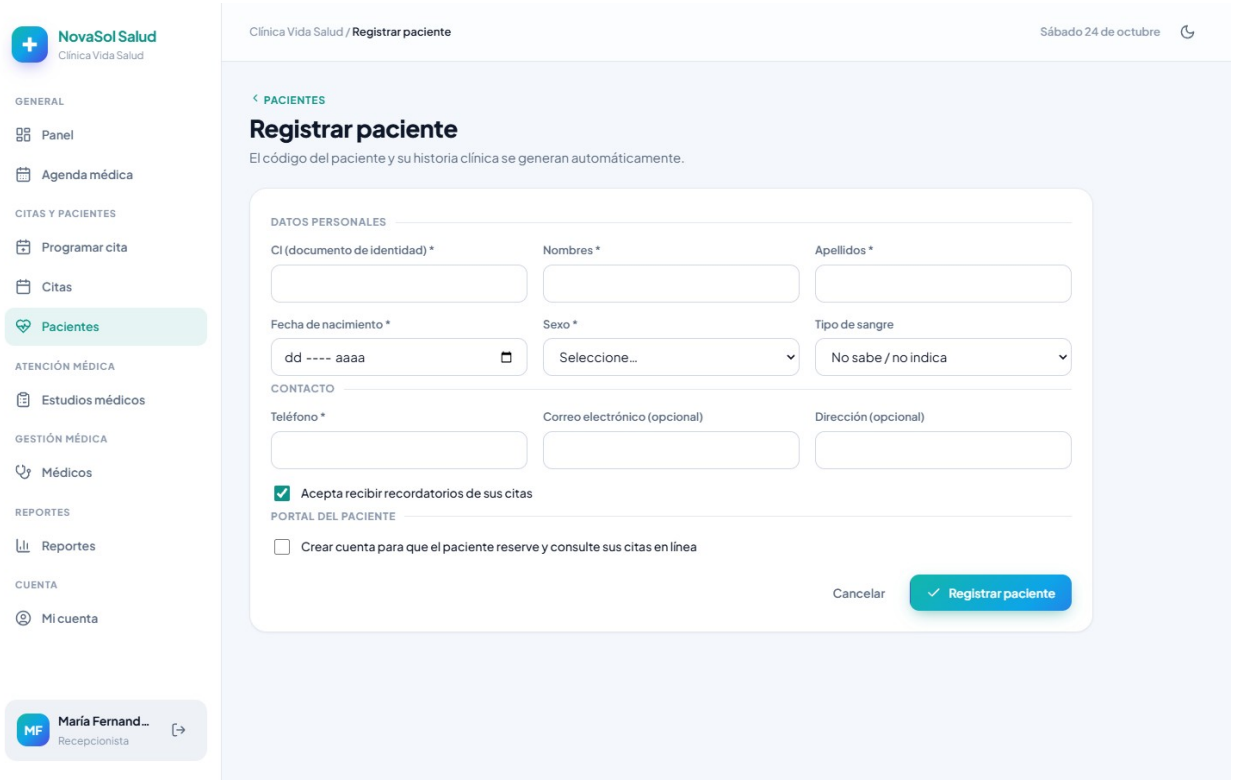
CU7: búsqueda de pacientes



Nota. Captura del sistema en funcionamiento.

Figura 28

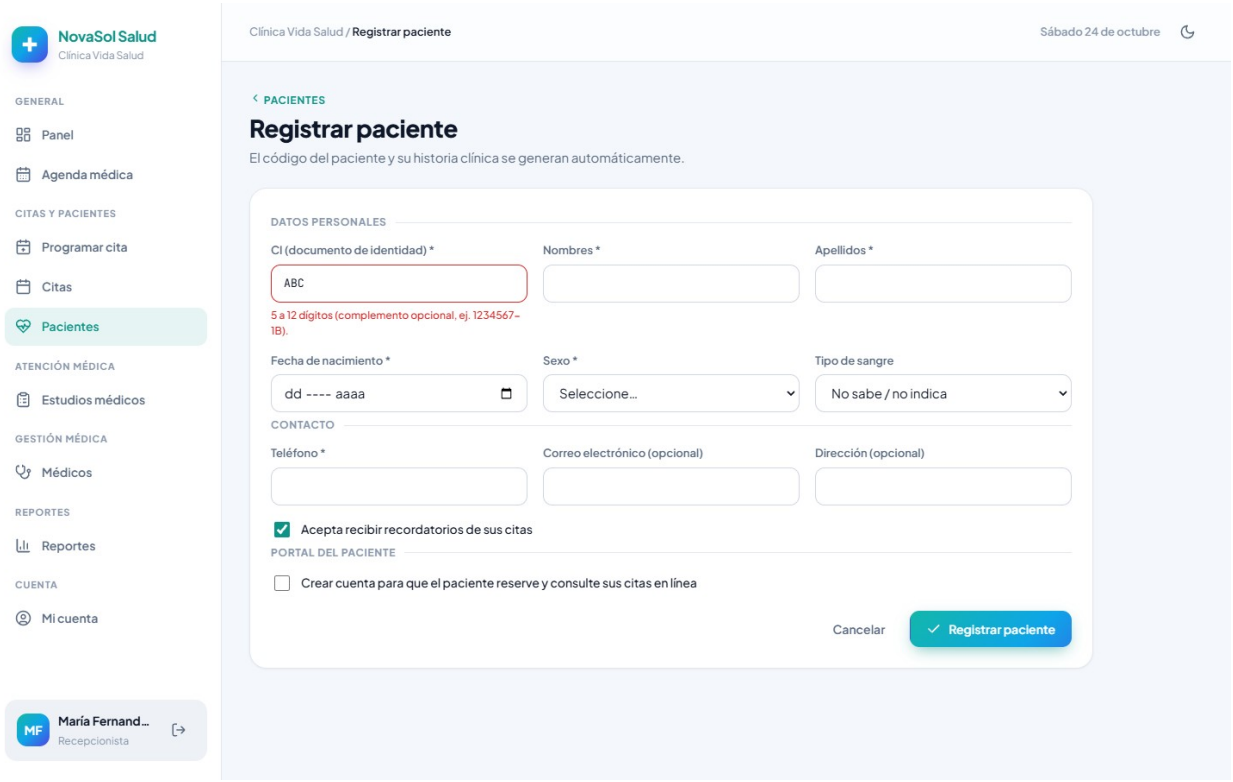
CU7: registro de paciente



Nota. Captura del sistema en funcionamiento.

Figura 29

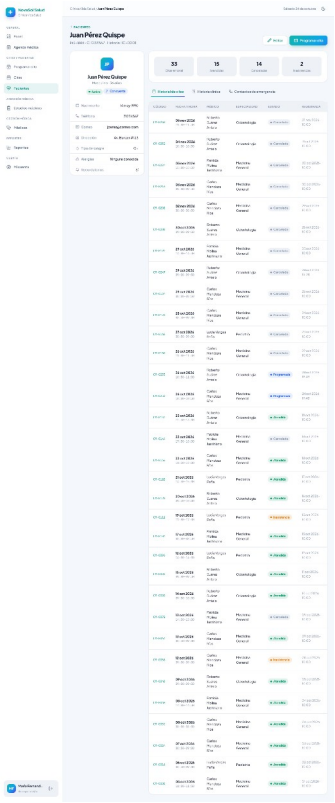
CU7: validación de datos



Nota. Captura del sistema en funcionamiento.

Figura 30

CU7: ficha del paciente



Nota. Captura del sistema en funcionamiento.

Figura 31

Página pública: portada de la clínica



Nota. Captura del sistema en funcionamiento.

Figura 32

Página pública: especialidades



Nota. Captura del sistema en funcionamiento.

Figura 33

Página pública: médicos con sus especialidades y horarios



Nota. Captura del sistema en funcionamiento.

Figura 34

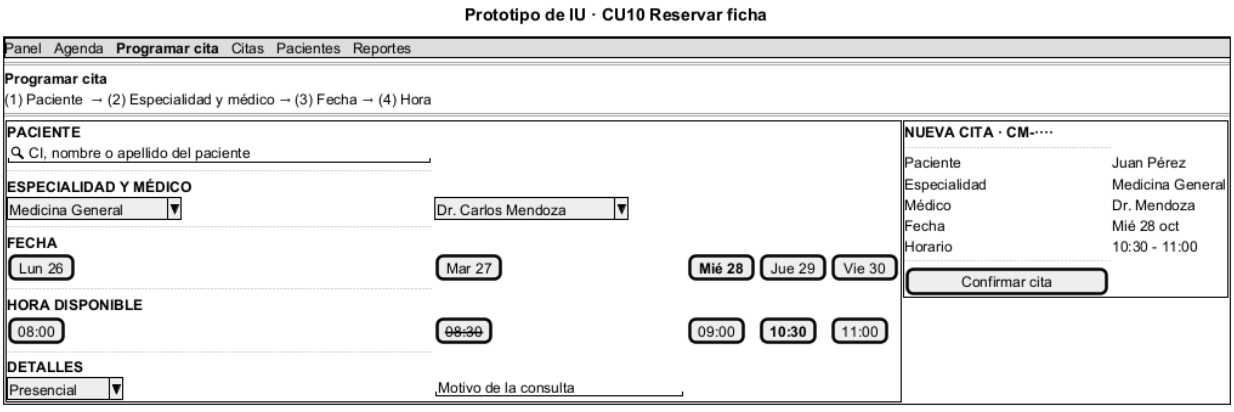
Página pública en un celular



Nota. Captura del sistema en funcionamiento.

Figura 35

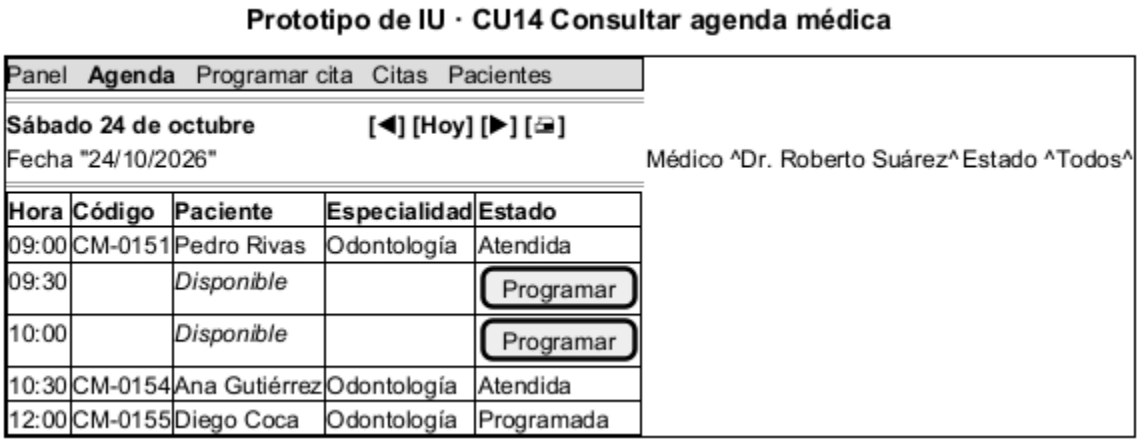
Boceto de la interfaz de CU10 Reservar ficha



Nota. Boceto elaborado con PlantUML (Salt).

Figura 36

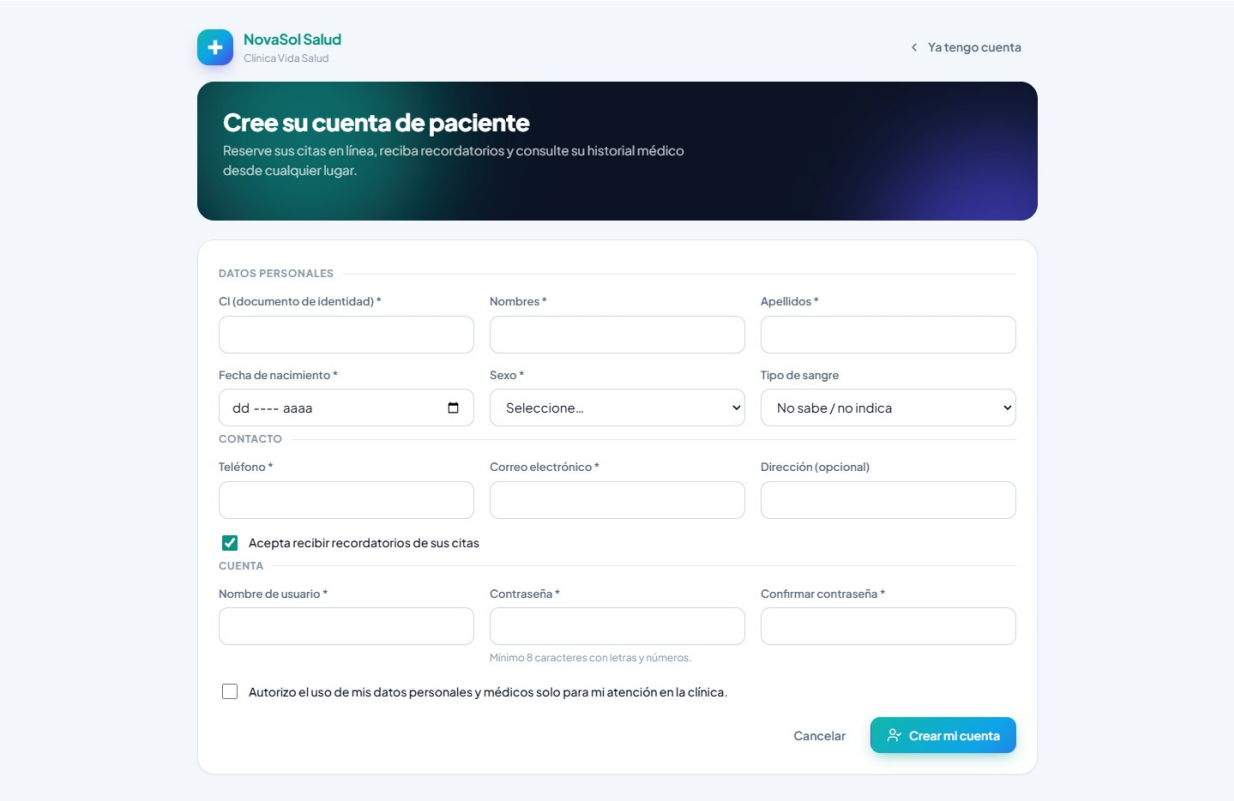
Boceto de la interfaz de CU14 Consultar agenda médica



Nota. Boceto elaborado con PlantUML (Salt).

Figura 37

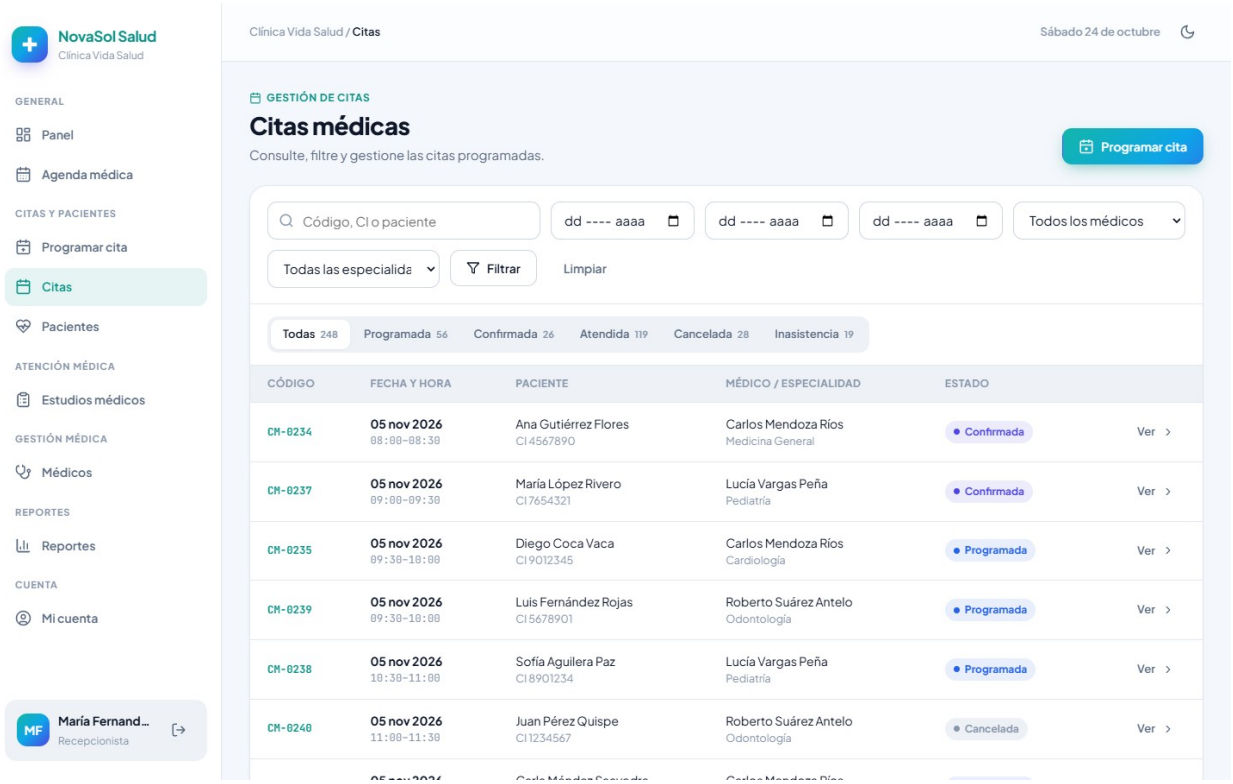
CU8: autoregistro del paciente



Nota. Captura del sistema en funcionamiento.

Figura 38

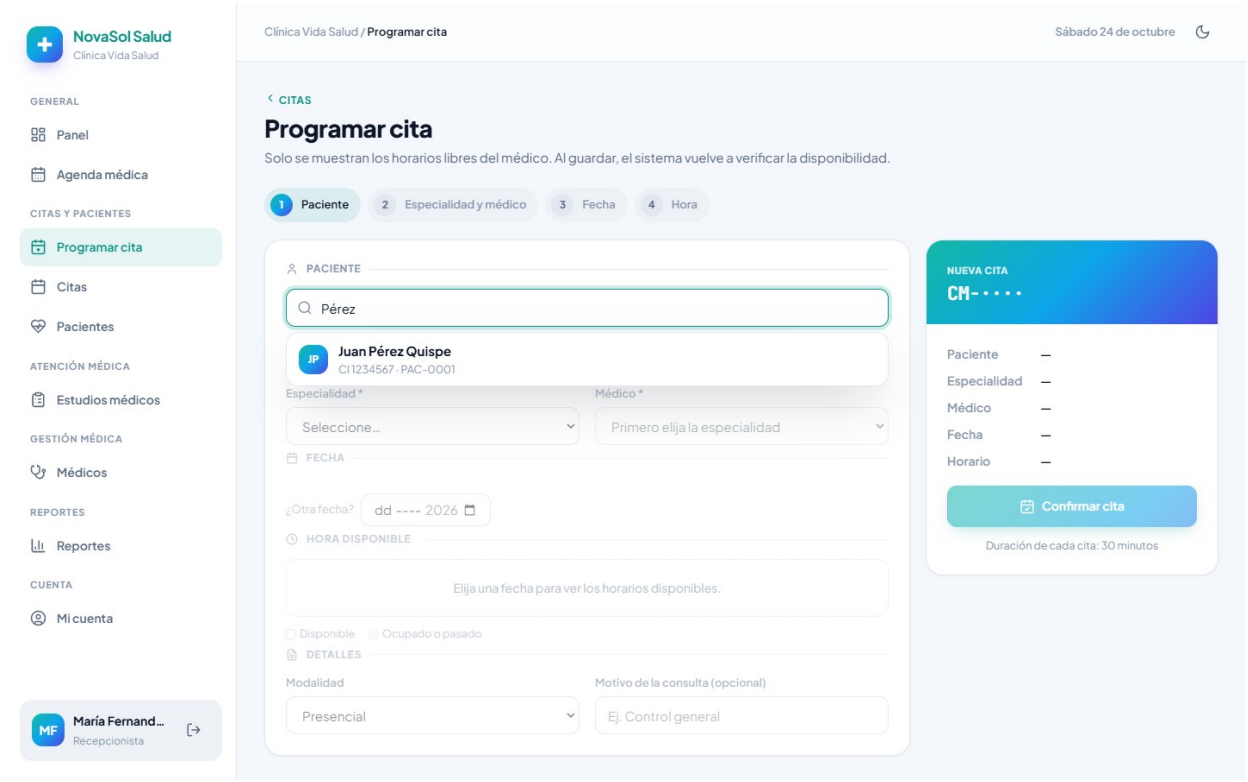
Consulta de citas con filtros



Nota. Captura del sistema en funcionamiento.

Figura 39

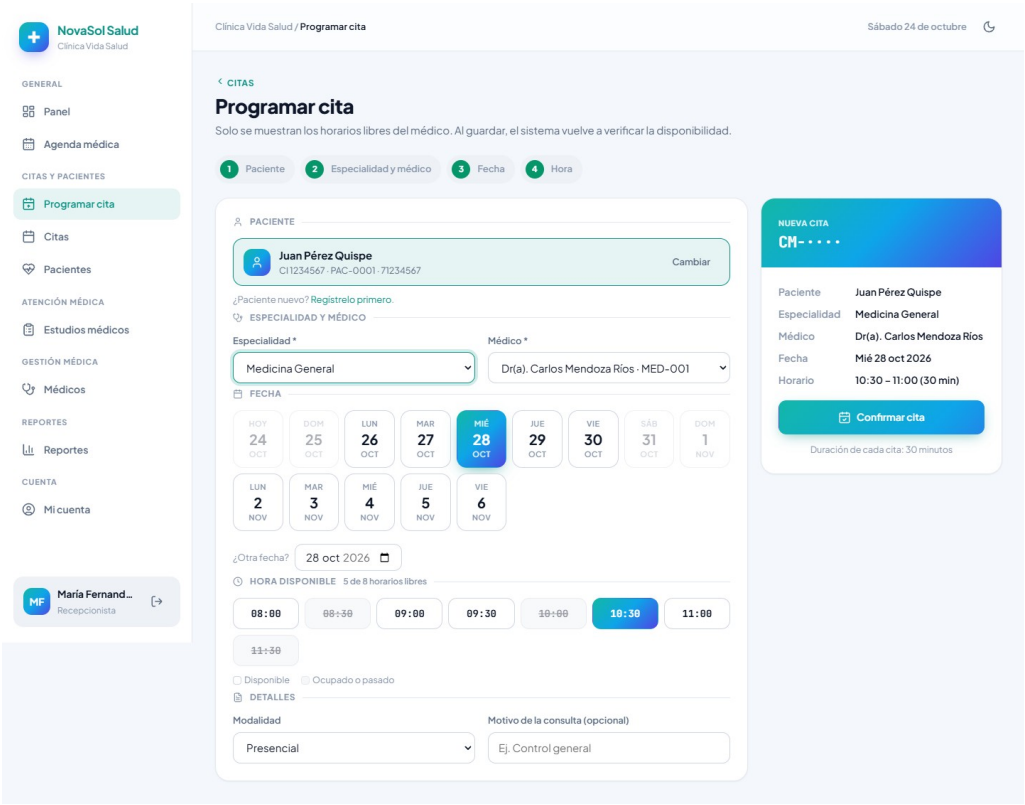
CU10: búsqueda del paciente



Nota. Captura del sistema en funcionamiento.

Figura 40

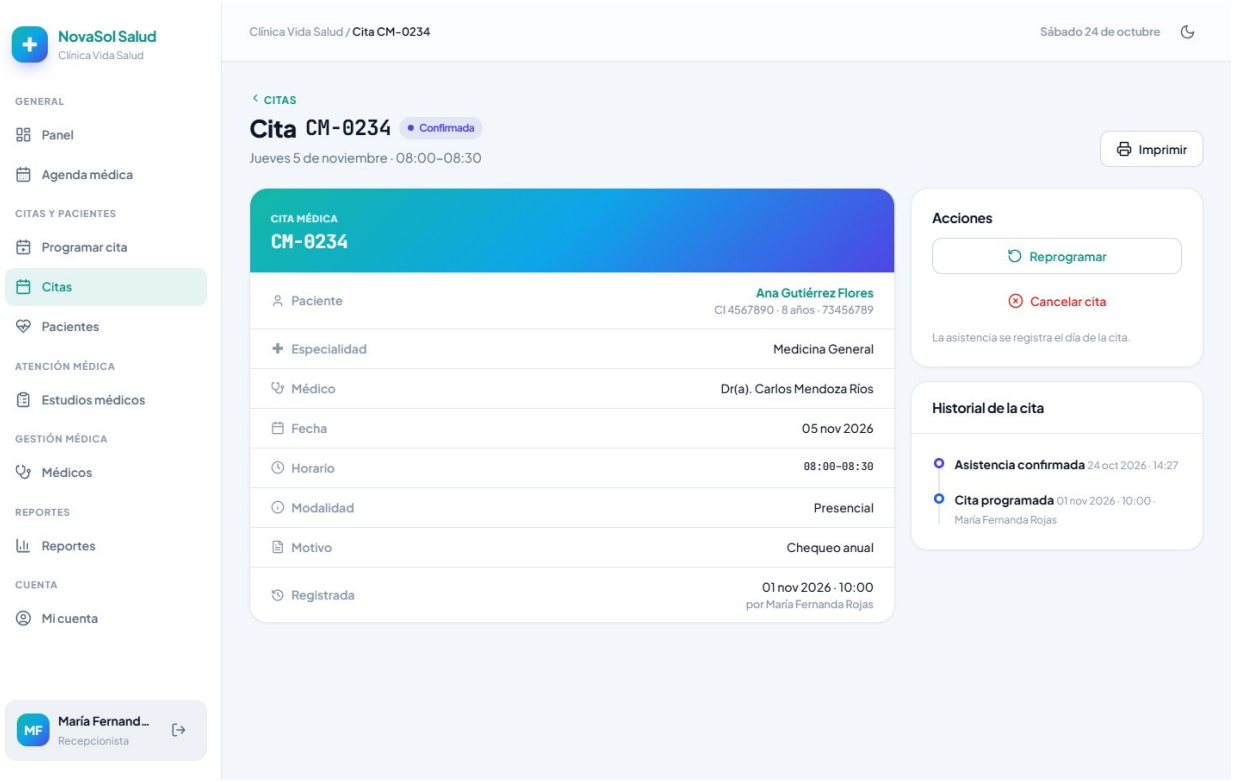
CU9 y CU10: programar cita con horarios libres



Nota. Captura del sistema en funcionamiento.

Figura 41

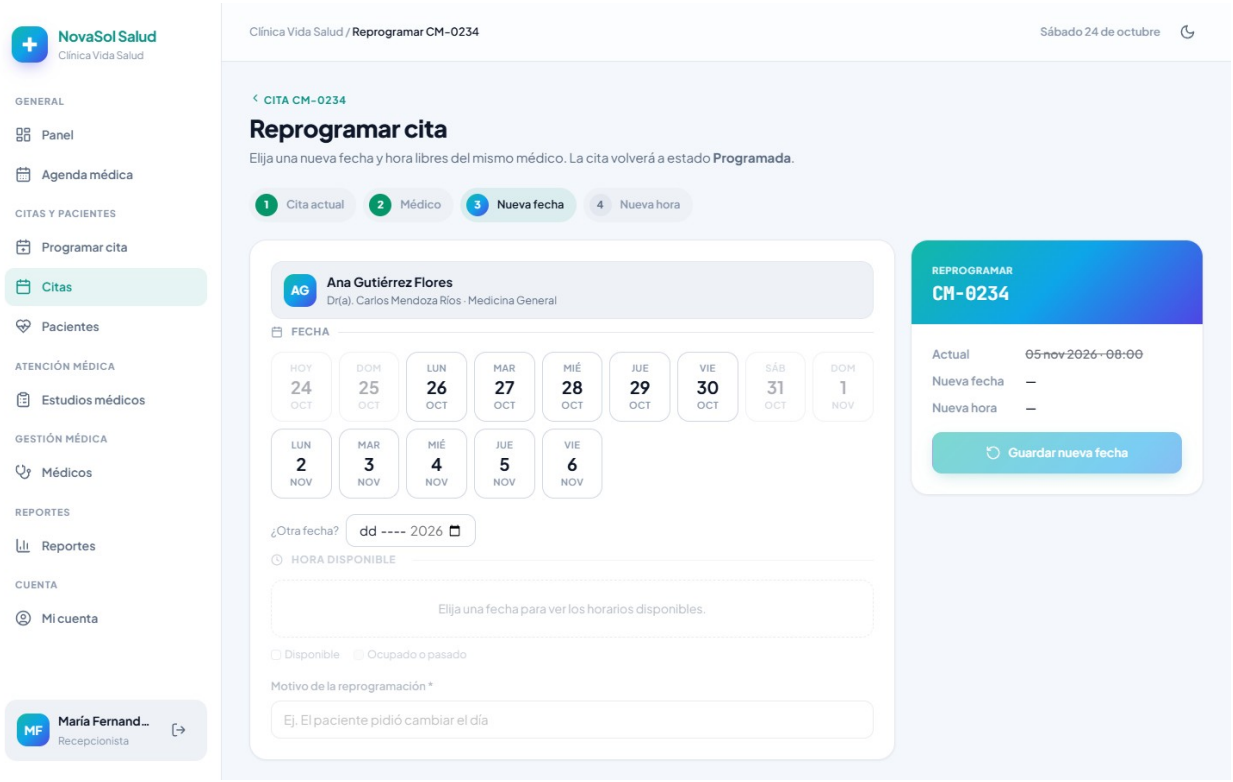
CU13: detalle de la cita y acciones



Nota. Captura del sistema en funcionamiento.

Figura 42

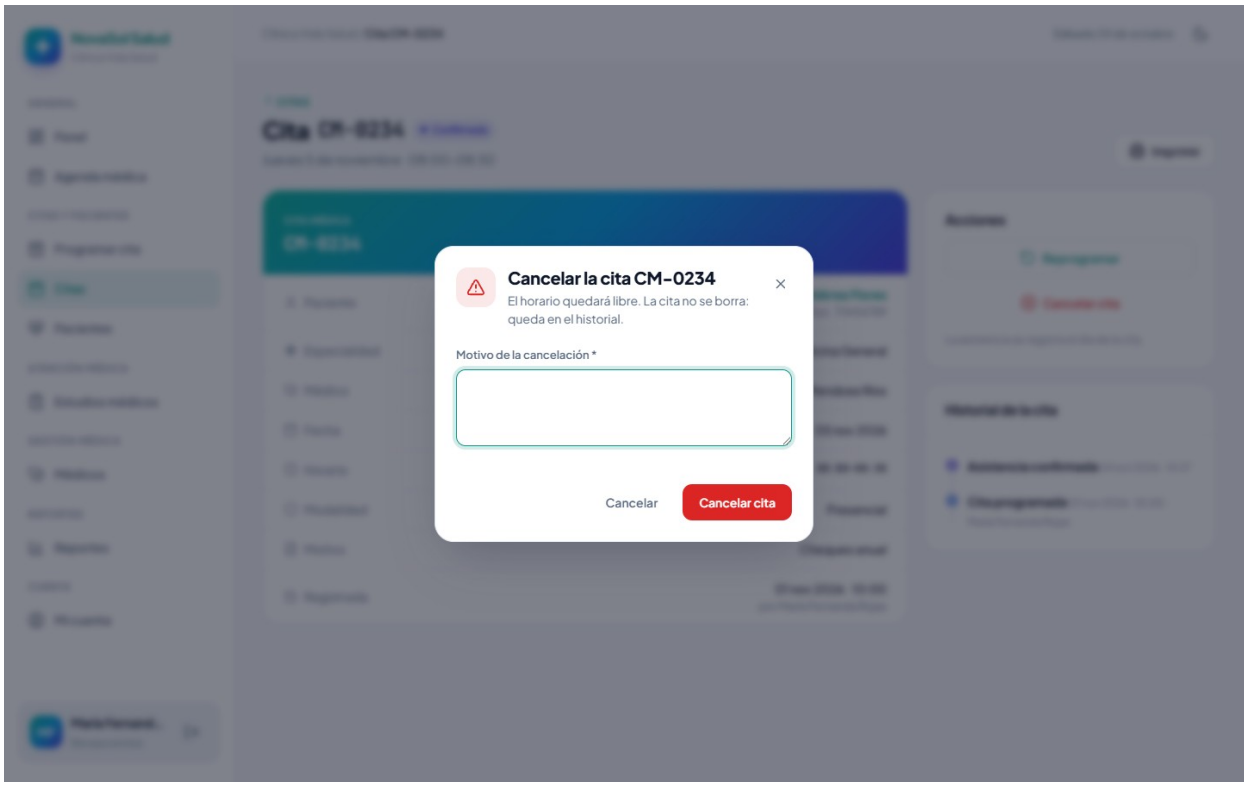
CU11: reprogramar cita



Nota. Captura del sistema en funcionamiento.

Figura 43

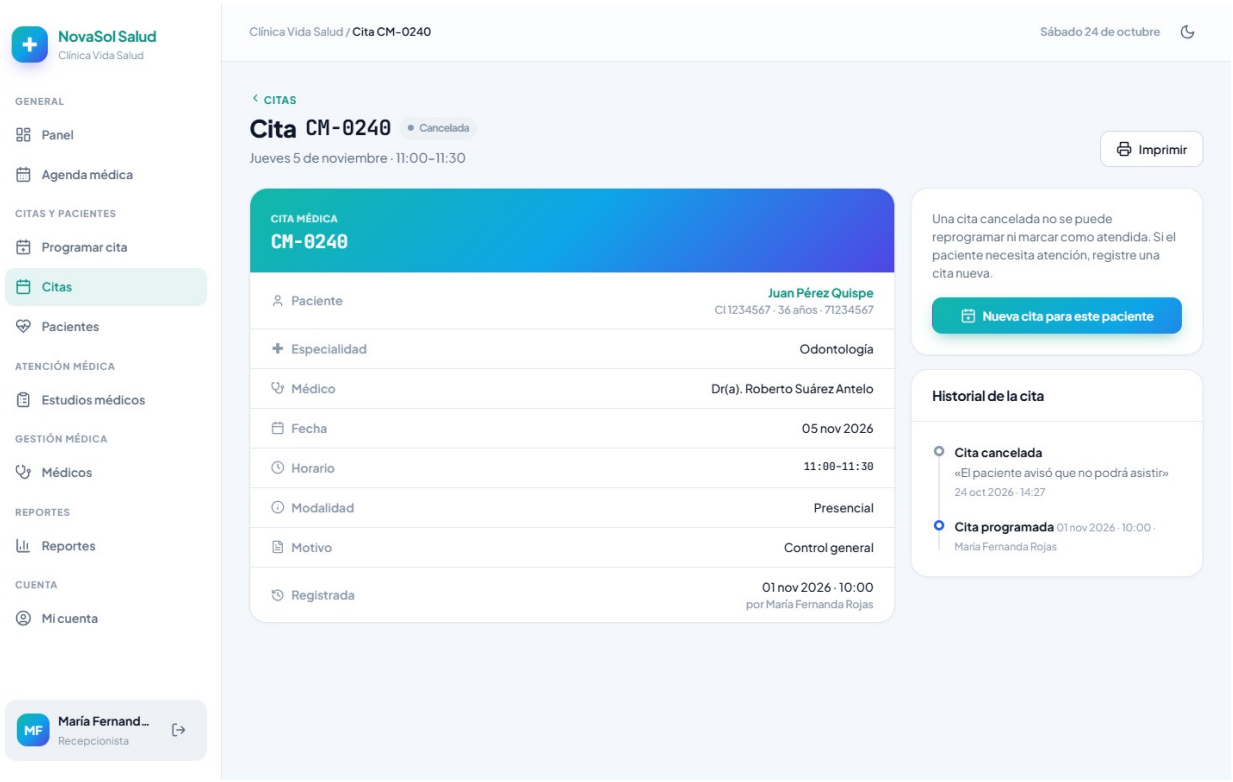
CU12: cancelar cita con motivo



Nota. Captura del sistema en funcionamiento.

Figura 44

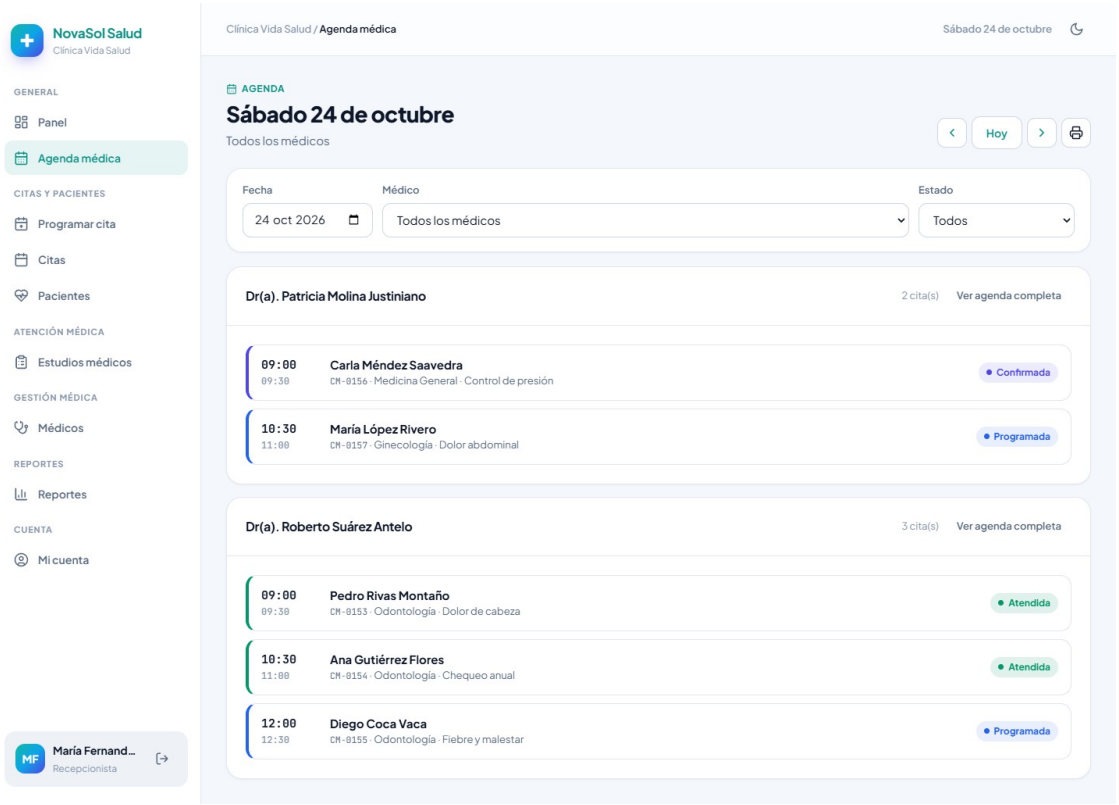
CU12: cita cancelada con su historial



Nota. Captura del sistema en funcionamiento.

Figura 45

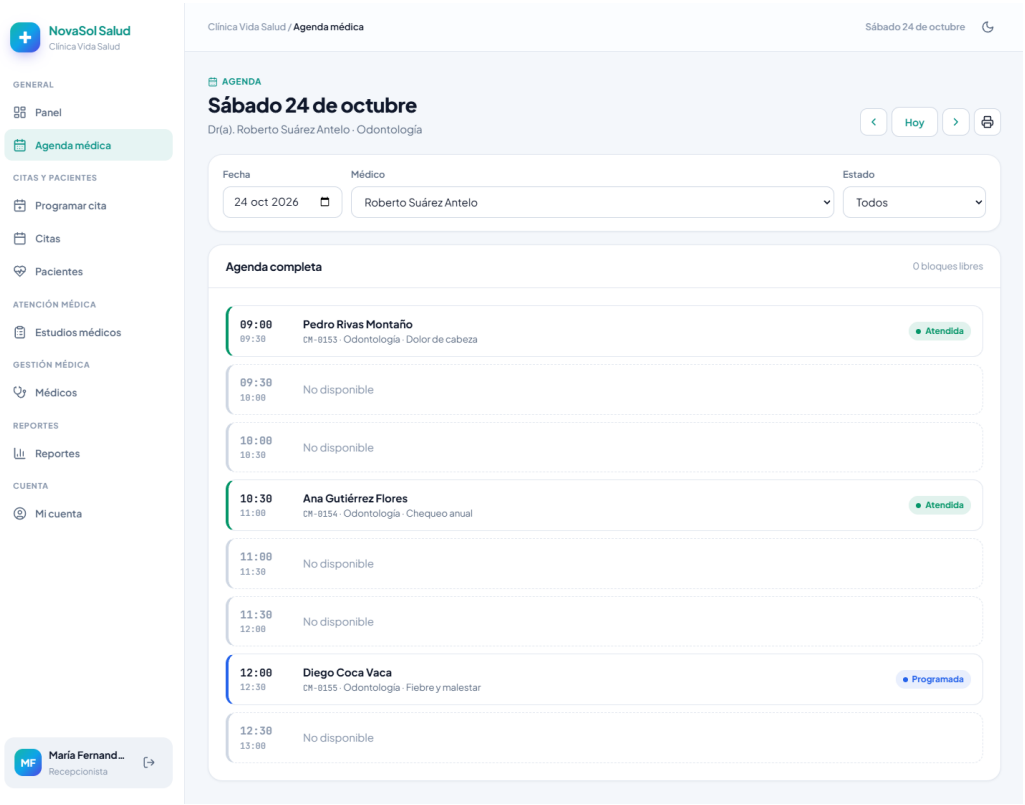
CU14: agenda de todos los médicos



Nota. Captura del sistema en funcionamiento.

Figura 46

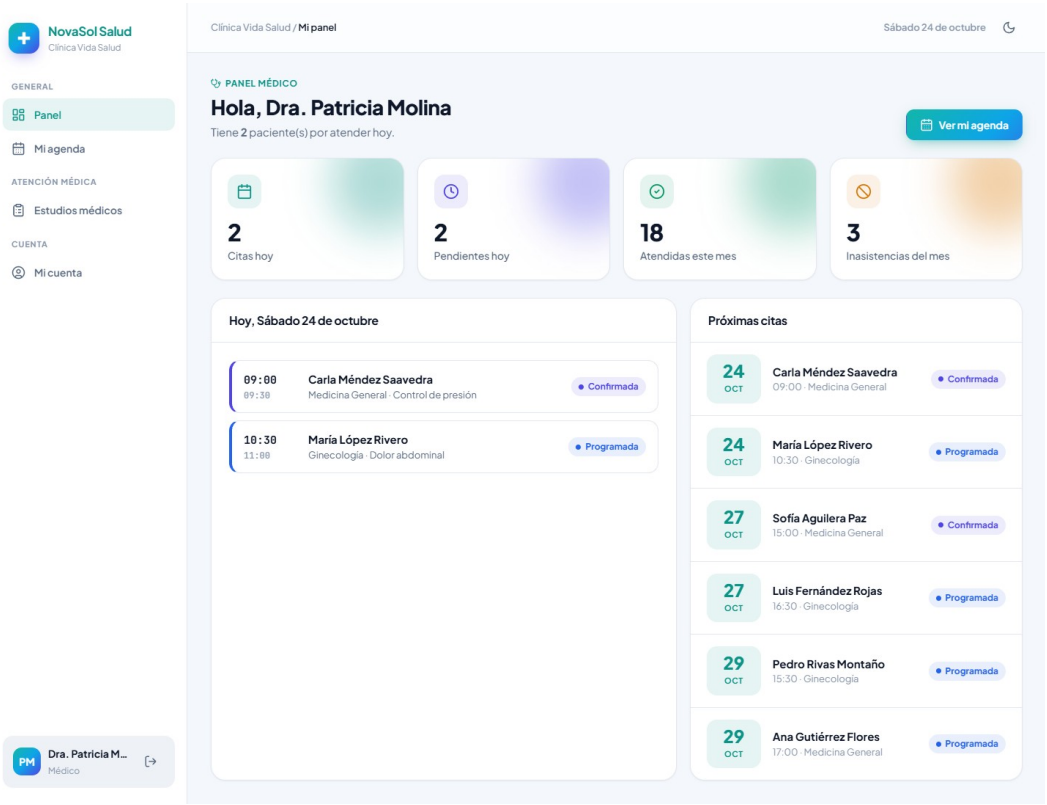
CU14: agenda de un médico



Nota. Captura del sistema en funcionamiento.

Figura 47

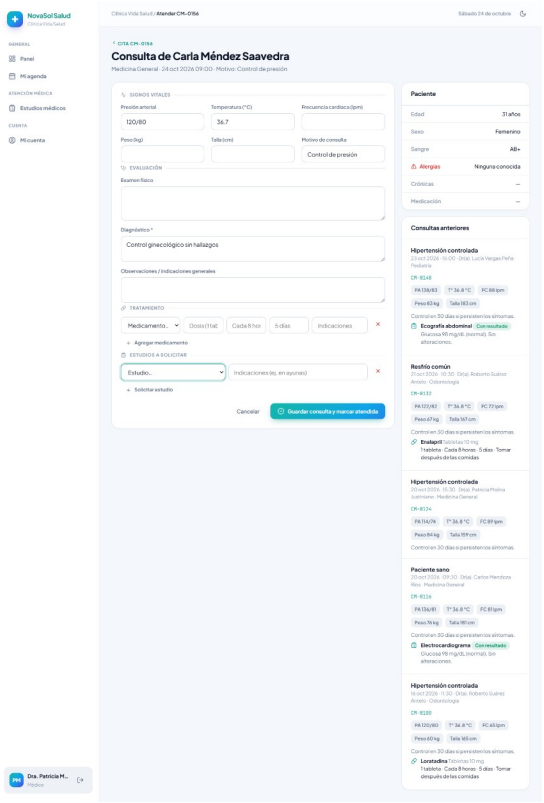
Panel del médico



Nota. Captura del sistema en funcionamiento.

Figura 48

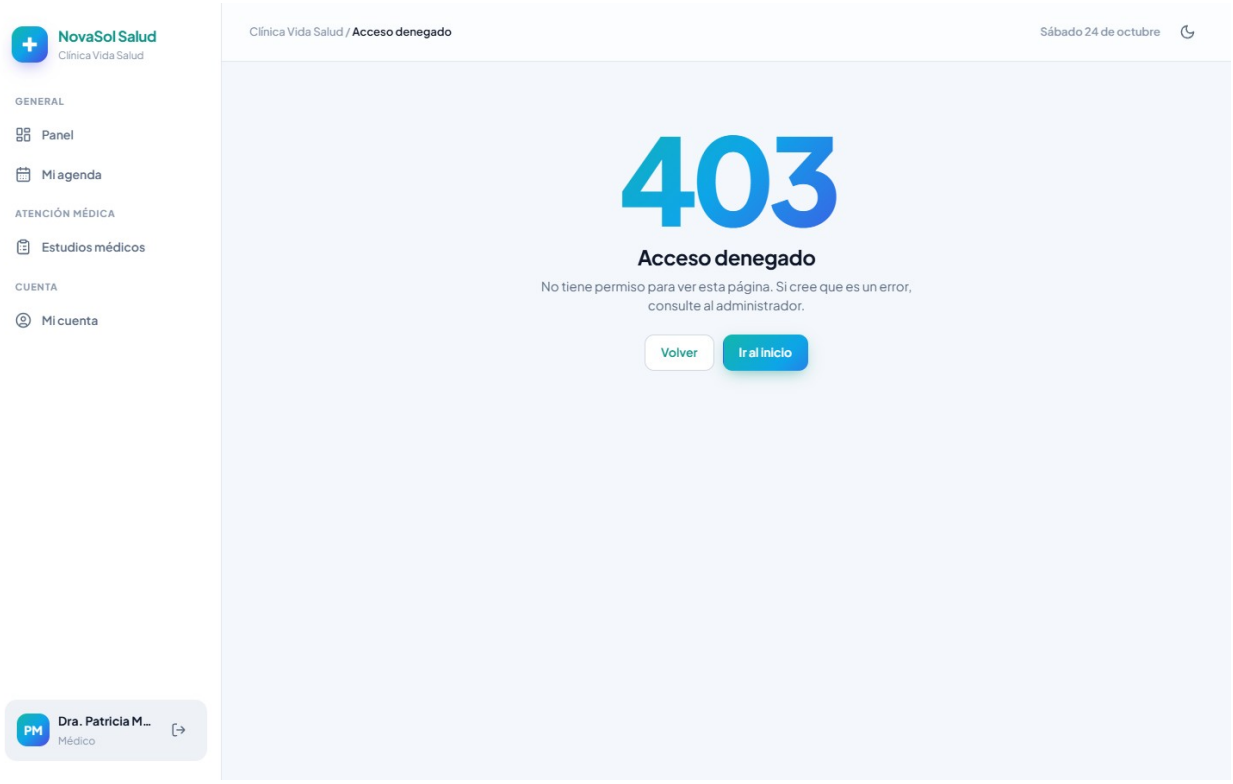
CU15, CU17 y CU18: registro de la consulta



Nota. Captura del sistema en funcionamiento.

Figura 49

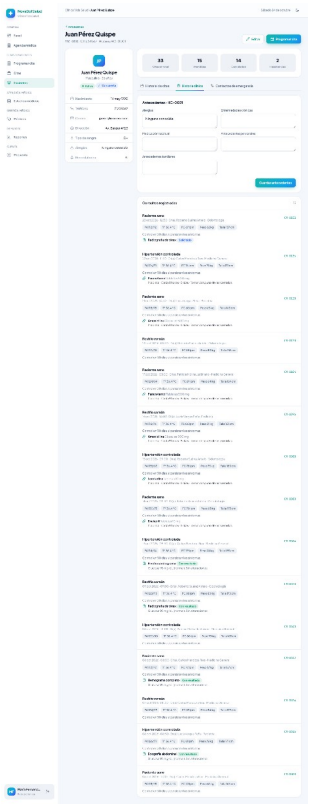
CU15: cita atendida



Nota. Captura del sistema en funcionamiento.

Figura 50

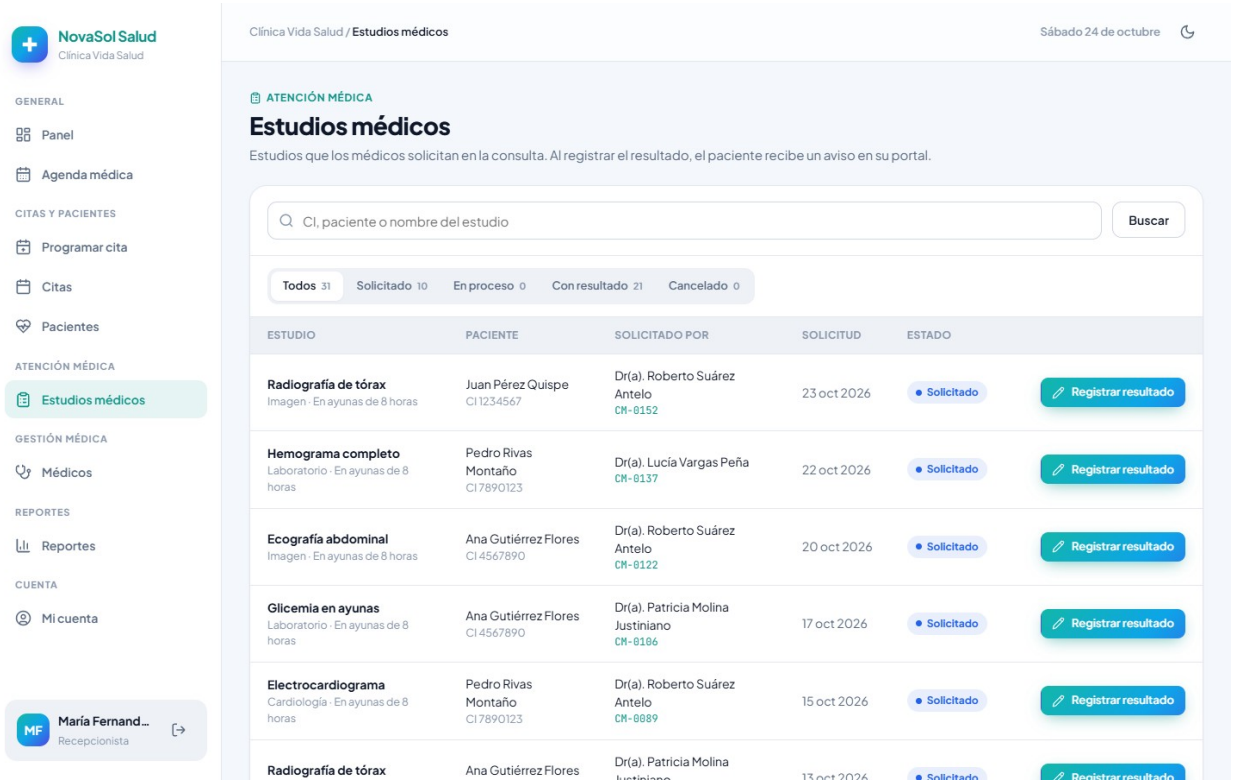
CU16: historia clínica



Nota. Captura del sistema en funcionamiento.

Figura 51

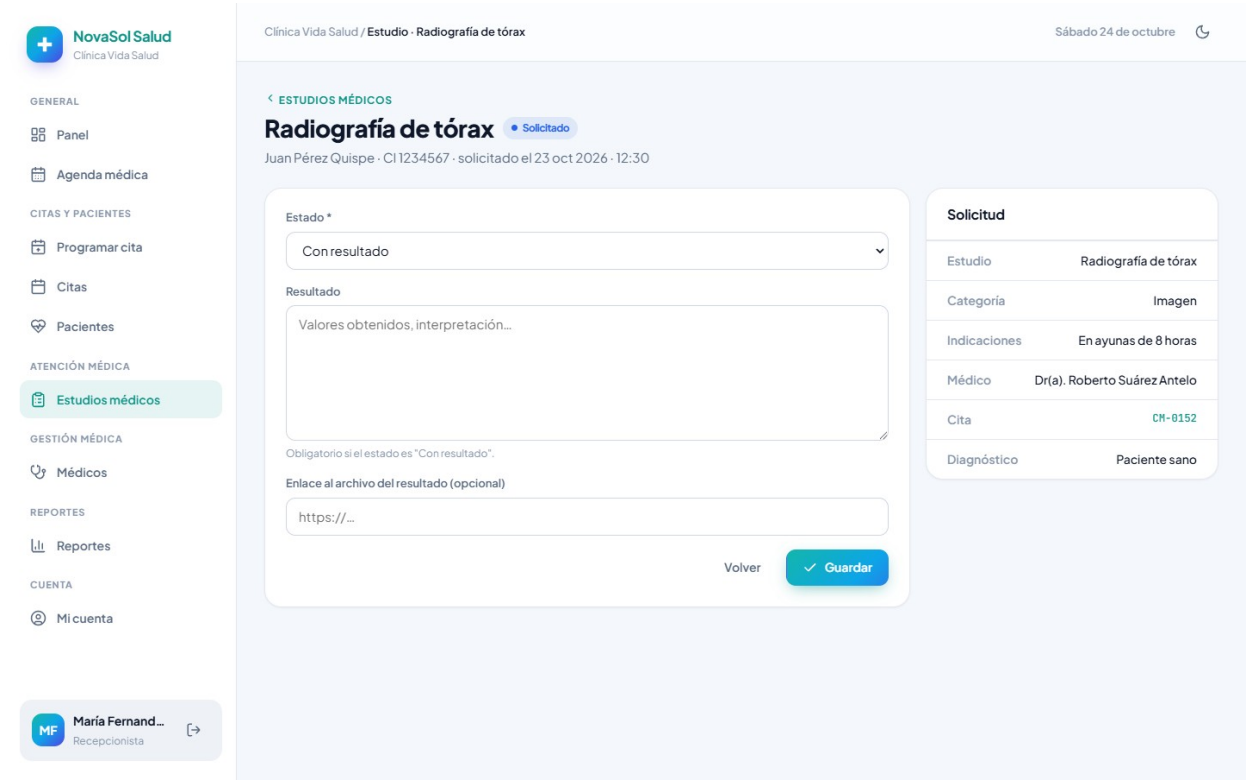
CU18: bandeja de estudios



Nota. Captura del sistema en funcionamiento.

Figura 52

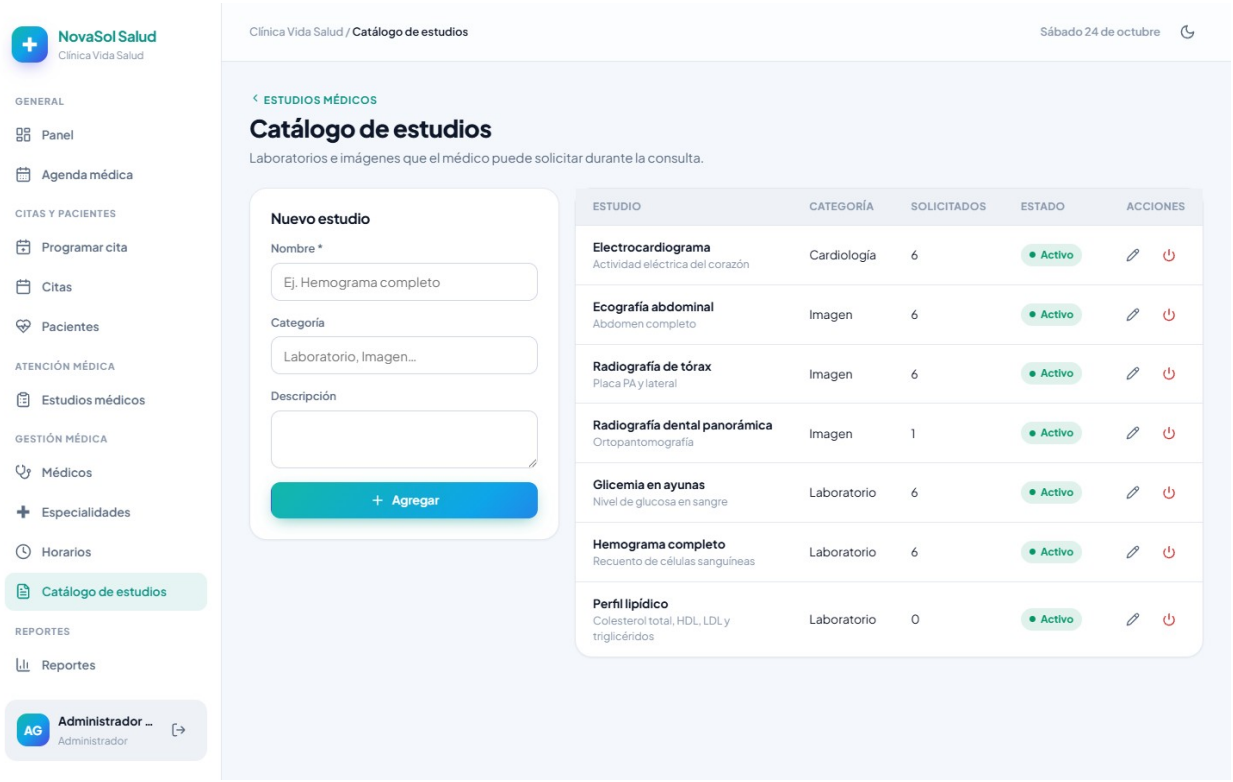
CU18: registro del resultado



Nota. Captura del sistema en funcionamiento.

Figura 53

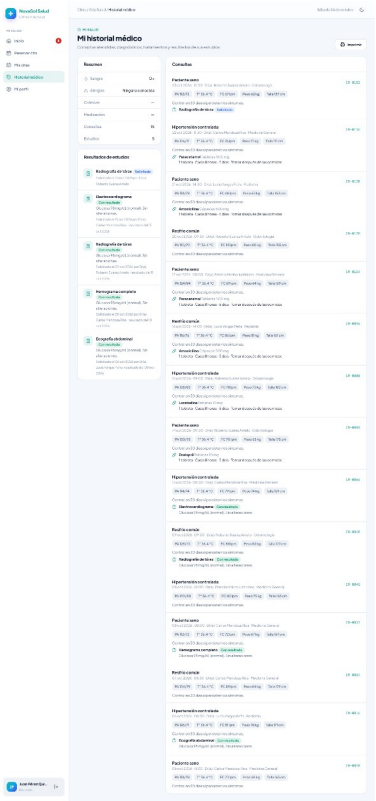
CU18: catálogo de estudios



Nota. Captura del sistema en funcionamiento.

Figura 54

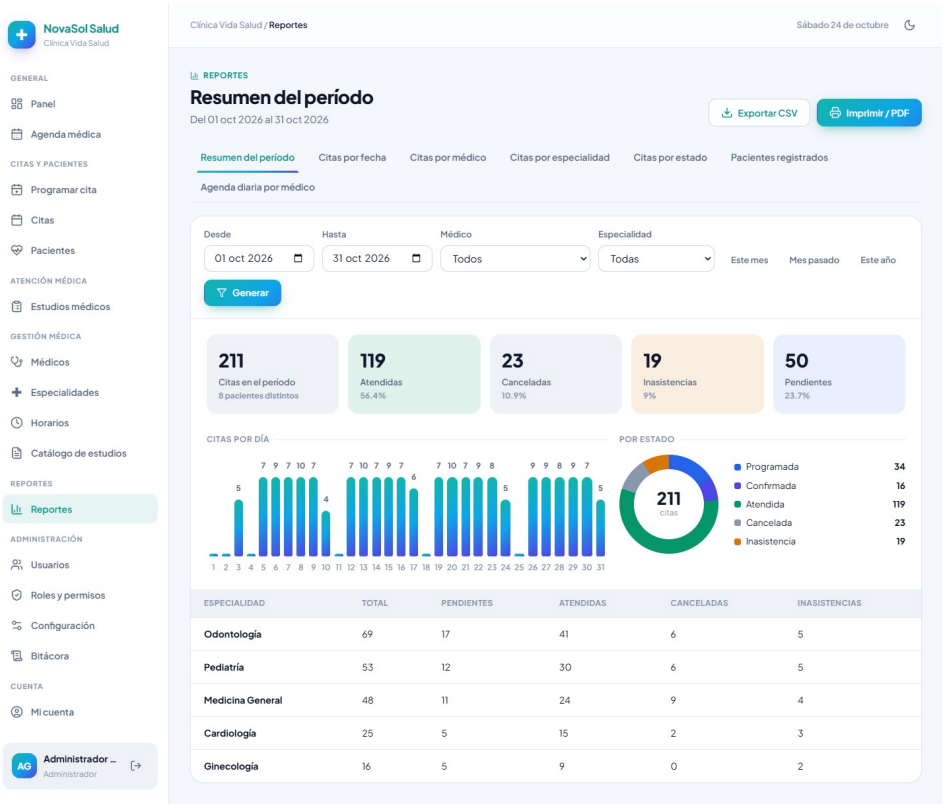
CU19: historial y resultados en el portal



Nota. Captura del sistema en funcionamiento.

Figura 55

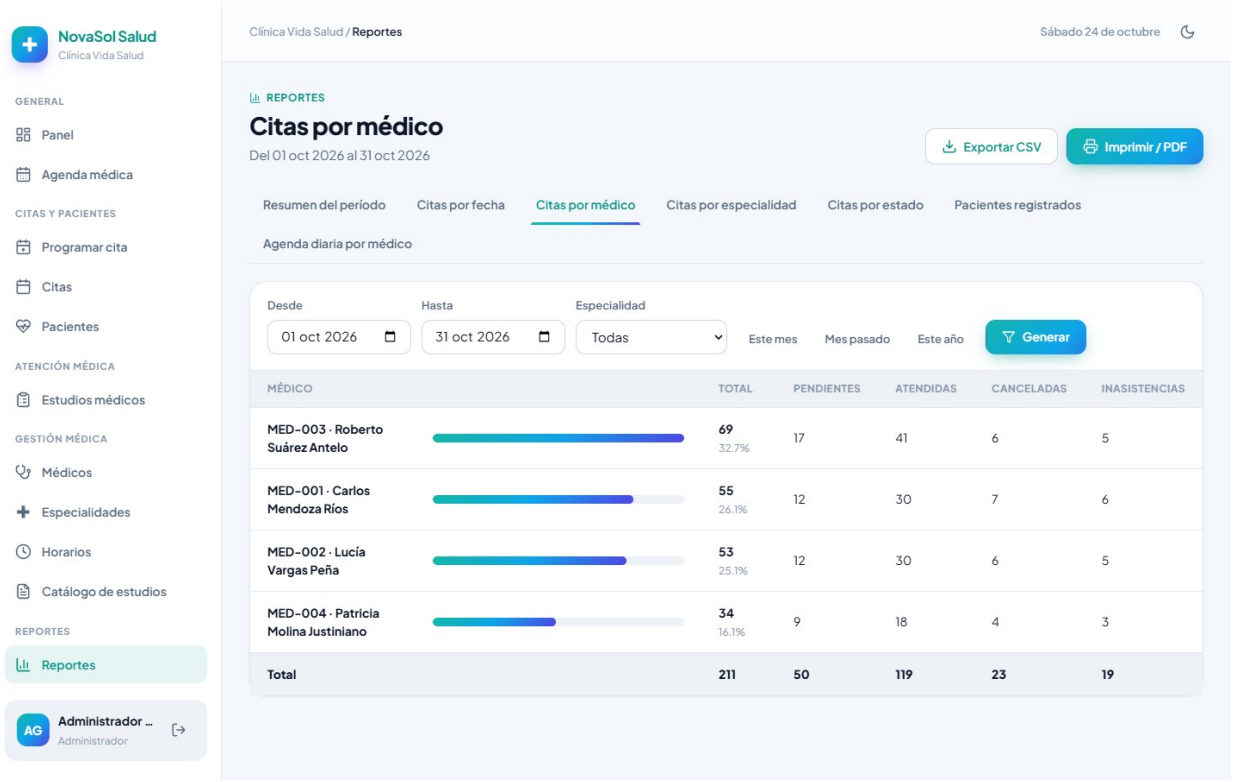
CU20: resumen del período



Nota. Captura del sistema en funcionamiento.

Figura 56

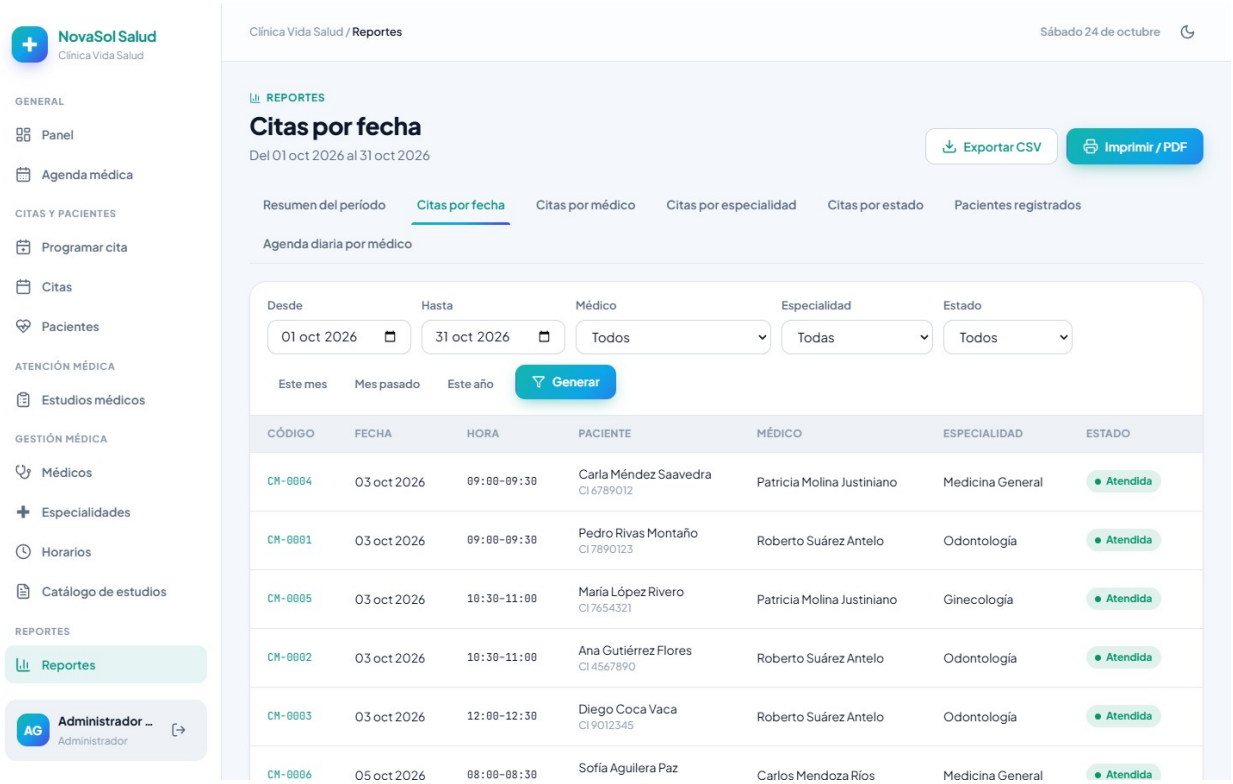
CU20: citas por médico



Nota. Captura del sistema en funcionamiento.

Figura 57

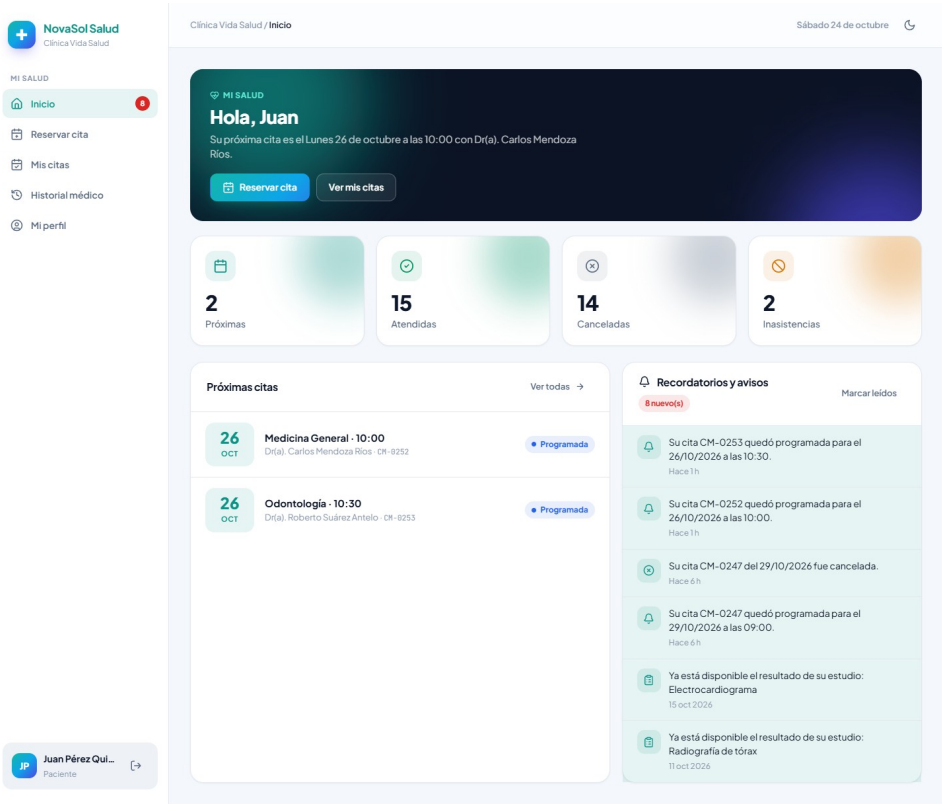
CU20: citas por fecha



Nota. Captura del sistema en funcionamiento.

Figura 58

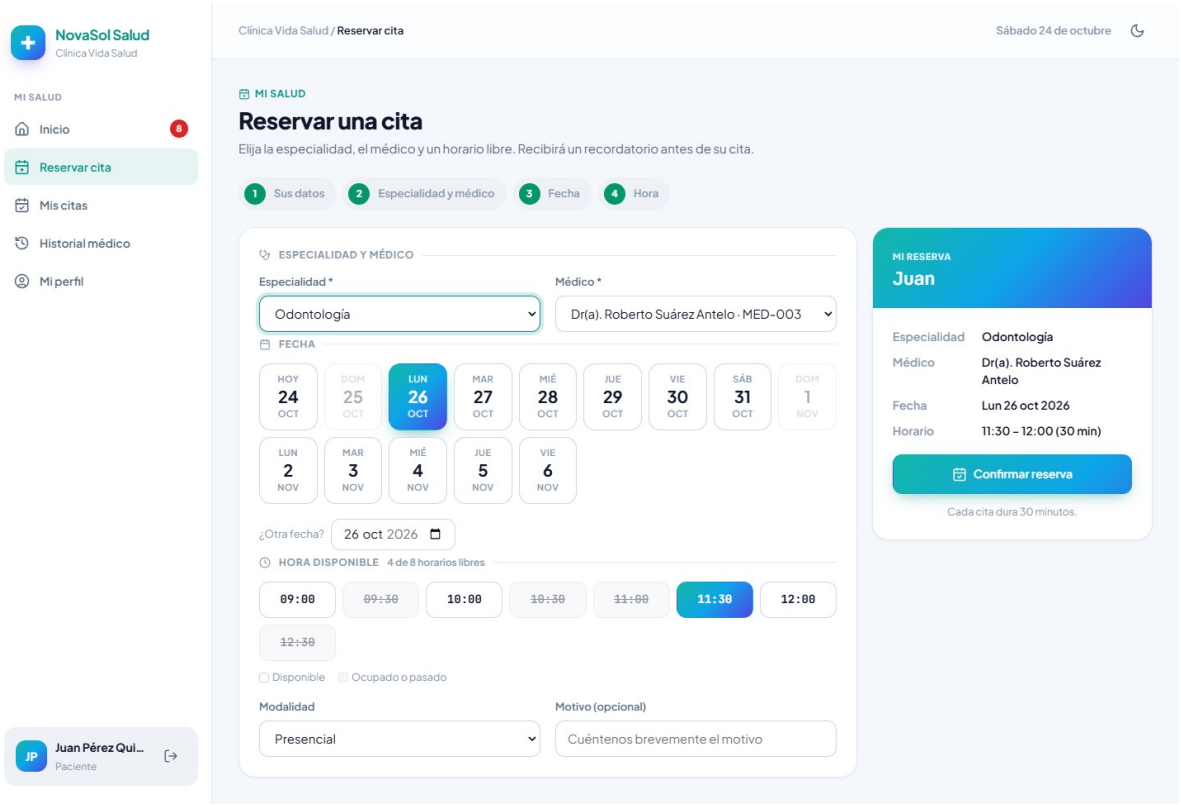
Portal del paciente: inicio



Nota. Captura del sistema en funcionamiento.

Figura 59

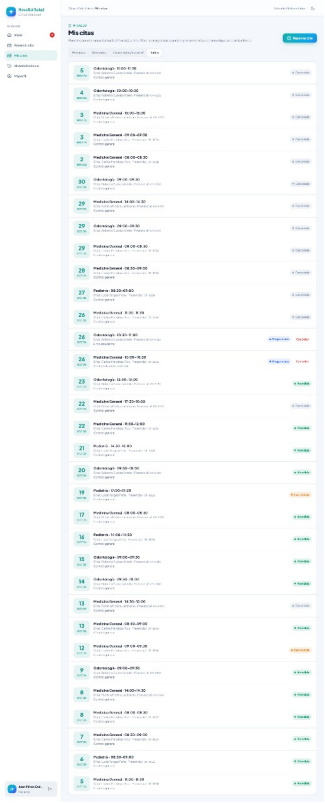
Portal: reserva de cita (CU10)



Nota. Captura del sistema en funcionamiento.

Figura 60

Portal: mis citas (CU12)



Nota. Captura del sistema en funcionamiento.

Figura 61

Diseño adaptable: agenda en un celular



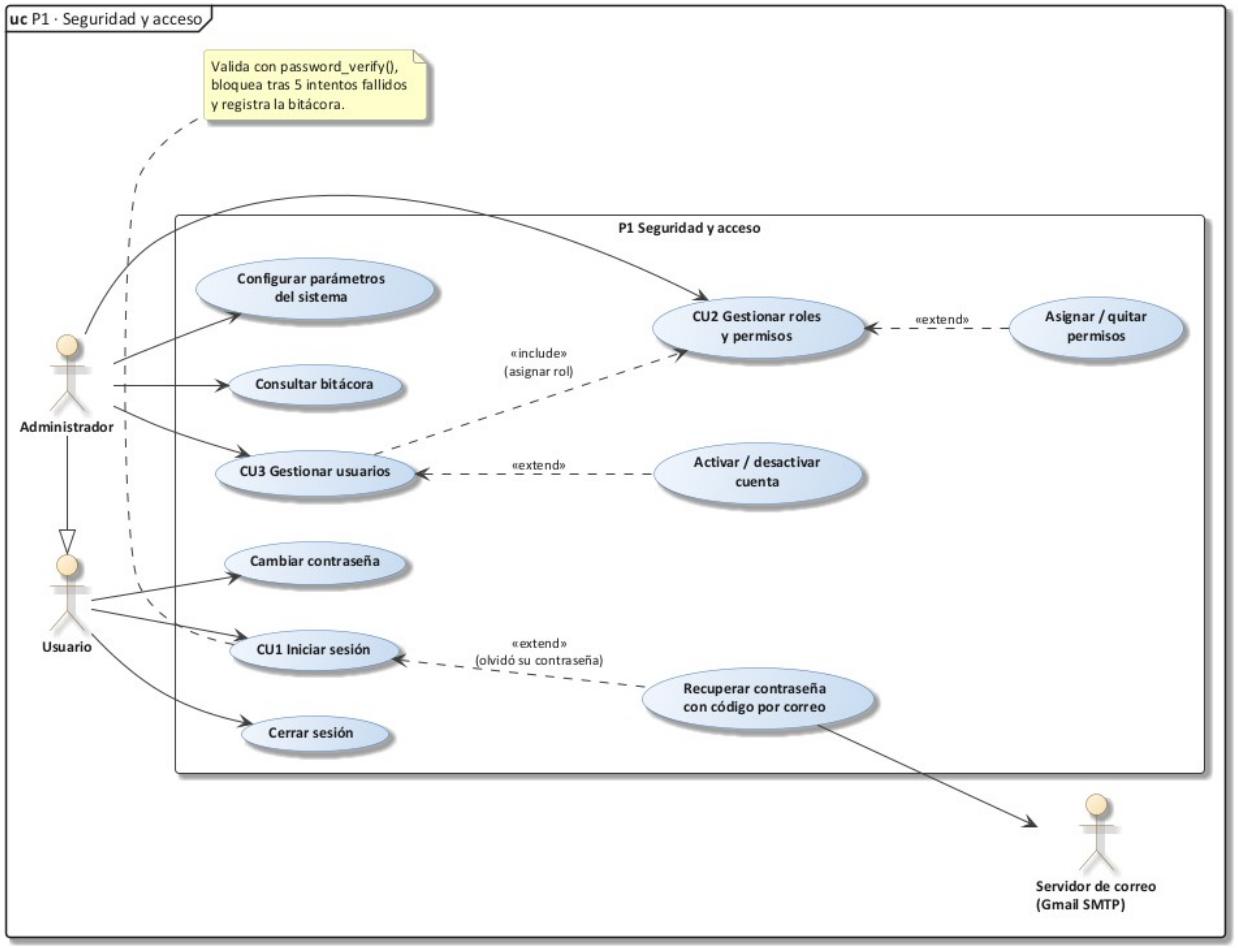
Nota. Captura del sistema en funcionamiento.

2.5 Estructurar modelo de casos de uso

El modelo se estructura por paquetes, con generalización de actores (el Administrador hereda los casos de uso del Recepcionista y todos heredan de Usuario) y relaciones «include» y «extend».

Figura 62

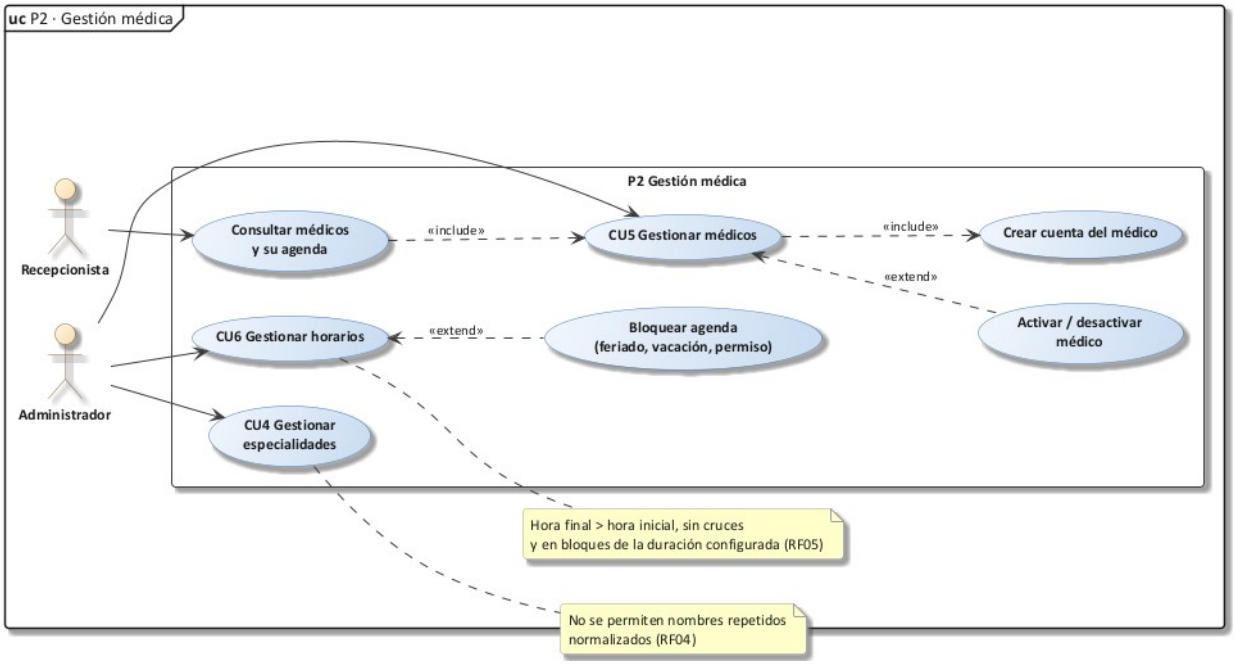
Modelo de casos de uso estructurado del paquete P1 Seguridad y acceso



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 63

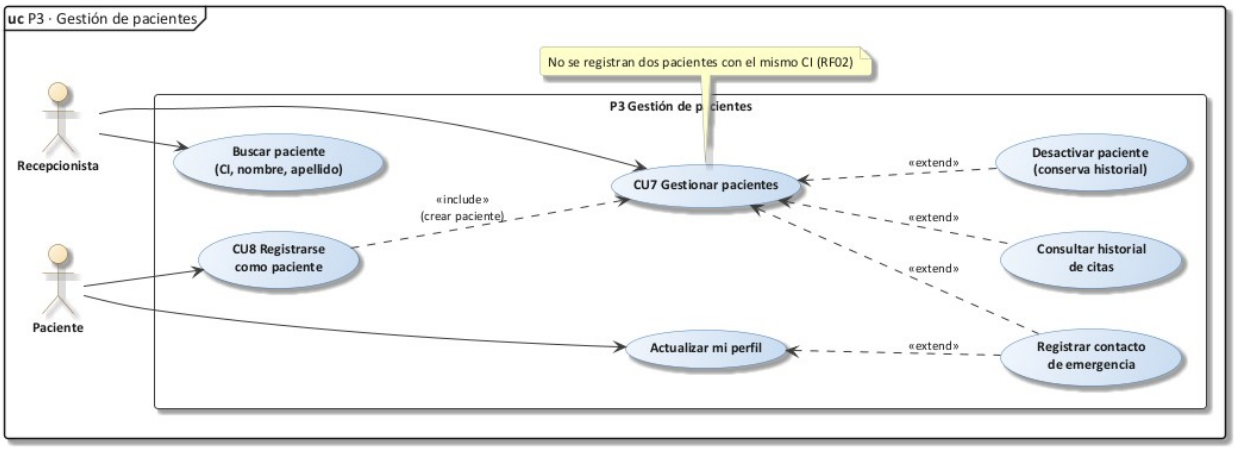
Modelo de casos de uso estructurado del paquete P2 Gestión médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 64

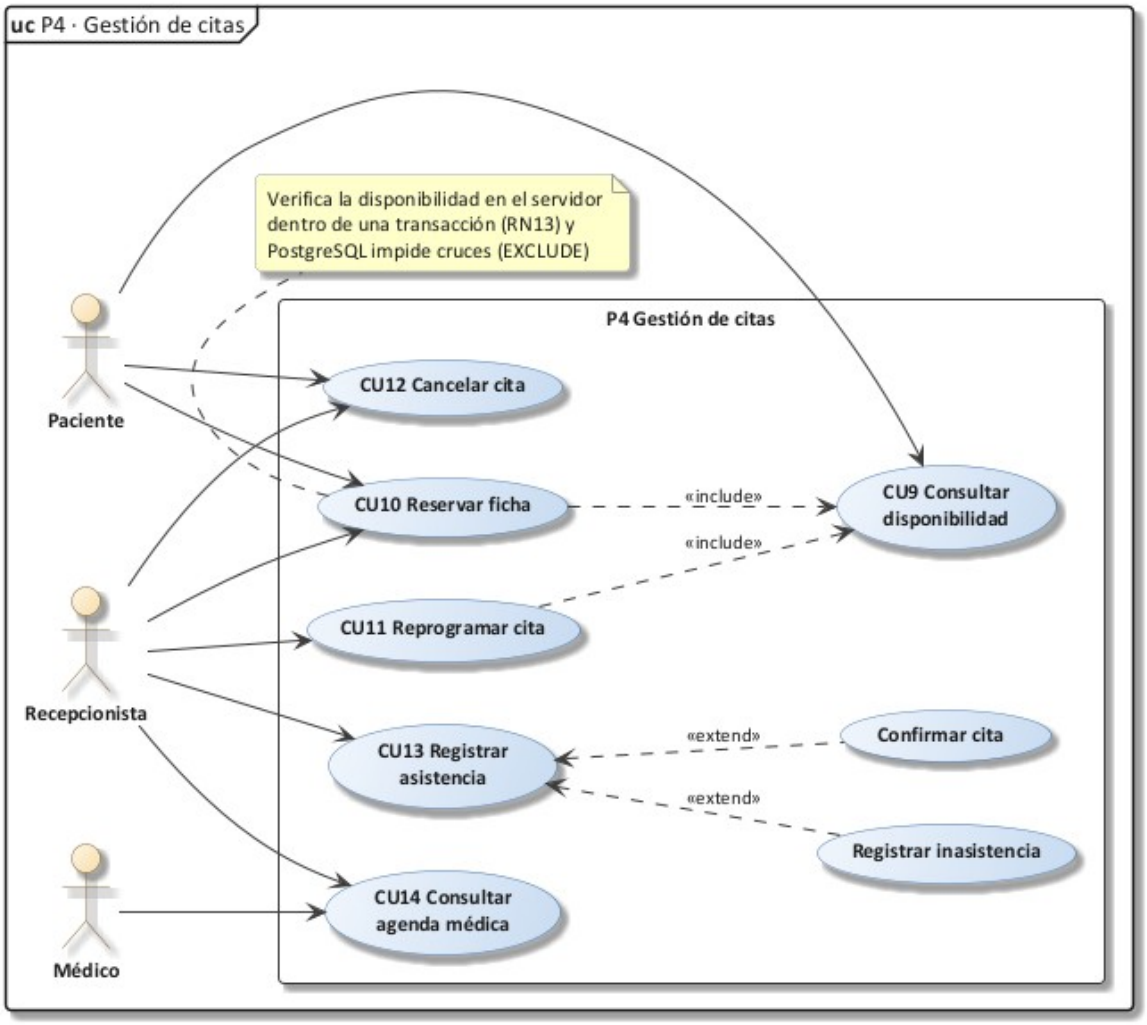
Modelo de casos de uso estructurado del paquete P3 Gestión de pacientes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 65

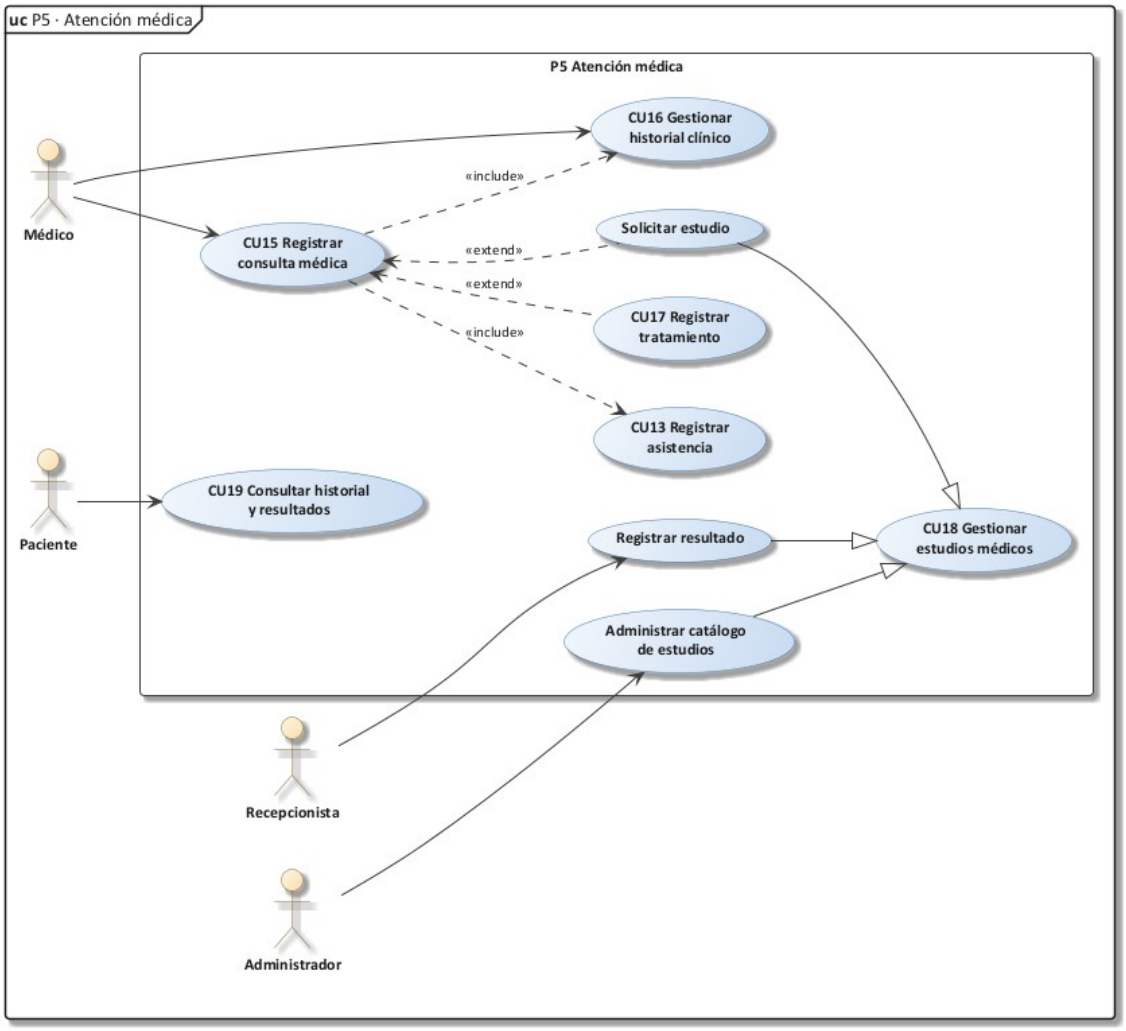
Modelo de casos de uso estructurado del paquete P4 Gestión de citas



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 66

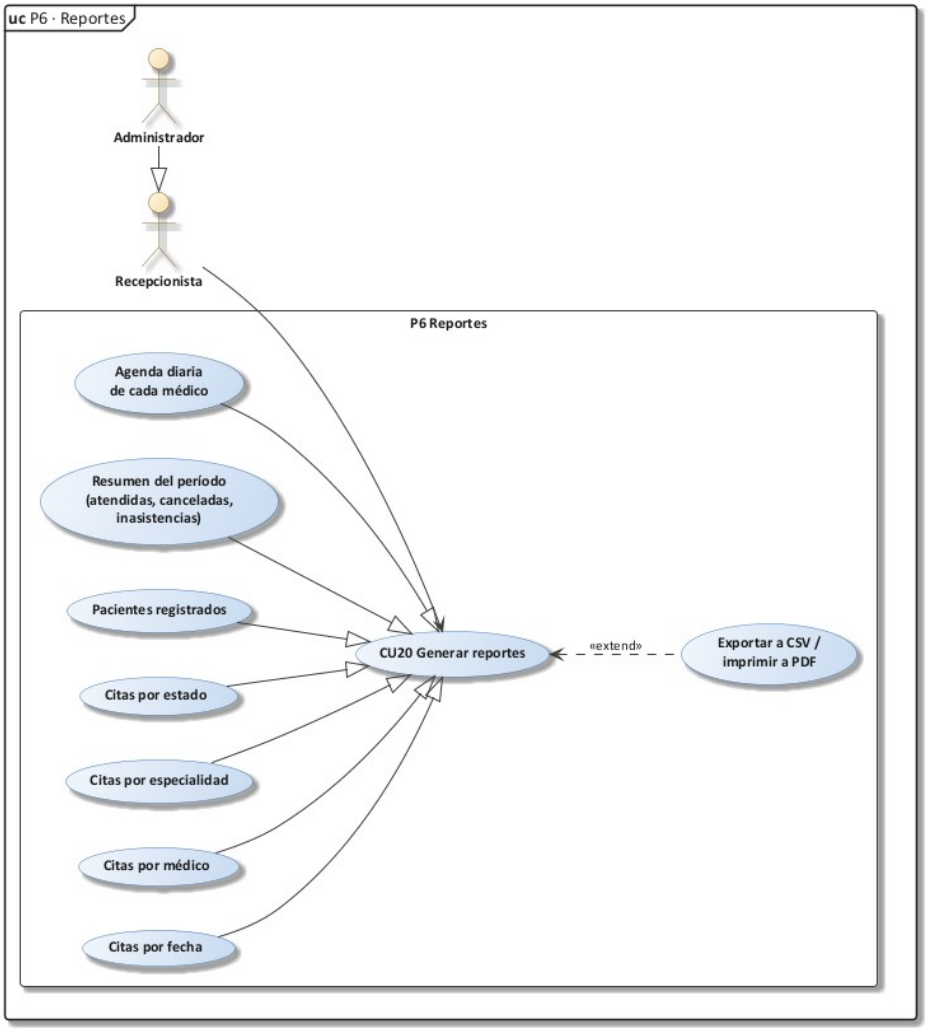
Modelo de casos de uso estructurado del paquete P5 Atención médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 67

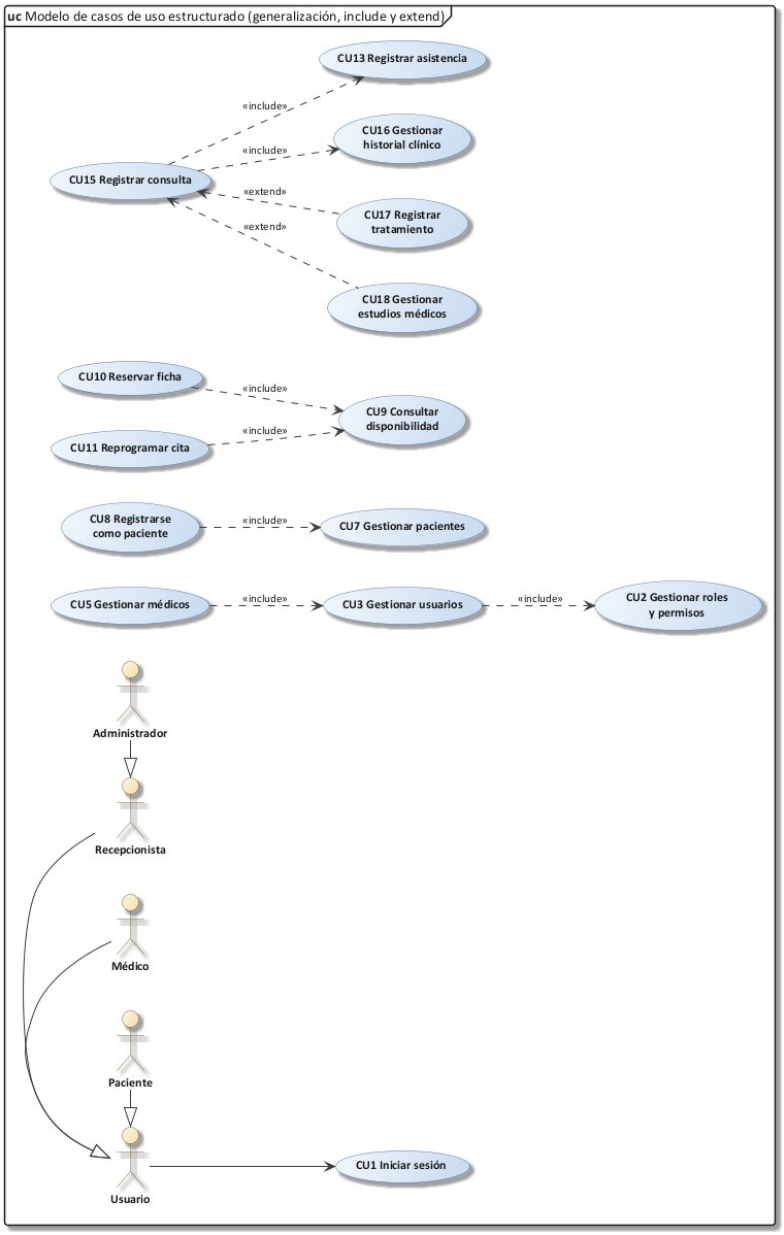
Modelo de casos de uso estructurado del paquete P6 Reportes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 68

Modelo de casos de uso estructurado completo



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Capítulo 3. Flujo de trabajo: análisis

3.1 Análisis de la arquitectura

3.1.1 Identificar paquetes

Los casos de uso se agrupan en paquetes de análisis según su afinidad funcional y los actores que los usan.

P1 Seguridad y acceso (Ciclo I): Controla el ingreso al sistema, los usuarios, los roles y permisos, la configuración y la bitácora. Es un paquete de servicio común del que dependen todos los demás.

P2 Gestión médica (Ciclo I): Administra las especialidades, los médicos y sus horarios de atención y bloqueos de agenda.

P3 Gestión de pacientes (Ciclo I y II): Registra a los pacientes, su historia clínica y contactos de emergencia; incluye el autoregistro desde el portal.

P4 Gestión de citas (Ciclo II): Calcula la disponibilidad y gestiona el ciclo de vida de la cita. Es el paquete central del negocio.

P5 Atención médica (Ciclo II): Registra la consulta, los tratamientos, los estudios y el historial clínico.

P6 Reportes (Ciclo II): Genera los reportes administrativos de citas.

3.1.2 Relacionar paquetes y casos de uso

CICLO I · CU1 a CU7: seguridad y datos maestros

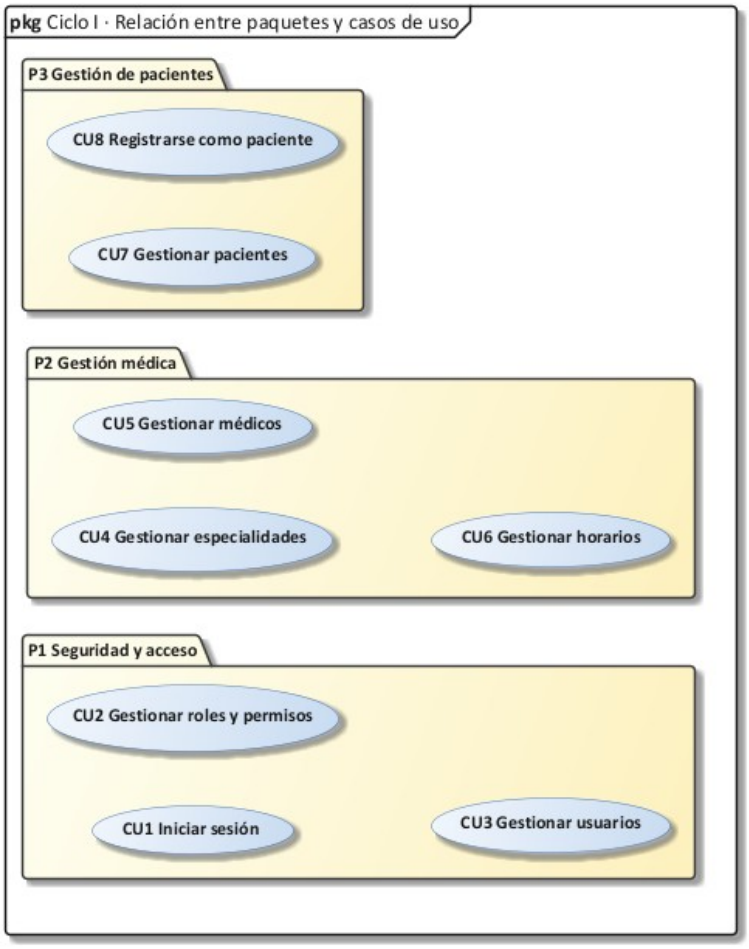
Tabla 56

Paquetes y casos de uso del Ciclo I

Paquete	Casos de uso
P1 Seguridad y acceso	CU1, CU2, CU3
P2 Gestión médica	CU4, CU5, CU6
P3 Gestión de pacientes	CU7

Figura 69

Relación entre paquetes y casos de uso del Ciclo I



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

CICLO II · CU8 a CU20: citas, atención, reportes y portal

Tabla 57

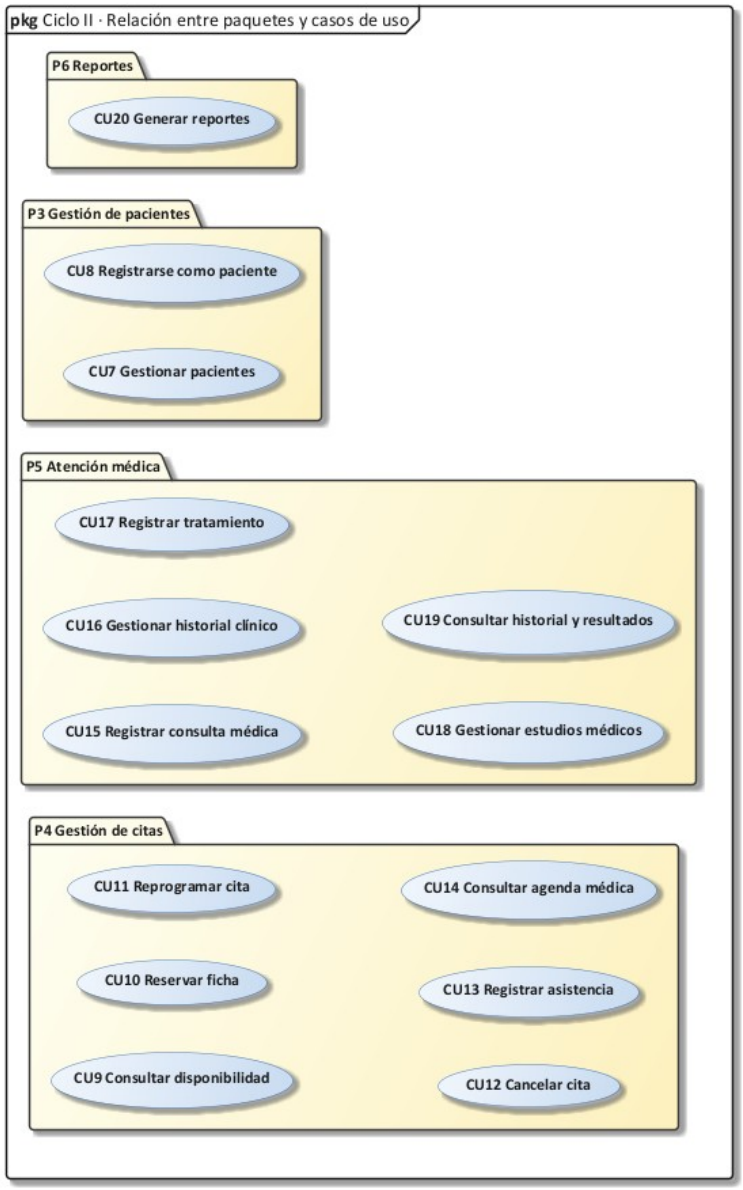
Paquetes y casos de uso del Ciclo II

Paquete	Casos de uso
P3 Gestión de pacientes	CU8
P4 Gestión de citas	CU9, CU10, CU11, CU12, CU13, CU14
P5 Atención médica	CU15, CU16, CU17, CU18, CU19

Paquete	Casos de uso
P6 Reportes	CU20

Figura 70

Relación entre paquetes y casos de uso del Ciclo II



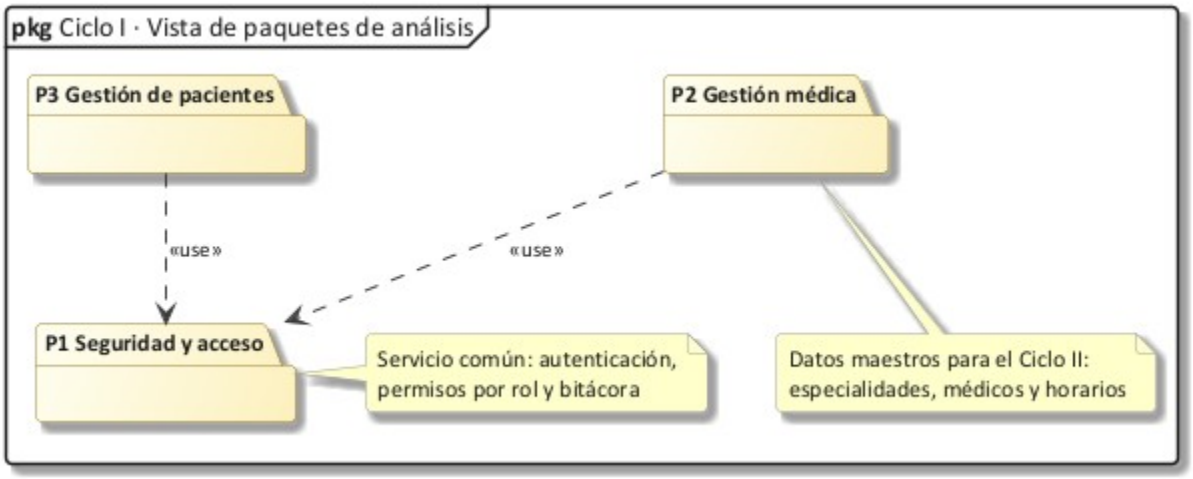
Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

3.1.3 Vista de paquetes

CICLO I · CU1 a CU7: seguridad y datos maestros

Figura 71

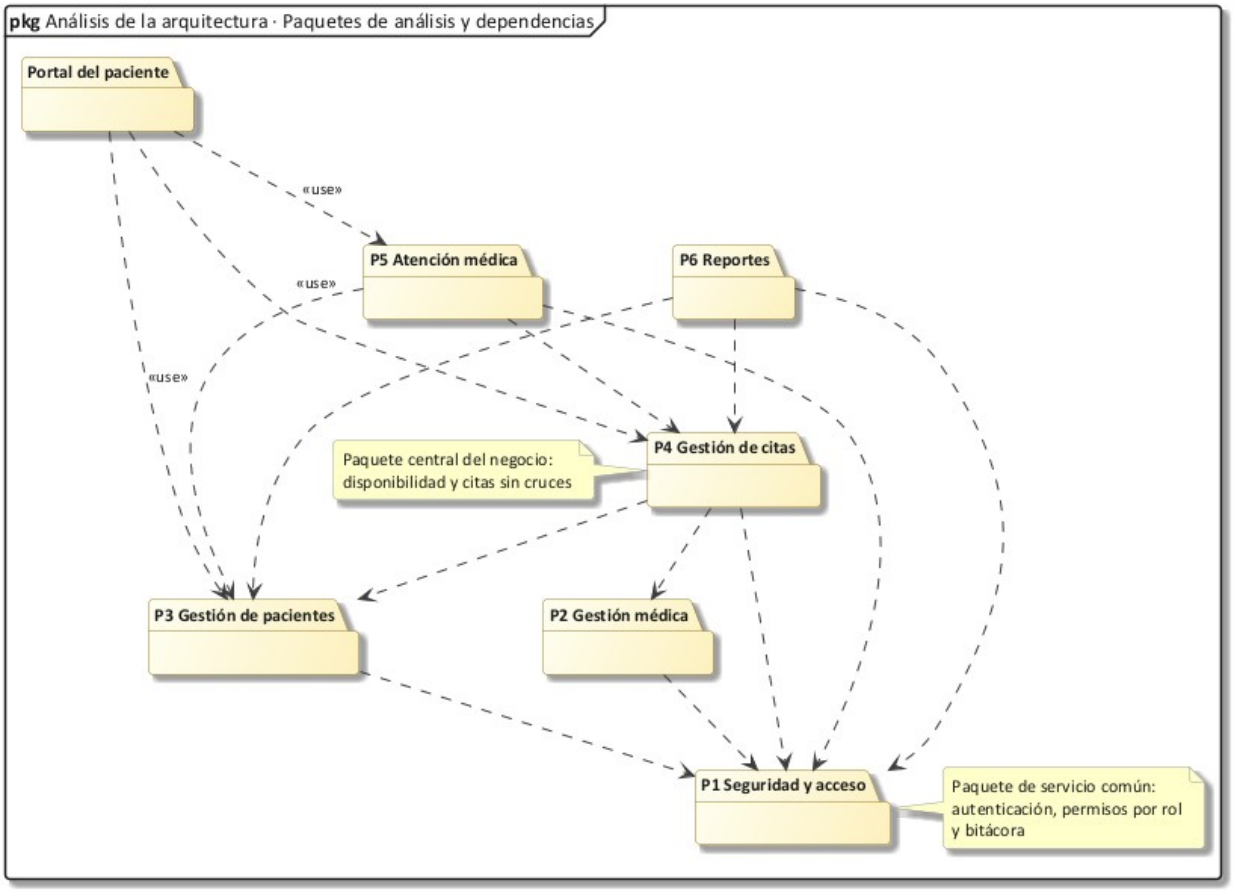
Vista de paquetes de análisis del Ciclo I



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 72

Vista de paquetes de análisis del Ciclo II (sistema completo)



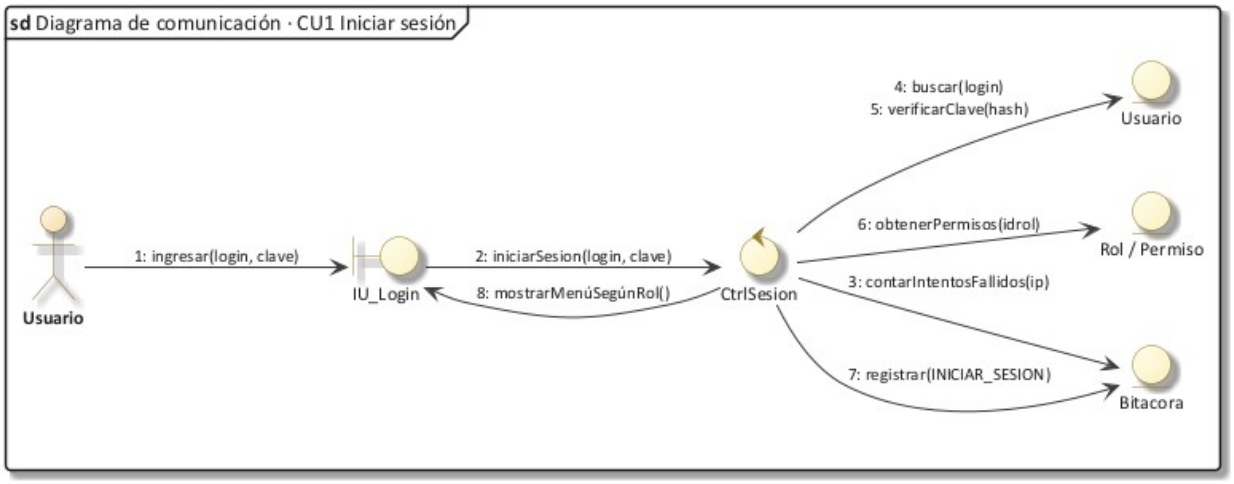
Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

3.2 Analizar un caso de uso

Para cada caso de uso se identificaron sus clases de análisis (interfaz, control y entidad) y los mensajes que intercambian, representados en diagramas de comunicación.

Figura 73

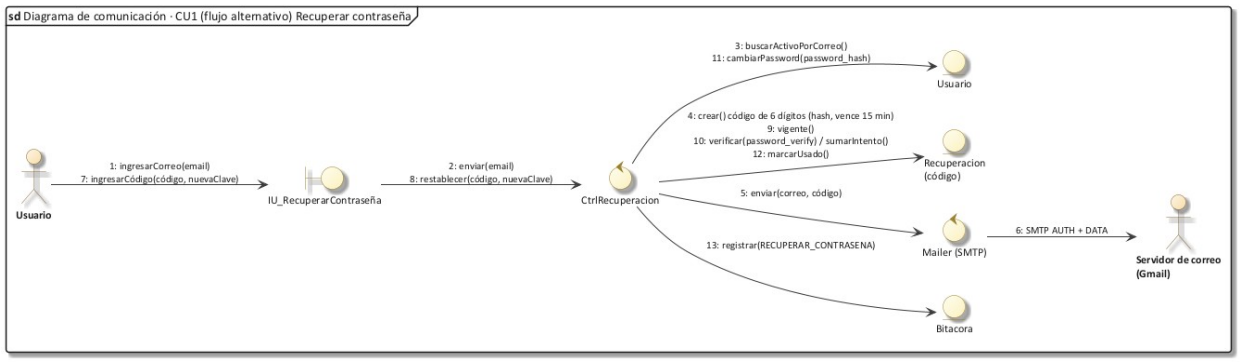
Diagrama de comunicación de CU1 Iniciar sesión



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 74

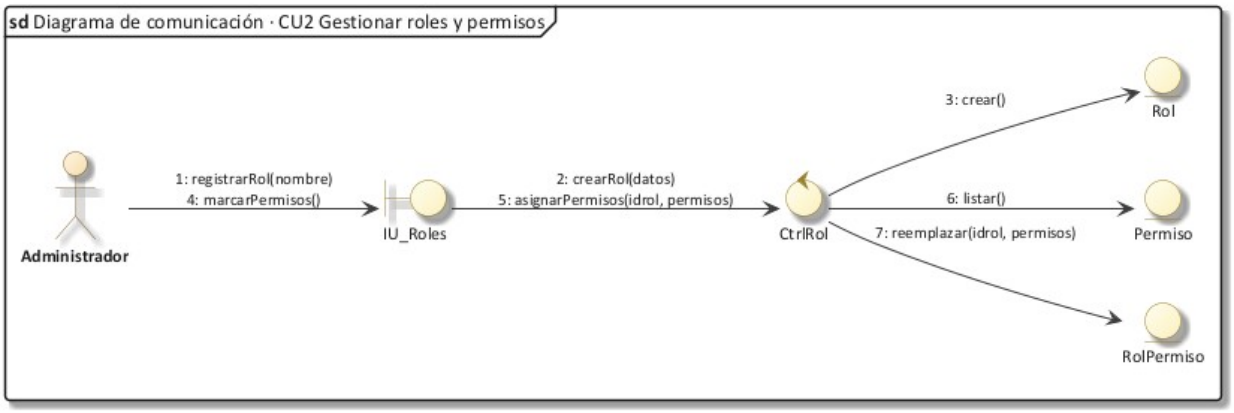
Diagrama de comunicación de CU1 (flujo alternativo): recuperar contraseña por correo



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 75

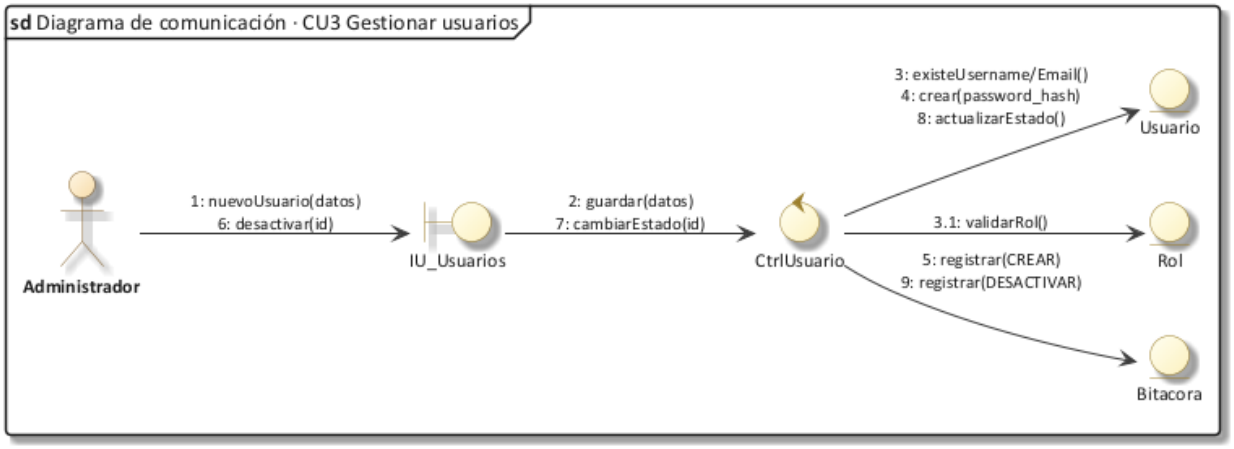
Diagrama de comunicación de CU2 Gestionar roles y permisos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 76

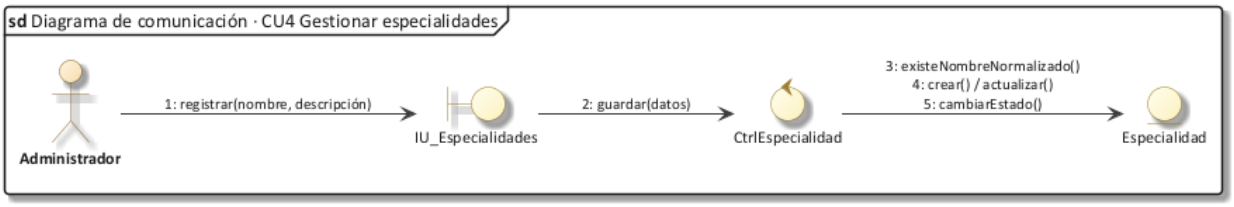
Diagrama de comunicación de CU3 Gestionar usuarios



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 77

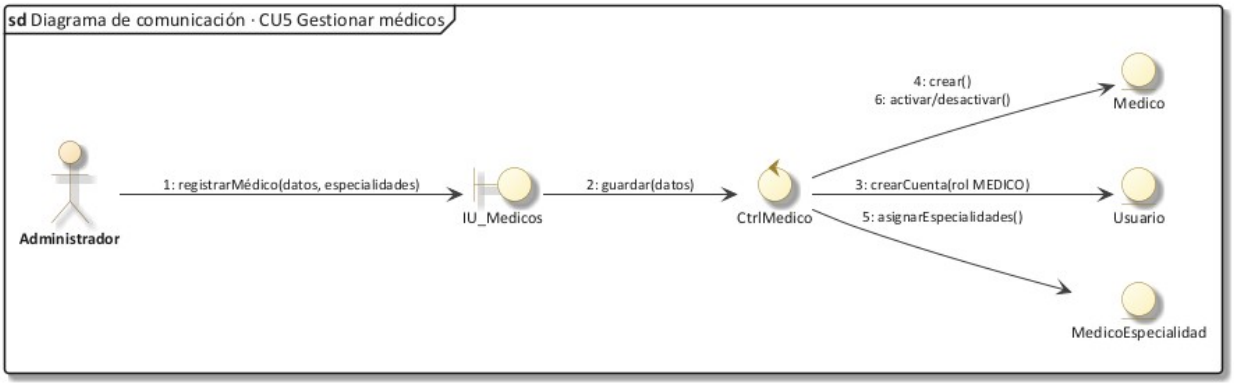
Diagrama de comunicación de CU4 Gestionar especialidades



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 78

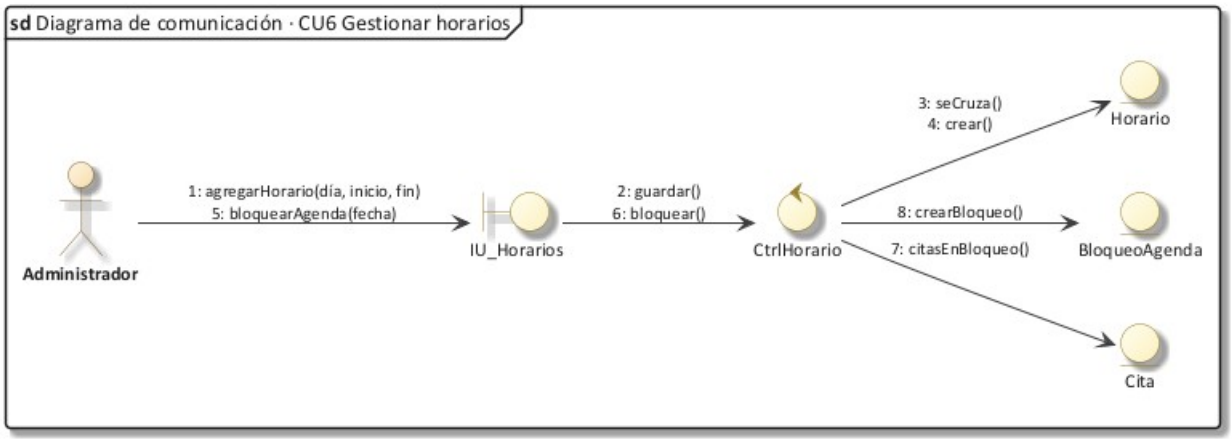
Diagrama de comunicación de CU5 Gestionar médicos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 79

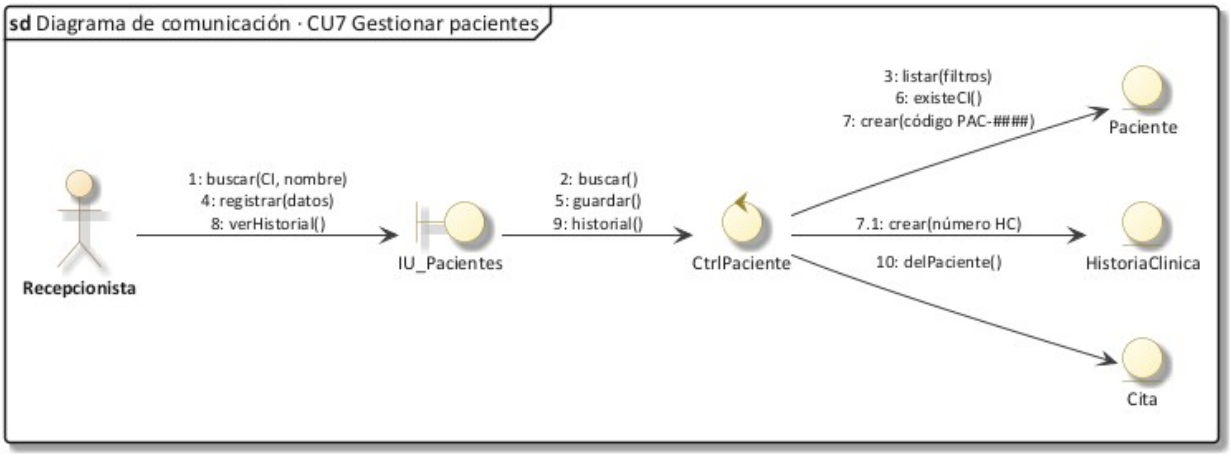
Diagrama de comunicación de CU6 Gestionar horarios



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 80

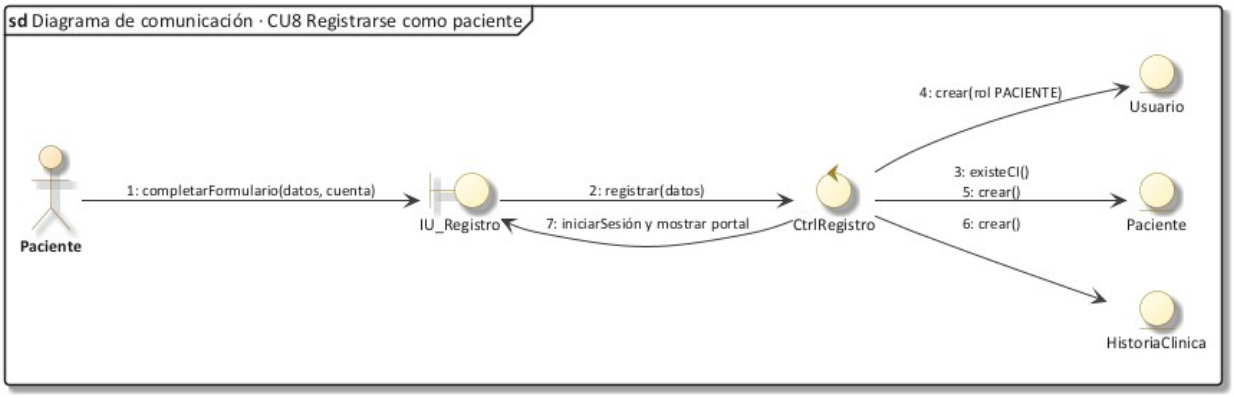
Diagrama de comunicación de CU7 Gestionar pacientes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 81

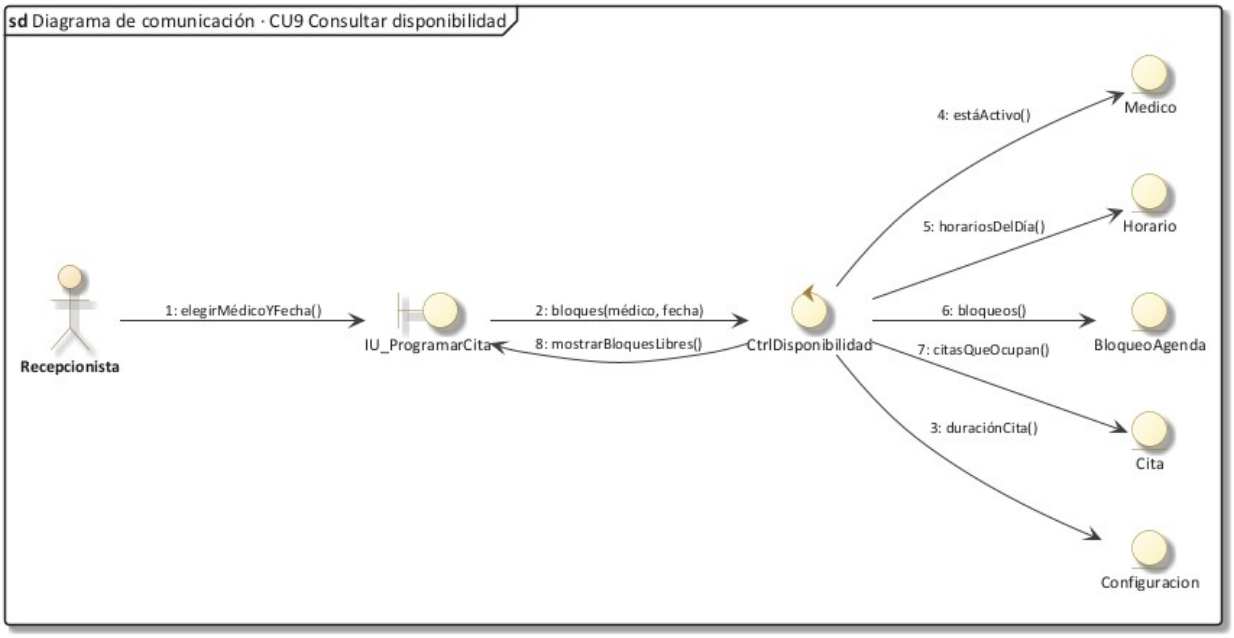
Diagrama de comunicación de CU8 Registrarse como paciente



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 82

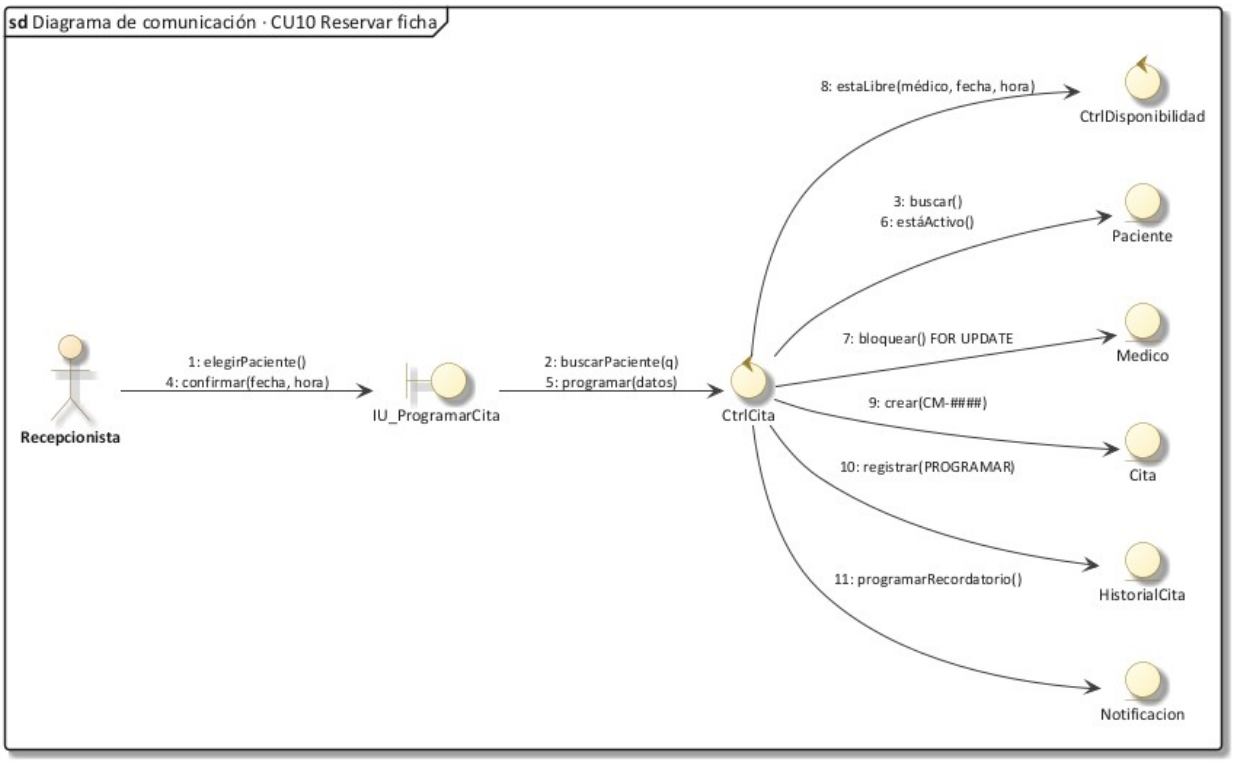
Diagrama de comunicación de CU9 Consultar disponibilidad



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 83

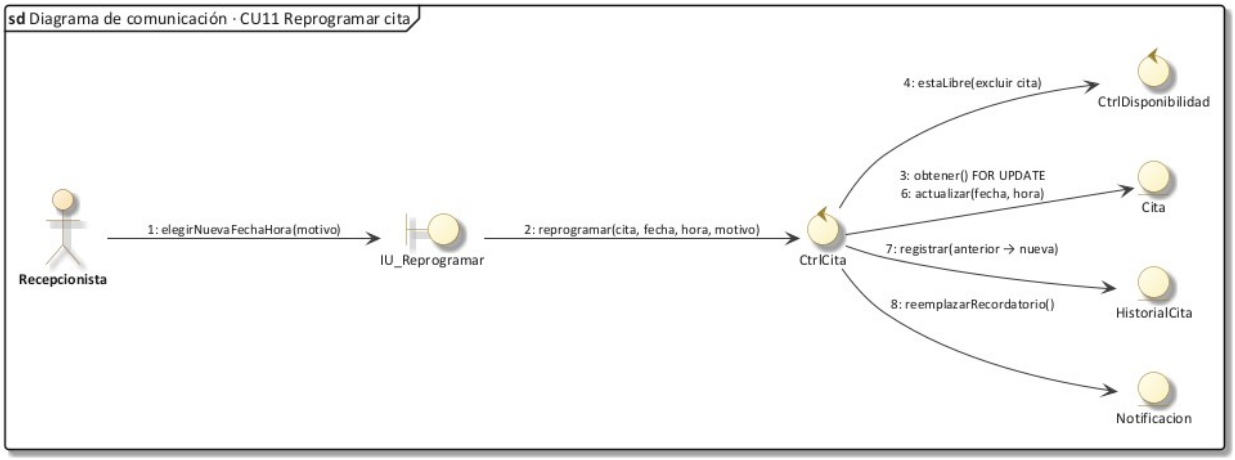
Diagrama de comunicación de CU10 Reservar ficha



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 84

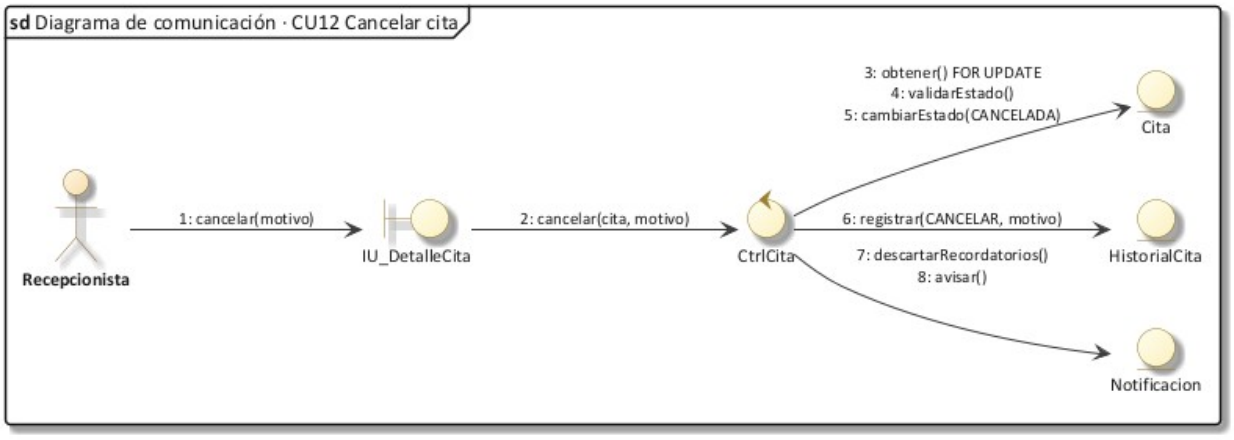
Diagrama de comunicación de CU11 Reprogramar cita



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 85

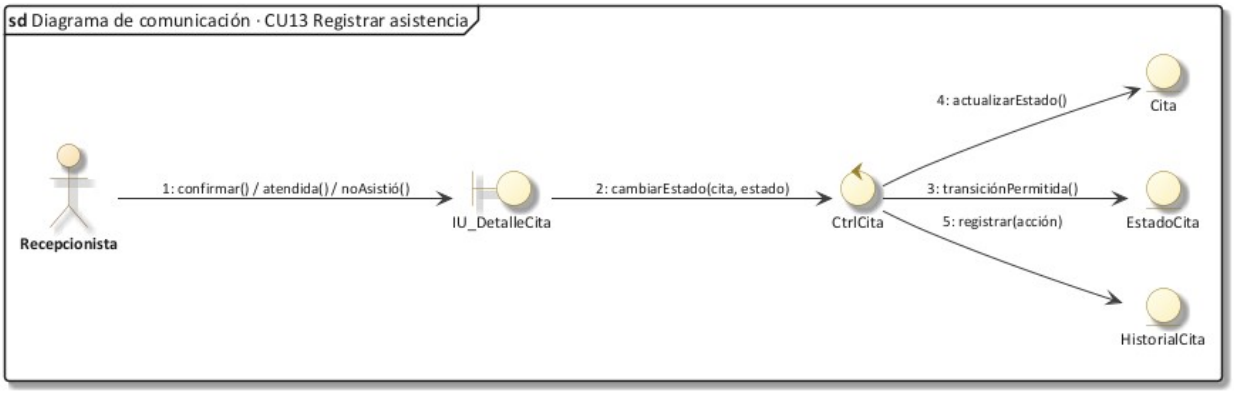
Diagrama de comunicación de CU12 Cancelar cita



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 86

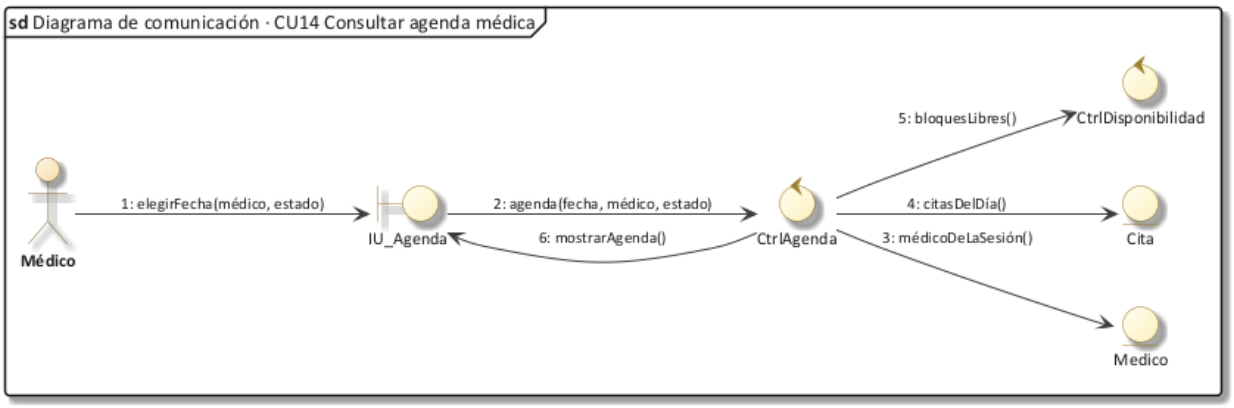
Diagrama de comunicación de CU13 Registrar asistencia



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 87

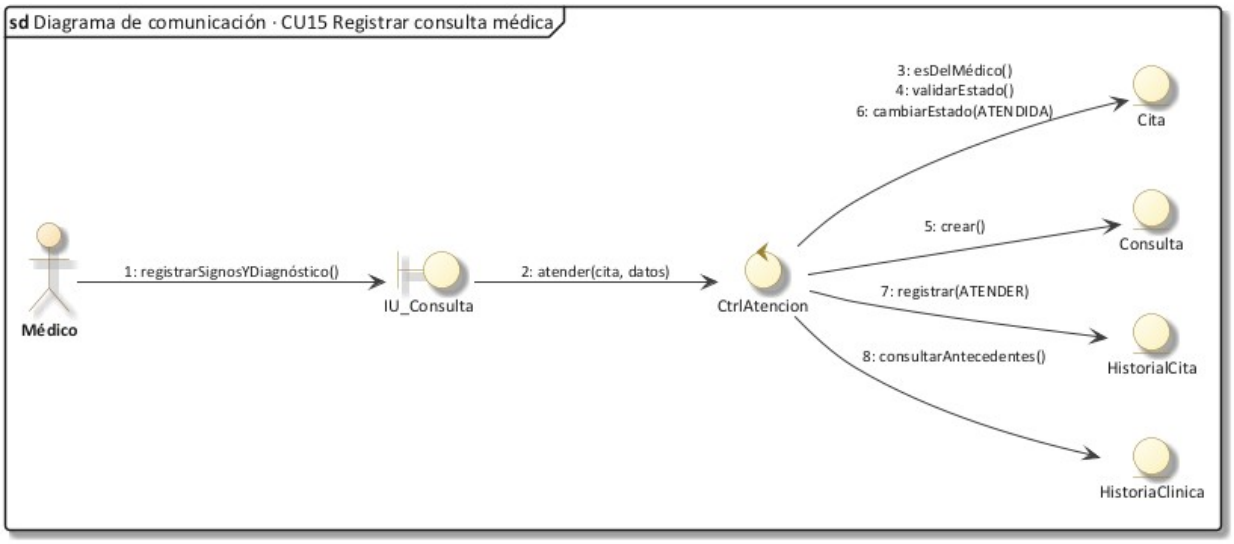
Diagrama de comunicación de CU14 Consultar agenda médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 88

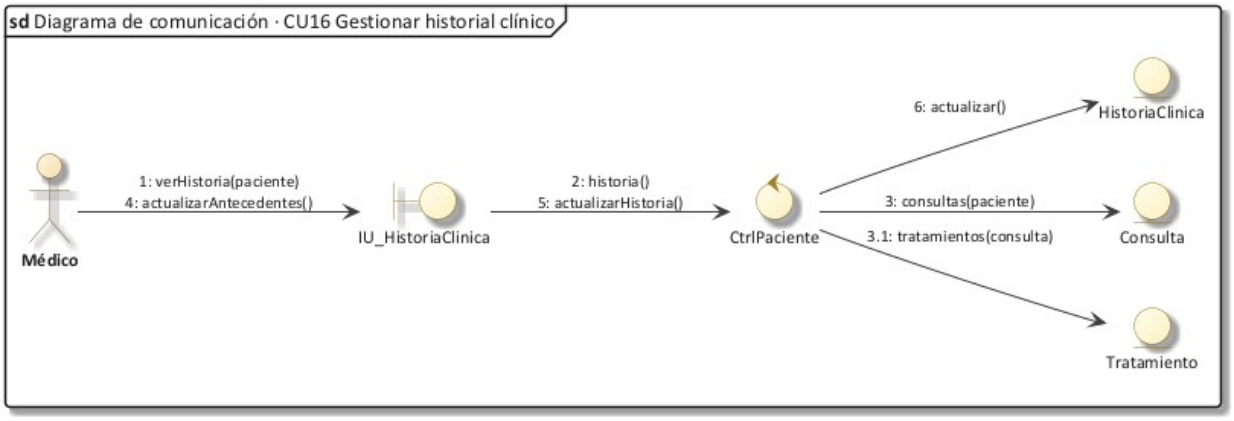
Diagrama de comunicación de CU15 Registrar consulta médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 89

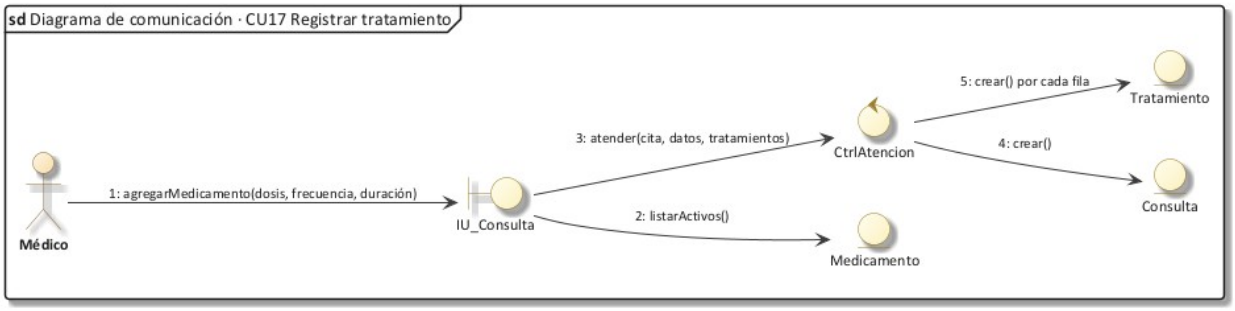
Diagrama de comunicación de CU16 Gestionar historial clínico



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 90

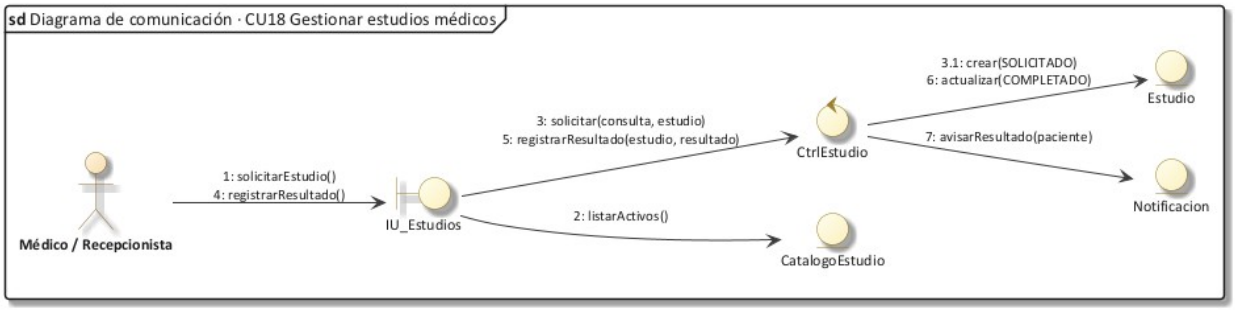
Diagrama de comunicación de CU17 Registrar tratamiento



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 91

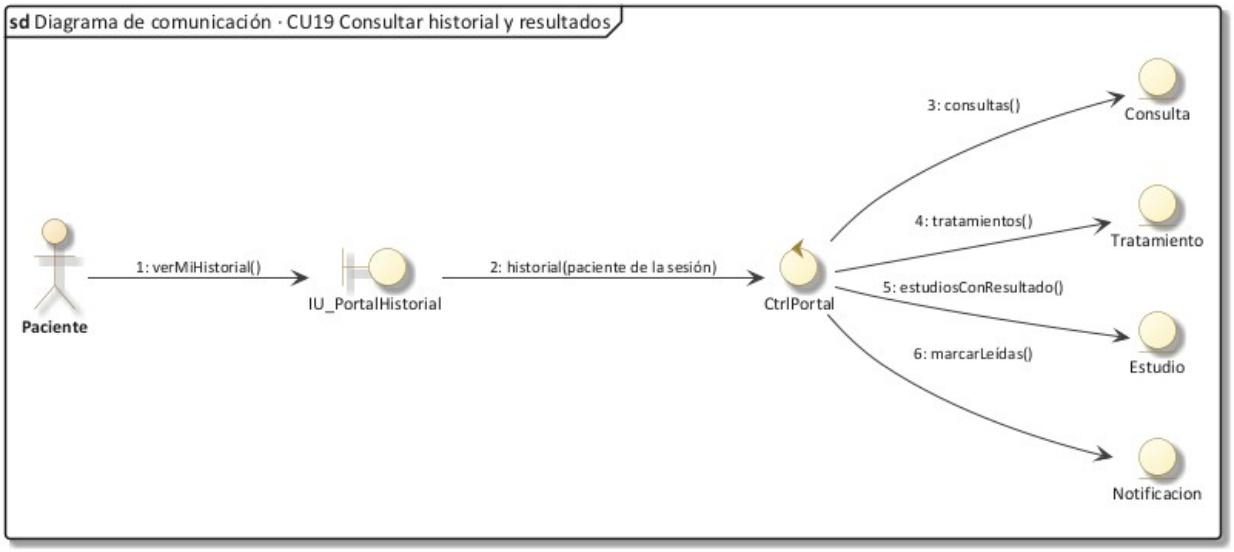
Diagrama de comunicación de CU18 Gestionar estudios médicos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 92

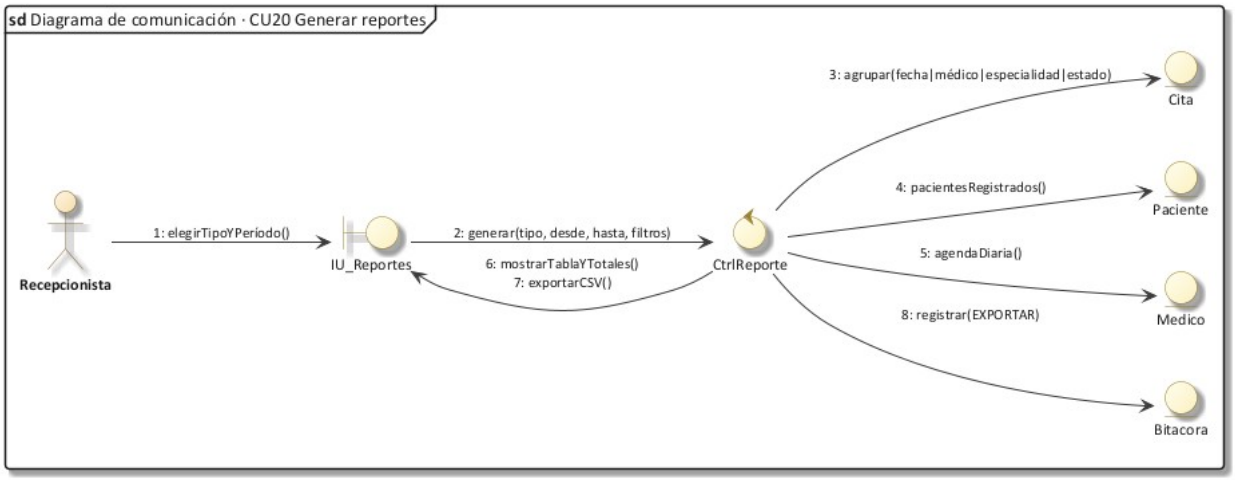
Diagrama de comunicación de CU19 Consultar historial y resultados



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 93

Diagrama de comunicación de CU20 Generar reportes



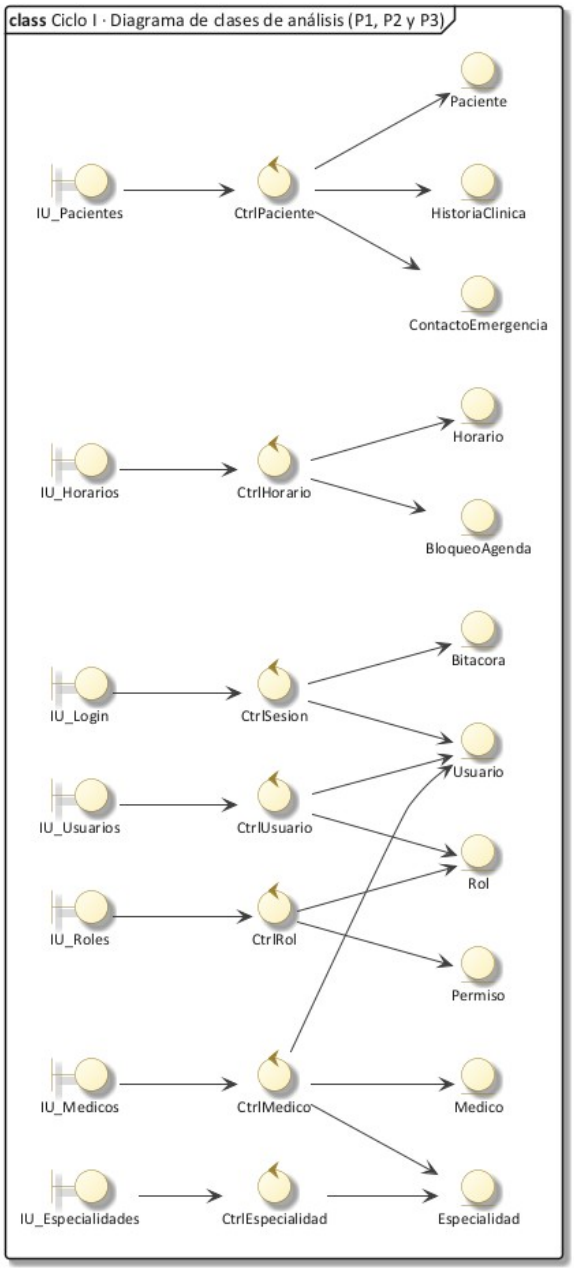
Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

3.3 Análisis de clase

Las clases de interfaz representan las pantallas, las de control coordinan cada caso de uso y las de entidad representan la información persistente.

Figura 94

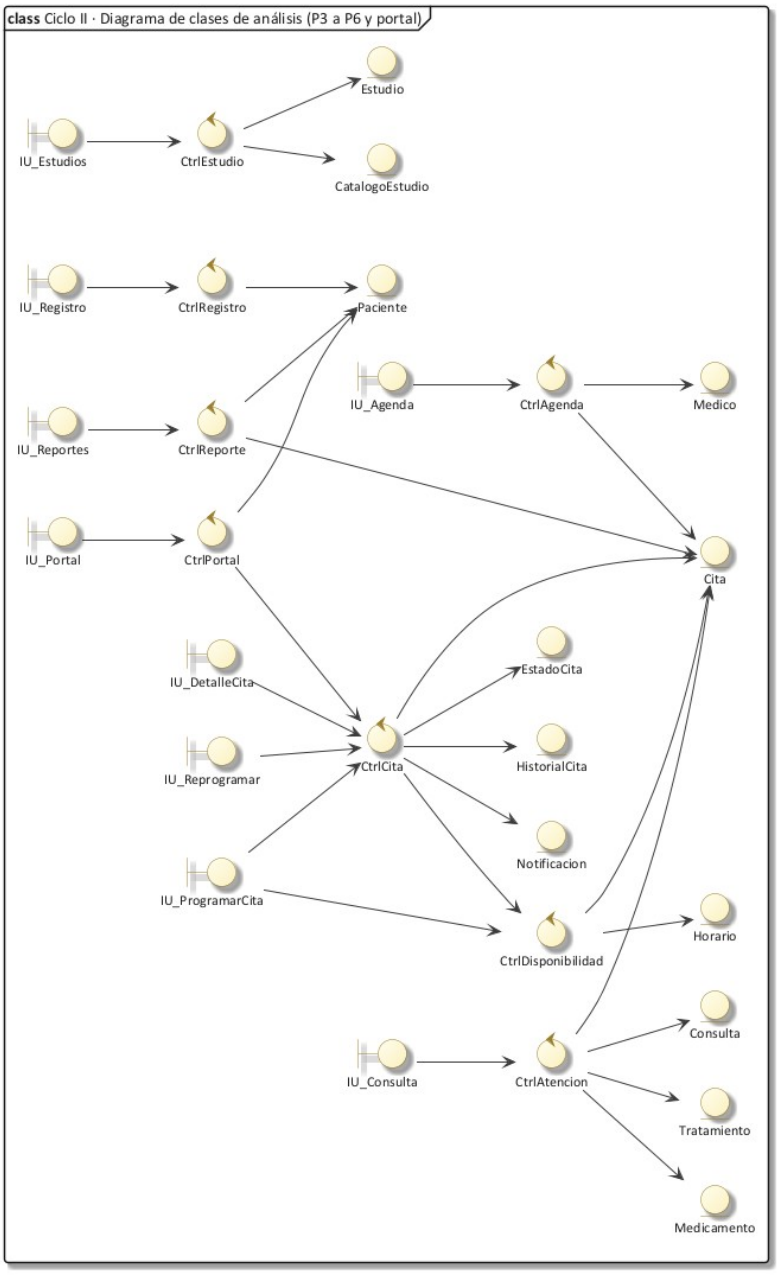
Diagrama de clases de análisis del Ciclo I



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 95

Diagrama de clases de análisis del Ciclo II

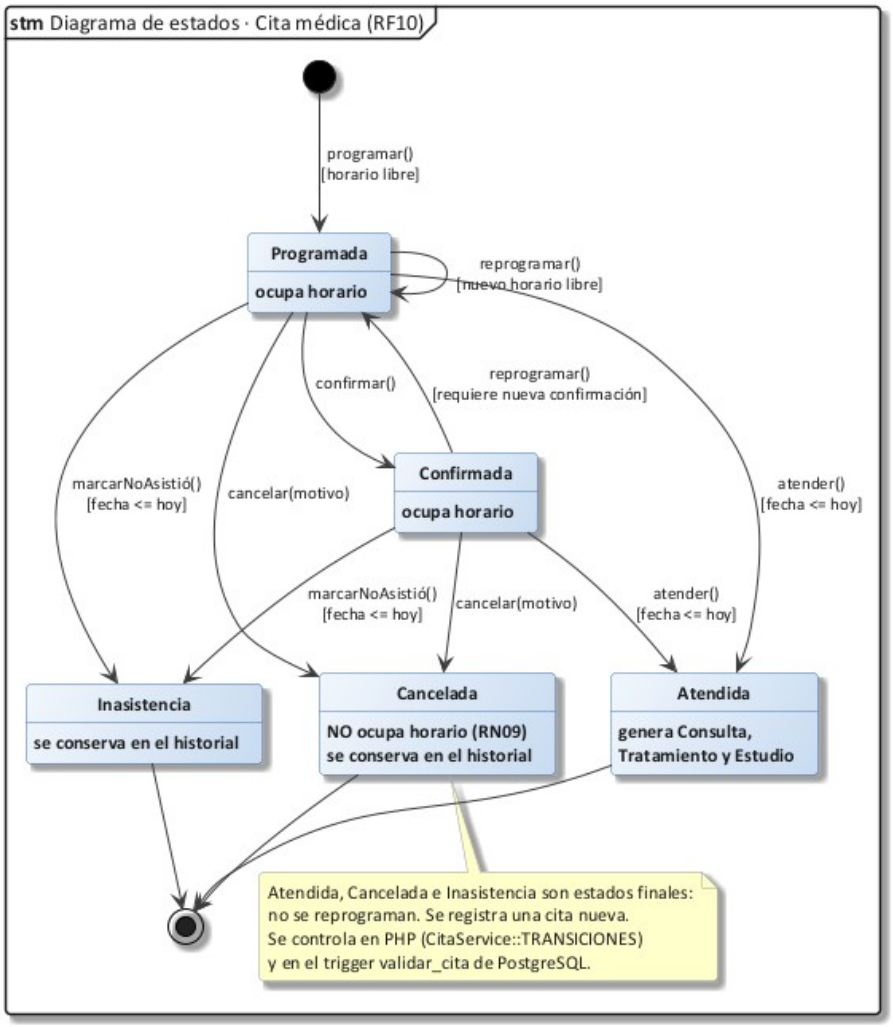


Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

La entidad **Cita** tiene un ciclo de vida con estados y transiciones controladas (RF10):

Figura 96

Diagrama de estados de la cita médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

3.4 Análisis de paquetes

Tabla 58

Contenido de los paquetes de análisis

Paquete	Interfaz	Control	Entidad	Ciclo
P1 Seguridad y acceso	IU_Login, IU_Usuarios, IU_Roles	CtrlSesion, CtrlUsuario, CtrlRol	Usuario, Rol, Permiso, Bitacora	I

Paquete	Interfaz	Control	Entidad	Ciclo
P2 Gestión médica	IU_Especialidades, IU_Medicos, IU_Horarios	CtrlEspecialidad, CtrlMedico, CtrlHorario	Especialidad, Medico, Horario, BloqueoAgenda	I
P3 Gestión de pacientes	IU_Pacientes, IU_Registro	CtrlPaciente, CtrlRegistro	Paciente, HistoriaClinica, ContactoEmergencia	I y II
P4 Gestión de citas	IU_ProgramarCita, IU_DetalleCita, IU_Agenda	CtrlCita, CtrlDisponibilidad, CtrlAgenda	Cita, EstadoCita, HistorialCita, Notificacion	II
P5 Atención médica	IU_Consulta, IU_Estudios, IU_Portal	CtrlAtencion, CtrlEstudio, CtrlPortal	Consulta, Tratamiento, Estudio	II
P6 Reportes	IU_Reportes	CtrlReporte	Cita, Paciente, Medico	II

Las dependencias entre paquetes van de los paquetes del negocio hacia P1, y de P4, P5 y P6 hacia los datos maestros de P2 y P3; no existen dependencias circulares, lo que permitió construir el Ciclo I antes que el Ciclo II.

Capítulo 4. Flujo de trabajo: diseño

4.1 Diseño de arquitectura

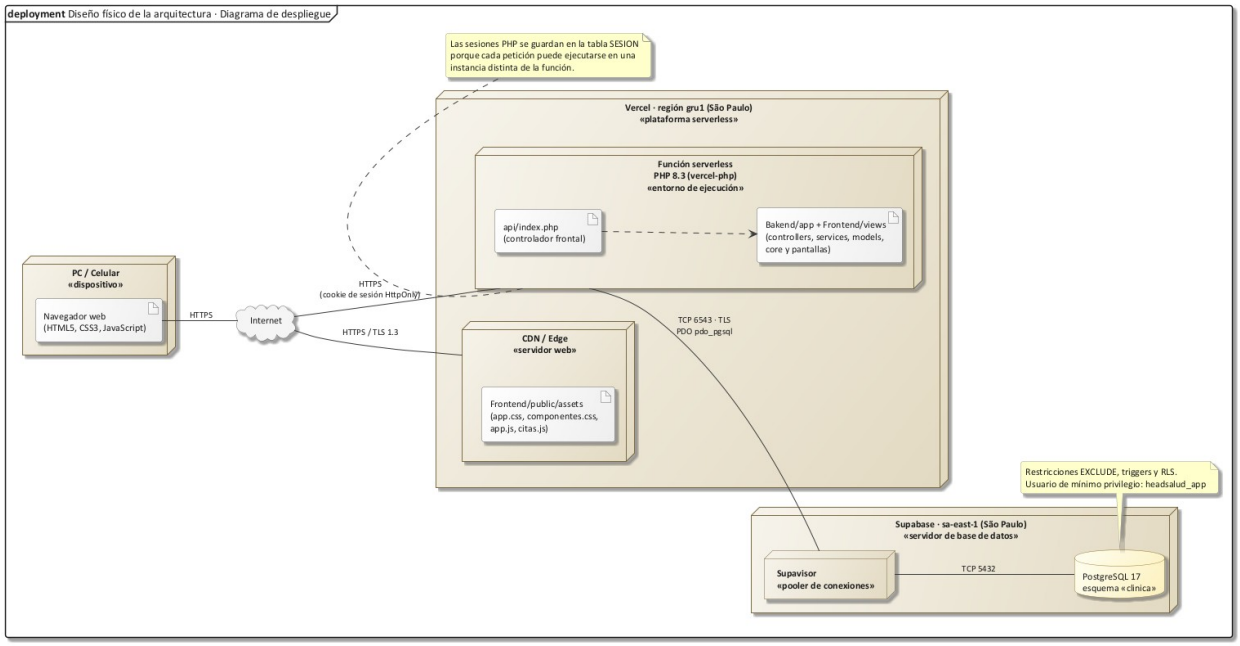
Se eligió una arquitectura **cliente-servidor en capas** basada en MVC: presentación (vistas y recursos estáticos), control (controlador frontal y controladores), lógica de negocio (servicios), acceso a datos (modelos) y un núcleo propio (enrutador, autenticación, sesiones, CSRF, validación y conexión a la base de datos).

4.1.1 Diseño físico de la arquitectura (diagrama de despliegue)

El navegador se comunica por HTTPS con Vercel, que sirve los archivos estáticos desde su CDN y ejecuta PHP 8.3 como función serverless. La función se conecta por TLS, mediante PDO, al pooler de Supabase, que da acceso a PostgreSQL 17 en la misma región.

Figura 97

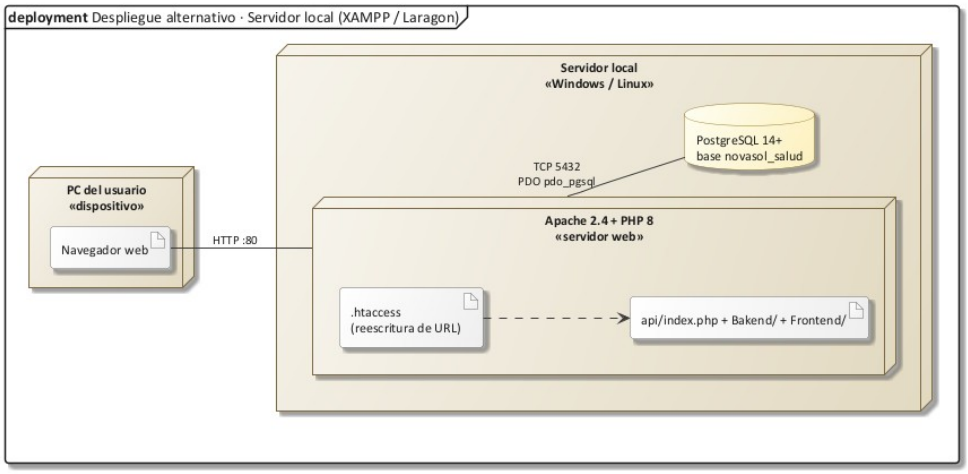
Diagrama de despliegue en producción



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 98

Diagrama de despliegue alternativo en un servidor local

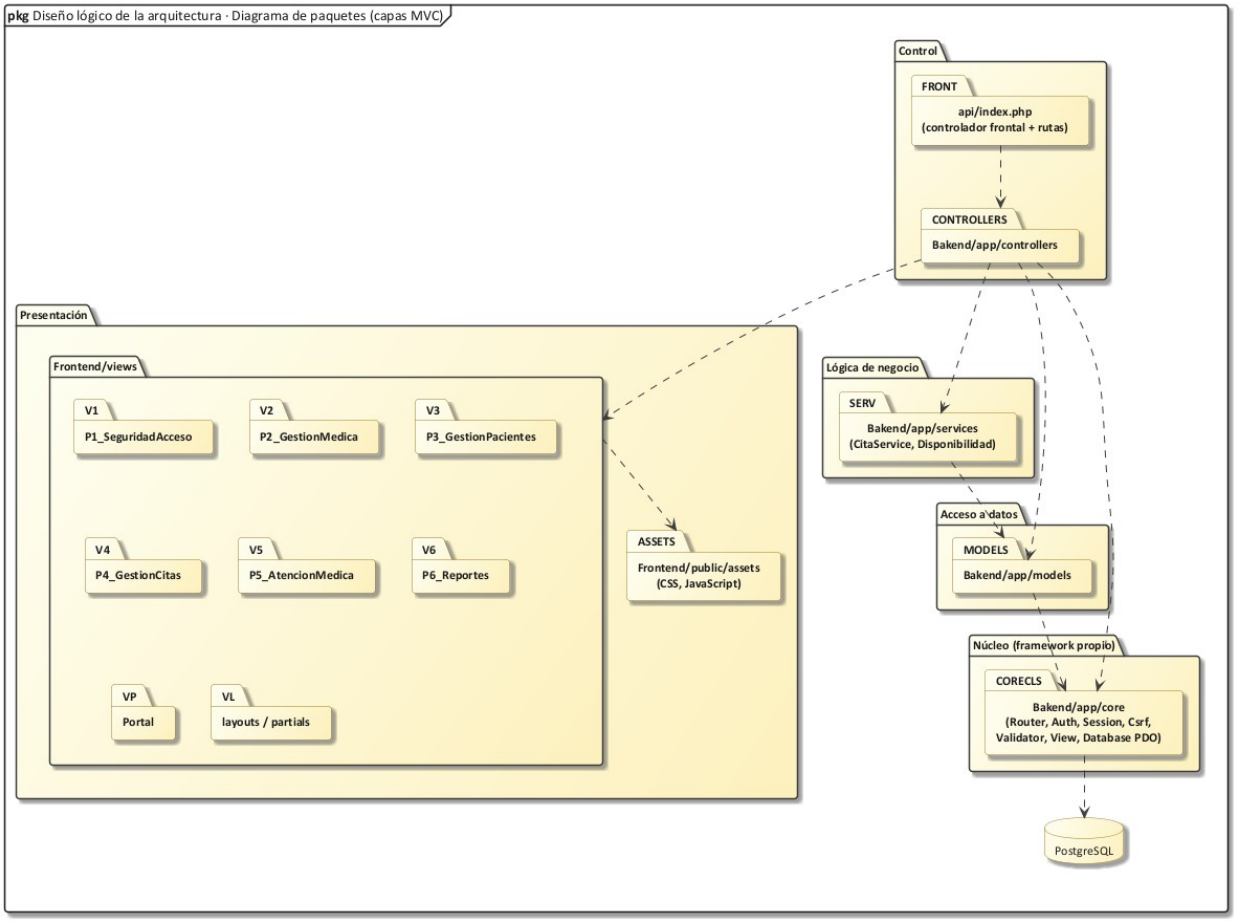


Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

4.1.2 Diseño lógico de la arquitectura (diagrama organizado en capas)

Figura 99

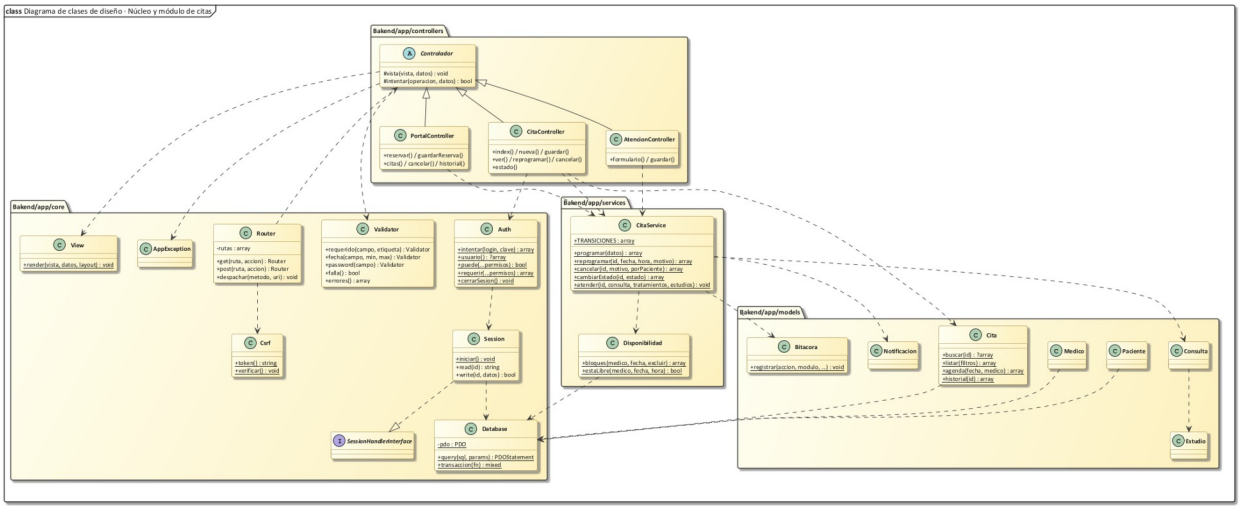
Diagrama de paquetes de diseño organizado en capas



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 100

Diagrama de clases de diseño del núcleo y del módulo de citas



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Capítulo 5. Diseño de datos

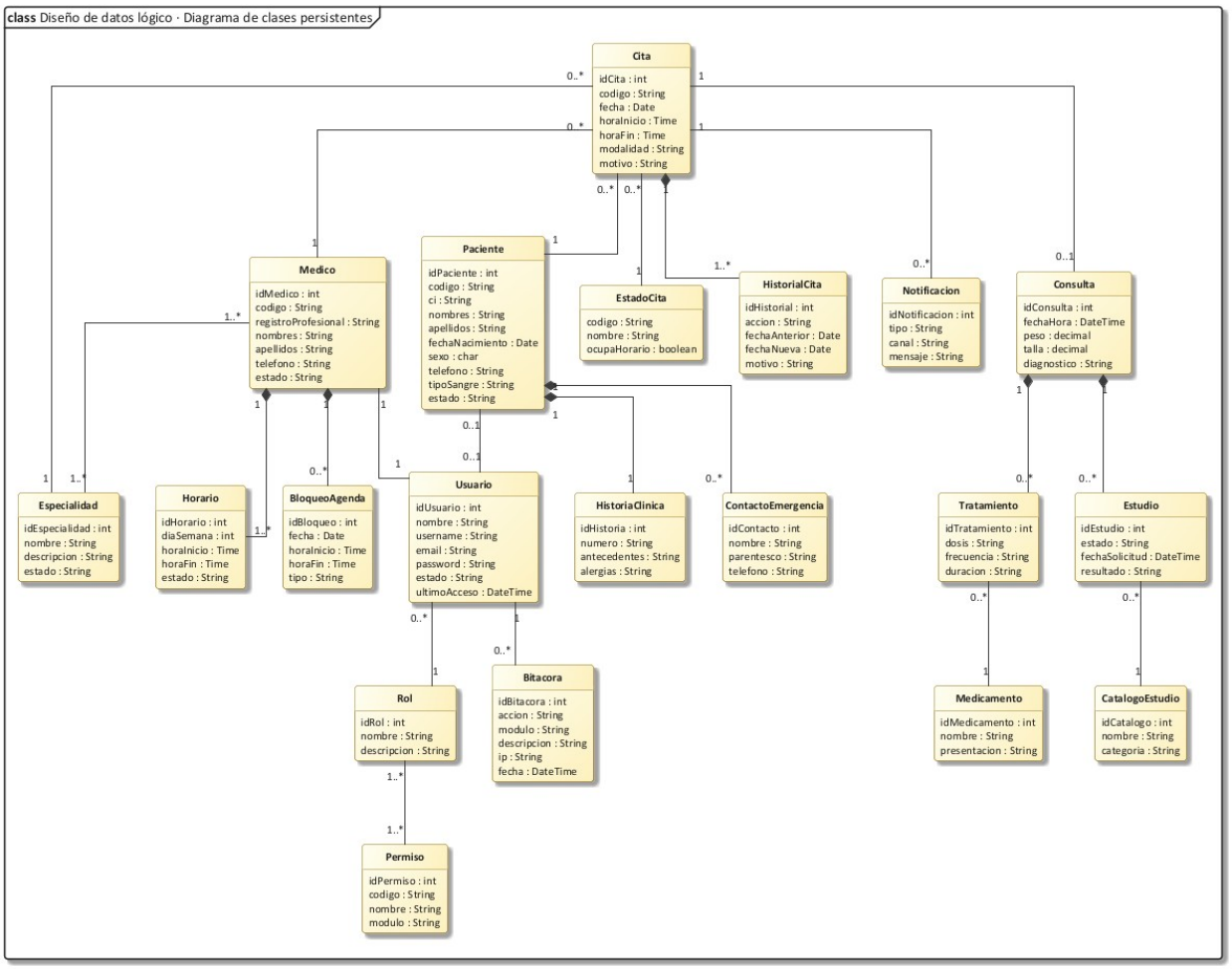
5.1 Diseño de datos lógico

Diagrama de clase

El diseño lógico parte del diagrama de clases persistentes, que contiene las clases cuya información se guarda en la base de datos, con sus atributos, tipos y multiplicidades.

Figura 101

Diagrama de clases persistentes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Mapeo

Cada clase persistente se transforma en una tabla. Las asociaciones de uno a muchos se representan con una clave foránea en la tabla del lado «muchos»; las de muchos a muchos con una tabla intermedia; y las composiciones con una clave foránea con eliminación en cascada.

Tabla 59

Mapeo de clases a tablas

Clase	Tabla	Clave primaria	Claves foráneas / regla de mapeo
Rol	rol	idrol	—
Permiso	permiso	idpermiso	—
Rol–Permiso (muchos a muchos)	rolpermiso	idrol, idpermiso	Tabla intermedia: idrol → rol, idpermiso → permiso
Usuario	usuario	idusuario	idrol → rol
Bitacora	bitacora	idbitacora	idusuario → usuario
Especialidad	especialidad	idespecialidad	—
Medico	medico	idmedico	idusuario → usuario (1 a 1, UNIQUE)
Medico–Especialidad (muchos a muchos)	medicoespecialidad	idmedico, idespecialidad	Tabla intermedia
Horario	horario	idhorario	idmedico → medico (composición, CASCADE)
BloqueoAgenda	bloqueoagenda	idbloqueo	idmedico → medico (composición, CASCADE)
Paciente	paciente	idpaciente	idusuario → usuario (opcional, UNIQUE)
HistoriaClinica	historiaclinica	idhistoria	idpaciente → paciente (1 a 1, UNIQUE, CASCADE)
ContactoEmergencia	contactoemergencia	idcontacto	idpaciente → paciente (CASCADE)
EstadoCita	estadocita	codigo	—
Cita	cita	idcita	idpaciente, idmedico, idespecialidad, codigoestado, idusuarioregistro
HistorialCita	historialcita	idhistorial	idcita → cita (CASCADE), codigoestado, idusuario
Notificacion	notificacion	idnotificacion	idcita, idestudio

Clase	Tabla	Clave primaria	Claves foráneas / regla de mapeo
Consulta	consulta	idconsulta	idcita → cita (1 a 1, UNIQUE)
Medicamento	medicamento	idmedicamento	—
Tratamiento	tratamiento	idtratamiento	idconsulta (CASCADE), idmedicamento
CatalogoEstudio	catalogoestudio	idcatalogo	—
Estudio	estudio	idestudio	idconsulta (CASCADE), idcatalogo
(soporte) Configuracion, Recuperacion, Sesion	configuracion, recuperacion, sesion	clave, idrecuperacion, id	Recuperacion: idusuario → usuario

Normalización

El modelo se normalizó hasta la tercera forma normal (Elmasri y Navathe, 2007):

- **Primera forma normal (1FN):** todos los atributos son atómicos. Por ejemplo, las especialidades de un médico no se guardan como una lista en la tabla MEDICO, sino en la tabla MEDICOESPECIALIDAD; los contactos de emergencia y los tratamientos están en sus propias tablas.
- **Segunda forma normal (2FN):** cada atributo depende de toda la clave primaria. En las tablas con clave compuesta (ROLPERMISO y MEDICOESPECIALIDAD) no hay atributos adicionales, y el resto de las tablas usa una clave simple.
- **Tercera forma normal (3FN):** no existen dependencias transitivas. El nombre del estado de una cita no se repite en CITA, sino que está en ESTADOCITA; el nombre de la especialidad, del medicamento y del estudio están en sus catálogos; los datos del paciente no se copian en la cita.

Tabla 60

Ejemplos de normalización aplicados

Forma	Antes (tabla sin normalizar)	Después
1FN	MEDICO(codigo, nombres, especialidades = "Cardiología, Medicina General")	MEDICO(idmedico, …) + MEDICOESPECIALIDAD(idmedico, idespecialidad)
2FN	CITA_PACIENTE(idcita, idpaciente, fecha, ci, nombres)	CITA(idcita, idpaciente, fecha) + PACIENTE(idpaciente, ci, nombres)
3FN	CITA(idcita, codigoestado, nombreestado, ocupahorario)	CITA(idcita, codigoestado) + ESTADOCITA(codigo, nombre, ocupahorario)

5.2 Diseño de datos físico

Tabla de volumen

La tabla de volumen estima el crecimiento de cada tabla. Se supone una clínica con 4 médicos que atienden unas 16 citas diarias cada uno durante 22 días al mes (alrededor de 17 000 citas al año). Los registros actuales corresponden a la base de datos publicada.

Tabla 61

Tabla de volumen de la base de datos

Tabla	Columnas	Registros actuales	Crecimiento anual	Registros a 5 años
bitacora	9	168	40.000	200.168
bloqueoagenda	7	1	60	301
catalogoestudio	5	7	10	57
cita	13	249	17.000	85.249
configuracion	3	5	Fijo	5
consulta	12	119	12.000	60.119
contactoemergencia	5	2	1.500	7.502
especialidad	4	6	2	16

Tabla	Columnas	Registros actuales	Crecimiento anual	Registros a 5 años
estadocita	4	5	Fijo	5
estudio	9	31	3.500	17.531
historiaclinica	8	9	2.500	12.509
historialcita	11	446	30.000	150.446
horario	6	19	10	69
medicamento	4	8	30	158
medico	9	4	2	14
medicoespecialidad	2	6	3	21
notificacion	9	112	17.000	85.112
paciente	14	9	2.500	12.509
permiso	4	21	2	31
recuperacion	8	2	300	1.502
rol	3	4	Fijo	4
rolpermiso	2	33	5	58
sesion	3	40	Fijo	40
tratamiento	7	59	9.000	45.059
usuario	8	11	1.500	7.511

Nota. Las sesiones se eliminan al expirar, por lo que su volumen se mantiene estable.

Script

La base de datos se crea con los scripts de la carpeta `Bakend/database` del repositorio: `01_esquema.sql` (tablas y restricciones), `02_reglas_negocio.sql` (triggers), `03_datos_iniciales.sql` (datos de prueba), `04_usuario_app.sql` (usuario con privilegios mínimos), `05_consultas.sql` (consultas) y `06_procedimientos_triggers.sql` (procedimientos almacenados y disparadores). A continuación se muestran las tablas principales; el script completo de las tablas se incluye en el Anexo F.

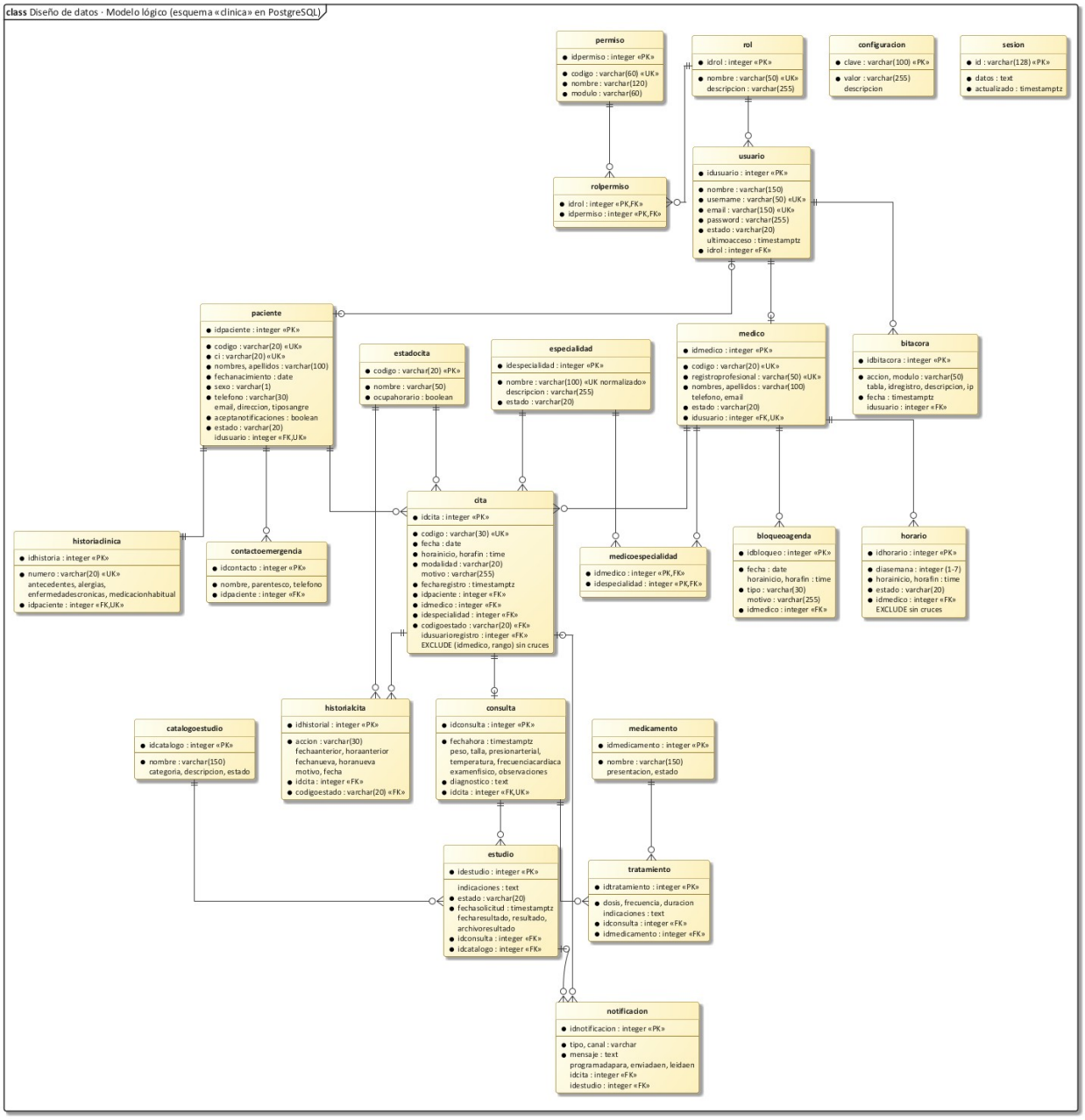
```
create table paciente (
    idpaciente            integer generated by default as identity primary key,
    codigo                varchar(20)  not null unique,
    ci                    varchar(20)  not null unique,        -- RF02: CI único
    nombres               varchar(100) not null,
    apellidos             varchar(100) not null,
    fechanacimiento       date not null check (fechanacimiento > date '1900-01-01'),
    sexo                  varchar(1) not null check (sexo in ('M', 'F', 'O')),
    telefono              varchar(30)  not null,
    email                 varchar(150),
    direccion             varchar(255),
    tiposangre            varchar(5) check (tiposangre in ('A+', 'A-', 'B+', 'B-', 'AB+',
'AB-', 'O+', 'O-')),
    aceptanotificaciones  boolean not null default true,
    estado                varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO')),
    idusuario             integer unique references usuario(idusuario)   -- cuenta del portal
(opcional)
);
create index paciente_nombre_idx on paciente (lower(apellidos), lower(nombres));
```

```
create table cita (
    idcita             integer generated by default as identity primary key,
    codigo             varchar(30) not null unique
                       default 'CM-' || lpad(nextval('cita_codigo_seq')::text, 4, '0'),
    fecha              date not null,
    horainicio         time not null,
    horafin            time not null,
    modalidad          varchar(20) not null default 'PRESENCIAL' check (modalidad in
('PRESENCIAL', 'VIRTUAL')),
    motivo             varchar(255),
    fecharegistro      timestamptz not null default now(),
    idpaciente         integer not null references paciente(idpaciente),        -- RN03 / RN11
    idmedico           integer not null references medico(idmedico),            -- RN03 / RN11
    idespecialidad     integer not null references especialidad(idespecialidad),
    codigoestado       varchar(20) not null default 'PROGRAMADA' references estadocita(codigo),
    idusuarioregistro  integer references usuario(idusuario),
    constraint cita_rango_ck check (horafin > horainicio),
    -- RF07 / RN07: un médico no puede tener citas superpuestas (las canceladas no ocupan
horario)
    constraint cita_sin_cruce_medico exclude using gist (
        idmedico with =, tsrange(fecha + horainicio, fecha + horafin) with &&
    ) where (codigoestado <> 'CANCELADA'),
    -- Un paciente tampoco puede estar en dos citas a la vez
    constraint cita_sin_cruce_paciente exclude using gist (
        idpaciente with =, tsrange(fecha + horainicio, fecha + horafin) with &&
    ) where (codigoestado <> 'CANCELADA')
);
create index cita_idpaciente_idx on cita(idpaciente);
create index cita_idmedico_fecha_idx on cita(idmedico, fecha);
create index cita_fecha_idx on cita(fecha);
create index cita_idespecialidad_idx on cita(idespecialidad);
create index cita_codigoestado_idx on cita(codigoestado);
create index cita_idusuarioregistro_idx on cita(idusuarioregistro);
```

Diagrama relacional

Figura 102

Diagrama relacional de la base de datos (esquema clinica)



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Actualización de tuplas (población de datos)

Los siguientes ejemplos se ejecutaron sobre una copia local de la base de datos, cargada con los scripts 01 a 03, dentro de una transacción que luego se revirtió.

Insert()

```
insert into paciente (codigo, ci, nombres, apellidos, fechanacimiento, sexo, telefono, email)
values ('PAC-0100', ' 9988776sc ', '  Rosa   Elena ', ' Justiniano   Vaca ', '1990-05-14', 'F',
        '76543210', ' Rosa.Justiniano@Correo.COM ')
returning idpaciente, codigo, ci, nombres, apellidos, email;
```

idpaciente	codigo	ci	nombres	apellidos	email
9	PAC-0100	9988776SC	Rosa Elena	Justiniano Vaca	rosa.justiniano@correo.com

Nota. El disparador `paciente_normalizar` quitó los espacios sobrantes, pasó el CI a mayúsculas y el correo a minúsculas.

```
insert into especialidad (nombre, descripcion) values ('Neurología', 'Sistema nervioso')
returning idespecialidad, nombre, estado;
```

idespecialidad	nombre	estado
7	Neurología	ACTIVO

```
insert into especialidad (nombre) values ('  neurologia ');   -- mismo nombre normalizado
```

Resultado: Rechazado: duplicate key value violates unique constraint "especialidad_nombre_uk".

```
insert into horario (diasemana, horainicio, horafin, idmedico) values (6, '08:00', '12:00', 1);
insert into horario (diasemana, horainicio, horafin, idmedico) values (6, '11:00', '13:00', 1);
-- se cruza
```

Resultado: primer horario: Correcto (1 fila); segundo horario: Rechazado: conflicting key value violates exclusion constraint "horario_sin_cruce".

Update()

```
update paciente set telefono = '71122334', direccion = 'Av. Banzer, 4.º anillo'
where codigo = 'PAC-0100' returning codigo, telefono, direccion;
```

codigo	telefono	direccion
PAC-0100	71122334	Av. Banzer, 4.º anillo

```
update configuracion set valor = '20' where clave = 'DURACION_CITA_MINUTOS' returning clave,
valor;
```

clave	valor
DURACION_CITA_MINUTOS	20

```
update cita set codigoestado = 'CONFIRMADA' where codigo = 'CM-0166' returning codigo,
codigoestado;
```

codigo	codigoestado
CM-0166	CONFIRMADA

Delete()

```
delete from paciente where codigo = 'PAC-0100';
delete from horario where diasemana = 6 and idmedico = 1 and horainicio = '08:00';
delete from especialidad where nombre = 'Neurología';
delete from medico where idmedico = 1;   -- tiene citas registradas
```

Sentencia	Resultado
delete from paciente (PAC-0100)	Correcto (1 fila)
delete from horario	Correcto (1 fila)
delete from especialidad (Neurología)	Correcto (1 fila)
delete from medico (con citas)	Rechazado: update or delete on table "medico" violates foreign key constraint "cita_idmedico_fkey" on table "cita"

Nota. RN11: un registro con historial no se elimina; el médico o el paciente se desactiva.

Consultas SQL

Se elaboraron 30 consultas sobre la base de datos (archivo `05_consultas.sql`): diez consultas simples, diez consultas de varias tablas y diez subconsultas. Todas se ejecutaron también sobre la base de datos publicada; los resultados que se muestran (las primeras filas) provienen de la copia local cargada con los datos iniciales del script 03.

Consultas simples

C01. Pacientes activos ordenados por apellido

```
select codigo, ci, nombres, apellidos, telefono
from paciente
where estado = 'ACTIVO'
order by apellidos, nombres;
```

codigo	ci	nombres	apellidos	telefono
PAC-0007	8901234	Sofía	Aguilera Paz	77890123
PAC-0008	9012345	Diego	Coca Vaca	78901234
PAC-0004	5678901	Luis	Fernández Rojas	74567890
PAC-0003	4567890	Ana	Gutiérrez Flores	73456789

Nota. Se muestran 4 de 8 filas.

C02. Especialidades que atiende la clínica

```
select idespecialidad, nombre, descripcion
from especialidad
where estado = 'ACTIVO'
order by nombre;
```

idespecialidad	nombre	descripcion
5	Cardiología	Corazón y sistema circulatorio
6	Dermatología	Piel, cabello y uñas
3	Ginecología	Salud de la mujer
1	Medicina General	Atención primaria, control y prevención

Nota. Se muestran 4 de 6 filas.

C03. Médicos activos con su registro profesional

```
select codigo, nombres || ' ' || apellidos as medico, registroprofesional, telefono
from medico
where estado = 'ACTIVO'
order by apellidos;
```

codigo	medico	registroprofesional	telefono
MED-001	Carlos Mendoza Ríos	RP-12345	70011223
MED-004	Patricia Molina Justiniano	RP-13579	70022334
MED-003	Roberto Suárez Antelo	RP-24680	70077889
MED-002	Lucía Vargas Peña	RP-67890	70044556

C04. Citas programadas o confirmadas de los próximos 7 días

```
select codigo, fecha, horainicio, horafin, codigoestado
from cita
where fecha between current_date and current_date + 7
  and codigoestado in ('PROGRAMADA', 'CONFIRMADA')
order by fecha, horainicio;
```

codigo	fecha	horainicio	horafin	codigoestado
CM-0153	2026-10-28	08:30	09:00	CONFIRMADA
CM-0159	2026-10-28	10:00	10:30	CONFIRMADA
CM-0154	2026-10-28	10:00	10:30	PROGRAMADA
CM-0155	2026-10-28	11:30	12:00	PROGRAMADA

Nota. Se muestran 4 de 51 filas.

C05. Cantidad de citas por estado

```
select codigoestado, count(*) as total
from cita
group by codigoestado
order by total desc;
```

codigoestado	total
ATENDIDA	117
PROGRAMADA	53

codigoestado	total
CANCELADA	27
CONFIRMADA	26

Nota. Se muestran 4 de 5 filas.

C06. Búsqueda de pacientes por CI, nombre o apellido (RF02)

```
select codigo, ci, nombres, apellidos
from paciente
where ci ilike '%123%' or nombres || ' ' || apellidos ilike '%pérez%';
```

codigo	ci	nombres	apellidos
PAC-0001	1234567	Juan	Pérez Quispe
PAC-0006	7890123	Pedro	Rivas Montaño
PAC-0007	8901234	Sofía	Aguilera Paz
PAC-0008	9012345	Diego	Coca Vaca

C07. Estados de cita que ocupan el horario del médico (RN09)

```
select codigo, nombre
from estadocita
where ocupahorario
order by codigo;
```

codigo	nombre
ATENDIDA	Atendida
CONFIRMADA	Confirmada
INASISTENCIA	Inasistencia
PROGRAMADA	Programada

C08. Citas registradas por mes

```
select to_char(date_trunc('month', fecha), 'YYYY-MM') as mes, count(*) as citas
from cita
group by date_trunc('month', fecha)
order by mes;
```

mes	citas
2026-10	184
2026-11	58

C09. Últimos cinco accesos al sistema

```
select username, nombre, ultimoacceso
from usuario
where ultimoacceso is not null
order by ultimoacceso desc
limit 5;
```

Resultado: la consulta no devuelve filas con los datos actuales.

C10. Horarios activos de los lunes

```
select idmedico, horainicio, horafin
from horario
where diasemana = 1 and estado = 'ACTIVO'
order by horainicio;
```

idmedico	horainicio	horafin
1	08:00	12:00
3	09:00	13:00
2	14:00	18:00

Consultas múltiples

C11. Agenda de la semana con paciente, médico y especialidad (RF11)

```
select c.codigo, c.fecha, c.horainicio,
       p.nombres || ' ' || p.apellidos as paciente,
       m.nombres || ' ' || m.apellidos as medico,
       e.nombre as especialidad, ec.nombre as estado
from cita c
join paciente p      on p.idpaciente = c.idpaciente
join medico m        on m.idmedico = c.idmedico
join especialidad e  on e.idespecialidad = c.idespecialidad
join estadocita ec   on ec.codigo = c.codigoestado
where c.fecha between current_date and current_date + 7
order by c.fecha, c.horainicio;
```

codigo	fecha	horainicio	paciente	medico	especialidad	estado
CM-0153	2026-10-28	08:30	Pedro Rivas Montaño	Carlos Mendoza Ríos	Cardiología	Confirmada
CM-0159	2026-10-28	10:00	Luis Fernández Rojas	Roberto Suárez Antelo	Odontología	Confirmada
CM-0154	2026-10-28	10:00	Ana Gutiérrez Flores	Carlos Mendoza Ríos	Medicina General	Programada
CM-0160	2026-10-28	11:30	Juan Pérez Quispe	Roberto Suárez Antelo	Odontología	Cancelada

Nota. Se muestran 4 de 58 filas.

C12. Médicos con todas sus especialidades

```
select m.codigo, m.nombres || ' ' || m.apellidos as medico,
       string_agg(e.nombre, ', ' order by e.nombre) as especialidades
from medico m
join medicoespecialidad me on me.idmedico = m.idmedico
join especialidad e        on e.idespecialidad = me.idespecialidad
group by m.idmedico, m.codigo, m.nombres, m.apellidos
order by medico;
```

codigo	medico	especialidades
MED-001	Carlos Mendoza Ríos	Cardiología, Medicina General
MED-002	Lucía Vargas Peña	Pediatría
MED-004	Patricia Molina Justiniano	Ginecología, Medicina General
MED-003	Roberto Suárez Antelo	Odontología

C13. Usuarios con su rol

```
select u.username, u.nombre, u.email, r.nombre as rol, u.estado
from usuario u
join rol r on r.idrol = u.idrol
order by r.nombre, u.username;
```

username	nombre	email	rol	estado
admin	Administrador General	admin@vidasalud.com	ADMINISTRADOR	ACTIVO
cmendoza	Dr. Carlos Mendoza	cmendoza@vidasalud.com	MEDICO	ACTIVO
lvargas	Dra. Lucía Vargas	lvargas@vidasalud.com	MEDICO	ACTIVO

username	nombre	email	rol	estado
pmolina	Dra. Patricia Molina	pmolina@vidasalud.com	MEDICO	ACTIVO

Nota. Se muestran 4 de 9 filas.

C14. Cantidad de permisos de cada rol

```
select r.nombre as rol, count(rp.idpermiso) as permisos
from rol r
left join rolpermiso rp on rp.idrol = r.idrol
group by r.idrol, r.nombre
order by permisos desc;
```

rol	permisos
ADMINISTRADOR	18
RECEPCIONISTA	10
MEDICO	4
PACIENTE	1

C15. Citas por médico y estado (RF13)

```
select m.nombres || ' ' || m.apellidos as medico,
       count(*) filter (where c.codigoestado = 'ATENDIDA')     as atendidas,
       count(*) filter (where c.codigoestado = 'CANCELADA')    as canceladas,
       count(*) filter (where c.codigoestado = 'INASISTENCIA') as inasistencias,
       count(c.idcita) as total
from medico m
left join cita c on c.idmedico = m.idmedico
group by m.idmedico, m.nombres, m.apellidos
order by total desc;
```

medico	atendidas	canceladas	inasistencias	total
Roberto Suárez Antelo	36	11	7	77
Lucía Vargas Peña	32	7	4	64
Carlos Mendoza Ríos	30	5	5	63
Patricia Molina Justiniano	19	4	3	38

C16. Consultas médicas con paciente y diagnóstico

```
select c.codigo as cita, co.fechahora, p.nombres || ' ' || p.apellidos as paciente,
co.diagnostico
from consulta co
join cita c     on c.idcita = co.idcita
join paciente p on p.idpaciente = c.idpaciente
order by co.fechahora desc
limit 10;
```

cita	fechahora	paciente	diagnostico
CM-0152	2026-10-27 16:30	Juan Pérez Quispe	Paciente sano
CM-0151	2026-10-27 15:00	Luis Fernández Rojas	Cefalea tensional
CM-0150	2026-10-27 12:00	Sofía Aguilera Paz	Resfrío común
CM-0149	2026-10-27 10:30	María López Rivero	Faringitis aguda

Nota. Se muestran 4 de 10 filas.

C17. Tratamientos indicados con el medicamento y el paciente

```
select p.nombres || ' ' || p.apellidos as paciente, me.nombre as medicamento,
       t.dosis, t.frecuencia, t.duracion
from tratamiento t
join medicamento me on me.idmedicamento = t.idmedicamento
join consulta co    on co.idconsulta = t.idconsulta
join cita c         on c.idcita = co.idcita
join paciente p     on p.idpaciente = c.idpaciente
order by paciente;
```

paciente	medicamento	dosis	frecuencia	duracion
Ana Gutiérrez Flores	Enalapril	1 tableta	Cada 8 horas	5 días
Ana Gutiérrez Flores	Paracetamol	1 tableta	Cada 8 horas	5 días
Ana Gutiérrez Flores	Paracetamol	1 tableta	Cada 8 horas	5 días
Ana Gutiérrez Flores	Amoxicilina	1 tableta	Cada 8 horas	5 días

Nota. Se muestran 4 de 58 filas.

C18. Estudios médicos con su estado y paciente (CU18)

```
select ce.nombre as estudio, ce.categoria, es.estado, es.fechasolicitud,
       p.nombres || ' ' || p.apellidos as paciente
from estudio es
join catalogoestudio ce on ce.idcatalogo = es.idcatalogo
join consulta co        on co.idconsulta = es.idconsulta
join cita c             on c.idcita = co.idcita
join paciente p         on p.idpaciente = c.idpaciente
order by es.fechasolicitud desc;
```

estudio	categoria	estado	fechasolicitud	paciente
Radiografía de tórax	Imagen	SOLICITADO	2026-10-27 16:30	Juan Pérez Quispe
Ecografía abdominal	Imagen	COMPLETADO	2026-10-27 09:00	Carla Méndez Saavedra
Electrocardiograma	Cardiología	COMPLETADO	2026-10-26 12:30	Sofía Aguilera Paz
Hemograma completo	Laboratorio	SOLICITADO	2026-10-26 11:00	Pedro Rivas Montaño

Nota. Se muestran 4 de 30 filas.

C19. Historial de cambios de las citas con el usuario que los hizo

```
select c.codigo, h.accion, ec.nombre as estado, h.motivo, h.fecha, u.username
from historialcita h
join cita c        on c.idcita = h.idcita
join estadocita ec on ec.codigo = h.codigoestado
left join usuario u on u.idusuario = h.idusuario
order by h.fecha desc
limit 15;
```

codigo	accion	estado	motivo	fecha	username
CM-0235	PROGRAMAR	Programada	—	2026-11-05 10:00	mrojas
CM-0238	PROGRAMAR	Programada	—	2026-11-05 10:00	mrojas
CM-0241	PROGRAMAR	Programada	—	2026-11-05 10:00	mrojas
CM-0240	PROGRAMAR	Programada	—	2026-11-05 10:00	mrojas

Nota. Se muestran 4 de 15 filas.

C20. Horario semanal de cada médico con el nombre del día

```
select m.nombres || ' ' || m.apellidos as medico,
       (array['Lunes','Martes','Miércoles','Jueves','Viernes','Sábado','Domingo'])[h.diasemana]
as dia,
       h.horainicio, h.horafin
from horario h
join medico m on m.idmedico = h.idmedico
where h.estado = 'ACTIVO'
order by medico, h.diasemana, h.horainicio;
```

medico	dia	horainicio	horafin
Carlos Mendoza Ríos	Lunes	08:00	12:00
Carlos Mendoza Ríos	Martes	08:00	12:00
Carlos Mendoza Ríos	Miércoles	08:00	12:00
Carlos Mendoza Ríos	Jueves	08:00	12:00

Nota. Se muestran 4 de 19 filas.

Subconsultas

C21. Pacientes que nunca sacaron una cita

```
select codigo, nombres, apellidos
from paciente p
where not exists (select 1 from cita c where c.idpaciente = p.idpaciente);
```

Resultado: la consulta no devuelve filas con los datos actuales.

C22. Médicos con más citas que el promedio de la clínica

```
select m.nombres || ' ' || m.apellidos as medico, count(*) as citas
from cita c
join medico m on m.idmedico = c.idmedico
group by m.idmedico, m.nombres, m.apellidos
having count(*) > (select avg(n) from (select count(*) as n from cita group by idmedico) x);
```

medico	citas
Lucía Vargas Peña	64
Roberto Suárez Antelo	77
Carlos Mendoza Ríos	63

C23. Pacientes con al menos tres citas atendidas

```
select codigo, nombres, apellidos
from paciente
where idpaciente in (
    select idpaciente from cita
    where codigoestado = 'ATENDIDA'
    group by idpaciente
    having count(*) >= 3
);
```

codigo	nombres	apellidos
PAC-0001	Juan	Pérez Quispe
PAC-0002	María	López Rivero
PAC-0003	Ana	Gutiérrez Flores

codigo	nombres	apellidos
PAC-0004	Luis	Fernández Rojas

Nota. Se muestran 4 de 8 filas.

C24. Especialidad más solicitada

```
select e.nombre, x.total
from (select idespecialidad, count(*) as total from cita group by idespecialidad) x
join especialidad e on e.idespecialidad = x.idespecialidad
where x.total = (select max(n) from (select count(*) as n from cita group by idespecialidad)
y);
```

nombre	total
Odontología	77

C25. Última cita de cada paciente (subconsulta correlacionada)

```
select p.nombres || ' ' || p.apellidos as paciente, c.codigo, c.fecha, c.codigoestado
from cita c
join paciente p on p.idpaciente = c.idpaciente
where c.fecha + c.horainicio = (
    select max(c2.fecha + c2.horainicio) from cita c2 where c2.idpaciente = c.idpaciente
);
```

paciente	codigo	fecha	codigoestado
Juan Pérez Quispe	CM-0240	2026-11-09	CANCELADA
María López Rivero	CM-0237	2026-11-09	CONFIRMADA
Ana Gutiérrez Flores	CM-0242	2026-11-09	PROGRAMADA
Luis Fernández Rojas	CM-0239	2026-11-09	PROGRAMADA

Nota. Se muestran 4 de 8 filas.

C26. Médicos activos que no atienden los sábados

```
select codigo, nombres, apellidos
from medico m
where estado = 'ACTIVO'
  and m.idmedico not in (select idmedico from horario where diasemana = 6 and estado =
'ACTIVO');
```

codigo	nombres	apellidos
MED-001	Carlos	Mendoza Ríos
MED-002	Lucía	Vargas Peña

C27. Porcentaje de asistencia de cada médico (citas ya cerradas)

```
select m.nombres || ' ' || m.apellidos as medico,
       (select count(*) from cita c where c.idmedico = m.idmedico and c.codigoestado =
'ATENDIDA') as atendidas,
       (select count(*) from cita c where c.idmedico = m.idmedico and c.codigoestado in
('ATENDIDA', 'INASISTENCIA')) as cerradas,
       round(100.0 * (select count(*) from cita c where c.idmedico = m.idmedico and
c.codigoestado = 'ATENDIDA')
             / nullif((select count(*) from cita c where c.idmedico = m.idmedico and
c.codigoestado in ('ATENDIDA', 'INASISTENCIA')), 0), 1) as asistencia_pct
from medico m
order by asistencia_pct desc nulls last;
```

medico	atendidas	cerradas	asistencia_pct
Lucía Vargas Peña	32	36	88.9
Patricia Molina Justiniano	19	22	86.4
Carlos Mendoza Ríos	30	35	85.7
Roberto Suárez Antelo	36	43	83.7

C28. Medicamentos que nunca se recetaron

```
select nombre, presentacion
from medicamento
where idmedicamento not in (select idmedicamento from tratamiento);
```

nombre	presentacion
Ibuprofeno	Tabletas 400 mg
Omeprazol	Cápsulas 20 mg
Salbutamol	Inhalador 100 mcg
Metformina	Tabletas 850 mg

C29. Pacientes con estudios todavía pendientes de resultado

```
select codigo, nombres, apellidos
from paciente p
```

```
where exists (
    select 1 from estudio es
    join consulta co on co.idconsulta = es.idconsulta
    join cita c      on c.idcita = co.idcita
    where c.idpaciente = p.idpaciente and es.estado in ('SOLICITADO', 'EN_PROCESO')
);
```

codigo	nombres	apellidos
PAC-0001	Juan	Pérez Quispe
PAC-0003	Ana	Gutiérrez Flores
PAC-0006	Pedro	Rivas Montaño
PAC-0008	Diego	Coca Vaca

C30. Citas de la especialidad con más inasistencias

```
select c.codigo, c.fecha, c.codigoestado
from cita c
where c.idespecialidad = (
    select idespecialidad from cita
    where codigoestado = 'INASISTENCIA'
    group by idespecialidad
    order by count(*) desc
    limit 1
)
order by c.fecha desc
limit 10;
```

codigo	fecha	codigoestado
CM-0240	2026-11-09	CANCELADA
CM-0242	2026-11-09	PROGRAMADA
CM-0241	2026-11-09	PROGRAMADA
CM-0230	2026-11-07	PROGRAMADA

Nota. Se muestran 4 de 10 filas.

Procedimientos almacenados

Se programaron cinco procedimientos y funciones en PL/pgSQL y SQL (archivo `06_procedimientos_triggers.sql`).

PA1. fn_horarios_libres: horarios libres de un médico en una fecha (CU9)

```
create or replace function fn_horarios_libres(p_idmedico integer, p_fecha date)
returns table (horainicio time, horafin time)
language sql stable set search_path = '' as $$
    with cfg as (
        select coalesce((select valor::int from clinica.configuracion where clave =
'DURACION_CITA_MINUTOS'), 30) as minutos
    ), bloques as (
        select (g)::time as ini, (g + make_interval(mins => cfg.minutos))::time as fin
        from clinica.horario h, cfg,
             generate_series(p_fecha + h.horainicio, p_fecha + h.horafin - make_interval(mins
=> cfg.minutos),
                             make_interval(mins => cfg.minutos)) g
        where h.idmedico = p_idmedico and h.estado = 'ACTIVO'
          and h.diasemana = extract(isodow from p_fecha)
    )
    select b.ini, b.fin
    from bloques b
    where p_fecha + b.ini > clinica.ahora_local()
      and not exists (select 1 from clinica.cita c
                      join clinica.estadocita e on e.codigo = c.codigoestado and e.ocupahorario
                      where c.idmedico = p_idmedico and c.fecha = p_fecha
                        and b.ini < c.horafin and b.fin > c.horainicio)
      and not exists (select 1 from clinica.bloqueoagenda k
                      where k.idmedico = p_idmedico and k.fecha = p_fecha
                        and (k.horainicio is null or (b.ini < k.horafin and b.fin >
k.horainicio)))
    order by b.ini
$$;
```

Ejecución:

```
select * from fn_horarios_libres(1, '2026-11-02');
```

horainicio	horafin
08:30	09:00
09:00	09:30
10:00	10:30
10:30	11:00
11:30	12:00

PA2. sp_cancelar_cita: cancelar una cita con su motivo (CU12)

```
create or replace procedure sp_cancelar_cita(p_idcita integer, p_motivo varchar, p_idusuario
integer)
language plpgsql set search_path = '' as $$
begin
```

```
    if coalesce(trim(p_motivo), '') = '' then
        raise exception 'Debe indicar el motivo de la cancelación';
    end if;
    update clinica.cita set codigoestado = 'CANCELADA' where idcita = p_idcita;
    if not found then
        raise exception 'La cita % no existe', p_idcita;
    end if;
    insert into clinica.historialcita (accion, motivo, idcita, codigoestado, idusuario)
    values ('CANCELAR', p_motivo, p_idcita, 'CANCELADA', p_idusuario);
end $$;
```

Ejecución:

```
call sp_cancelar_cita(<id de CM-0161>, 'El paciente viajará', 1);
```

codigo	codigoestado	accion	motivo
CM-0161	CANCELADA	CANCELAR	El paciente viajará

PA3. sp_desactivar_paciente: desactivar un paciente (RN11)

```
create or replace procedure sp_desactivar_paciente(p_idpaciente integer)
language plpgsql set search_path = '' as $$
begin
    if exists (select 1 from clinica.cita
               where idpaciente = p_idpaciente and codigoestado in ('PROGRAMADA', 'CONFIRMADA')
                 and fecha >= clinica.ahora_local()::date) then
        raise exception 'El paciente tiene citas pendientes; cancélelas antes de desactivarlo';
    end if;
    update clinica.paciente set estado = 'INACTIVO' where idpaciente = p_idpaciente;
end $$;
```

Ejecución:

```
call sp_desactivar_paciente(<id de PAC-0003>);
```

PA4. fn_resumen_citas: resumen de citas por estado (CU20)

```
create or replace function fn_resumen_citas(p_desde date, p_hasta date)
returns table (estado varchar, total bigint, porcentaje numeric)
language sql stable set search_path = '' as $$
    select e.nombre, count(c.idcita),
           round(100.0 * count(c.idcita) / nullif(sum(count(c.idcita)) over (), 0), 1)
    from clinica.estadocita e
    left join clinica.cita c on c.codigoestado = e.codigo and c.fecha between p_desde and
p_hasta
    group by e.codigo, e.nombre
```

```
    order by count(c.idcita) desc
$$;
```

Ejecución:

```
select * from fn_resumen_citas(current_date - 30, current_date + 30);
```

estado	total	porcentaje
Atendida	117	48.3
Programada	52	21.5
Cancelada	28	11.6
Confirmada	26	10.7
Inasistencia	19	7.9

PA5. fn_edad_paciente: edad de un paciente

```
create or replace function fn_edad_paciente(p_idpaciente integer)
returns integer
language sql stable set search_path = '' as $$
    select extract(year from age(clinica.ahora_local()::date, fechanacimiento))::int
    from clinica.paciente where idpaciente = p_idpaciente
$$;
```

Ejecución:

```
select codigo, nombres, apellidos, fechanacimiento, fn_edad_paciente(idpaciente) as edad
from paciente where idpaciente in (1, 2, 3);
```

codigo	nombres	apellidos	fechanacimiento	edad
PAC-0001	Juan	Pérez Quispe	1990-05-14	36
PAC-0002	María	López Rivero	1985-11-02	40
PAC-0003	Ana	Gutiérrez Flores	2018-03-21	8

Resultado de PA2 sin motivo: Rechazado: Debe indicar el motivo de la cancelación.

Resultado de PA3 con un paciente que tiene citas pendientes (PAC-0003 Ana Gutiérrez Flores):

Rechazado: El paciente tiene citas pendientes; cancélelas antes de desactivarlo.

Triggers (disparadores)

La aplicación PHP valida los datos antes de guardarlos; los disparadores son la última línea de defensa en la base de datos, por si un dato llega sin pasar por la aplicación.

TR1. cita_validar: reglas de la cita (RN04 a RN08, RF08, RF10)

```
create or replace function validar_cita() returns trigger
language plpgsql set search_path = '' as $$
declare
    v_duracion integer := coalesce((select valor::int from clinica.configuracion where clave =
'DURACION_CITA_MINUTOS'), 30);
    v_cambia_horario boolean := tg_op = 'INSERT'
        or new.fecha is distinct from old.fecha
        or new.horainicio is distinct from old.horainicio
        or new.horafin is distinct from old.horafin
        or new.idmedico is distinct from old.idmedico;
begin
    -- RF08: solo se reprogramen citas vigentes
    if tg_op = 'UPDATE' and v_cambia_horario and old.codigoestado not in ('PROGRAMADA',
'CONFIRMADA') then
        raise exception 'Solo se pueden reprogramar citas programadas o confirmadas';
    end if;

    if v_cambia_horario then
        -- RN04: el médico debe estar activo
        if not exists (select 1 from clinica.medico where idmedico = new.idmedico and estado =
'ACTIVO') then
            raise exception 'El médico no está activo y no puede recibir nuevas citas';
        end if;
        if tg_op = 'INSERT' and not exists (select 1 from clinica.paciente where idpaciente =
new.idpaciente and estado = 'ACTIVO') then
            raise exception 'El paciente está inactivo';
        end if;
        -- El médico debe atender la especialidad elegida
        if not exists (select 1 from clinica.medicoespecialidad
                       where idmedico = new.idmedico and idespecialidad = new.idespecialidad)
then
            raise exception 'El médico no atiende la especialidad seleccionada';
        end if;
        -- RN06: nada en el pasado
        if new.fecha + new.horainicio <= clinica.ahora_local() then
            raise exception 'No se pueden registrar citas en fechas u horas pasadas';
        end if;
        -- RN08: duración configurada
        if new.horafin - new.horainicio <> make_interval(mins => v_duracion) then
            raise exception 'La cita debe durar % minutos', v_duracion;
        end if;
        -- RN05: dentro de un horario de atención y alineada a sus intervalos
        if not exists (
            select 1 from clinica.horario h
            where h.idmedico = new.idmedico and h.estado = 'ACTIVO'
              and h.diasemana = extract(isodow from new.fecha)
              and new.horainicio >= h.horainicio and new.horafin <= h.horafin
              and (extract(epoch from new.horainicio - h.horainicio)::int / 60) % v_duracion =
0
        ) then
```

```
            raise exception 'El horario elegido está fuera de la atención del médico';
        end if;
        -- Bloqueos de agenda (feriados, vacaciones, permisos)
        if exists (
            select 1 from clinica.bloqueoagenda b
            where b.idmedico = new.idmedico and b.fecha = new.fecha
              and (b.horainicio is null or (new.horainicio < b.horafin and new.horafin >
b.horainicio))
        ) then
            raise exception 'El médico tiene la agenda bloqueada en ese horario';
        end if;
    end if;

    -- RF10: transiciones de estado permitidas
    if tg_op = 'UPDATE' and new.codigoestado is distinct from old.codigoestado then
        if not (
            (old.codigoestado = 'PROGRAMADA' and new.codigoestado in ('CONFIRMADA',
'CANCELADA', 'ATENDIDA', 'INASISTENCIA')) or
            (old.codigoestado = 'CONFIRMADA' and new.codigoestado in ('PROGRAMADA',
'CANCELADA', 'ATENDIDA', 'INASISTENCIA'))
        ) then
            raise exception 'No se puede pasar una cita de % a %', old.codigoestado,
new.codigoestado;
        end if;
        if new.codigoestado in ('ATENDIDA', 'INASISTENCIA') and new.fecha >
clinica.ahora_local()::date then
            raise exception 'No se puede registrar la asistencia de una cita futura';
        end if;
    end if;
    return new;
end $$;

create trigger cita_validar before insert or update on cita
    for each row execute function validar_cita();
```

Tabla 62

Pruebas del disparador cita_validar

Prueba	Resultado
Cita en una fecha pasada	Rechazado: No se pueden registrar citas en fechas u horas pasadas
Cita de 45 minutos	Rechazado: La cita debe durar 30 minutos
Cita fuera del horario del médico	Rechazado: El horario elegido está fuera de la atención del médico
Especialidad que el médico no atiende	Rechazado: El médico no atiende la especialidad seleccionada
Cambiar una cita ATENDIDA a PROGRAMADA	Rechazado: No se puede pasar una cita de ATENDIDA a PROGRAMADA
Cita a la misma hora que otra del mismo médico	Rechazado: conflicting key value violates exclusion constraint "cita_sin_cruce_medico"
Cita válida en un horario libre	Correcto: CM-0248 programada el 2026-11-02 a las 08:30

Nota. La cita superpuesta la rechaza la restricción EXCLUDE cita_sin_cruce_medico, que actúa junto con el disparador.

TR2. rol_proteger: los roles del sistema no se renombran ni se eliminan

```
create or replace function proteger_rol() returns trigger
language plpgsql set search_path = '' as $$
begin
    if old.nombre in ('ADMINISTRADOR', 'RECEPCIONISTA', 'MEDICO', 'PACIENTE') then
        if tg_op = 'DELETE' then
            raise exception 'El rol % es del sistema y no se puede eliminar', old.nombre;
        elsif new.nombre <> old.nombre then
            raise exception 'El rol % es del sistema y no se puede renombrar', old.nombre;
        end if;
    end if;
    return coalesce(new, old);
end $$;

create trigger rol_proteger before update or delete on rol
    for each row execute function proteger_rol();
```

Prueba	Resultado
delete from rol where nombre = 'ADMINISTRADOR'	Rechazado: El rol ADMINISTRADOR es del sistema y no se puede eliminar
update rol set nombre = 'ADMIN' …	Rechazado: El rol ADMINISTRADOR es del sistema y no se puede renombrar

TR3. paciente_normalizar: limpia los datos del paciente

```
create or replace function tg_normalizar_paciente() returns trigger
language plpgsql set search_path = '' as $$
begin
    new.nombres   := regexp_replace(trim(new.nombres), '\s+', ' ', 'g');
    new.apellidos := regexp_replace(trim(new.apellidos), '\s+', ' ', 'g');
    new.ci        := upper(trim(new.ci));
    new.email     := nullif(lower(trim(new.email)), '');
    return new;
end $$;

create trigger paciente_normalizar before insert or update on paciente
    for each row execute function tg_normalizar_paciente();
```

Resultado: al guardar el correo " JPerez@Correo.com " se almacena "jperez@correo.com".

TR4. medico_auditar_estado: registra en la bitácora la activación o desactivación de un médico

```
create or replace function tg_auditar_estado_medico() returns trigger
language plpgsql set search_path = '' as $$
begin
    insert into clinica.bitacora (accion, modulo, tabla, idregistro, descripcion)
    values (case when new.estado = 'ACTIVO' then 'ACTIVAR' else 'DESACTIVAR' end,
            'GESTION_MEDICA', 'medico', new.idmedico,
            format('El médico %s %s pasó de %s a %s', new.nombres, new.apellidos, old.estado,
new.estado));
    return new;
end $$;

create trigger medico_auditar_estado after update of estado on medico
    for each row when (old.estado is distinct from new.estado)
    execute function tg_auditar_estado_medico();
```

accion	modulo	tabla	descripcion
DESACTIVAR	GESTION_MEDICA	medico	El médico Patricia Molina Justiniano pasó de ACTIVO a INA…

TR5. estudio_fecha_resultado: fecha del resultado de un estudio

```
create or replace function tg_fecha_resultado_estudio() returns trigger
language plpgsql set search_path = '' as $$
begin
    if new.estado = 'COMPLETADO' and new.fecharesultado is null then
        new.fecharesultado := now();
    end if;
    return new;
end $$;

create trigger estudio_fecha_resultado before insert or update on estudio
    for each row execute function tg_fecha_resultado_estudio();

revoke execute on all functions in schema clinica from public;
revoke execute on all procedures in schema clinica from public;
-- La aplicación (headsalud_app, ver 04_usuario_app.sql) puede usar las funciones nuevas
do $$
begin
    if exists (select 1 from pg_roles where rolname = 'headsalud_app') then
        execute 'grant execute on all functions in schema clinica to headsalud_app';
        execute 'grant execute on all procedures in schema clinica to headsalud_app';
    end if;
end $$;
```

idestudio	estado	tiene_fecha
3	COMPLETADO	true

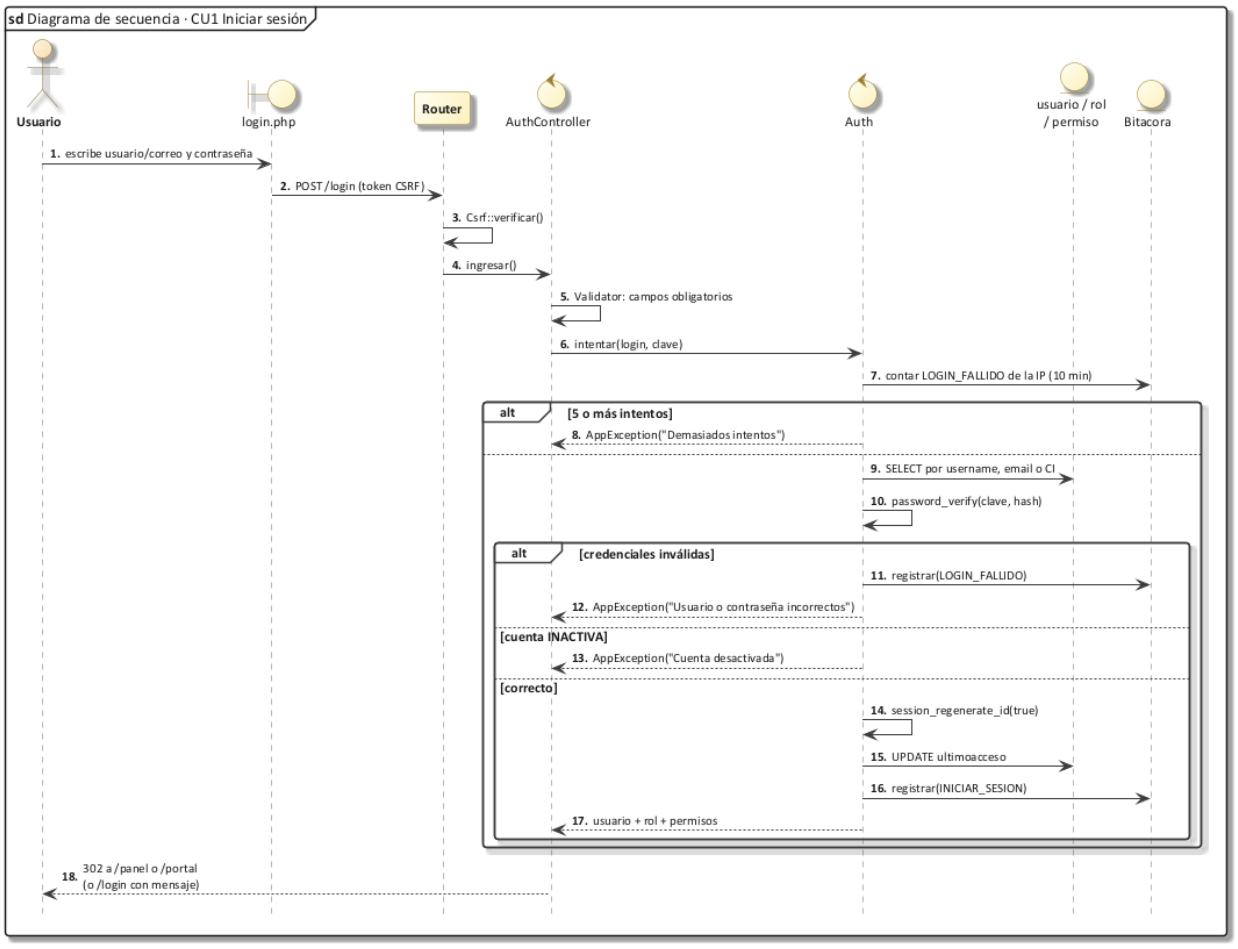
5.3 Diseñar casos de uso

Los diagramas de secuencia muestran, para cada caso de uso, la interacción entre las vistas, los controladores, los servicios, los modelos y la base de datos tal como están implementados.

CICLO I · CU1 a CU7: seguridad y datos maestros

Figura 103

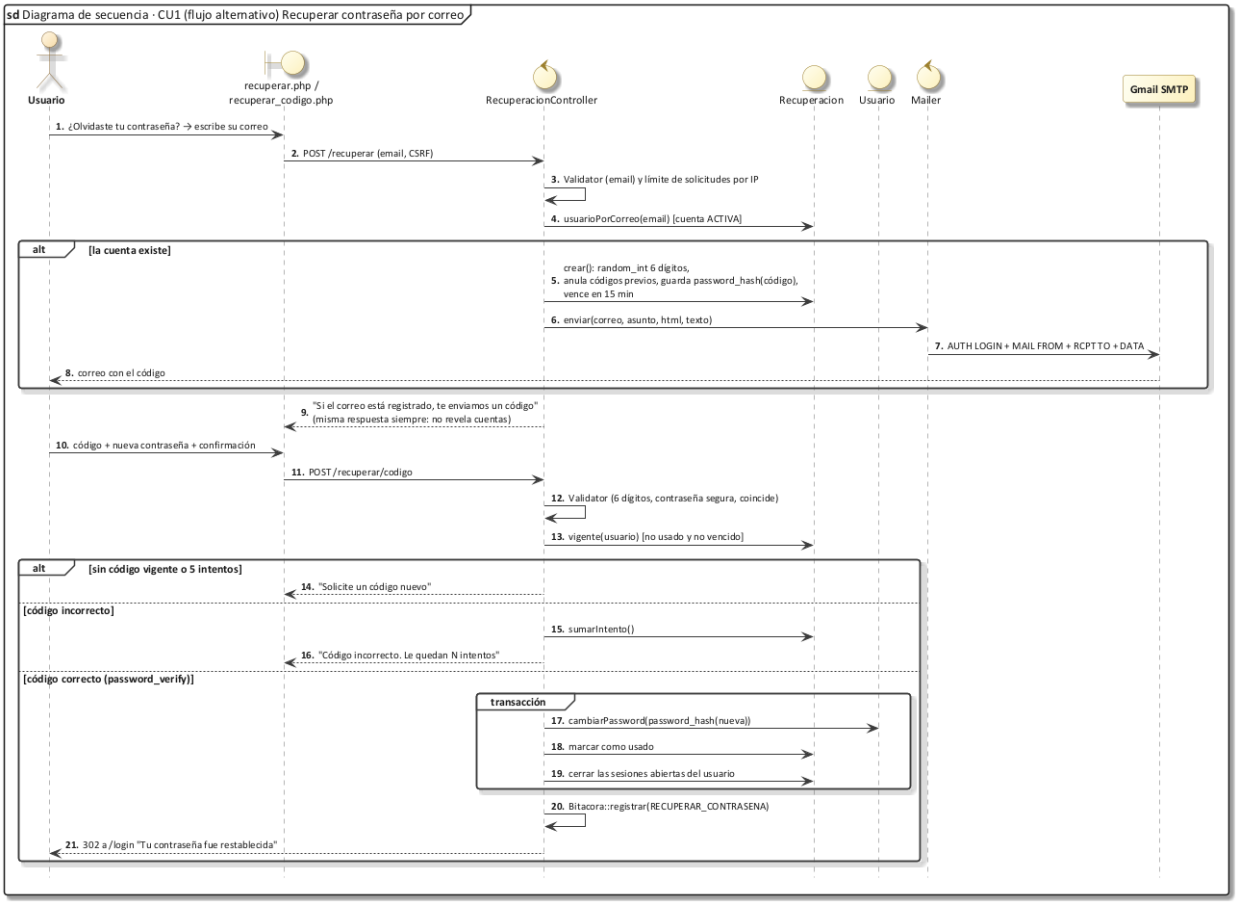
Diagrama de secuencia de CU1 Iniciar sesión



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 104

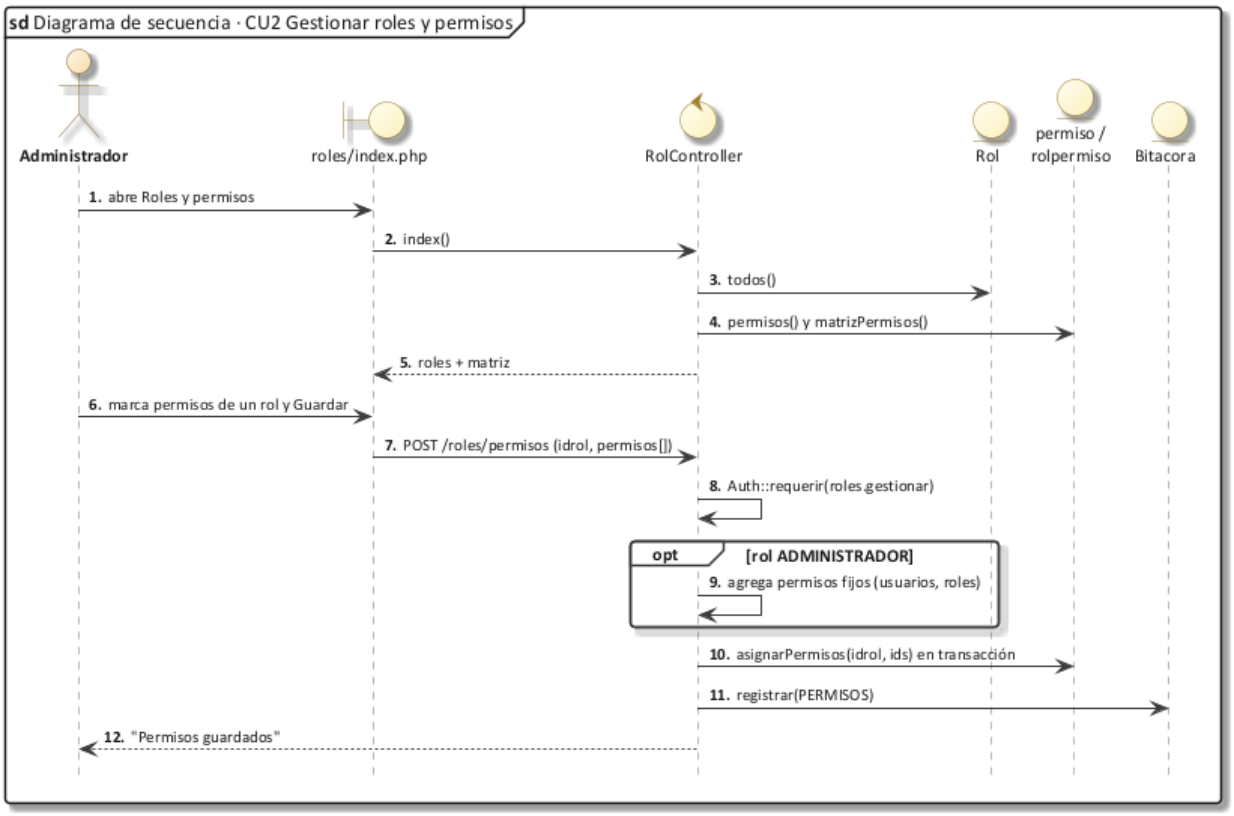
Diagrama de secuencia de CU1 (flujo alternativo): recuperar contraseña por correo



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 105

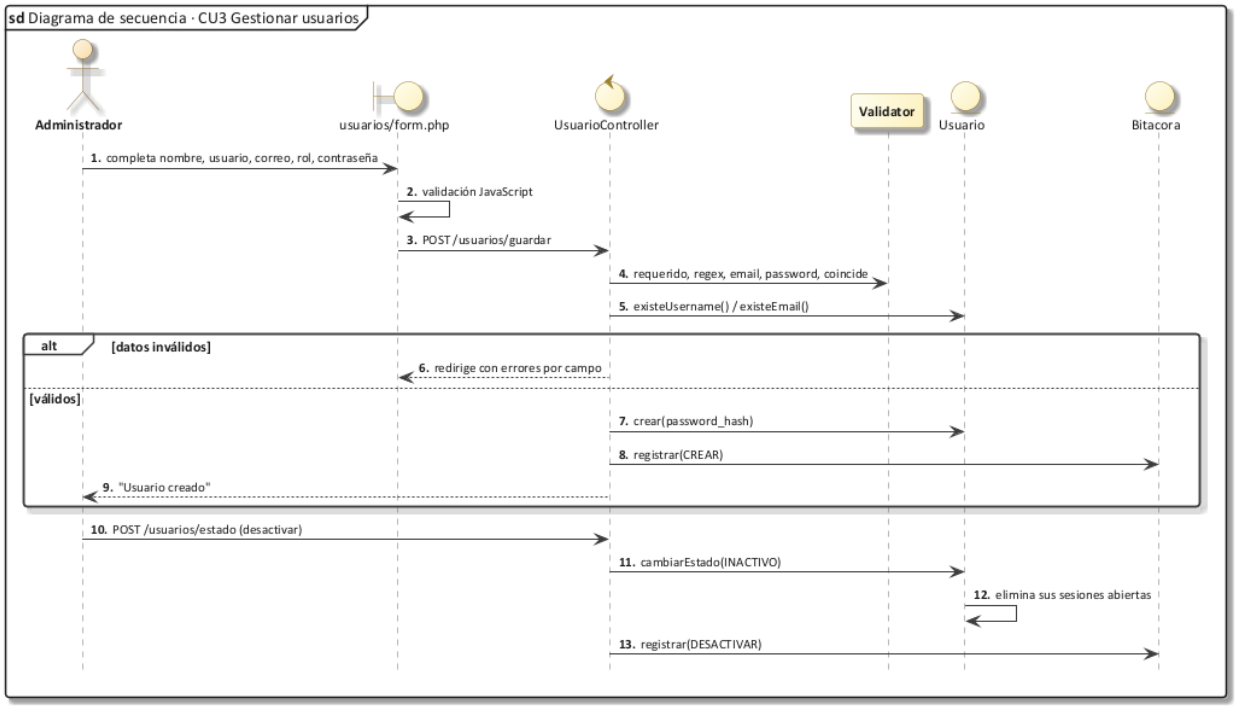
Diagrama de secuencia de CU2 Gestionar roles y permisos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 106

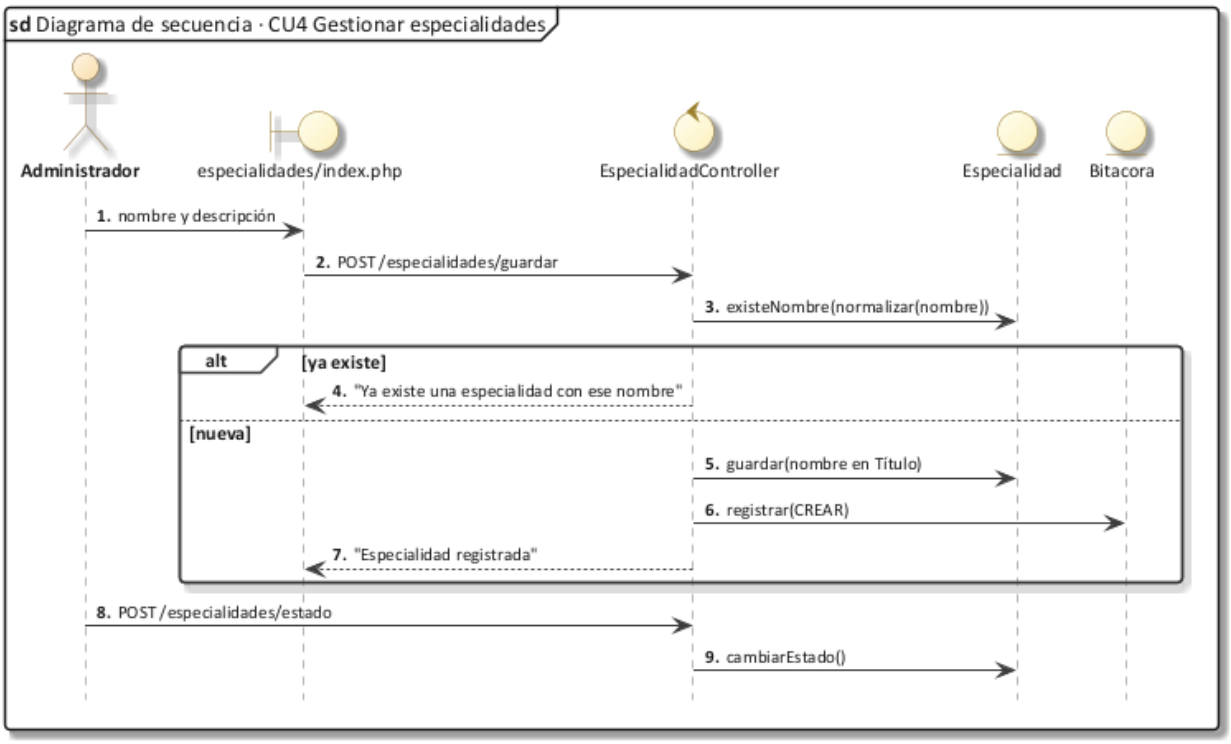
Diagrama de secuencia de CU3 Gestionar usuarios



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 107

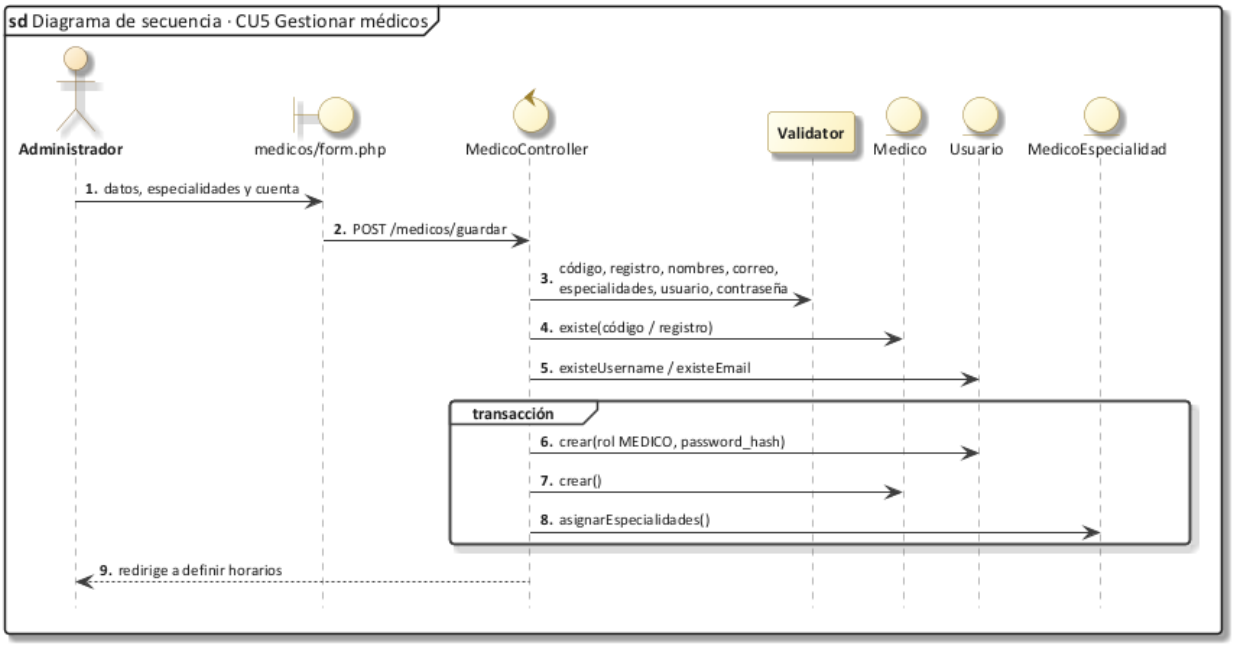
Diagrama de secuencia de CU4 Gestionar especialidades



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 108

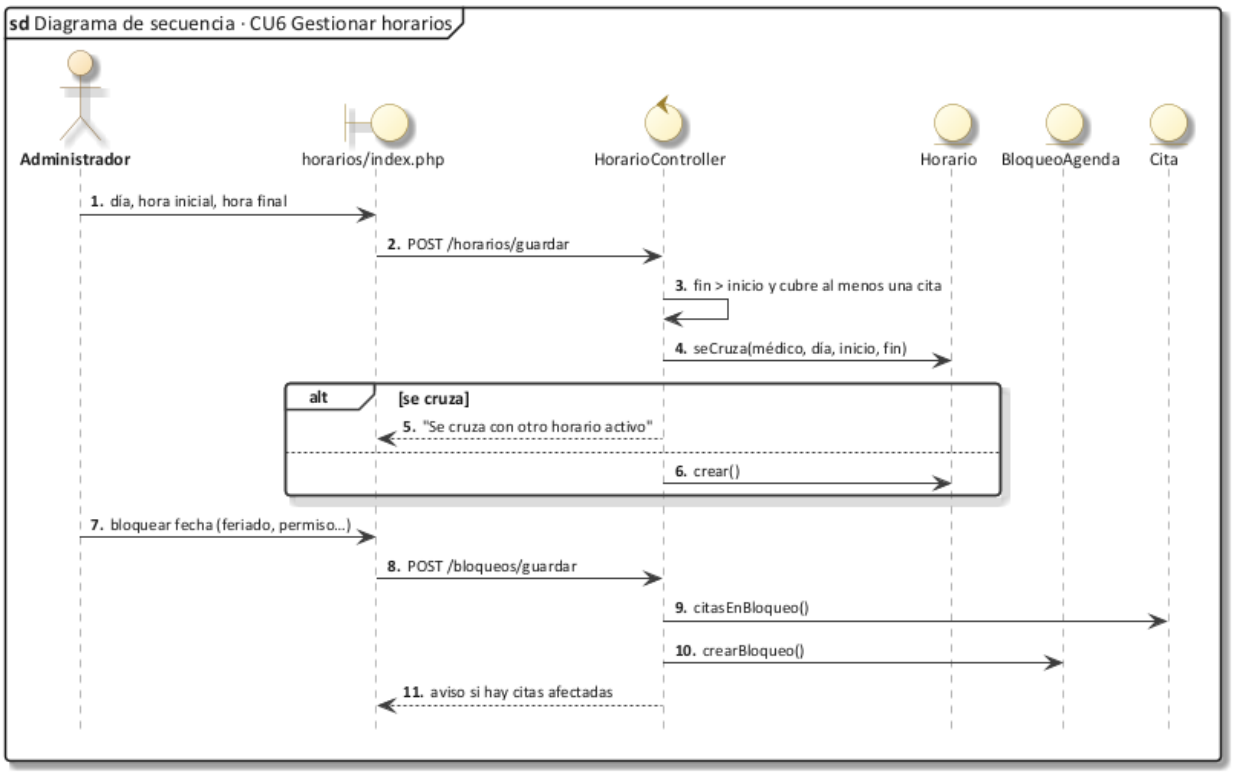
Diagrama de secuencia de CU5 Gestionar médicos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 109

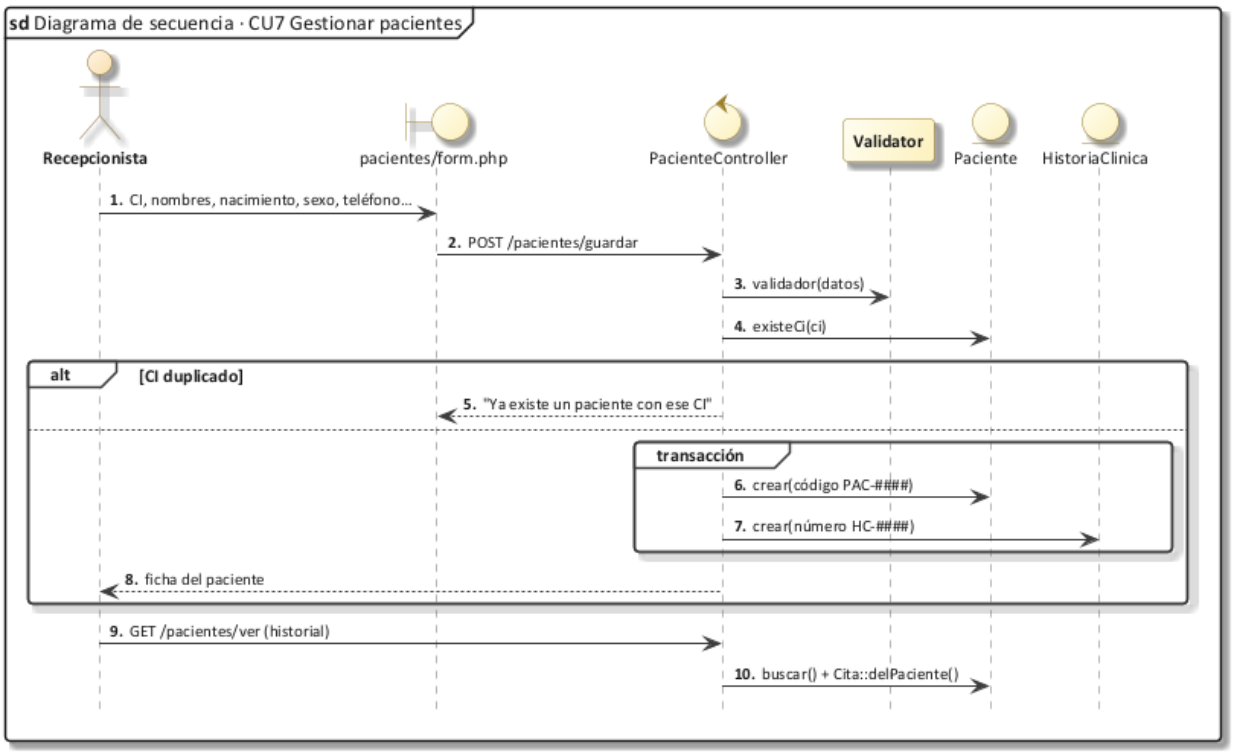
Diagrama de secuencia de CU6 Gestionar horarios



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 110

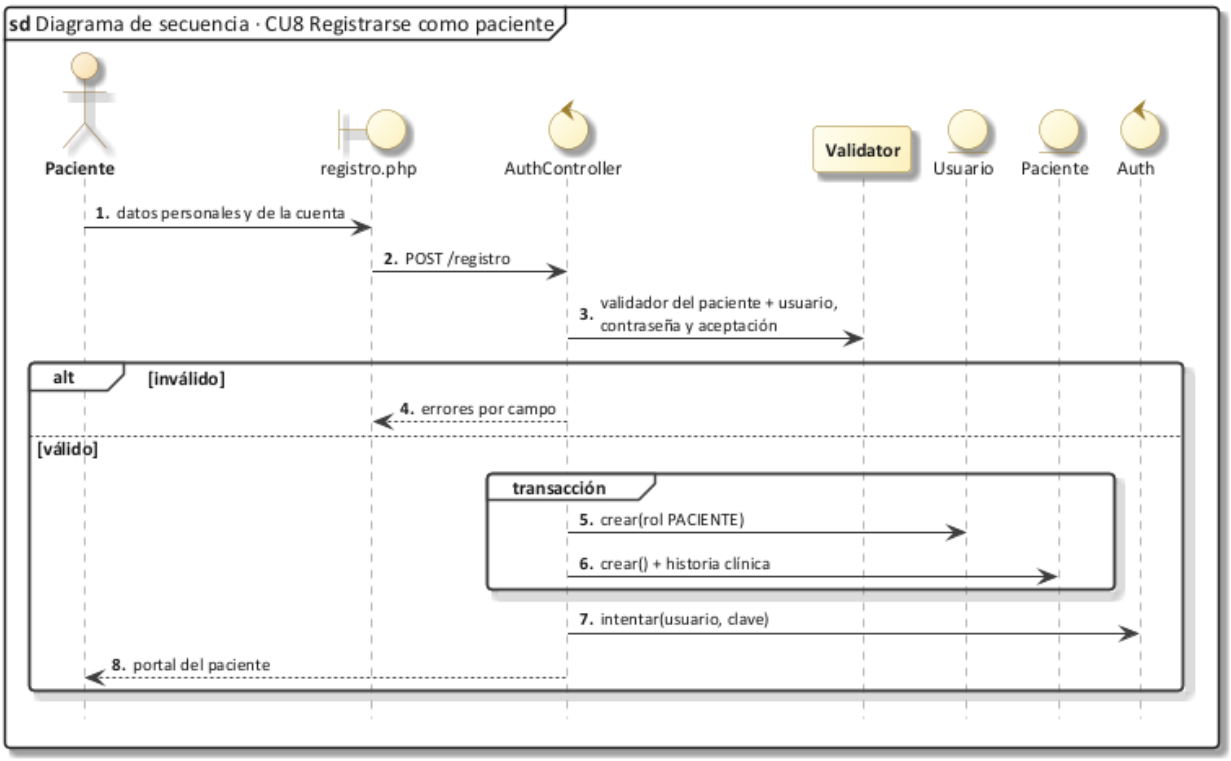
Diagrama de secuencia de CU7 Gestionar pacientes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 111

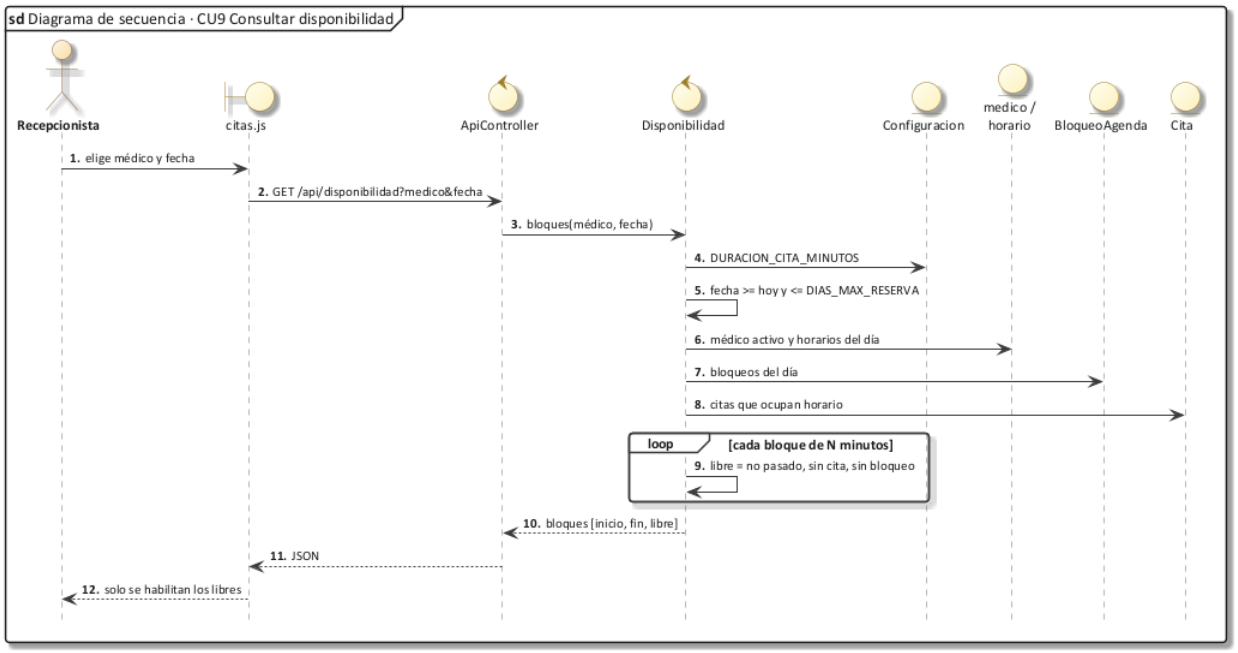
Diagrama de secuencia de CU8 Registrarse como paciente



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 112

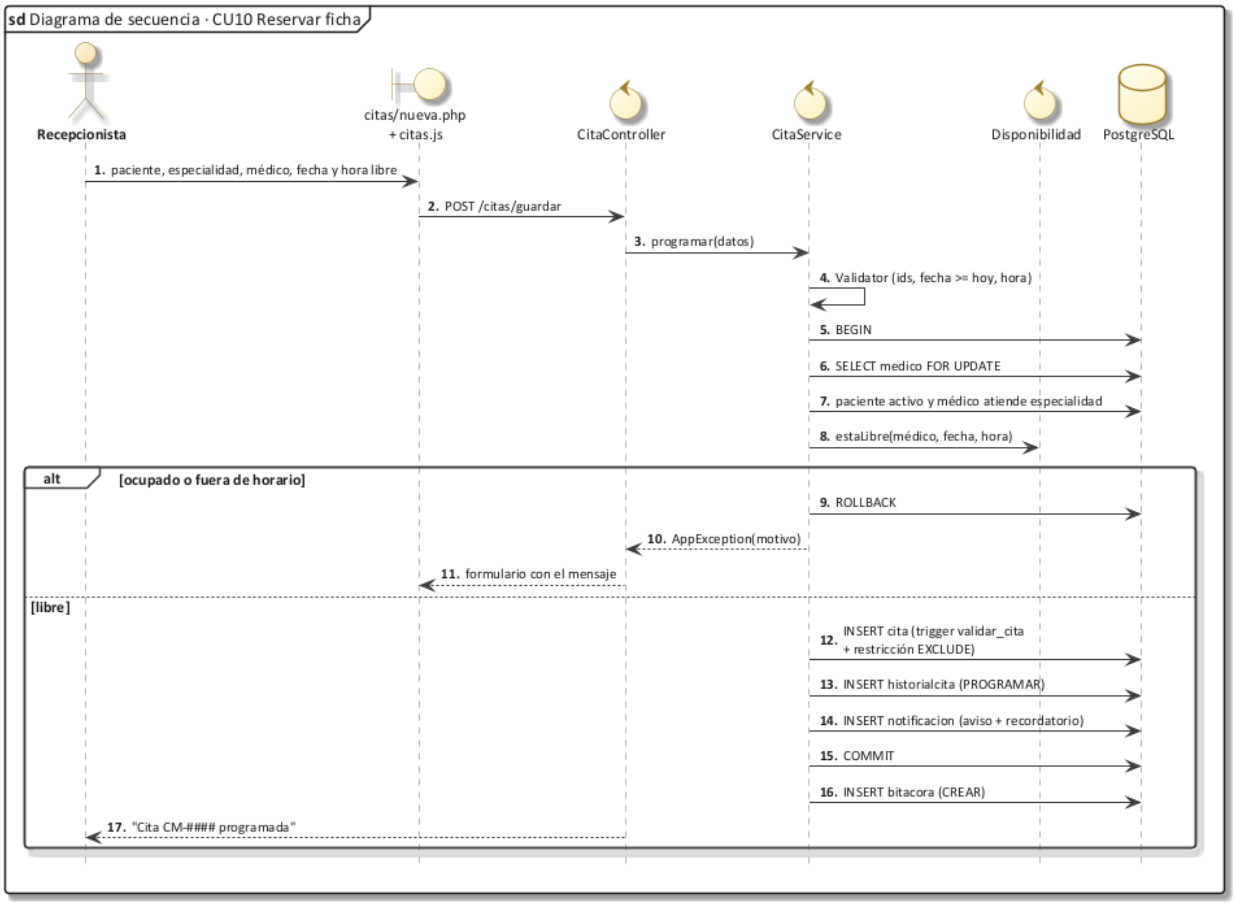
Diagrama de secuencia de CU9 Consultar disponibilidad



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 113

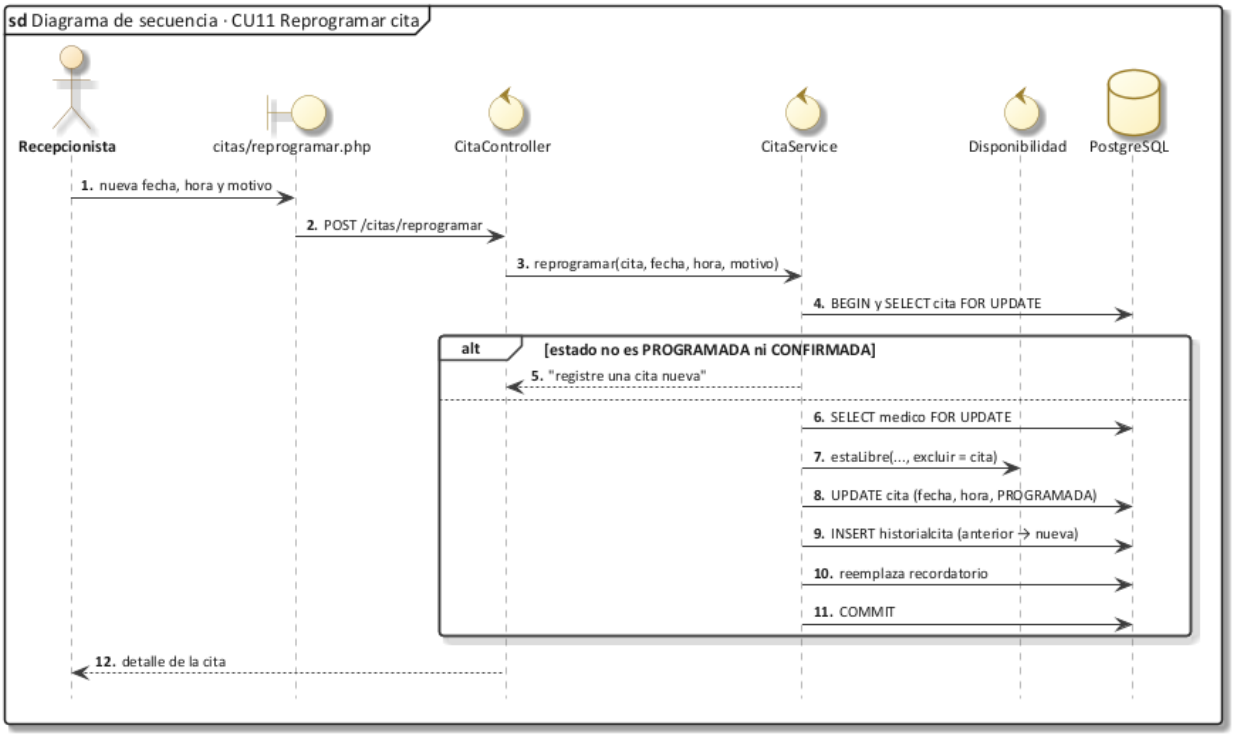
Diagrama de secuencia de CU10 Reservar ficha



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 114

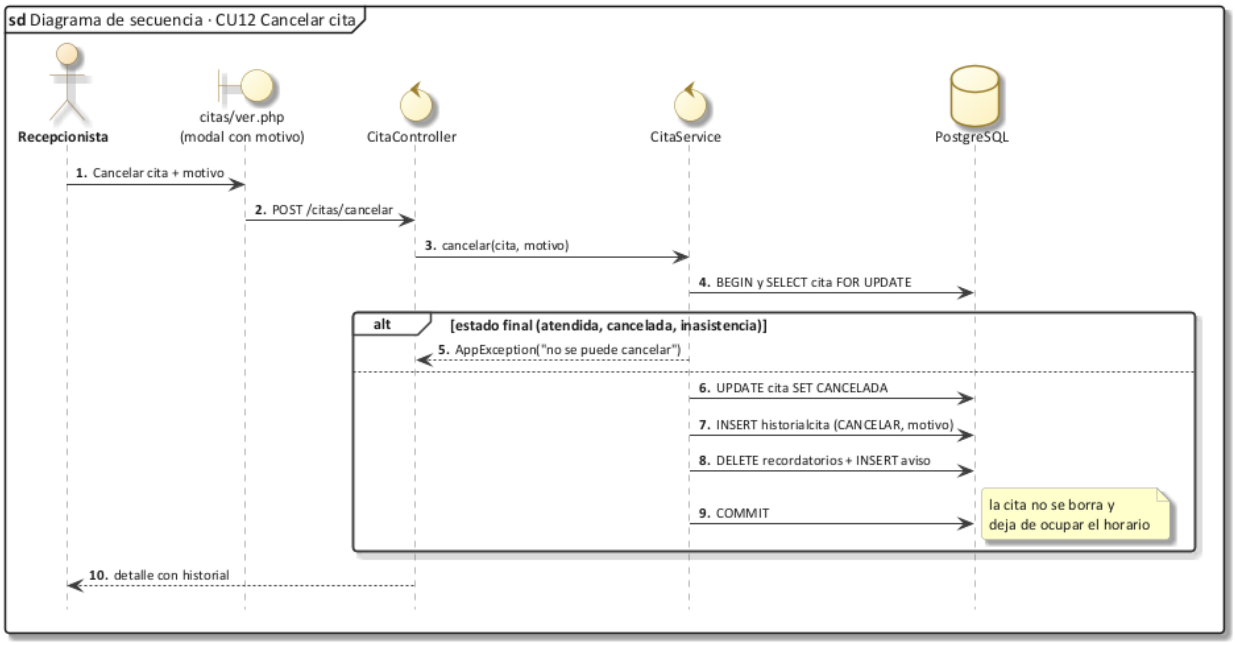
Diagrama de secuencia de CU11 Reprogramar cita



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 115

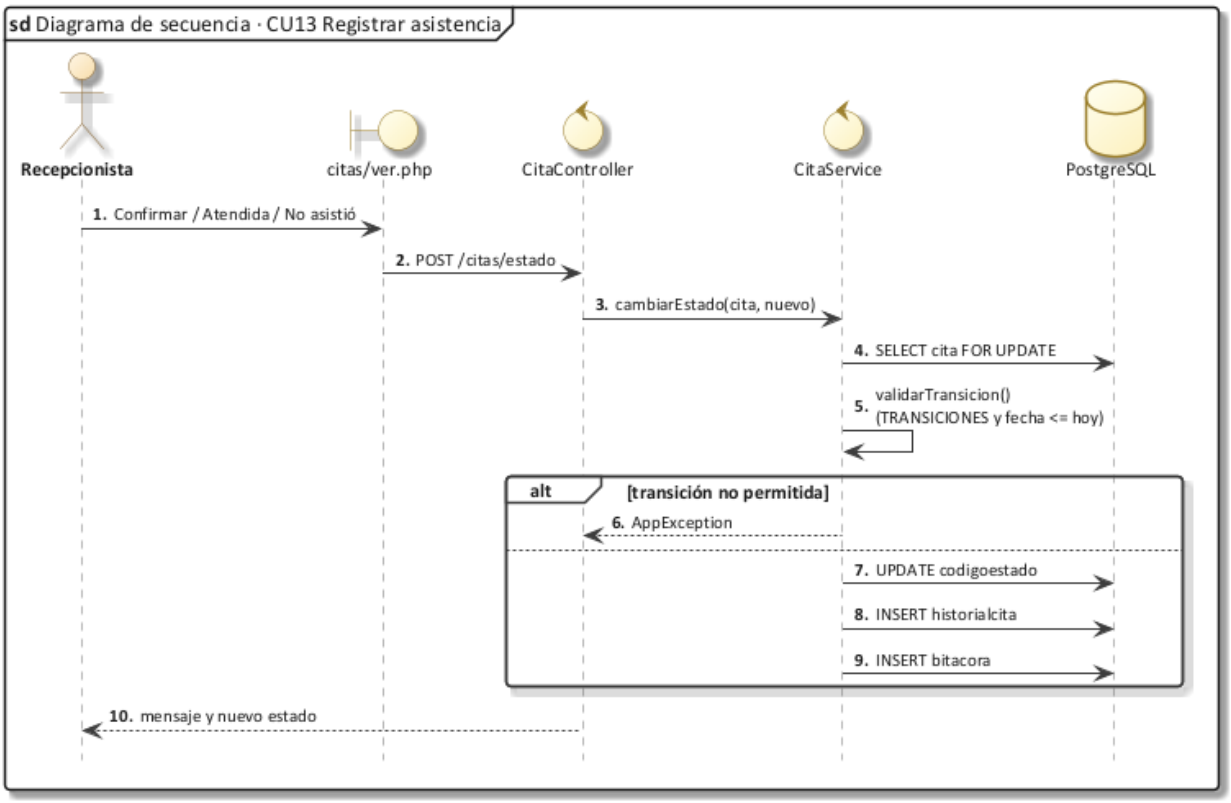
Diagrama de secuencia de CU12 Cancelar cita



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 116

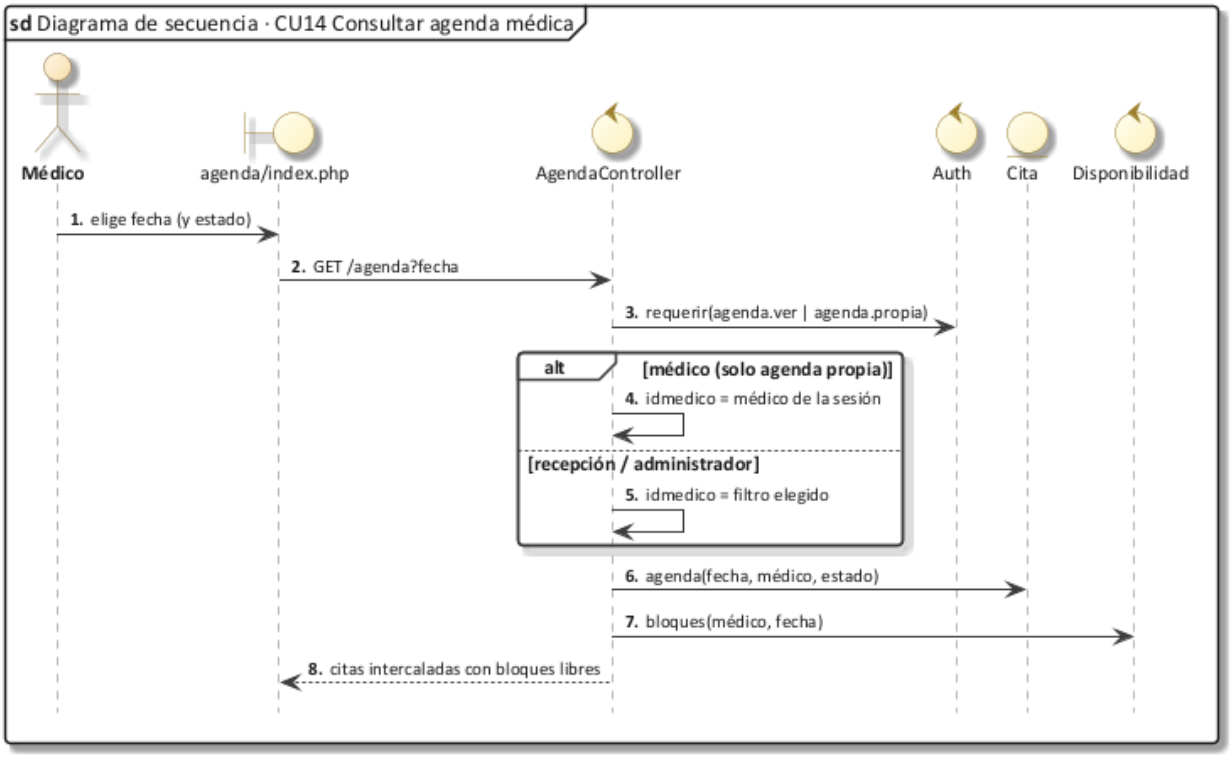
Diagrama de secuencia de CU13 Registrar asistencia



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 117

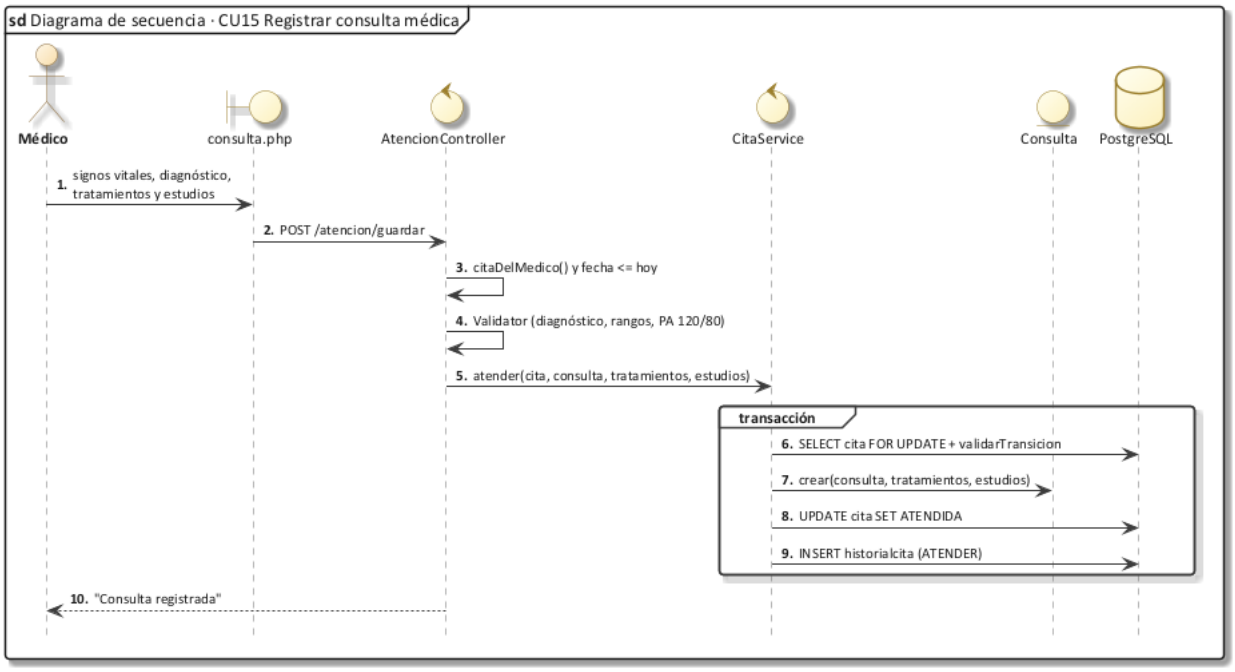
Diagrama de secuencia de CU14 Consultar agenda médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 118

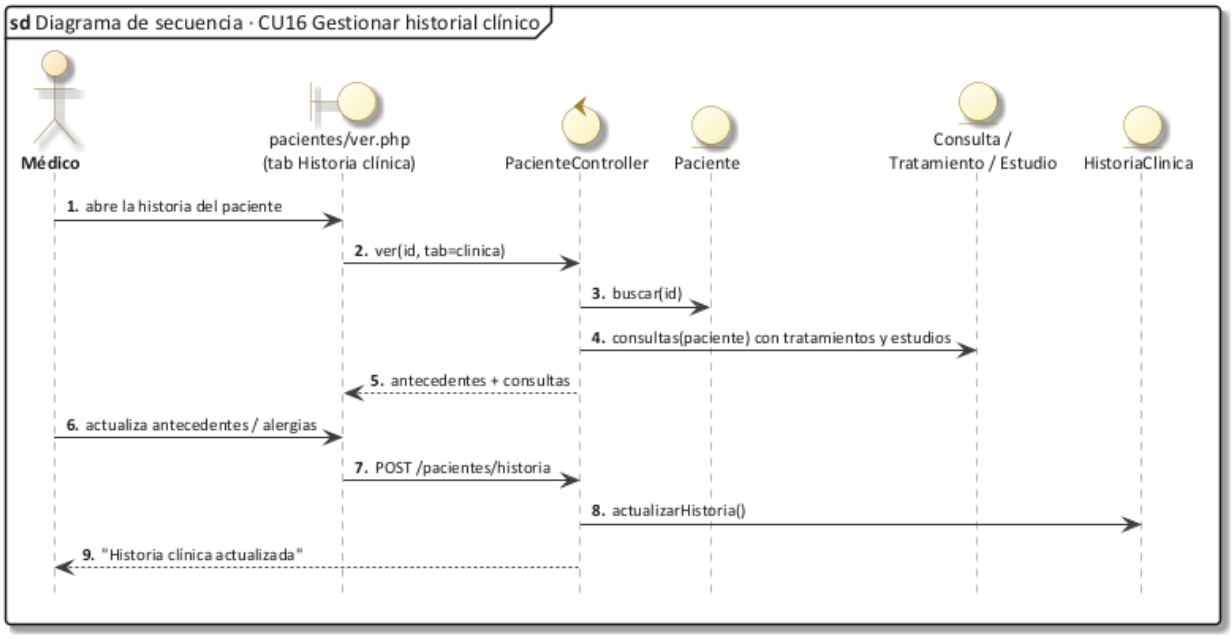
Diagrama de secuencia de CU15 Registrar consulta médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 119

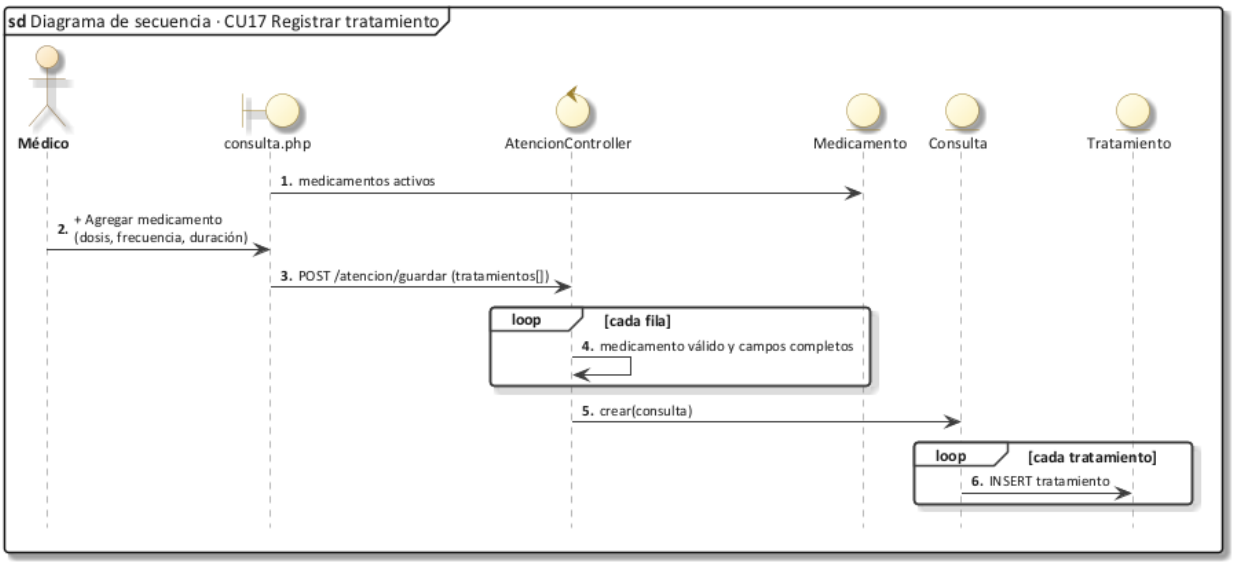
Diagrama de secuencia de CU16 Gestionar historial clínico



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 120

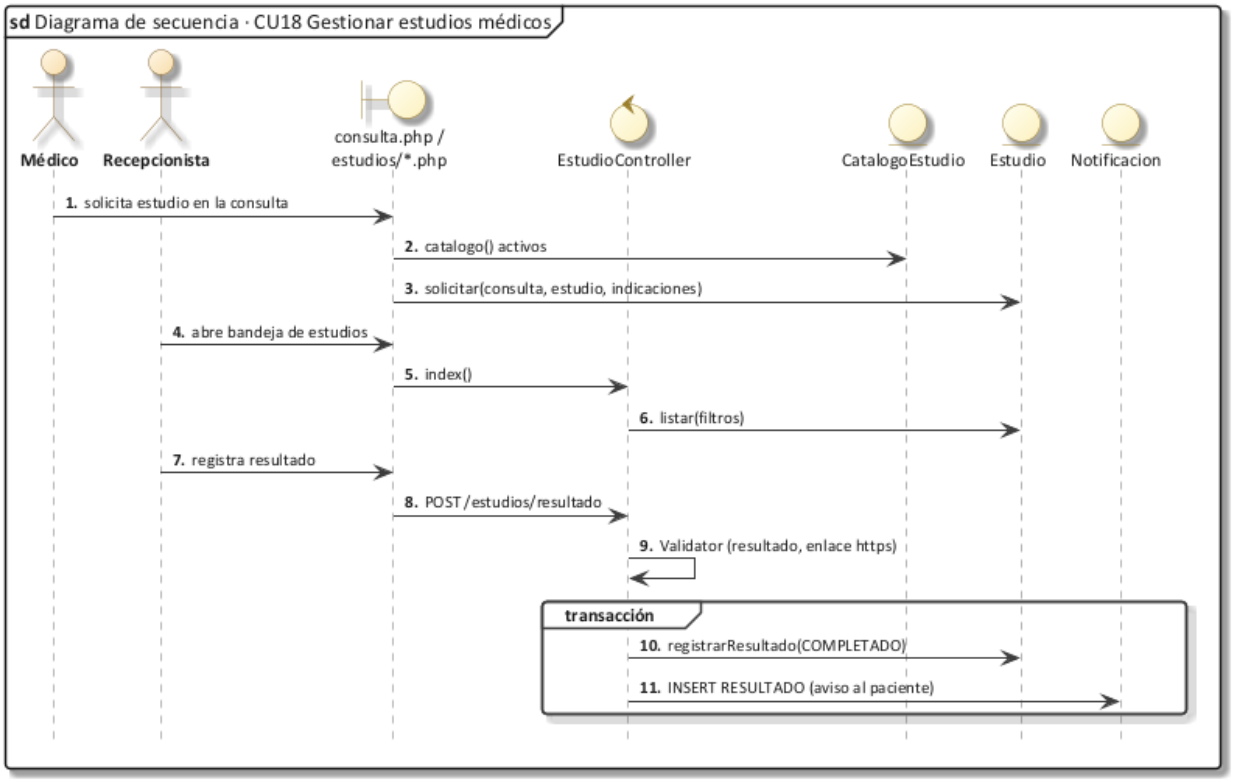
Diagrama de secuencia de CU17 Registrar tratamiento



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 121

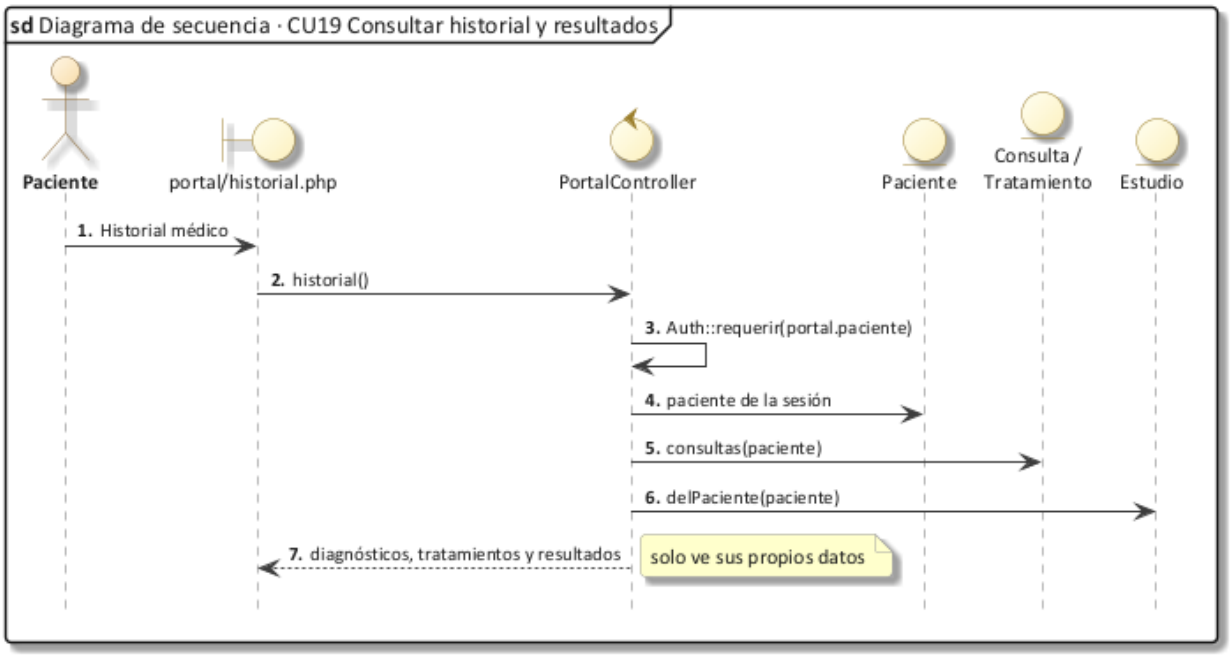
Diagrama de secuencia de CU18 Gestionar estudios médicos



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 122

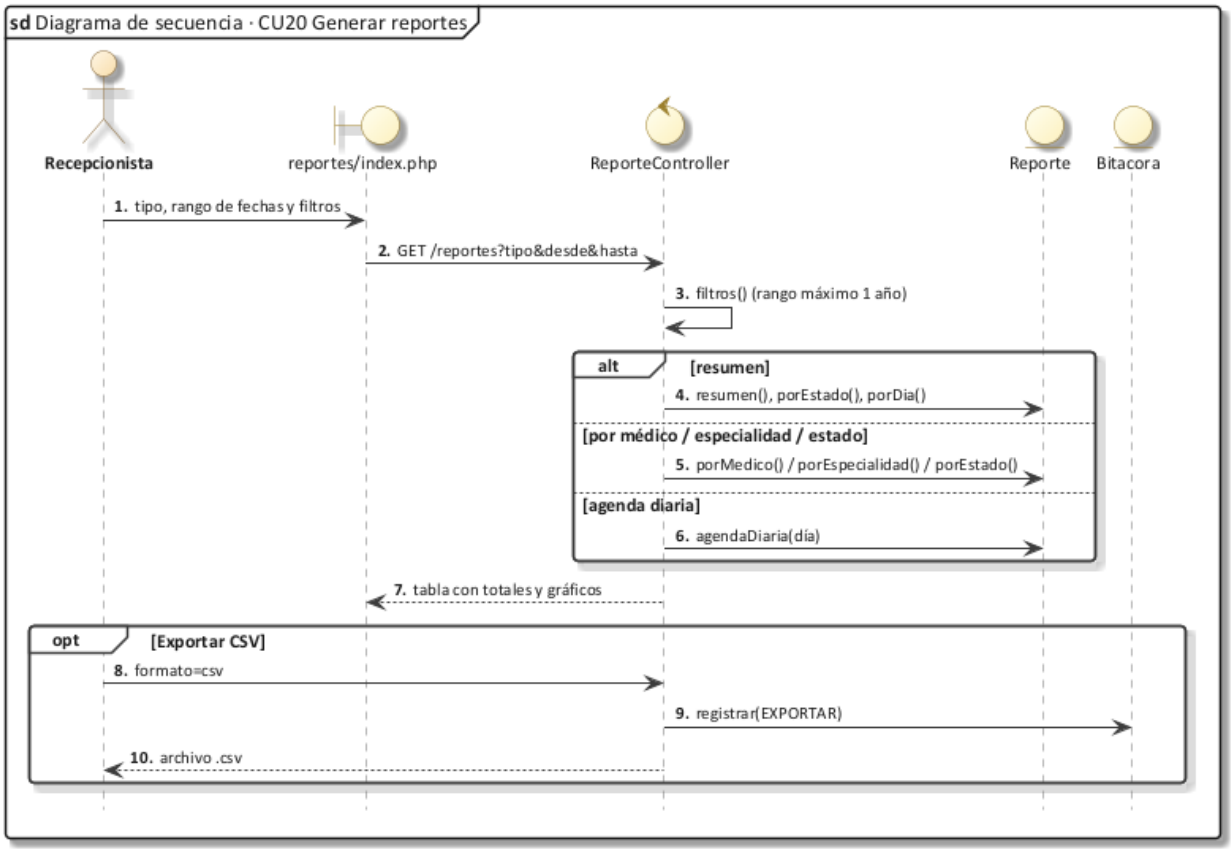
Diagrama de secuencia de CU19 Consultar historial y resultados



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 123

Diagrama de secuencia de CU20 Generar reportes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Capítulo 6. Flujo de trabajo: implementación

6.1 Elección de plataforma de desarrollo del software

Tabla 63

Plataforma de desarrollo

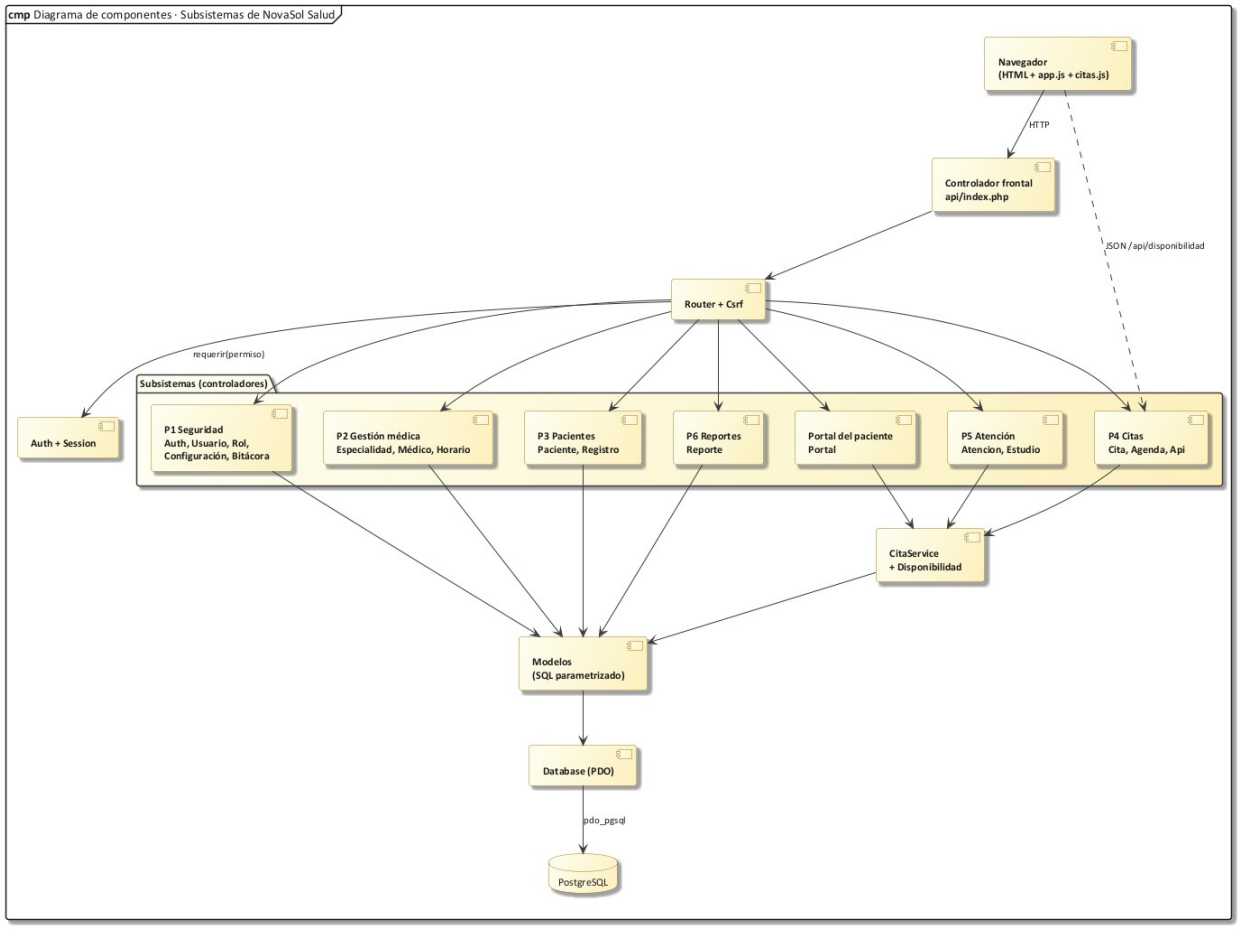
Elemento	Elección	Motivo
Lenguaje de programación	PHP 8.3 (sin framework), HTML5, CSS3 y JavaScript	Exigido por el enunciado; amplio soporte y funciones de seguridad incluidas (password_hash, PDO).
Base de datos	PostgreSQL 17 en Supabase	Restricciones avanzadas (EXCLUDE), disparadores y procedimientos; plan gratuito en la nube.
Sistemas operativos	Windows 11 (desarrollo), Linux (Vercel y Supabase)	El código es portable: funciona igual en ambos.
Otros	Vercel, GitHub, Enterprise Architect, PlantUML, Gmail SMTP	Publicación automática, control de versiones, diagramas y correo de recuperación.

6.2 Implementación de la arquitectura del sistema

El diagrama de componentes principal muestra los subsistemas del sistema y sus dependencias. Toda petición entra por el controlador frontal `api/index.php`, pasa por el enrutador (que verifica el token CSRF) y la autenticación, y llega al controlador del subsistema, que usa los servicios y modelos para acceder a PostgreSQL mediante PDO.

Figura 124

Diagrama de componentes principal del sistema



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

```
novasol-salud/
├── Bakend/                 Lo que el usuario NO ve
│   ├── app/
│   │   ├── bootstrap.php   Arranque: entorno, errores, carga de clases, sesión
│   │   ├── core/           Router, Auth, Csrf, Session, Validator, View, Database, Mailer
│   │   ├── controllers/    Un controlador por caso de uso o módulo (incluye la página
pública)
│   │   ├── models/         Acceso a datos (SQL parametrizado)
│   │   └── services/       CitaService y Disponibilidad
│   └── database/           Scripts SQL 01 a 06
├── Frontend/               Lo que el usuario SÍ ve
│   ├── views/              Publico, P1_SeguridadAcceso … P6_Reportes, Portal, layouts
│   └── public/             CSS, JavaScript e ícono (lo único que el navegador abre directo)
├── Docs/                   Informe, diagramas, modelo de Enterprise Architect y capturas
├── api/index.php           Entrada de todas las peticiones y tabla de rutas
└── router.php · .htaccess · vercel.json
```

Bakend decide *qué* pasa (permisos, reglas de negocio y base de datos) y **Frontend** decide *cómo se ve*. Los casos de uso no son una carpeta: cada uno se reparte entre su controlador y sus pantallas. `api/`, `router.php`, `.htaccess` y `vercel.json` quedan en la raíz porque arrancan el sistema en Vercel, en el servidor local de PHP y en Apache.

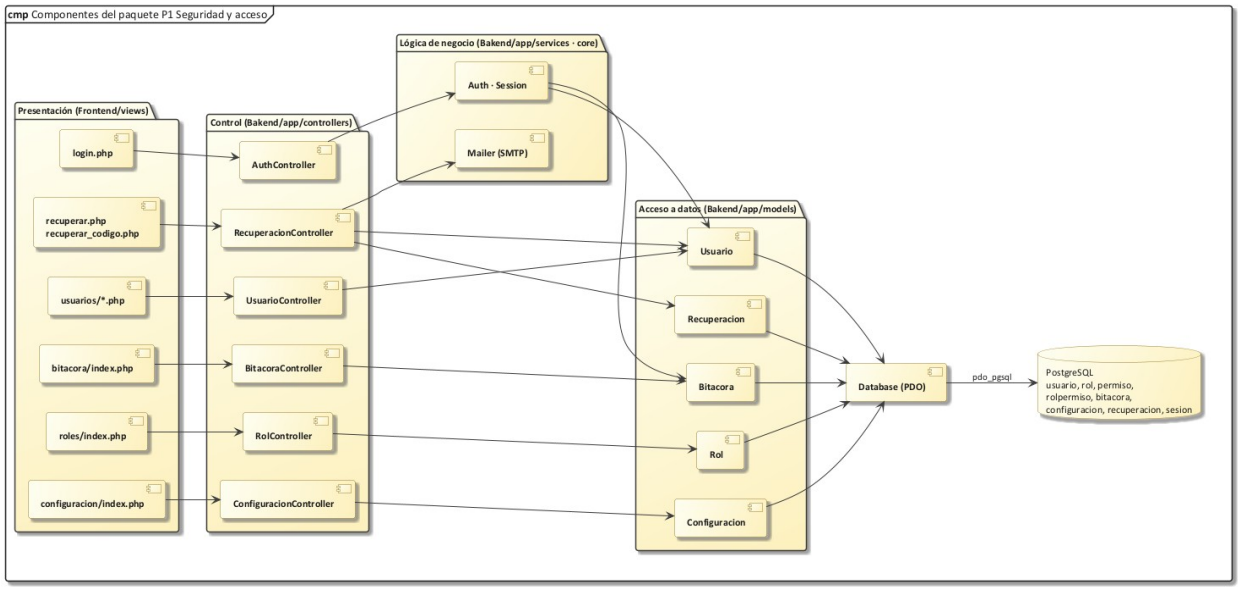
6.3 Implementación de la arquitectura del subsistema

Cada paquete se implementa como un subsistema con sus vistas, controladores, servicios y modelos.

CICLO I · CU1 a CU7: seguridad y datos maestros

Figura 125

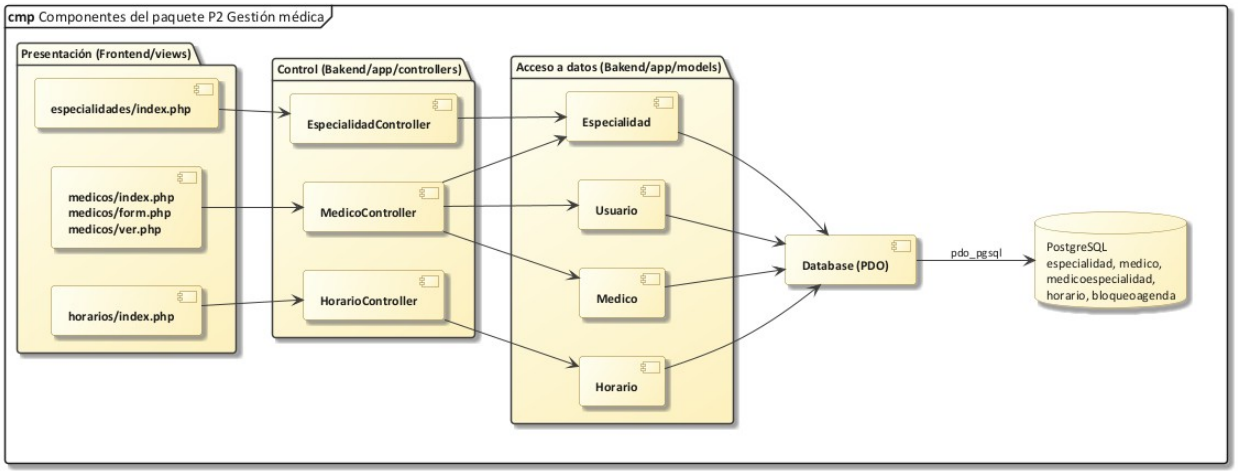
Diagrama de componentes del paquete P1 Seguridad y acceso



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 126

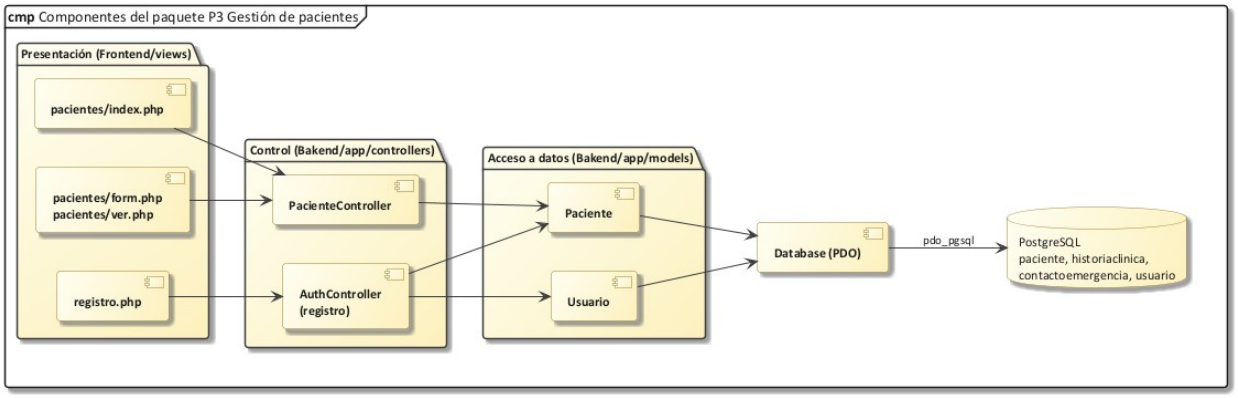
Diagrama de componentes del paquete P2 Gestión médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 127

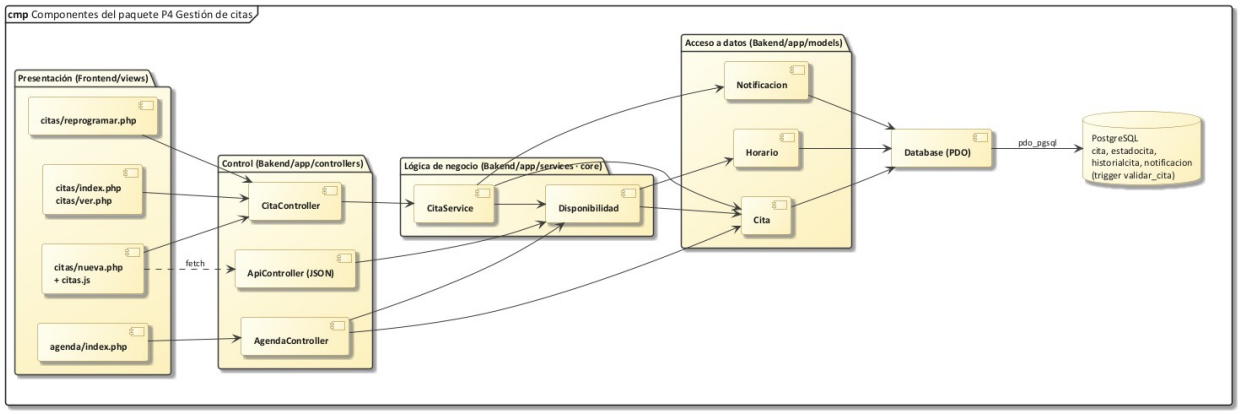
Diagrama de componentes del paquete P3 Gestión de pacientes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 128

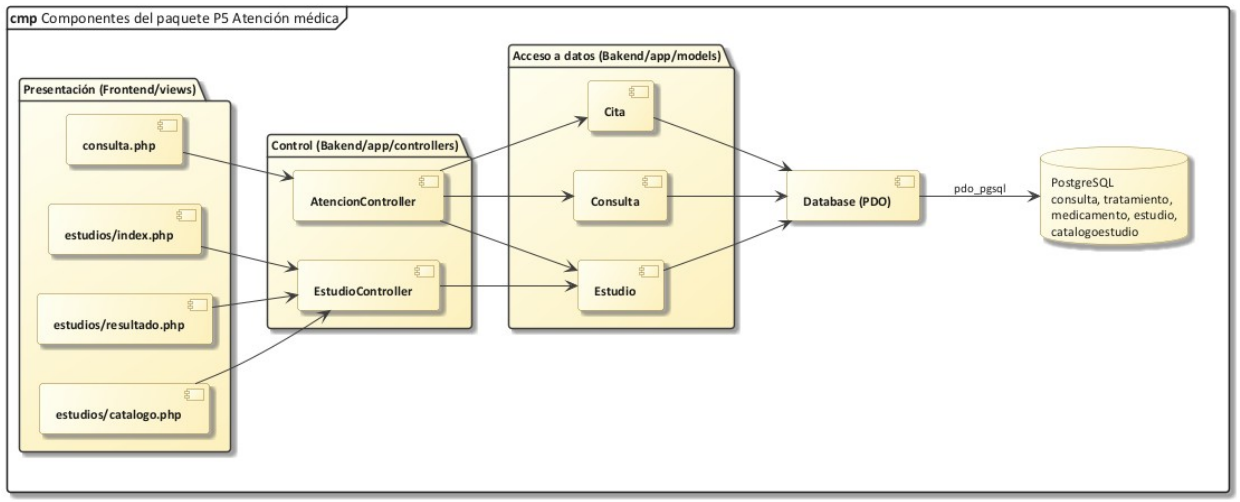
Diagrama de componentes del paquete P4 Gestión de citas



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 129

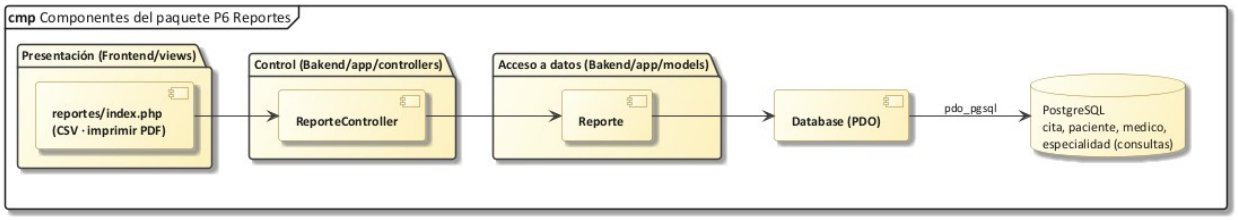
Diagrama de componentes del paquete P5 Atención médica



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 130

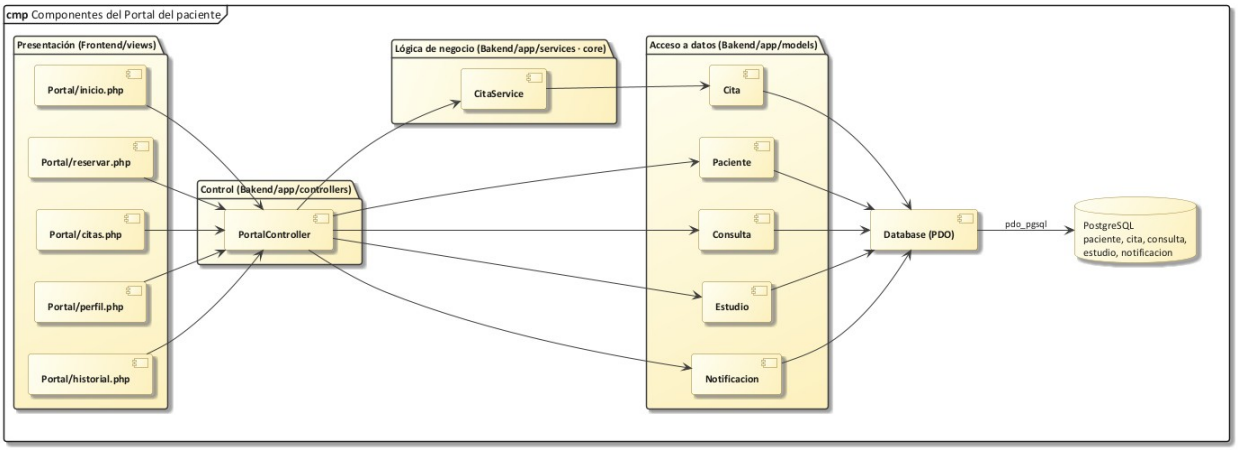
Diagrama de componentes del paquete P6 Reportes



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

Figura 131

Diagrama de componentes del Portal del paciente



Nota. Elaboración propia en notación UML con el estilo de Enterprise Architect.

7. Conclusiones

- Se desarrolló el sistema web NovaSol Salud con PHP 8 y PostgreSQL, conectados mediante PDO con consultas parametrizadas, que cumple los trece requerimientos funcionales (RF01 a RF13) y las catorce reglas de negocio (RN01 a RN14) del enunciado.
- El método de Ishikawa permitió identificar que la causa raíz de los problemas de la clínica es la falta de un sistema centralizado, y orientar las alternativas de cambio hacia los módulos del sistema.
- La organización del trabajo en dos ciclos del Proceso Unificado permitió construir primero la base (seguridad y datos maestros, CU1 a CU7) y después el núcleo del negocio (citas, atención, reportes y portal, CU8 a CU20) sin rehacer lo anterior.
- Se mantuvo la trazabilidad desde los casos de uso hasta el código: cada caso de uso tiene su descripción, su diagrama de comunicación, su diagrama de secuencia y sus componentes.
- La duplicación de citas se resolvió en tres niveles: el sistema solo ofrece horarios libres, el servidor vuelve a verificar la disponibilidad dentro de una transacción y PostgreSQL impide las citas superpuestas con restricciones de exclusión y disparadores.
- La base de datos quedó normalizada hasta la tercera forma normal, con 30 consultas, cinco procedimientos almacenados y cinco disparadores probados.

8. Recomendaciones

- Configurar en producción la cuenta de Gmail con contraseña de aplicación para el envío del código de recuperación.
- Integrar el envío real de recordatorios por correo o WhatsApp mediante una tarea programada.
- Almacenar los archivos de resultados de estudios en un servicio de almacenamiento como Supabase Storage.
- Agregar un calendario semanal y mensual a la agenda y una lista de espera cuando no haya horarios disponibles.
- Implementar la autenticación en dos pasos para el personal.
- Realizar copias de seguridad periódicas de la base de datos y capacitar al personal antes de la puesta en marcha.

9. Bibliografía

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Pearson Addison Wesley.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Elmasri, R., & Navathe, S. B. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Pearson Educación.

Ishikawa, K. (1986). *¿Qué es el control total de calidad? La modalidad japonesa*. Norma.

Jacobson, I., Booch, G., & Rumbaugh, J. (2000). *El proceso unificado de desarrollo de software*. Addison Wesley.

Larman, C. (2003). *UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2.ª ed.). Prentice Hall.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

The PHP Group. (2026). *Manual de PHP*. https://www.php.net/manual/es/

PlantUML. (2025). *PlantUML language reference guide*. https://plantuml.com/es/

The PostgreSQL Global Development Group. (2026). *PostgreSQL 17 documentation*. https://www.postgresql.org/docs/17/

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Supabase. (2026). *Supabase documentation*. https://supabase.com/docs

Vercel. (2026). *Vercel documentation*. https://vercel.com/docs

Anexos

Anexo A. Repositorio y sistema en línea

Tabla 64

Enlaces del proyecto

Sistema en línea	https://novasol-salud.vercel.app
Repositorio GitHub	https://github.com/Kato-001/novasol-salud
Base de datos	PostgreSQL 17 en Supabase (esquema clinica)
Documentación técnica	README.md y carpeta Docs/ del repositorio

Anexo B. Cuentas de acceso

Contraseña de todas las cuentas de demostración: **HeadSalud2026**. Se puede ingresar con el usuario o el correo; los pacientes también con su CI.

Tabla 65

Cuentas de demostración

Rol	Persona	Usuario	Correo
Administrador	Administrador General	admin	admin@vidasalud.com
Recepcionista	María Fernanda Rojas	mrojas	recepcion@vidasalud.com
Recepcionista	Jorge Cuéllar	jcuellar	jcuellar@vidasalud.com
Médico	Dr. Carlos Mendoza	cmendoza	cmendoza@vidasalud.com
Médico	Dra. Lucía Vargas	lvargas	lvargas@vidasalud.com
Médico	Dr. Roberto Suárez	rsuarez	rsuarez@vidasalud.com
Médico	Dra. Patricia Molina	pmolina	pmolina@vidasalud.com
Paciente	Juan Pérez Quispe (CI 1234567)	jperez	jperez@correo.com
Paciente	María López Rivero (CI 7654321)	mlopez	mlopez@correo.com

Anexo C. Pruebas del sistema

Tabla 66

Pruebas funcionales y de seguridad

N.º	Prueba	Resultado esperado	Resultado obtenido
1	Registrar un paciente nuevo	El paciente queda guardado	Correcto: código e historia clínica generados
2	Paciente con CI duplicado	Se rechaza el duplicado	Correcto: "Ya existe un paciente registrado con ese CI"
3	Cita en horario disponible	La cita queda registrada	Correcto
4	Cita superpuesta, mismo médico	Se impide el conflicto	Correcto: "El horario ya no está disponible"
5	Cita fuera del horario	Se rechaza la cita	Correcto
6	Cancelar una cita	Cambia a cancelada y conserva el historial	Correcto
7	Historial de un paciente	Aparecen sus citas	Correcto
8	Citas de un médico por fecha	Resultados correctos	Correcto
9	Página interna sin sesión	Acceso denegado	Correcto: redirige a /login
10	Datos inválidos	El servidor valida y rechaza	Correcto, aun sin JavaScript
11	Recepcionista abre /usuarios	Acceso denegado	Correcto: 403
12	Formulario sin token CSRF	Petición rechazada	Correcto: 419
13	Recuperar contraseña con código correcto	Contraseña restablecida	Correcto
14	Reutilizar un código ya usado	Código rechazado	Correcto

Anexo D. Manual breve de uso

Tabla 67

Manual breve de uso

Tarea	Pasos
Programar una cita (recepción)	Programar cita → buscar paciente por CI → especialidad → médico → día → hora libre → Confirmar cita.
Reprogramar o cancelar	Citas → abrir la cita → Reprogramar (nueva fecha y motivo) o Cancelar cita (motivo).
Atender (médico)	Mi agenda → abrir la cita → Atender → diagnóstico, tratamiento y estudios → Guardar.
Reservar (paciente)	Portal → Reservar cita → especialidad → médico → día → hora → Confirmar reserva.
Recuperar la contraseña	Login → ¿Olvidaste tu contraseña? → correo → escribir el código de 6 dígitos y la nueva contraseña.
Generar un reporte	Reportes → pestaña → fechas y filtros → Generar → Exportar CSV o Imprimir / PDF.

Anexo E. Modelo en Enterprise Architect

El modelo UML del proyecto se entrega también como proyecto de Enterprise Architect 17.2 (Sparx Systems), en el archivo `Docs/EA/NovaSol_Salud.qea` del repositorio. Se abre con Enterprise Architect (*File → Open Project*) y contiene los elementos con sus relaciones, organizados igual que este informe:

Tabla 68

Contenido del modelo en Enterprise Architect

Paquete del modelo	Contenido	Diagramas
01 Modelo de dominio	19 clases conceptuales con sus asociaciones y multiplicidades	Modelo de dominio
02 Captura de requisitos	Actores (con generalización a Usuario); Ciclo I y Ciclo II, cada uno con sus paquetes P1 a P6 y los 20 casos de uso; asociaciones actor–caso de uso y relaciones «include» y «extend»	Casos de uso del Ciclo I, casos de uso del Ciclo II y un diagrama por paquete

Paquete del modelo	Contenido	Diagramas
03 Análisis	Paquetes P1 a P6 y Portal del paciente con sus dependencias «use»	Vista de paquetes de análisis
04 Diseño de datos	20 clases persistentes con atributos, tipos, asociaciones y composiciones	Diagrama de clases persistentes

Nota. El modelo se genera con los scripts `Docs/EA/extraer_modelo.py` y `Docs/EA/crear_modelo_ea.ps1` (API de automatización de Enterprise Architect). Las figuras de este informe usan la notación y el estilo de Enterprise Architect.

Anexo F. Script de creación de las tablas

```
-- NovaSol Salud · Sistema Web de Gestión de Citas Médicas
-- 01 · Esquema de la base de datos (PostgreSQL)
-- ----------------------------------------------------------------------
-- Todo vive en el esquema "clinica" (no expuesto por la API pública de Supabase).
-- Implementa el diagrama de clases (Docs/Diagrama de Clinica.svg) más
-- tres tablas de soporte: PERMISO y ROLPERMISO (permisos por rol) y
-- SESION (sesiones PHP persistentes, necesarias en Vercel).

create schema if not exists clinica;
create schema if not exists extensions;   -- ya existe en Supabase; se crea en PostgreSQL local
set search_path = clinica, extensions, public;

create extension if not exists btree_gist with schema extensions;   -- restricciones EXCLUDE
(citas sin cruces)

-- Normaliza un texto para comparar nombres: minúsculas, sin tildes y sin espacios repetidos
create or replace function normalizar(t text) returns text
language sql immutable parallel safe set search_path = '' as $$
    select translate(lower(regexp_replace(trim(t), '\s+', ' ', 'g')), 'áéíóúüñ', 'aeiouun')
$$;

-- P1 · Seguridad y acceso
create table rol (
    idrol        integer generated by default as identity primary key,
    nombre       varchar(50)  not null unique,
    descripcion  varchar(255)
);

create table permiso (
    idpermiso    integer generated by default as identity primary key,
    codigo       varchar(60)  not null unique,       -- ej. citas.gestionar
    nombre       varchar(120) not null,
    modulo       varchar(60)  not null
);

create table rolpermiso (
    idrol        integer not null references rol(idrol) on delete cascade,
    idpermiso    integer not null references permiso(idpermiso) on delete cascade,
    primary key (idrol, idpermiso)
);
create index rolpermiso_idpermiso_idx on rolpermiso(idpermiso);
```

```
create table usuario (
    idusuario     integer generated by default as identity primary key,
    nombre        varchar(150) not null,
    username      varchar(50)  not null,
    email         varchar(150) not null,
    password      varchar(255) not null,               -- hash de password_hash() (bcrypt)
    estado        varchar(20)  not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO')),
    ultimoacceso  timestamptz,
    idrol         integer not null references rol(idrol)
);
create unique index usuario_username_uk on usuario (lower(username));
create unique index usuario_email_uk on usuario (lower(email));
create index usuario_idrol_idx on usuario(idrol);

create table bitacora (
    idbitacora   integer generated by default as identity primary key,
    accion       varchar(50) not null,
    modulo       varchar(50) not null,
    tabla        varchar(50),
    idregistro   integer,
    descripcion  text,
    ip           varchar(64),
    fecha        timestamptz not null default now(),
    idusuario    integer references usuario(idusuario)
);
create index bitacora_idusuario_idx on bitacora(idusuario);
create index bitacora_fecha_idx on bitacora(fecha desc);

create table configuracion (
    clave        varchar(100) primary key,
    valor        varchar(255) not null,
    descripcion  varchar(255)
);

-- Códigos de verificación para recuperar la contraseña por correo
-- (el código se guarda con password_hash(), vence a los 15 minutos y admite 5 intentos)
create table recuperacion (
    idrecuperacion  integer generated by default as identity primary key,
    idusuario       integer not null references usuario(idusuario) on delete cascade,
    codigohash      varchar(255) not null,
    expira          timestamptz not null,
    intentos        integer not null default 0 check (intentos >= 0),
    usado           boolean not null default false,
    ip              varchar(64),
    creado          timestamptz not null default now()
);
create index recuperacion_idusuario_idx on recuperacion(idusuario, creado desc);

-- Sesiones PHP (session_set_save_handler): permiten que la sesión sobreviva entre
-- las distintas instancias serverless de Vercel
create table sesion (
    id           varchar(128) primary key,
    datos        text not null default '',
    actualizado  timestamptz not null default now()
);
create index sesion_actualizado_idx on sesion(actualizado);

-- P2 · Gestión médica
create table especialidad (
    idespecialidad  integer generated by default as identity primary key,
    nombre          varchar(100) not null,
    descripcion     varchar(255),
```

```
    estado          varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO'))
);
-- RF04: no se repiten nombres normalizados ("Pediatría" = "pediatria ")
create unique index especialidad_nombre_uk on especialidad (normalizar(nombre));

create table medico (
    idmedico             integer generated by default as identity primary key,
    codigo               varchar(20)  not null unique,
    registroprofesional  varchar(50)  not null unique,
    nombres              varchar(100) not null,
    apellidos            varchar(100) not null,
    telefono             varchar(30),
    email                varchar(150),
    estado               varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO')),
    idusuario            integer not null unique references usuario(idusuario)
);

create table medicoespecialidad (
    idmedico        integer not null references medico(idmedico) on delete cascade,
    idespecialidad  integer not null references especialidad(idespecialidad),
    primary key (idmedico, idespecialidad)
);
create index medicoespecialidad_idespecialidad_idx on medicoespecialidad(idespecialidad);

create table horario (
    idhorario   integer generated by default as identity primary key,
    diasemana   integer not null check (diasemana between 1 and 7),   -- 1 = lunes … 7 =
domingo (ISO)
    horainicio  time not null,
    horafin     time not null,
    estado      varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    idmedico    integer not null references medico(idmedico) on delete cascade,
    constraint horario_rango_ck check (horafin > horainicio),
    -- RF05: un médico no puede tener dos horarios activos que se crucen el mismo día
    constraint horario_sin_cruce exclude using gist (
        idmedico with =, diasemana with =,
        tsrange(date '2000-01-01' + horainicio, date '2000-01-01' + horafin) with &&
    ) where (estado = 'ACTIVO')
);
create index horario_idmedico_idx on horario(idmedico);

create table bloqueoagenda (
    idbloqueo   integer generated by default as identity primary key,
    fecha       date not null,
    horainicio  time,
    horafin     time,
    tipo        varchar(30) not null check (tipo in ('FERIADO', 'VACACION', 'PERMISO',
'OTRO')),
    motivo      varchar(255),
    idmedico    integer not null references medico(idmedico) on delete cascade,
    constraint bloqueo_rango_ck check ((horainicio is null and horafin is null)
        or (horainicio is not null and horafin is not null and horafin > horainicio))
);
create index bloqueoagenda_idmedico_fecha_idx on bloqueoagenda(idmedico, fecha);

-- P3 · Gestión de pacientes
create table paciente (
    idpaciente            integer generated by default as identity primary key,
    codigo                varchar(20)  not null unique,
    ci                    varchar(20)  not null unique,        -- RF02: CI único
    nombres               varchar(100) not null,
```

```
    apellidos             varchar(100) not null,
    fechanacimiento       date not null check (fechanacimiento > date '1900-01-01'),
    sexo                  varchar(1) not null check (sexo in ('M', 'F', 'O')),
    telefono              varchar(30)  not null,
    email                 varchar(150),
    direccion             varchar(255),
    tiposangre            varchar(5) check (tiposangre in ('A+', 'A-', 'B+', 'B-', 'AB+',
'AB-', 'O+', 'O-')),
    aceptanotificaciones  boolean not null default true,
    estado                varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO')),
    idusuario             integer unique references usuario(idusuario)   -- cuenta del portal
(opcional)
);
create index paciente_nombre_idx on paciente (lower(apellidos), lower(nombres));

create table contactoemergencia (
    idcontacto  integer generated by default as identity primary key,
    nombre      varchar(150) not null,
    parentesco  varchar(50)  not null,
    telefono    varchar(30)  not null,
    idpaciente  integer not null references paciente(idpaciente) on delete cascade
);
create index contactoemergencia_idpaciente_idx on contactoemergencia(idpaciente);

create table historiaclinica (
    idhistoria              integer generated by default as identity primary key,
    numero                  varchar(20) not null unique,
    antecedentespersonales  text,
    antecedentesfamiliares  text,
    alergias                text,
    enfermedadescronicas    text,
    medicacionhabitual      text,
    idpaciente              integer not null unique references paciente(idpaciente) on delete
cascade
);

-- P4 · Gestión de citas
create table estadocita (
    codigo        varchar(20) primary key,
    nombre        varchar(50) not null,
    descripcion   varchar(255),
    ocupahorario  boolean not null default true
);

create sequence cita_codigo_seq;

create table cita (
    idcita             integer generated by default as identity primary key,
    codigo             varchar(30) not null unique
                       default 'CM-' || lpad(nextval('cita_codigo_seq')::text, 4, '0'),
    fecha              date not null,
    horainicio         time not null,
    horafin            time not null,
    modalidad          varchar(20) not null default 'PRESENCIAL' check (modalidad in
('PRESENCIAL', 'VIRTUAL')),
    motivo             varchar(255),
    fecharegistro      timestamptz not null default now(),
    idpaciente         integer not null references paciente(idpaciente),        -- RN03 / RN11
    idmedico           integer not null references medico(idmedico),            -- RN03 / RN11
    idespecialidad     integer not null references especialidad(idespecialidad),
    codigoestado       varchar(20) not null default 'PROGRAMADA' references estadocita(codigo),
    idusuarioregistro  integer references usuario(idusuario),
```

```
    constraint cita_rango_ck check (horafin > horainicio),
    -- RF07 / RN07: un médico no puede tener citas superpuestas (las canceladas no ocupan
horario)
    constraint cita_sin_cruce_medico exclude using gist (
        idmedico with =, tsrange(fecha + horainicio, fecha + horafin) with &&
    ) where (codigoestado <> 'CANCELADA'),
    -- Un paciente tampoco puede estar en dos citas a la vez
    constraint cita_sin_cruce_paciente exclude using gist (
        idpaciente with =, tsrange(fecha + horainicio, fecha + horafin) with &&
    ) where (codigoestado <> 'CANCELADA')
);
create index cita_idpaciente_idx on cita(idpaciente);
create index cita_idmedico_fecha_idx on cita(idmedico, fecha);
create index cita_fecha_idx on cita(fecha);
create index cita_idespecialidad_idx on cita(idespecialidad);
create index cita_codigoestado_idx on cita(codigoestado);
create index cita_idusuarioregistro_idx on cita(idusuarioregistro);

create table historialcita (
    idhistorial    integer generated by default as identity primary key,
    accion         varchar(30) not null,
    fechaanterior  date,
    horaanterior   time,
    fechanueva     date,
    horanueva      time,
    motivo         varchar(255),
    fecha          timestamptz not null default now(),
    idcita         integer not null references cita(idcita) on delete cascade,
    codigoestado   varchar(20) not null references estadocita(codigo),
    idusuario      integer references usuario(idusuario)
);
create index historialcita_idcita_idx on historialcita(idcita);
create index historialcita_codigoestado_idx on historialcita(codigoestado);
create index historialcita_idusuario_idx on historialcita(idusuario);

-- P5 · Atención médica
create table consulta (
    idconsulta          integer generated by default as identity primary key,
    fechahora           timestamptz not null default now(),
    motivo              varchar(255),
    peso                numeric(5,2) check (peso between 0.5 and 400),
    talla               numeric(5,2) check (talla between 20 and 250),
    presionarterial     varchar(20),
    temperatura         numeric(4,1) check (temperatura between 30 and 45),
    frecuenciacardiaca  integer check (frecuenciacardiaca between 20 and 250),
    examenfisico        text,
    diagnostico         text not null,
    observaciones       text,
    idcita              integer not null unique references cita(idcita)
);

create table medicamento (
    idmedicamento  integer generated by default as identity primary key,
    nombre         varchar(150) not null,
    presentacion   varchar(100),
    estado         varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO',
'INACTIVO'))
);

create table tratamiento (
    idtratamiento  integer generated by default as identity primary key,
    dosis          varchar(100) not null,
    frecuencia     varchar(100) not null,
```

```
    duracion       varchar(100) not null,
    indicaciones   text,
    idconsulta     integer not null references consulta(idconsulta) on delete cascade,
    idmedicamento  integer not null references medicamento(idmedicamento)
);
create index tratamiento_idconsulta_idx on tratamiento(idconsulta);
create index tratamiento_idmedicamento_idx on tratamiento(idmedicamento);

create table catalogoestudio (
    idcatalogo   integer generated by default as identity primary key,
    nombre       varchar(150) not null,
    categoria    varchar(100),
    descripcion  varchar(255),
    estado       varchar(20) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);

create table estudio (
    idestudio         integer generated by default as identity primary key,
    indicaciones      text,
    estado            varchar(20) not null default 'SOLICITADO'
                      check (estado in ('SOLICITADO', 'EN_PROCESO', 'COMPLETADO',
'CANCELADO')),
    fechasolicitud    timestamptz not null default now(),
    fecharesultado    timestamptz,
    resultado         text,
    archivoresultado  varchar(500),
    idconsulta        integer not null references consulta(idconsulta) on delete cascade,
    idcatalogo        integer not null references catalogoestudio(idcatalogo)
);
create index estudio_idconsulta_idx on estudio(idconsulta);
create index estudio_idcatalogo_idx on estudio(idcatalogo);

create table notificacion (
    idnotificacion  integer generated by default as identity primary key,
    tipo            varchar(30) not null,
    canal           varchar(20) not null default 'APP' check (canal in ('APP', 'EMAIL', 'SMS',
'WHATSAPP')),
    mensaje         text not null,
    programadapara  timestamptz,
    enviadaen       timestamptz,
    leidaen         timestamptz,
    idcita          integer references cita(idcita) on delete cascade,
    idestudio       integer references estudio(idestudio) on delete cascade
);
create index notificacion_idcita_idx on notificacion(idcita);
create index notificacion_idestudio_idx on notificacion(idestudio);
```